ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Αναγνώριση Ελληνικών Παραδοσιακών Φορεσιών

ΜΕ ΤΕΧΝΙΚΕΣ ΜΗΧΑΝΙΚΗΣ ΜΑΘΗΣΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΙΚΗΣ ΟΡΑΣΗΣ

Recognition of Greek Traditional Costumes

USING MACHINE LEARNING AND COMPUTER VISION TECHNIQUES



Εκπονήθηκε από:
Ελένη Ευαγγελία Κούμπου

Επιβλέπων Καθηγητής:
Διονύσιος Πολίτης

Θεσσαλονίκη
2026

Περίλιψη

Η παρούσα πτυχιακή εργασία πραγματεύεται την ανάπτυξη ενός συστήματος αυτόματης αναγνώρισης της γεωγραφικής προέλευσης ελληνικών παραδοσιακών φορεσιών από εικόνες, με αξιοποίηση τεχνικών Μηχανικής Μάθησης, Βαθιάς Μάθησης και Υπολογιστικής Όρασης. Για τον σκοπό αυτό δημιουργήθηκε και οργανώθηκε κατάλληλο σύνολο δεδομένων, ενώ μελετήθηκαν και συγκρίθηκαν διαφορετικές αρχιτεκτονικές νευρωνικών δικτύων και στρατηγικές εκπαίδευσης. Η αξιολόγηση των μοντέλων πραγματοποιήθηκε με κατάλληλες μετρικές απόδοσης, ώστε να επιλεγεί η καταλληλότερη προσέγγιση για το συγκεκριμένο πρόβλημα ταξινόμησης εικόνων. Το τελικό μοντέλο ενσωματώθηκε σε διαδικτυακή εφαρμογή, μέσω της οποίας ο χρήστης μπορεί να εισάγει μία εικόνα παραδοσιακής φορεσιάς και να λαμβάνει τις πιθανότερες γεωγραφικές περιοχές προέλευσής της. Η εργασία συνδυάζει τη μελέτη σύγχρονων τεχνικών τεχνητής νοημοσύνης με μία εφαρμογή σχετική με την ελληνική πολιτιστική κληρονομιά.

Abstract

This thesis focuses on the development of a system for the automatic recognition of the geographical origin of Greek traditional costumes from images, using Machine Learning, Deep Learning, and Computer Vision techniques. For this purpose, an appropriate dataset was created and organized, while different neural network architectures and training strategies were studied and compared. The models were evaluated using suitable performance metrics in order to identify the most appropriate approach for the specific image classification problem. The final model was integrated into a web application through which the user can upload an image of a traditional costume and receive the most likely geographical regions of origin. The thesis combines the study of modern Artificial Intelligence techniques with an application related to Greek cultural heritage.

Περιεχόμενα

Περίληψη **1**

Abstract **1**

ΚΕΦΑΛΑΙΟ 1 - ΕΙΣΑΓΩΓΗ **6**

1.1 Εισαγωγή και αντικείμενο της εργασίας 6

1.2 Περιγραφή του προβλήματος και κίνητρο 6

1.3 Σκοπός και στόχοι της εργασίας 7

1.4 Μεθοδολογική προσέγγιση 8

1.5 Δομή της πτυχιακής εργασίας 8

ΚΕΦΑΛΑΙΟ 2 - ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ **10**

2.1 Εισαγωγή 10

2.2 Μηχανική Μάθηση και Βαθιά Μάθηση 10

2.3 Τεχνητά νευρωνικά δίκτυα 11

2.4 Ψηφιακές εικόνες και Υπολογιστική Όραση 12

2.5 Συνελικτικά Νευρωνικά Δίκτυα 13

2.5.1 Συνέλιξη και φίλτρα 13

2.5.2 Stride και Padding 14

2.5.3 Pooling και μείωση χωρικών διαστάσεων 14

2.5.4 Ιεραρχική εξαγωγή χαρακτηριστικών 14

2.6 Εκπαίδευση Νευρωνικών Δικτύων για ταξινόμηση εικόνων 15

2.6.1 Logits και Softmax 15

2.6.2 Συνάρτηση απώλειας 15

2.6.3 Backpropagation και βελτιστοποίηση 15

2.6.4 Batches και Epochs 16

2.7 Transfer Learning, Feature Extraction και Partial Fine- Tuning 16

2.7.1 Feature Extraction 16

2.7.2 Fine-Tuning και Partial Fine-Tuning 17

2.8 ImageNet και Προεκπαιδευμένα Μοντέλα 17

2.9 Αρχιτεκτονικές Βαθιάς Μάθησης 18

2.9.1 ResNet και Residual Learning 18

2.9.2 EfficientNet και Compound Scaling 19

2.9.3 ConvNeXt 20

2.9.4 Σύνδεση των αρχιτεκτονικών με την πειραματική διαδικασία 20

2.10 Μετρικές Αξιολόγησης Μοντέλων Ταξινόμησης 21

2.10.1 Confusion Matrix 21

2.10.2 Accuracy 22

2.10.3 Precision 22
2.10.4 Recall 23
2.10.5 F1-score 23
2.9.6 Macro και Weighted Average 23
2.10.7 Top-1 και Top-3 Accuracy 24
2.11 Συμπεράσματα Κεφαλαίου 25
ΚΕΦΑΛΑΙΟ 3 - ΣΥΝΟΛΟ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΠΡΟΕΠΕΞΕΡΓΑΣΙΑ 26
3.1 Εισαγωγή 26
3.2 Συλλογη και οργανωση 26
3.2.1 Ορισμός του προβλήματος και των γεωγραφικών κλάσεων 26
3.2.2 Πηγή και διαδικασία συλλογής των εικόνων 27
3.2.3 Σύνθεση και κατανομή του αρχικού dataset 28
3.3 Προεπεξεργασία και Έλεγχος Ποιότητας του Συνόλου Δεδομένων 29
3.3.1 Αρχική τυποποίηση των εικόνων 29
3.3.2 Αυτοματοποιημένος έλεγχος εγκυρότητας 30
3.3.3 Έλεγχος διπλοτύπων και οπτικής ομοιότητας 30
3.4 Διαχωρισμός του Dataset και Αποφυγή Διαρροής Δεδομένων 31
3.4.1 Δημιουργία των Training, Validation και Test Sets 31
3.4.2 Ομαδοποίηση παρόμοιων εικόνων 31
3.4.3 Group-aware Split και τελική κατανομή 32
3.5 Προετοιμασία των Δεδομένων για τα Μοντέλα Βαθιάς Μάθησης 33
3.5.1 Φόρτωση και αριθμητική αναπαράσταση των εικόνων 33
3.5.2 Resize, Tensors και ImageNet Normalization 34
3.6 Εμπλουτισμός και Φόρτωση των Δεδομένων Κατά την Εκπαίδευση 35
3.6.1 Data Augmentation 35
3.6.2 On-the-fly Augmentation και DataLoaders 37
3.7 Έλεγχος και Επαλήθευση της Τελικής Ροής Δεδομένων 37
3.7.1 Προγραμματιστικός και οπτικός έλεγχος 37
3.7.2 Συνολική ροή προετοιμασίας του dataset 39
3.8 Περιορισμοί του Dataset και Συμπεράσματα 40
3.8.1 Περιορισμοί και δυνατότητες επέκτασης 40
3.9 Συμπεράσματα κεφαλαίου 41
ΚΕΦΑΛΑΙΟ 4 - ΜΕΘΟΔΟΛΟΓΙΑ ΚΑΙ ΠΡΟΣΑΡΜΟΓΗ ΤΩΝ ΜΟΝΤΕΛΩΝ 42
4.1 Εισαγωγή 42
4.2 Υπολογιστικό Περιβάλλον και Βιβλιοθήκες 42
4.3 Επιλογή και Αρχικοποίηση Μοντέλων 44

4.4 Προσαρμογή των μοντέλων στο πρόβλημα Ταξινόμησης 46
4.5 Στρατιγική Feature Extraction 48
4.6 Στρατιγική Partial Fine-Tuning 50
4.7 Οι έξι πειραματικές διαμορφώσεισ 53
4.8 Έλεγχος της ορθής προσαρμογής των μοντέλων 55
4.9 Συμπεράσματα κεφαλάιου 58
ΚΕΦΑΛΑΙΟ 5 - ΕΚΠΑΙΔΕΥΣΗ ΚΑΙ ΠΕΙΡΑΜΑΤΙΚΗ ΔΙΑΔΙΚΑΣΙΑ 60
5.1 Εισαγωγή 60
5.2 Κοινές Ρυθμίσεις της Πειραματικής Εκπαίδευσης 61
5.3 Συνάρτηση Απώλειας και Βελτιστοποίηση 61
5.4 Διαδικασία Εκπαίδευσης και Επικύρωσης 63
5.5 Early Stopping και Αποθήκευση Καλύτερου Μοντέλου 65
5.6 Εκτέλεση και Παρακολούθηση των Έξι Πειραμάτων 66
5.7 Συμπεράσματα Κεφαλαίου 68
ΚΕΦΑΛΑΙΟ 6 - ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ 69
6.1 Εισαγωγή 69
6.2 Πλαίσιο και Μετρικές Αξιολόγησης 69
6.3 Συνολικά Αποτελέσματα των Έξι Πειραμάτων 71
6.4 Αποτελέσματα ανά Αρχιτεκτονική και Στρατηγική Μεταφοράς Μάθησης 73
6.5 Αξιολόγηση ανά Γεωγραφική Κατηγορία 75
6.6 Ανάλυση Πίνακα Σύγχυσης και Σφαλμάτων Ταξινόμησης 76
6.7 Αξιολόγηση Top-1 και Top-3 Προβλέψεων 78
6.8 Συγκριτική Αποτίμηση και Επιλογή Τελικού Μοντέλου 79
6.9 Συμπεράσματα Κεφαλαίου 80
ΚΕΦΑΛΑΙΟ 7 - ΑΝΑΠΤΥΞΗ ΔΙΑΔΙΚΤΥΑΚΗΣ ΕΦΑΡΜΟΓΗΣ 82
7.1 Εισαγωγή 82
7.2 Αρχιτεκτονική και Οργάνωση της Εφαρμογής 83
7.3 Backend και Δρομολόγηση με Flask 85
7.4 Ενσωμάτωση του Εκπαιδευμένου Μοντέλου και Διαδικασία Πρόβλεψης 87
7.5 Υπολογισμός και Παρουσίαση των Top-3 Προβλέψεων 91
7.6 Σχεδίαση της Διεπαφής Χρήστη 94
7.7 Εκπαιδευτικό Περιεχόμενο και Έλεγχος Λειτουργίας 99
7.8 Συμπεράσματα Κεφαλαίου 101
ΚΕΦΑΛΑΙΟ 8 - ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΑΙ ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ 103
8.1 Συνολική Αποτίμηση της Εργασίας 103
8.2 Βασικά Συμπεράσματα 104

8.3 Περιορισμοί της Παρούσας Προσέγγισης 104
8.4 Μελλοντικές Επεκτάσεις 105
8.5 Τελικό Συμπέρασμα 106
ΒΙΒΛΙΟΓΡΑΦΙΑ ΚΑΙ ΠΗΓΕΣ 107
Βιβλιογραφία 107
Πηγές 107

ΚΕΦΑΛΑΙΟ 1 - ΕΙΣΑΓΩΓΗ

1.1 ΕΙΣΑΓΩΓΗ ΚΑΙ ΑΝΤΙΚΕΙΜΕΝΟ ΤΗΣ ΕΡΓΑΣΙΑΣ

Η συνεχής ανάπτυξη της Τεχνητής Νοημοσύνης (Artificial Intelligence – AI) και της Μηχανικής Μάθησης (Machine Learning – ML) έχει οδηγήσει στη δημιουργία υπολογιστικών συστημάτων ικανών να επεξεργάζονται μεγάλους όγκους δεδομένων και να αναγνωρίζουν σύνθετα πρότυπα. Ιδιαίτερα σημαντική είναι η πρόοδος που έχει σημειωθεί στον τομέα της Υπολογιστικής Όρασης (Computer Vision), ο οποίος ασχολείται με την ανάπτυξη μεθόδων που επιτρέπουν σε υπολογιστικά συστήματα να εξάγουν και να αξιοποιούν πληροφορία από ψηφιακές εικόνες και άλλο οπτικό περιεχόμενο.

Η Βαθιά Μάθηση (Deep Learning) έχει διαδραματίσει καθοριστικό ρόλο στην εξέλιξη της Υπολογιστικής Όρασης. Μέσω νευρωνικών δικτύων με πολλαπλά επίπεδα επεξεργασίας είναι δυνατή η αυτόματη εκμάθηση οπτικών χαρακτηριστικών από τα διαθέσιμα δεδομένα, περιορίζοντας την ανάγκη για χειροκίνητο καθορισμό των χαρακτηριστικών που πρέπει να εξετάζονται. Η προσέγγιση αυτή έχει ιδιαίτερη σημασία σε προβλήματα ταξινόμησης εικόνων, όπου το ζητούμενο είναι η αντιστοίχιση μιας εικόνας σε μία από προκαθορισμένες κατηγορίες.

Παράλληλα με την τεχνολογική εξέλιξη, η ψηφιοποίηση πολιτιστικού υλικού δημιουργεί νέα πεδία εφαρμογής για τις τεχνικές της Πληροφορικής. Ψηφιακές συλλογές εικόνων μπορούν να οργανωθούν, να αναλυθούν και να αξιοποιηθούν μέσω τεχνικών Μηχανικής Μάθησης, επιτρέποντας την ανάπτυξη εφαρμογών που συνδυάζουν την τεχνολογία με την παρουσίαση και μελέτη της πολιτιστικής κληρονομιάς.

Στο πλαίσιο αυτό, η παρούσα πτυχιακή εργασία εξετάζει την εφαρμογή τεχνικών Βαθιάς Μάθησης και Υπολογιστικής Όρασης για την αυτόματη αναγνώριση της γεωγραφικής προέλευσης ελληνικών παραδοσιακών φορεσιών από ψηφιακές εικόνες. Οι παραδοσιακές φορεσιές επιλέχθηκαν ως πεδίο εφαρμογής λόγω της σημαντικής οπτικής ποικιλομορφίας που παρουσιάζουν, αλλά και της ύπαρξης κοινών χαρακτηριστικών μεταξύ διαφορετικών γεωγραφικών περιοχών, τα οποία καθιστούν την αυτόματη διάκρισή τους ένα ενδιαφέρον πρόβλημα ταξινόμησης εικόνων.

Το πρόβλημα διατυπώνεται υπολογιστικά ως πρόβλημα πολυταξικής ταξινόμησης εικόνων (multi-class image classification). Κάθε εικόνα πρέπει να αντιστοιχιστεί σε μία από εννέα προκαθορισμένες γεωγραφικές κλάσεις: Ήπειρος, Κρήτη, Μακεδονία, Νησιά Αιγαίου, Νησιά Ιονίου, Πελοπόννησος, Στερεά Ελλάδα, Θεσσαλία και Θράκη. Οι διαφορετικές τοπικές φορεσιές που προέρχονται από την ίδια γεωγραφική περιοχή δεν αποτελούν ξεχωριστές κλάσεις, αλλά αντιμετωπίζονται ως διαφορετικά δείγματα της αντίστοιχης γεωγραφικής κατηγορίας. Για την αντιμετώπιση του προβλήματος δημιουργήθηκε και οργανώθηκε κατάλληλο σύνολο δεδομένων εικόνων, πραγματοποιήθηκε συστηματική προεπεξεργασία και διαχωρισμός των δεδομένων και αξιοποιήθηκαν μοντέλα βαθιάς μάθησης με τεχνικές transfer learning. Η απόδοσή τους εξετάστηκε μέσω κατάλληλων μετρικών ταξινόμησης, με στόχο την αξιολόγηση της δυνατότητάς τους να γενικεύουν σε εικόνες που δεν χρησιμοποιήθηκαν κατά την εκπαίδευση. Η εργασία ολοκληρώνεται με την ανάπτυξη διαδικτυακής εφαρμογής, μέσω της οποίας το εκπαιδευμένο μοντέλο ενσωματώνεται σε ένα λειτουργικό περιβάλλον αλληλεπίδρασης με τον χρήστη. Ο χρήστης μπορεί να εισάγει μία εικόνα παραδοσιακής φορεσιάς και το σύστημα να επιστρέφει τις πιθανότερες γεωγραφικές περιοχές προέλευσης. Με τον τρόπο αυτό, η διαδικασία δεν περιορίζεται στην πειραματική εκπαίδευση ενός μοντέλου, αλλά επεκτείνεται στην ενσωμάτωσή του σε μία ολοκληρωμένη εφαρμογή λογισμικού.

1.2 ΠΕΡΙΓΡΑΦΗ ΤΟΥ ΠΡΟΒΛΗΜΑΤΟΣ ΚΑΙ ΚΙΝΗΤΡΟ

Η ταξινόμηση εικόνων αποτελεί ένα από τα βασικά προβλήματα της Υπολογιστικής Όρασης. Δεδομένης μιας εικόνας εισόδου και ενός προκαθορισμένου συνόλου κατηγοριών, το υπολογιστικό σύστημα καλείται να προσδιορίσει την κατηγορία στην οποία είναι πιθανότερο να ανήκει η εικόνα.

Παρότι η γενική διατύπωση του προβλήματος είναι απλή, η αποτελεσματική επίλυσή του εξαρτάται σε μεγάλο βαθμό από την ποιότητα των δεδομένων, τη δυσκολία διάκρισης μεταξύ των κλάσεων και την ικανότητα του μοντέλου να εξάγει χαρακτηριστικά που είναι χρήσιμα για την ταξινόμηση.

Στην περίπτωση των ελληνικών παραδοσιακών φορεσιών, το πρόβλημα παρουσιάζει πρόσθετες δυσκολίες. Η γεωγραφική προέλευση μιας φορεσιάς δεν καθορίζεται απαραίτητα από ένα μοναδικό και εύκολα αναγνωρίσιμο οπτικό χαρακτηριστικό. Η συνολική εμφάνιση μπορεί να προκύπτει από συνδυασμό διαφορετικών στοιχείων, όπως η μορφή των ενδυμάτων, οι χρωματικοί συνδυασμοί, τα κεντήματα, τα διακοσμητικά στοιχεία, τα καλύμματα κεφαλής και άλλα επιμέρους χαρακτηριστικά.

Επιπλέον, διαφορετικές τοπικές φορεσιές της ίδιας γεωγραφικής περιοχής μπορούν να παρουσιάζουν σημαντικές μεταξύ τους διαφοροποιήσεις. Επομένως, το μοντέλο δεν καλείται απλώς να απομνημονεύσει μία συγκεκριμένη εμφάνιση για κάθε περιοχή, αλλά να μάθει οπτικά χαρακτηριστικά που μπορούν να συνδέσουν διαφορετικές φορεσιές με την ίδια γεωγραφική κλάση.

Παράλληλα, φορεσιές διαφορετικών περιοχών ενδέχεται να εμφανίζουν οπτικές ομοιότητες. Παρόμοια είδη ενδυμάτων, χρωματικοί συνδυασμοί ή διακοσμητικά χαρακτηριστικά μπορούν να εμφανίζονται σε περισσότερες από μία γεωγραφικές κατηγορίες. Το γεγονός αυτό δημιουργεί πιθανή οπτική επικάλυψη μεταξύ των κλάσεων (inter-class similarity) και αυξάνει τη δυσκολία του προβλήματος ταξινόμησης.
Μια ακόμη πρόκληση αφορά το περιορισμένο μέγεθος του διαθέσιμου συνόλου δεδομένων. Η εκπαίδευση σύγχρονων μοντέλων βαθιάς μάθησης συνδέεται συνήθως με τη χρήση μεγάλου αριθμού εικόνων. Στην παρούσα εργασία, αντίθετα, το διαθέσιμο dataset είναι σημαντικά μικρότερο. Το γεγονός αυτό καθιστά ιδιαίτερα σημαντικές τεχνικές όπως το transfer learning και το data augmentation, οι οποίες επιτρέπουν την αποτελεσματικότερη αξιοποίηση των διαθέσιμων δεδομένων.

Το ενδιαφέρον της εργασίας από πλευράς Πληροφορικής βρίσκεται επομένως στη μελέτη ενός ολοκληρωμένου προβλήματος επιβλεπόμενης ταξινόμησης εικόνων με περιορισμένο σύνολο δεδομένων. Το πρόβλημα περιλαμβάνει όχι μόνο την επιλογή και εκπαίδευση ενός μοντέλου, αλλά ολόκληρη την υπολογιστική αλυσίδα: δημιουργία και έλεγχο του dataset, προεπεξεργασία εικόνων, αντιμετώπιση πιθανών πηγών διαρροής δεδομένων, εκπαίδευση μοντέλων, ποσοτική αξιολόγηση των αποτελεσμάτων και τελική ενσωμάτωση του επιλεγμένου μοντέλου σε διαδικτυακή εφαρμογή. Το κίνητρο της εργασίας δεν περιορίζεται συνεπώς στην επίτευξη μιας υψηλής τιμής ακρίβειας. Εξίσου σημαντική είναι η διερεύνηση του κατά πόσο μία ολοκληρωμένη μεθοδολογία Μηχανικής Μάθησης μπορεί να εφαρμοστεί σε ένα εξειδικευμένο πρόβλημα πολιτιστικού περιεχομένου, όπου τα διαθέσιμα δεδομένα είναι περιορισμένα και οι κατηγορίες παρουσιάζουν σημαντική οπτική πολυπλοκότητα.

1.3 ΣΚΟΠΟΣ ΚΑΙ ΣΤΟΧΟΙ ΤΗΣ ΕΡΓΑΣΙΑΣ

Βασικός σκοπός της παρούσας πτυχιακής εργασίας είναι η σχεδίαση, ανάπτυξη και αξιολόγηση ενός συστήματος αυτόματης αναγνώρισης της γεωγραφικής προέλευσης ελληνικών παραδοσιακών φορεσιών από εικόνες, με χρήση τεχνικών Μηχανικής Μάθησης και Υπολογιστικής Όρασης.
Για την επίτευξη του παραπάνω σκοπού τέθηκαν επιμέρους στόχοι που καλύπτουν ολόκληρο τον κύκλο ανάπτυξης ενός συστήματος ταξινόμησης εικόνων. Αρχικά απαιτήθηκε η δημιουργία ενός κατάλληλα οργανωμένου συνόλου δεδομένων, στο οποίο οι εικόνες αντιστοιχίζονται σε εννέα γεωγραφικές κλάσεις. Ιδιαίτερη σημασία δόθηκε στον έλεγχο της ποιότητας και της ομοιότητας των εικόνων, καθώς και στον ορθό διαχωρισμό τους σε training, validation και test sets. Στη συνέχεια, στόχος ήταν η κατάλληλη προετοιμασία των εικόνων για την είσοδό τους στα μοντέλα βαθιάς μάθησης. Η διαδικασία περιλαμβάνει την αλλαγή των διαστάσεων, τη μετατροπή σε tensors, την κανονικοποίηση και την εφαρμογή ελεγχόμενου data augmentation στα δεδομένα εκπαίδευσης.

Σε επίπεδο Μηχανικής Μάθησης, επιδιώκεται η αξιοποίηση και σύγκριση κατάλληλων προεκπαιδευμένων αρχιτεκτονικών νευρωνικών δικτύων, με προσαρμογή τους στο συγκεκριμένο πρόβλημα μέσω transfer learning. Η αξιολόγηση δεν περιορίζεται αποκλειστικά στη συνολική ακρίβεια, αλλά εξετάζει την απόδοση των μοντέλων με κατάλληλες μετρικές, ώστε να είναι δυνατή η πληρέστερη κατανόηση της συμπεριφοράς τους.

Παράλληλα, στόχος είναι η εξέταση των προβλέψεων του μοντέλου όχι μόνο ως μία μοναδική τελική κατηγορία, αλλά και μέσω της παρουσίασης των τριών πιθανότερων γεωγραφικών περιοχών (Top-3). Η προσέγγιση αυτή είναι ιδιαίτερα χρήσιμη στο συγκεκριμένο πρόβλημα, καθώς επιτρέπει να παρουσιαστεί στον χρήστη περισσότερη πληροφορία σχετικά με την κατανομή των προβλέψεων του μοντέλου.

Τελικός στόχος είναι η πρακτική αξιοποίηση του εκπαιδευμένου μοντέλου μέσω της ανάπτυξης μιας διαδικτυακής εφαρμογής. Η εφαρμογή λειτουργεί ως διεπαφή μεταξύ του χρήστη και του μοντέλου Μηχανικής Μάθησης, επιτρέποντας την εισαγωγή μιας εικόνας και την παρουσίαση των αποτελεσμάτων αναγνώρισης με κατανοητό τρόπο.

Συνολικά, η εργασία επιδιώκει να καλύψει όχι μόνο το στάδιο της εκπαίδευσης ενός μοντέλου, αλλά την ανάπτυξη μιας ολοκληρωμένης υπολογιστικής λύσης, από την αρχική διαχείριση των δεδομένων μέχρι την τελική διάθεση της λειτουργικότητας μέσω μιας εφαρμογής λογισμικού.

1.4 ΜΕΘΟΔΟΛΟΓΙΚΗ ΠΡΟΣΕΓΓΙΣΗ

Η υλοποίηση της εργασίας οργανώθηκε σε διαδοχικά στάδια, ώστε κάθε μέρος της διαδικασίας να μπορεί να ελεγχθεί και να αξιολογηθεί ανεξάρτητα πριν από τη μετάβαση στο επόμενο. Το πρώτο στάδιο αφορούσε τη συλλογή και οργάνωση του οπτικού υλικού. Οι εικόνες κατηγοριοποιήθηκαν σύμφωνα με τη γεωγραφική τους προέλευση και οργανώθηκαν στις εννέα κλάσεις του προβλήματος. Ακολούθησε διαδικασία τυποποίησης και ελέγχου του dataset, με στόχο τον εντοπισμό μη έγκυρων αρχείων, ακριβών διπλοτύπων και εικόνων με υψηλή οπτική ομοιότητα.

Ιδιαίτερη προσοχή δόθηκε στον διαχωρισμό των δεδομένων. Η ύπαρξη πολύ όμοιων εικόνων σε διαφορετικά σύνολα θα μπορούσε να οδηγήσει σε διαρροή δεδομένων (data leakage) και να δημιουργήσει υπερβολικά αισιόδοξη εικόνα για την πραγματική ικανότητα γενίκευσης ενός μοντέλου. Για τον λόγο αυτό, η πληροφορία οπτικής ομοιότητας χρησιμοποιήθηκε για την ομαδοποίηση σχετικών εικόνων πριν από τον διαχωρισμό του dataset σε training, validation και test sets.

Στο επόμενο στάδιο πραγματοποιήθηκε η προετοιμασία των δεδομένων για τα μοντέλα βαθιάς μάθησης μέσω της PyTorch. Οι εικόνες μετατράπηκαν στην απαιτούμενη αριθμητική μορφή και εφαρμόστηκαν οι απαραίτητοι μετασχηματισμοί. Στο training set χρησιμοποιήθηκε ελεγχόμενο data augmentation, ενώ τα validation και test sets διατηρήθηκαν χωρίς τυχαίους μετασχηματισμούς, ώστε η διαδικασία αξιολόγησης να παραμένει σταθερή και συγκρίσιμη.

Για την ταξινόμηση των εικόνων αξιοποιήθηκαν προεκπαιδευμένα μοντέλα βαθιάς μάθησης, τα οποία προσαρμόστηκαν στις εννέα κλάσεις του συγκεκριμένου προβλήματος. Η χρήση transfer learning επιτρέπει την αξιοποίηση οπτικών χαρακτηριστικών που έχουν ήδη αποκτηθεί μέσω προεκπαίδευσης σε μεγάλο σύνολο εικόνων και είναι ιδιαίτερα χρήσιμη όταν το διαθέσιμο dataset του νέου προβλήματος είναι περιορισμένο.

Ακολούθησε η πειραματική εκπαίδευση και αξιολόγηση των μοντέλων. Η απόδοσή τους εξετάστηκε σε δεδομένα που δεν χρησιμοποιήθηκαν για την εκπαίδευση, μέσω κατάλληλων μετρικών ταξινόμησης. Παράλληλα, η ανάλυση των αποτελεσμάτων επιτρέπει την εξέταση όχι μόνο της συνολικής απόδοσης, αλλά και των διαφορών που εμφανίζονται μεταξύ των γεωγραφικών κλάσεων.

Το τελικό στάδιο της εργασίας αφορά την ενσωμάτωση του επιλεγμένου μοντέλου σε διαδικτυακή εφαρμογή. Για την ανάπτυξη του backend χρησιμοποιήθηκε το Flask, ενώ δημιουργήθηκε κατάλληλη διεπαφή μέσω της οποίας ο χρήστης μπορεί να υποβάλει μια εικόνα προς αναγνώριση. Η εικόνα υποβάλλεται στην απαιτούμενη προεπεξεργασία, παρέχεται στο εκπαιδευμένο μοντέλο και τα αποτελέσματα επιστρέφονται στη διεπαφή της εφαρμογής με τη μορφή των πιθανότερων γεωγραφικών περιοχών.

Η μεθοδολογία καλύπτει επομένως ολόκληρο τον κύκλο ανάπτυξης της εφαρμογής: δεδομένα → προεπεξεργασία → μοντέλο → εκπαίδευση → αξιολόγηση → ενσωμάτωση σε εφαρμογή. Η αναλυτική παρουσίαση κάθε σταδίου πραγματοποιείται στα αντίστοιχα κεφάλαια της εργασίας.

1.5 ΔΟΜΗ ΤΗΣ ΠΤΥΧΙΑΚΗΣ ΕΡΓΑΣΙΑΣ

Η πτυχιακή εργασία οργανώνεται σε οκτώ κεφάλαια, τα οποία ακολουθούν τη λογική πορεία από τη θεωρητική θεμελίωση του προβλήματος έως την ανάπτυξη και αξιολόγηση της τελικής εφαρμογής.
Το **Κεφάλαιο 1** παρουσιάζει το αντικείμενο της εργασίας, το πρόβλημα που εξετάζεται, το κίνητρο, τους βασικούς στόχους και τη συνολική μεθοδολογική προσέγγιση.

Στο **Κεφάλαιο 2** παρουσιάζεται το απαραίτητο θεωρητικό υπόβαθρο. Αναλύονται βασικές έννοιες της Μηχανικής Μάθησης και της Βαθιάς Μάθησης, η ταξινόμηση εικόνων, τα νευρωνικά και συνελικτικά νευρωνικά δίκτυα, το transfer learning και οι βασικές αρχές αξιολόγησης μοντέλων ταξινόμησης.
Το **Κεφάλαιο 3** επικεντρώνεται στο **σύνολο δεδομένων και στην προεπεξεργασία του**. Παρουσιάζονται η διαδικασία συλλογής και οργάνωσης των εικόνων, οι έλεγχοι ποιότητας και ομοιότητας, η αντιμετώπιση πιθανής διαρροής δεδομένων, ο διαχωρισμός σε training, validation και test sets, καθώς και η τελική προετοιμασία των εικόνων μέσω της PyTorch.
Στο **Κεφάλαιο 4** παρουσιάζονται η μεθοδολογία Μηχανικής Μάθησης και οι αρχιτεκτονικές που χρησιμοποιήθηκαν στο συγκεκριμένο πρόβλημα. Αναλύεται ο τρόπος προσαρμογής των προεκπαιδευμένων μοντέλων στις εννέα γεωγραφικές κλάσεις και οι βασικές σχεδιαστικές επιλογές της υλοποίησης.
Το **Κεφάλαιο 5** περιγράφει την **εκπαίδευση και την πειραματική διαδικασία**. Παρουσιάζονται οι παράμετροι εκπαίδευσης, οι διαφορετικές πειραματικές ρυθμίσεις και η διαδικασία που ακολουθήθηκε για τη συστηματική σύγκριση των μοντέλων.
Στο **Κεφάλαιο 6** παρουσιάζονται και αναλύονται τα **αποτελέσματα της αξιολόγησης**. Εξετάζεται η συνολική απόδοση των μοντέλων, η συμπεριφορά τους ανά γεωγραφική κλάση και τα σημαντικότερα μοτίβα σφαλμάτων που προκύπτουν από τις προβλέψεις.
Το **Κεφάλαιο 7** αφορά την **ανάπτυξη της διαδικτυακής εφαρμογής** και την ενσωμάτωση του εκπαιδευμένου μοντέλου σε αυτή. Παρουσιάζονται η αρχιτεκτονική της εφαρμογής, η σύνδεση του Flask backend με το μοντέλο, η διαδικασία εισαγωγής και επεξεργασίας μιας εικόνας και η παρουσίαση των αποτελεσμάτων στον τελικό χρήστη.
Τέλος, στο **Κεφάλαιο 8** παρουσιάζονται τα **συμπεράσματα της πτυχιακής εργασίας**, οι βασικοί περιορισμοί της προσέγγισης και πιθανές κατευθύνσεις για μελλοντική επέκταση, όπως η διεύρυνση του dataset, η αξιοποίηση πρόσθετων αρχιτεκτονικών και η μετάβαση από την αναγνώριση γεωγραφικής περιοχής στην αναγνώριση συγκεκριμένης τοπικής φορεσιάς.

ΚΕΦΑΛΑΙΟ 2 - ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ

2.1 ΕΙΣΑΓΩΓΗ

Το παρόν κεφάλαιο παρουσιάζει το θεωρητικό υπόβαθρο που είναι απαραίτητο για την κατανόηση της μεθοδολογίας που ακολουθείται στην παρούσα εργασία. Αρχικά παρουσιάζονται οι βασικές έννοιες της Μηχανικής Μάθησης και της Βαθιάς Μάθησης και στη συνέχεια αναλύεται η λειτουργία των τεχνητών νευρωνικών δικτύων και η εφαρμογή τους στην επεξεργασία ψηφιακών εικόνων. Ιδιαίτερη έμφαση δίνεται στα Συνελικτικά Νευρωνικά Δίκτυα, στις βασικές αρχές εκπαίδευσης ενός μοντέλου ταξινόμησης και στη μεταφορά μάθησης. Παρουσιάζονται επίσης οι έννοιες του feature extraction και του fine-tuning, καθώς αποτελούν τις δύο βασικές στρατηγικές που εξετάζονται πειραματικά στην εργασία. Τέλος, περιγράφονται οι οικογένειες αρχιτεκτονικών ResNet, EfficientNet και ConvNeXt και παρουσιάζονται οι βασικές μετρικές που χρησιμοποιούνται για την αξιολόγηση μοντέλων πολυταξικής ταξινόμησης εικόνων.

2.2 ΜΗΧΑΝΙΚΗ ΜΑΘΗΣΗ ΚΑΙ ΒΑΘΙΑ ΜΑΘΗΣΗ

Η **Τεχνητή Νοημοσύνη (Artificial Intelligence – AI)** αποτελεί έναν ευρύ κλάδο της Πληροφορικής που ασχολείται με τη σχεδίαση και ανάπτυξη υπολογιστικών συστημάτων ικανών να εκτελούν εργασίες που απαιτούν επεξεργασία πληροφοριών, αναγνώριση προτύπων, λήψη αποφάσεων ή άλλες λειτουργίες που συνδέονται με μορφές νοήμονος συμπεριφοράς. Στο ευρύτερο πεδίο της Τεχνητής Νοημοσύνης εντάσσεται η **Μηχανική Μάθηση (Machine Learning – ML)**, η οποία επιτρέπει σε ένα υπολογιστικό μοντέλο να προσαρμόζει τη συμπεριφορά του με βάση διαθέσιμα δεδομένα, αντί η συνολική λειτουργία του να καθορίζεται αποκλειστικά από κανόνες που έχουν προγραμματιστεί εκ των προτέρων.

Κατά τη διαδικασία της Μηχανικής Μάθησης, ένα μοντέλο εκτίθεται σε ένα σύνολο παραδειγμάτων και επιχειρεί να εντοπίσει σχέσεις και πρότυπα που μπορούν να χρησιμοποιηθούν για την πραγματοποίηση προβλέψεων. Η διαδικασία κατά την οποία προσαρμόζονται οι παράμετροι του μοντέλου ονομάζεται **εκπαίδευση (training)**. Μετά την ολοκλήρωσή της, το ζητούμενο είναι το μοντέλο να μπορεί να επεξεργαστεί νέα δεδομένα που δεν χρησιμοποιήθηκαν κατά την εκπαίδευσή του και να παράγει αξιόπιστες προβλέψεις.

Μία από τις βασικότερες κατηγορίες Μηχανικής Μάθησης είναι η **επιβλεπόμενη μάθηση (supervised learning)**. Σε αυτήν, κάθε παράδειγμα εκπαίδευσης συνοδεύεται από τη σωστή έξοδο ή **ετικέτα (label)**. Το μοντέλο λαμβάνει τα δεδομένα εισόδου μαζί με τις αντίστοιχες ετικέτες και προσπαθεί να μάθει τη σχέση που τα συνδέει. Μετά την εκπαίδευση μπορεί να χρησιμοποιήσει τη σχέση αυτή για την πρόβλεψη της ετικέτας νέων παραδειγμάτων.

Δύο βασικά είδη προβλημάτων επιβλεπόμενης μάθησης είναι η **παλινδρόμηση (regression)** και η **ταξινόμηση (classification)**. Στην παλινδρόμηση η επιθυμητή έξοδος είναι συνήθως μία συνεχής αριθμητική τιμή, ενώ στην ταξινόμηση η έξοδος αντιστοιχεί σε μία προκαθορισμένη κατηγορία. Όταν υπάρχουν περισσότερες από δύο διαθέσιμες κατηγορίες και κάθε παράδειγμα αντιστοιχεί σε μία μόνο από αυτές, το πρόβλημα χαρακτηρίζεται ως **πολυταξική ταξινόμηση (multi-class classification)**.

Η παρούσα εργασία αποτελεί χαρακτηριστικό παράδειγμα επιβλεπόμενης πολυταξικής ταξινόμησης. Η είσοδος του συστήματος είναι μία ψηφιακή εικόνα ελληνικής παραδοσιακής φορεσιάς και η επιθυμητή έξοδος είναι μία από τις εννέα γεωγραφικές κλάσεις που έχουν οριστεί. Κατά την εκπαίδευση είναι γνωστή η πραγματική κλάση κάθε εικόνας και το μοντέλο χρησιμοποιεί την πληροφορία αυτή για να προσαρμόσει τις παραμέτρους του.

Κεντρική έννοια στη Μηχανική Μάθηση αποτελεί η **γενίκευση (generalization)**. Στόχος ενός μοντέλου δεν είναι απλώς να προβλέπει σωστά τα παραδείγματα που έχει ήδη δει, αλλά να έχει καλή απόδοση και σε νέα δεδομένα. Εάν ένα μοντέλο προσαρμοστεί υπερβολικά στα δεδομένα εκπαίδευσης, μπορεί να εμφανίσει το φαινόμενο της **υπερπροσαρμογής (overfitting)**. Σε αυτή την περίπτωση επιτυγχάνει υψηλή απόδοση στο training set, αλλά η απόδοσή του μειώνεται σε δεδομένα που δεν χρησιμοποιήθηκαν για την εκπαίδευσή του.

Το αντίθετο φαινόμενο ονομάζεται **υποπροσαρμογή (underfitting)** και εμφανίζεται όταν το μοντέλο δεν καταφέρνει να μάθει επαρκώς τα βασικά πρότυπα των δεδομένων.

Η επίτευξη ισορροπίας μεταξύ της επαρκούς προσαρμογής στα δεδομένα εκπαίδευσης και της δυνατότητας γενίκευσης αποτελεί βασικό στόχο κατά την ανάπτυξη ενός συστήματος Μηχανικής Μάθησης.Η **Βαθιά Μάθηση (Deep Learning)** αποτελεί υποπεδίο της Μηχανικής Μάθησης και βασίζεται κυρίως στη χρήση τεχνητών νευρωνικών δικτύων με πολλαπλά επίπεδα επεξεργασίας. Ένα από τα σημαντικότερα χαρακτηριστικά της είναι η δυνατότητα αυτόματης εκμάθησης κατάλληλων αναπαραστάσεων από τα δεδομένα. Στην επεξεργασία εικόνων αυτό σημαίνει ότι δεν είναι απαραίτητο να προσδιοριστούν χειροκίνητα όλα τα οπτικά χαρακτηριστικά που θα χρησιμοποιηθούν για την ταξινόμηση. Το ίδιο το δίκτυο μπορεί να μάθει ποια χαρακτηριστικά είναι χρήσιμα μέσα από τη διαδικασία εκπαίδευσης.
Η δυνατότητα αυτή είναι ιδιαίτερα σημαντική για ένα πρόβλημα όπως η ταξινόμηση παραδοσιακών φορεσιών. Η διάκριση μεταξύ γεωγραφικών περιοχών μπορεί να εξαρτάται από συνδυασμούς πολλών οπτικών στοιχείων και όχι από έναν απλό προκαθορισμένο κανόνα. Ένα μοντέλο βαθιάς μάθησης μπορεί να μάθει αναπαραστάσεις που αξιοποιούν διαφορετικά χαρακτηριστικά της εικόνας και να τα συνδυάζει για την πραγματοποίηση της τελικής πρόβλεψης.

2.3 ΤΕΧΝΗΤΑ ΝΕΥΡΩΝΙΚΑ ΔΙΚΤΥΑ

Τα **Τεχνητά Νευρωνικά Δίκτυα (Artificial Neural Networks – ANNs)** αποτελούν υπολογιστικά μοντέλα που οργανώνονται σε διασυνδεδεμένες μονάδες επεξεργασίας, οι οποίες συνήθως αναφέρονται ως τεχνητοί νευρώνες. Η λειτουργία τους βασίζεται στη διαδοχική επεξεργασία αριθμητικών δεδομένων μέσα από επίπεδα, με τις παραμέτρους των επιμέρους συνδέσεων να προσαρμόζονται κατά τη διαδικασία της εκπαίδευσης.

Ένας τεχνητός νευρώνας δέχεται ένα σύνολο εισόδων $x_1, x_{2,\dots} x_n$. Κάθε είσοδος συνδέεται με ένα αντίστοιχο **βάρος (weight)** w_i, το οποίο καθορίζει τη συνεισφορά της συγκεκριμένης εισόδου στον υπολογισμό του νευρώνα. Στο σταθμισμένο άθροισμα των εισόδων προστίθεται μία επιπλέον παράμετρος, η **πόλωση (bias)** b. Η βασική αυτή λειτουργία μπορεί να εκφραστεί μαθηματικά ως:

$$\boldsymbol{z = \sum_{i=1}^{n} w_i x_i + b}$$

Η τιμή z που προκύπτει δεν αποτελεί απαραίτητα την τελική έξοδο του νευρώνα. Συνήθως εφαρμόζεται σε αυτή μία **συνάρτηση ενεργοποίησης (activation function)**, με αποτέλεσμα:

$$\boldsymbol{a = f(z)}$$

όπου f είναι η συνάρτηση ενεργοποίησης και a η τελική έξοδος του νευρώνα. Η χρήση συναρτήσεων ενεργοποίησης επιτρέπει την εισαγωγή μη γραμμικότητας στο δίκτυο, γεγονός που είναι απαραίτητο για την εκμάθηση σύνθετων σχέσεων μεταξύ των δεδομένων. Χωρίς μη γραμμικές συναρτήσεις ενεργοποίησης, η διαδοχική σύνδεση πολλών γραμμικών επιπέδων θα εξακολουθούσε να ισοδυναμεί με έναν γραμμικό μετασχηματισμό, περιορίζοντας σημαντικά την ικανότητα αναπαράστασης του δικτύου.

Μία ιδιαίτερα διαδεδομένη συνάρτηση ενεργοποίησης στα νευρωνικά δίκτυα είναι η **Rectified Linear Unit (ReLU)**, η οποία ορίζεται ως:

$$\boldsymbol{ReLU(x) = \max(0, x)}$$

Η ReLU επιστρέφει την τιμή της εισόδου όταν αυτή είναι θετική και μηδέν όταν είναι αρνητική. Η απλή μορφή της και η αποτελεσματική υπολογιστική συμπεριφορά της έχουν συμβάλει στην ευρεία χρήση της σε βαθιά νευρωνικά δίκτυα, αν και διαφορετικές σύγχρονες αρχιτεκτονικές μπορούν να χρησιμοποιούν και άλλες συναρτήσεις ενεργοποίησης.

Ένα νευρωνικό δίκτυο οργανώνεται συνήθως σε ένα **επίπεδο εισόδου (input layer)**, ένα ή περισσότερα **κρυφά επίπεδα (hidden layers)** και ένα **επίπεδο εξόδου (output layer)**. Το επίπεδο εισόδου λαμβάνει τα αρχικά δεδομένα, ενώ τα κρυφά επίπεδα πραγματοποιούν διαδοχικούς μετασχηματισμούς των πληροφοριών. Το επίπεδο εξόδου παράγει την τελική αναπαράσταση που απαιτείται από το συγκεκριμένο πρόβλημα.

Κατά την επεξεργασία ενός παραδείγματος, τα δεδομένα μεταφέρονται διαδοχικά από το επίπεδο εισόδου προς τα επόμενα επίπεδα μέχρι την παραγωγή της εξόδου. Η διαδικασία αυτή ονομάζεται **προώθηση προς τα εμπρός (forward propagation)**. Σε κάθε επίπεδο πραγματοποιούνται μαθηματικοί μετασχηματισμοί με βάση τις τρέχουσες τιμές των παραμέτρων του δικτύου και τα αποτελέσματα χρησιμοποιούνται ως είσοδοι για το επόμενο επίπεδο.

Σε ένα πρόβλημα πολυταξικής ταξινόμησης, το τελικό επίπεδο του δικτύου πρέπει να παράγει πληροφορία για καθεμία από τις διαθέσιμες κλάσεις. Για ένα πρόβλημα με *C* κατηγορίες, η έξοδος μπορεί να εκφραστεί ως ένα διάνυσμα:

$$\boldsymbol{z} = [\boldsymbol{z_1}, \boldsymbol{z_2}, \dots \boldsymbol{z_C}]$$

Οι τιμές αυτές ονομάζονται **logits** και αποτελούν μη κανονικοποιημένες βαθμολογίες που παράγει το μοντέλο για τις διαφορετικές κλάσεις. Στην παρούσα εργασία, όπου το πρόβλημα περιλαμβάνει εννέα γεωγραφικές κλάσεις, το τελικό επίπεδο ταξινόμησης των μοντέλων προσαρμόζεται ώστε να παράγει εννέα αντίστοιχες εξόδους.

Οι τιμές των βαρών και των biases δεν είναι γνωστές εκ των προτέρων, αλλά προσαρμόζονται κατά τη διαδικασία της εκπαίδευσης. Για να πραγματοποιηθεί η προσαρμογή, πρέπει να υπολογιστεί αρχικά πόσο διαφέρει η πρόβλεψη του δικτύου από την πραγματική ετικέτα του παραδείγματος. Η διαφορά αυτή εκφράζεται μέσω μιας συνάρτησης απώλειας και στη συνέχεια χρησιμοποιείται για τον υπολογισμό των μεταβολών που πρέπει να πραγματοποιηθούν στις εκπαιδεύσιμες παραμέτρους του δικτύου. Η διαδικασία εκπαίδευσης αναλύεται περισσότερο στην Ενότητα 2.5.

Όσο αυξάνεται ο αριθμός των επιπέδων ενός δικτύου, είναι δυνατή η δημιουργία περισσότερο σύνθετων εσωτερικών αναπαραστάσεων των δεδομένων. Η ιδιότητα αυτή βρίσκεται στη βάση της Βαθιάς Μάθησης. Ωστόσο, στην περίπτωση των ψηφιακών εικόνων, η χρήση μόνο πλήρως συνδεδεμένων επιπέδων δεν αξιοποιεί αποτελεσματικά τη χωρική σχέση μεταξύ των pixels και μπορεί να οδηγήσει σε πολύ μεγάλο αριθμό παραμέτρων. Για τον λόγο αυτό αναπτύχθηκαν εξειδικευμένες αρχιτεκτονικές, όπως τα **Συνελικτικά Νευρωνικά Δίκτυα (Convolutional Neural Networks – CNNs)**, τα οποία αξιοποιούν τη χωρική δομή των εικόνων και παρουσιάζονται αναλυτικά στην Ενότητα 2.4.

2.4 ΨΗΦΙΑΚΕΣ ΕΙΚΟΝΕΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΙΚΗ ΌΡΑΣΗ

Η **Υπολογιστική Όραση (Computer Vision)** αποτελεί πεδίο της Πληροφορικής και της Τεχνητής Νοημοσύνης που ασχολείται με την ανάπτυξη υπολογιστικών μεθόδων για την επεξεργασία, ανάλυση και εξαγωγή πληροφορίας από οπτικά δεδομένα. Τα δεδομένα αυτά μπορούν να περιλαμβάνουν ψηφιακές εικόνες και βίντεο, ενώ οι εφαρμογές της Υπολογιστικής Όρασης καλύπτουν προβλήματα όπως η ταξινόμηση εικόνων, η ανίχνευση αντικειμένων, η αναγνώριση οπτικών προτύπων και η τμηματοποίηση εικόνων.

Για έναν υπολογιστή, μία ψηφιακή εικόνα αναπαρίσταται ως οργανωμένο σύνολο αριθμητικών τιμών. Η βασική μονάδα της είναι το **pixel (picture element)**, το οποίο αντιστοιχεί σε μία συγκεκριμένη χωρική θέση. Σε μία έγχρωμη εικόνα RGB, η χρωματική πληροφορία κάθε pixel περιγράφεται μέσω τριών καναλιών: κόκκινου (**Red**), πράσινου (**Green**) και μπλε (**Blue**). Ο συνδυασμός των τιμών των τριών καναλιών καθορίζει το χρώμα που αντιστοιχεί στη συγκεκριμένη θέση της εικόνας.

Μία εικόνα ύψους *H* και πλάτους *W* μπορεί επομένως να αναπαρασταθεί ως πολυδιάστατη αριθμητική δομή. Στα συστήματα βαθιάς μάθησης και ειδικότερα στην PyTorch, μία μεμονωμένη εικόνα μπορεί να έχει τη μορφή:

$$[\boldsymbol{C}, \boldsymbol{H}, \boldsymbol{W}]$$

όπου C είναι ο αριθμός των καναλιών, *H* το ύψος και *W* το πλάτος. Για μία RGB εικόνα ισχύει C=3.

Η **ανάλυση (resolution)** μιας ψηφιακής εικόνας συνδέεται με τις χωρικές διαστάσεις της και επομένως με τον αριθμό των pixels που χρησιμοποιούνται για την αναπαράστασή της. Οι εικόνες ενός dataset μπορεί αρχικά να έχουν διαφορετικές διαστάσεις.

Τα νευρωνικά δίκτυα ταξινόμησης, ωστόσο, συνήθως επεξεργάζονται τις εικόνες σύμφωνα με συγκεκριμένες απαιτήσεις εισόδου. Για τον λόγο αυτό η αλλαγή μεγέθους αποτελεί συνηθισμένο στάδιο της προεπεξεργασίας.

Η αριθμητική αναπαράσταση επιτρέπει την εφαρμογή μαθηματικών πράξεων στα δεδομένα μιας εικόνας. Ωστόσο, η σημασιολογική πληροφορία που αντιλαμβάνεται ένας άνθρωπος δεν βρίσκεται αποθηκευμένη ως μία μεμονωμένη αριθμητική τιμή. Αντικείμενα, σχήματα, υφές και άλλα χαρακτηριστικά σχηματίζονται από τη χωρική σχέση πολλών pixels. Ένα σύστημα Υπολογιστικής Όρασης πρέπει επομένως να μπορεί να εντοπίζει και να συνδυάζει τέτοιες δομές.

Μία σημαντική ιδιότητα των εικόνων είναι ότι τα γειτονικά pixels παρουσιάζουν **χωρικές σχέσεις**. Ομάδες γειτονικών pixels μπορούν να σχηματίζουν ακμές, γραμμές, χρωματικές περιοχές και υφές. Καθώς οι πληροφορίες αυτές συνδυάζονται, μπορούν να προκύψουν περισσότερο σύνθετες οπτικές αναπαραστάσεις. Η αξιοποίηση της χωρικής δομής διαφοροποιεί την επεξεργασία εικόνων από προβλήματα στα οποία οι μεταβλητές εισόδου μπορούν να αντιμετωπιστούν περισσότερο ανεξάρτητα.

Στην **ταξινόμηση εικόνων (image classification)**, ο στόχος είναι η αντιστοίχιση ολόκληρης της εικόνας σε μία από ένα προκαθορισμένο σύνολο κατηγοριών. Η εργασία αυτή διαφέρει, για παράδειγμα, από την **ανίχνευση αντικειμένων (object detection)**, όπου απαιτείται τόσο η αναγνώριση των αντικειμένων όσο και ο προσδιορισμός της θέσης τους μέσα στην εικόνα, και από τη **σημασιολογική τμηματοποίηση (semantic segmentation)**, όπου πραγματοποιείται ταξινόμηση σε επίπεδο pixels.

Η παρούσα εργασία αντιμετωπίζει το πρόβλημα ως **ταξινόμηση ολόκληρης της εικόνας**, καθώς για κάθε εικόνα παραδοσιακής φορεσιάς επιδιώκεται η πρόβλεψη μίας γεωγραφικής κλάσης. Το σύστημα δεν επιχειρεί να εντοπίσει χωριστά τα επιμέρους τμήματα της φορεσιάς ούτε να επισημάνει τη θέση τους. Αντίθετα, το μοντέλο επεξεργάζεται τη συνολική οπτική πληροφορία της εικόνας και παράγει μία πρόβλεψη για τη γεωγραφική της κατηγορία.

Η ανάγκη αποτελεσματικής επεξεργασίας της χωρικής πληροφορίας των εικόνων οδήγησε στην ανάπτυξη εξειδικευμένων νευρωνικών αρχιτεκτονικών. Μεταξύ αυτών, τα **Συνελικτικά Νευρωνικά Δίκτυα** αποτέλεσαν θεμελιώδη προσέγγιση για την ταξινόμηση εικόνων και βρίσκονται στη βάση και των τριών οικογενειών μοντέλων που εξετάζονται στην παρούσα εργασία.

2.5 ΣΥΝΕΛΙΚΤΙΚΑ ΝΕΥΡΩΝΙΚΑ ΔΙΚΤΥΑ

Τα **Συνελικτικά Νευρωνικά Δίκτυα (Convolutional Neural Networks – CNNs)** αποτελούν κατηγορία νευρωνικών δικτύων σχεδιασμένη για την επεξεργασία δεδομένων με χωρική δομή, όπως οι ψηφιακές εικόνες. Σε αντίθεση με μία προσέγγιση που θα αντιμετώπιζε κάθε pixel ως ανεξάρτητη είσοδο, τα CNNs αξιοποιούν τις τοπικές σχέσεις μεταξύ γειτονικών περιοχών της εικόνας και μπορούν να μαθαίνουν οπτικά χαρακτηριστικά απευθείας από τα δεδομένα.

2.5.1 Συνέλιξη και φίλτρα

Η βασική λειτουργία ενός CNN είναι η **συνέλιξη (convolution)**. Κατά τη διαδικασία αυτή, ένα μικρό σύνολο αριθμητικών βαρών, το οποίο ονομάζεται **φίλτρο ή kernel**, εφαρμόζεται διαδοχικά σε διαφορετικές τοπικές περιοχές της εισόδου. Αντί επομένως κάθε μονάδα να συνδέεται με ολόκληρη την εικόνα, η επεξεργασία πραγματοποιείται τοπικά.

Σε απλοποιημένη δισδιάστατη μορφή, η σχετική λειτουργία μπορεί να αποδοθεί ως:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

όπου X είναι η είσοδος, K το φίλτρο και Y η παραγόμενη έξοδος. Καθώς το φίλτρο εφαρμόζεται στις διαφορετικές θέσεις της εικόνας, δημιουργείται ένας **χάρτης χαρακτηριστικών (feature map)**, ο οποίος περιγράφει την απόκριση του συγκεκριμένου φίλτρου στις αντίστοιχες χωρικές περιοχές.

Οι τιμές των φίλτρων δεν καθορίζονται χειροκίνητα, αλλά αποτελούν εκπαιδεύσιμες παραμέτρους του δικτύου. Κατά συνέπεια, μέσα από την εκπαίδευση το CNN μπορεί να μάθει ποια τοπικά μοτίβα είναι χρήσιμα

για την επίλυση του συγκεκριμένου προβλήματος. Διαφορετικά φίλτρα μπορούν να αποκρίνονται σε διαφορετικούς τύπους οπτικών χαρακτηριστικών.

Μία σημαντική ιδιότητα της συνέλιξης είναι η **κοινή χρήση βαρών (weight sharing)**. Το ίδιο φίλτρο χρησιμοποιείται σε πολλές διαφορετικές χωρικές θέσεις της εικόνας. Έτσι το δίκτυο μπορεί να αναζητά ένα χαρακτηριστικό ανεξάρτητα από την ακριβή θέση στην οποία εμφανίζεται, ενώ παράλληλα χρησιμοποιεί σημαντικά λιγότερες παραμέτρους από όσες θα απαιτούσε μία πλήρως συνδεδεμένη επεξεργασία όλων των pixels.

2.5.2 Stride και Padding

Δύο βασικές παράμετροι της συνελικτικής λειτουργίας είναι το **stride** και το **padding**. Το stride καθορίζει το βήμα με το οποίο μετακινείται το φίλτρο πάνω στην είσοδο. Για παράδειγμα, με stride ίσο με 1 το φίλτρο μετακινείται κατά μία θέση κάθε φορά, ενώ με μεγαλύτερο stride παραλείπονται ενδιάμεσες θέσεις και μειώνονται ταχύτερα οι χωρικές διαστάσεις της παραγόμενης αναπαράστασης.

Το **padding** αφορά την προσθήκη επιπλέον τιμών γύρω από τα όρια της εισόδου πριν εφαρμοστεί η συνέλιξη. Μέσω κατάλληλου padding μπορεί να ελεγχθεί η χωρική διάσταση της εξόδου και να διατηρηθεί καλύτερα η πληροφορία που βρίσκεται κοντά στα όρια της εικόνας.

Για μία διάσταση εισόδου N, μέγεθος φίλτρου F, padding P και stride S, η αντίστοιχη διάσταση εξόδου υπολογίζεται ως:

$$N_{out} = \left\lfloor \frac{N + 2P - F}{S} \right\rfloor + 1$$

Η σχέση αυτή δείχνει ότι οι παράμετροι μιας συνέλιξης επηρεάζουν τόσο τον τρόπο επεξεργασίας όσο και τις διαστάσεις των feature maps που μεταφέρονται στα επόμενα επίπεδα.

2.5.3 Pooling και μείωση χωρικών διαστάσεων

Σε πολλές CNN αρχιτεκτονικές χρησιμοποιούνται λειτουργίες **pooling**, οι οποίες μειώνουν τις χωρικές διαστάσεις των feature maps. Μία από τις πιο γνωστές είναι το **max pooling**, όπου από κάθε μικρή περιοχή διατηρείται η μεγαλύτερη τιμή.

Η σταδιακή μείωση των χωρικών διαστάσεων περιορίζει τον όγκο των δεδομένων που μεταφέρεται στα βαθύτερα επίπεδα και αυξάνει σταδιακά το τμήμα της αρχικής εικόνας που μπορεί να επηρεάζει μία βαθύτερη ενεργοποίηση. Ωστόσο, δεν χρησιμοποιούν όλες οι σύγχρονες αρχιτεκτονικές την ίδια ακριβώς στρατηγική. Η μείωση της χωρικής ανάλυσης μπορεί επίσης να πραγματοποιείται μέσω συνελίξεων με κατάλληλο stride.

2.5.4 Ιεραρχική εξαγωγή χαρακτηριστικών

Ένα από τα σημαντικότερα χαρακτηριστικά των CNNs είναι η δυνατότητα **ιεραρχικής εκμάθησης οπτικών αναπαραστάσεων**. Στα πρώτα επίπεδα μπορούν να σχηματίζονται αναπαραστάσεις σχετικά απλών οπτικών μοτίβων, όπως ακμές, μεταβολές φωτεινότητας ή υφές. Καθώς η πληροφορία μεταφέρεται σε βαθύτερα επίπεδα, οι προηγούμενες αναπαραστάσεις συνδυάζονται και μπορούν να σχηματιστούν περισσότερο σύνθετα χαρακτηριστικά.

Στην περίπτωση των παραδοσιακών φορεσιών, η λογική αυτή είναι ιδιαίτερα χρήσιμη, καθώς η γεωγραφική ταξινόμηση δεν βασίζεται απαραίτητα σε ένα μόνο χαρακτηριστικό. Το δίκτυο μπορεί να αξιοποιήσει συνδυασμούς οπτικών πληροφοριών που σχετίζονται με χρώματα, υφές, σχήματα, διακοσμητικά μοτίβα και τη συνολική μορφή της φορεσιάς. Δεν είναι απαραίτητο κάθε εσωτερική αναπαράσταση που μαθαίνει το δίκτυο να αντιστοιχεί άμεσα σε ένα χαρακτηριστικό που μπορεί να περιγραφεί από τον άνθρωπο.

Η διαδοχική εξαγωγή χαρακτηριστικών καταλήγει τελικά σε μία περισσότερο συμπυκνωμένη αναπαράσταση της εικόνας, η οποία μπορεί να χρησιμοποιηθεί από το τελικό τμήμα του δικτύου για την πραγματοποίηση της ταξινόμησης. Η βασική αυτή φιλοσοφία συναντάται, με διαφορετικές σχεδιαστικές επιλογές, στις αρχιτεκτονικές **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny** που εξετάζονται στην παρούσα εργασία.

2.6 ΕΚΠΑΙΔΕΥΣΗ ΝΕΥΡΩΝΙΚΩΝ ΔΙΚΤΥΩΝ ΓΙΑ ΤΑΞΙΝΟΜΗΣΗ ΕΙΚΟΝΩΝ

Η εκπαίδευση ενός νευρωνικού δικτύου αποτελεί επαναληπτική διαδικασία κατά την οποία οι εκπαιδεύσιμες παράμετροι του μοντέλου προσαρμόζονται με στόχο τη μείωση της απόκλισης μεταξύ των προβλέψεών του και των πραγματικών ετικετών των δεδομένων. Η γενική διαδικασία περιλαμβάνει την προώθηση των δεδομένων μέσα από το δίκτυο, τον υπολογισμό της απώλειας, την οπισθοδιάδοση της πληροφορίας του σφάλματος και την ενημέρωση των παραμέτρων.

2.6.1 Logits και Softmax

Σε ένα πρόβλημα πολυταξικής ταξινόμησης, το τελευταίο επίπεδο του μοντέλου παράγει συνήθως μία αριθμητική τιμή, ή **logit**, για κάθε διαθέσιμη κλάση. Τα logits δεν αποτελούν από μόνα τους πιθανότητες, καθώς δεν είναι απαραίτητο να βρίσκονται μεταξύ 0 και 1 ούτε το άθροισμά τους να ισούται με 1.

Η συνάρτηση **Softmax** μπορεί να μετατρέψει ένα σύνολο logits σε κανονικοποιημένη κατανομή:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

όπου z_i είναι το logit της κλάσης i, C ο συνολικός αριθμός κλάσεων και p_i η αντίστοιχη κανονικοποιημένη τιμή. Οι τιμές p_i είναι μη αρνητικές και το άθροισμά τους ισούται με 1.

Η κλάση με τη μεγαλύτερη τιμή αποτελεί την **Top-1 πρόβλεψη** του μοντέλου. Εάν διατηρηθούν οι k κλάσεις με τις μεγαλύτερες τιμές, προκύπτουν οι **Top-k προβλέψεις**. Η έννοια αυτή έχει άμεση σχέση με την παρούσα εφαρμογή, στην οποία παρουσιάζονται στον χρήστη οι τρεις πιθανότερες γεωγραφικές περιοχές.

2.6.2 Συνάρτηση απώλειας

Για να πραγματοποιηθεί η εκπαίδευση απαιτείται μία αριθμητική μέτρηση της απόκλισης μεταξύ της πρόβλεψης και της πραγματικής ετικέτας. Η μέτρηση αυτή παρέχεται από τη **συνάρτηση απώλειας (loss function)**. Για προβλήματα πολυταξικής ταξινόμησης χρησιμοποιείται ευρέως η **Cross-Entropy Loss**.

Για ένα παράδειγμα του οποίου η πραγματική κλάση είναι yy, η βασική μορφή της απώλειας μπορεί να εκφραστεί ως:

$$L = -log(p_y)$$

όπου p_y είναι η κανονικοποιημένη τιμή που αντιστοιχεί στην πραγματική κλάση. Όσο μεγαλύτερη τιμή αποδίδει το μοντέλο στη σωστή κλάση, τόσο μικρότερη γίνεται η αντίστοιχη απώλεια.

2.6.3 Backpropagation και βελτιστοποίηση

Μετά τον υπολογισμό της απώλειας πρέπει να προσδιοριστεί με ποιον τρόπο πρέπει να μεταβληθούν οι εκπαιδεύσιμες παράμετροι ώστε να μειωθεί η τιμή της. Η διαδικασία **οπισθοδιάδοσης (backpropagation)** χρησιμοποιεί τον κανόνα της αλυσίδας για τον υπολογισμό των παραγώγων της απώλειας ως προς τις παραμέτρους του δικτύου.

Οι παράγωγοι χρησιμοποιούνται από έναν **αλγόριθμο βελτιστοποίησης (optimizer)** για την ενημέρωση των παραμέτρων. Σε μία απλοποιημένη μορφή καθόδου κλίσης, η ενημέρωση ενός βάρους μπορεί να εκφραστεί ως:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

όπου η είναι ο **ρυθμός μάθησης (learning rate)**. Ο ρυθμός μάθησης καθορίζει το μέγεθος της μεταβολής των παραμέτρων σε κάθε βήμα της βελτιστοποίησης και αποτελεί μία από τις σημαντικές υπερπαραμέτρους της διαδικασίας εκπαίδευσης.

2.6.4 Batches και Epochs

Στην πράξη, οι εικόνες του training set οργανώνονται σε μικρότερες ομάδες που ονομάζονται **batches**. Ο αριθμός των παραδειγμάτων που περιέχει κάθε batch ονομάζεται **batch size**. Η επεξεργασία των δεδομένων σε batches επιτρέπει αποτελεσματικότερη αξιοποίηση των υπολογιστικών πόρων και την επαναληπτική ενημέρωση των παραμέτρων κατά τη διάρκεια της εκπαίδευσης.

Μία πλήρης διέλευση όλων των παραδειγμάτων του training set από το μοντέλο ονομάζεται **epoch**. Η εκπαίδευση πραγματοποιείται συνήθως για περισσότερα epochs, ώστε το μοντέλο να έχει τη δυνατότητα να προσαρμόσει σταδιακά τις παραμέτρους του. Η συμπεριφορά του στα δεδομένα εκπαίδευσης και επικύρωσης μπορεί να παρακολουθείται κατά τη διάρκεια αυτής της διαδικασίας, ώστε να αξιολογείται η εξέλιξη της μάθησης και η πιθανή εμφάνιση overfitting.

Οι έννοιες που παρουσιάζονται στην παρούσα ενότητα περιγράφουν τη γενική θεωρητική λειτουργία της εκπαίδευσης ενός μοντέλου ταξινόμησης. Οι συγκεκριμένες επιλογές που χρησιμοποιήθηκαν στα πειράματα της εργασίας, όπως optimizer, learning rate, batch size και αριθμός epochs, παρουσιάζονται στο κεφάλαιο της πειραματικής διαδικασίας.

2.7 TRANSFER LEARNING, FEATURE EXTRACTION ΚΑΙ PARTIAL FINE- TUNING

Η εκπαίδευση ενός βαθιού νευρωνικού δικτύου από την αρχή μπορεί να απαιτεί μεγάλο αριθμό δεδομένων και σημαντικούς υπολογιστικούς πόρους. Όταν το διαθέσιμο dataset είναι περιορισμένο, μία ιδιαίτερα χρήσιμη προσέγγιση είναι η **μεταφορά μάθησης (transfer learning)**. Στη μεταφορά μάθησης αξιοποιείται η γνώση που έχει αποκτήσει ένα μοντέλο κατά την προηγούμενη εκπαίδευσή του σε ένα διαφορετικό και συνήθως πολύ μεγαλύτερο σύνολο δεδομένων.

Αντί όλες οι παράμετροι ενός δικτύου να αρχικοποιηθούν τυχαία, χρησιμοποιούνται **προεκπαιδευμένα βάρη (pretrained weights)** ως σημείο εκκίνησης. Το μοντέλο διαθέτει έτσι ήδη αναπαραστάσεις που έχουν προκύψει από την επεξεργασία μεγάλου αριθμού εικόνων και οι οποίες μπορούν να αξιοποιηθούν σε ένα νέο πρόβλημα.

Η δυνατότητα μεταφοράς αυτής της γνώσης βασίζεται στο γεγονός ότι αρκετά οπτικά χαρακτηριστικά είναι χρήσιμα σε διαφορετικά προβλήματα. Για παράδειγμα, αναπαραστάσεις που σχετίζονται με ακμές, σχήματα και υφές δεν περιορίζονται απαραίτητα στις κατηγορίες στις οποίες εκπαιδεύτηκε αρχικά το μοντέλο. Έτσι, ένα προεκπαιδευμένο δίκτυο μπορεί να αποτελέσει κατάλληλο σημείο εκκίνησης για ένα πιο εξειδικευμένο πρόβλημα ταξινόμησης.

Στο transfer learning υπάρχουν διαφορετικές στρατηγικές ως προς το ποια τμήματα του δικτύου επιτρέπεται να ενημερώνονται κατά τη νέα εκπαίδευση. Στην παρούσα εργασία ιδιαίτερη σημασία έχουν δύο στρατηγικές: το **feature extraction** και το **partial fine-tuning**, καθώς εφαρμόζονται ξεχωριστά στις τρεις αρχιτεκτονικές που εξετάζονται.

2.7.1 Feature Extraction

Στο **feature extraction**, το προεκπαιδευμένο τμήμα του μοντέλου χρησιμοποιείται ως εξαγωγέας οπτικών χαρακτηριστικών. Οι παράμετροι αυτού του τμήματος διατηρούνται σταθερές, δηλαδή **παγώνουν (freeze)** και δεν ενημερώνονται κατά το backpropagation. Αντίθετα, το τελικό τμήμα ταξινόμησης προσαρμόζεται στις κατηγορίες του νέου προβλήματος και οι αντίστοιχες παράμετροι εκπαιδεύονται με τα νέα δεδομένα.
↳ Εάν οι παράμετροι του feature extractor συμβολιστούν ως θ_f και οι παράμετροι του νέου classifier ως θ_c, η βασική λογική μπορεί να αποδοθεί ως:

$$\boldsymbol{\theta_f = \sigma\tau\alpha\theta\varepsilon\rho\acute{\varepsilon}\varsigma}, \qquad \boldsymbol{\theta_c = \varepsilon\kappa\pi\alpha\iota\delta\varepsilon\acute{\upsilon}\sigma\iota\mu\varepsilon\varsigma}$$

Η στρατηγική αυτή μειώνει τον αριθμό των παραμέτρων που πρέπει να ενημερωθούν κατά την εκπαίδευση και αξιοποιεί σε μεγάλο βαθμό τις αναπαραστάσεις που έχει ήδη μάθει το προεκπαιδευμένο μοντέλο. Για τον λόγο αυτό αποτελεί ιδιαίτερα χρήσιμη προσέγγιση όταν το νέο σύνολο δεδομένων είναι σχετικά περιορισμένο.

2.7.2 Fine-Tuning και Partial Fine-Tuning

Στο **fine-tuning**, επιτρέπεται η ενημέρωση προεκπαιδευμένων παραμέτρων με βάση τα δεδομένα του νέου προβλήματος. Με τον τρόπο αυτό οι ήδη υπάρχουσες αναπαραστάσεις δεν χρησιμοποιούνται απλώς ως σταθερός feature extractor, αλλά μπορούν να προσαρμοστούν περαιτέρω στο νέο πεδίο εφαρμογής.

Όταν η ενημέρωση περιορίζεται σε ένα επιλεγμένο μέρος του προεκπαιδευμένου δικτύου, η στρατηγική χαρακτηρίζεται ως **partial fine-tuning**. Ένα μέρος των επιπέδων παραμένει παγωμένο, ενώ επιλεγμένα επίπεδα επιτρέπεται να εκπαιδευτούν μαζί με το νέο τμήμα ταξινόμησης. Με αυτόν τον τρόπο επιδιώκεται ένας συμβιβασμός μεταξύ της διατήρησης της ήδη αποκτημένης γνώσης και της προσαρμογής των αναπαραστάσεων στις ιδιαιτερότητες του νέου προβλήματος.

Το partial fine-tuning προσφέρει μεγαλύτερη δυνατότητα προσαρμογής από το feature extraction, αλλά ταυτόχρονα αυξάνει τον αριθμό των εκπαιδεύσιμων παραμέτρων. Αυτό σημαίνει ότι οι δύο στρατηγικές δεν είναι ταυτόσημες ως προς τη διαδικασία εκπαίδευσης και η σχετική αποτελεσματικότητά τους μπορεί να εξαρτάται από το μοντέλο, το μέγεθος και τη φύση του dataset και τις υπόλοιπες πειραματικές ρυθμίσεις.

Η σύγκριση των δύο στρατηγικών αποτελεί βασικό στοιχείο της παρούσας εργασίας. Για καθεμία από τις αρχιτεκτονικές **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny** πραγματοποιείται ένα πείραμα με feature extraction και ένα με partial fine-tuning. Προκύπτουν επομένως **έξι βασικές πειραματικές διαμορφώσεις**, οι οποίες επιτρέπουν να εξεταστεί τόσο η επίδραση της αρχιτεκτονικής όσο και η επίδραση της στρατηγικής transfer learning.

2.8 IMAGENET ΚΑΙ ΠΡΟΕΚΠΑΙΔΕΥΜΕΝΑ ΜΟΝΤΕΛΑ

Το **ImageNet** αποτελεί ένα μεγάλης κλίμακας σύνολο δεδομένων εικόνων που έχει διαδραματίσει σημαντικό ρόλο στην ανάπτυξη και αξιολόγηση μοντέλων Υπολογιστικής Όρασης. Η χρήση του συνέβαλε στην ανάπτυξη βαθιών αρχιτεκτονικών ικανών να μαθαίνουν πλούσιες οπτικές αναπαραστάσεις, οι οποίες μπορούν να αξιοποιηθούν και σε προβλήματα διαφορετικά από εκείνα της αρχικής εκπαίδευσης.

Η σημασία ενός προεκπαιδευμένου μοντέλου δεν περιορίζεται στην ικανότητά του να αναγνωρίζει τις κατηγορίες στις οποίες εκπαιδεύτηκε αρχικά. Κατά την προεκπαίδευση, τα ενδιάμεσα επίπεδα του δικτύου μαθαίνουν αναπαραστάσεις που μπορούν να περιγράφουν γενικότερα οπτικά χαρακτηριστικά. Μέσω του transfer learning, οι αναπαραστάσεις αυτές μπορούν να μεταφερθούν σε ένα νέο dataset και να αποτελέσουν το αρχικό σημείο για την εκπαίδευση ενός διαφορετικού προβλήματος.

Η προσέγγιση είναι ιδιαίτερα σημαντική όταν το νέο dataset είναι περιορισμένο. Η εκπαίδευση ενός βαθιού δικτύου αποκλειστικά από τυχαία αρχικοποιημένα βάρη με μικρό αριθμό εικόνων μπορεί να δυσκολέψει την εκμάθηση αναπαραστάσεων που γενικεύουν αποτελεσματικά. Αντίθετα, η χρήση pretrained weights επιτρέπει την εκκίνηση από παραμέτρους που έχουν ήδη προκύψει από εκτεταμένη προηγούμενη εκπαίδευση.

Η χρήση προεκπαιδευμένων βαρών συνδέεται επίσης με την προετοιμασία των δεδομένων εισόδου. Όταν χρησιμοποιείται ένα προεκπαιδευμένο μοντέλο, η προεπεξεργασία των εικόνων πρέπει να είναι συμβατή με τις απαιτήσεις των αντίστοιχων weights. Για τον λόγο αυτό, όπως αναλύθηκε στο Κεφάλαιο 3, στην παρούσα εργασία εφαρμόζεται κανονικοποίηση των RGB καναλιών με τις αντίστοιχες τιμές μέσης τιμής και τυπικής απόκλισης που χρησιμοποιούνται για τα συγκεκριμένα ImageNet pretrained weights.

Το ImageNet και το transfer learning συνδέονται επομένως άμεσα με την πειραματική μεθοδολογία της εργασίας. Η προεκπαίδευση παρέχει την αρχική γνώση των μοντέλων, ενώ το feature extraction και το partial fine-tuning καθορίζουν τον τρόπο με τον οποίο αυτή η γνώση αξιοποιείται και προσαρμόζεται στο πρόβλημα ταξινόμησης των ελληνικών παραδοσιακών φορεσιών.

2.9 ΑΡΧΙΤΕΚΤΟΝΙΚΕΣ ΒΑΘΙΑΣ ΜΑΘΗΣΗΣ

Η εξέλιξη της Υπολογιστικής Όρασης έχει συνοδευτεί από την ανάπτυξη διαφορετικών αρχιτεκτονικών βαθιών νευρωνικών δικτύων, οι οποίες επιδιώκουν αποτελεσματικότερη εξαγωγή οπτικών χαρακτηριστικών, ευκολότερη εκπαίδευση και καλύτερη αξιοποίηση των διαθέσιμων υπολογιστικών πόρων. Παρότι οι αρχιτεκτονικές αυτές βασίζονται στη γενικότερη φιλοσοφία των Συνελικτικών Νευρωνικών Δικτύων, διαφοροποιούνται ως προς την εσωτερική τους δομή και τον τρόπο με τον οποίο οργανώνουν και επεξεργάζονται τα χαρακτηριστικά μιας εικόνας.

Στην παρούσα εργασία χρησιμοποιούνται τρεις αρχιτεκτονικές: **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny**. Η επιλογή τους επιτρέπει την εξέταση διαφορετικών σχεδιαστικών προσεγγίσεων των CNNs. Η ResNet βασίζεται στην έννοια της υπολειμματικής μάθησης και των residual connections, η EfficientNet δίνει ιδιαίτερη έμφαση στην αποδοτική κλιμάκωση της αρχιτεκτονικής, ενώ η ConvNeXt αποτελεί μία νεότερη προσέγγιση εκσυγχρονισμού του σχεδιασμού των συνελικτικών δικτύων. Στις ακόλουθες υποενότητες παρουσιάζονται οι βασικές αρχές των τριών οικογενειών, ενώ οι συγκεκριμένες ρυθμίσεις και ο τρόπος προσαρμογής τους στο dataset της εργασίας παρουσιάζονται στα επόμενα κεφάλαια.

2.9.1 ResNet και Residual Learning

Η αύξηση του βάθους ενός νευρωνικού δικτύου μπορεί να ενισχύσει την ικανότητά του να σχηματίζει σύνθετες αναπαραστάσεις, δεν συνεπάγεται όμως ότι ένα βαθύτερο δίκτυο εκπαιδεύεται πάντοτε αποτελεσματικότερα. Καθώς αυξάνεται ο αριθμός των επιπέδων, η διαδικασία βελτιστοποίησης μπορεί να γίνει δυσκολότερη. Η οικογένεια **Residual Networks (ResNet)** σχεδιάστηκε για την αντιμετώπιση αυτής της δυσκολίας μέσω της εισαγωγής της έννοιας της **υπολειμματικής μάθησης (residual learning)**.

Βασικό δομικό στοιχείο της ResNet αποτελεί το **residual block**. Αντί ένα σύνολο επιπέδων να επιχειρεί να μάθει απευθείας μία επιθυμητή απεικόνιση $H(x)$, διατυπώνεται η υπολειμματική συνάρτηση:

$$\boldsymbol{F(x) = H(x) - x}$$

οπότε η επιθυμητή απεικόνιση μπορεί να γραφτεί ως:

$$\boldsymbol{H(x) = F(x) + x}$$

και η έξοδος του residual block εκφράζεται ως:

$$\boldsymbol{y = F(x) + x}$$

Η είσοδος x μεταφέρεται επομένως απευθείας προς την έξοδο του block μέσω μιας **shortcut connection**, η οποία παρακάμπτει τα ενδιάμεσα επίπεδα, και προστίθεται στην έξοδο $F(x)$ αυτών των επιπέδων. Όταν οι διαστάσεις εισόδου και εξόδου είναι συμβατές, η σύνδεση αυτή μπορεί να πραγματοποιηθεί ως identity mapping. Όταν διαφέρουν, μπορεί να χρησιμοποιηθεί κατάλληλη προβολή ώστε να καταστεί δυνατή η πρόσθεση.

Οι shortcut connections δημιουργούν άμεσες διαδρομές για τη μεταφορά της πληροφορίας και των gradients μέσα στο δίκτυο και διευκολύνουν τη βελτιστοποίηση βαθύτερων αρχιτεκτονικών. Η σημαντική συνεισφορά της ResNet ήταν επομένως ότι επέτρεψε την αποτελεσματικότερη εκπαίδευση πολύ βαθιών συνελικτικών δικτύων χωρίς να απαιτείται κάθε διαδοχικό σύνολο επιπέδων να μαθαίνει εξ ολοκλήρου μία νέα απεικόνιση.

Η οικογένεια ResNet περιλαμβάνει διαφορετικές εκδόσεις, οι οποίες διαφοροποιούνται κυρίως ως προς το βάθος και τη δομή των residual blocks. Η **ResNet18** αποτελεί μία από τις μικρότερες εκδόσεις και χρησιμοποιεί τα λεγόμενα **Basic Blocks**. Η βασική της δομή περιλαμβάνει ένα αρχικό συνελικτικό τμήμα και στη συνέχεια τέσσερα διαδοχικά στάδια residual blocks, στα οποία αυξάνεται σταδιακά ο αριθμός των καναλιών ενώ μειώνονται οι χωρικές διαστάσεις των feature maps. Προς το τέλος της αρχιτεκτονικής εφαρμόζεται συγκέντρωση της χωρικής πληροφορίας και ένα τελικό πλήρως συνδεδεμένο επίπεδο χρησιμοποιείται για την ταξινόμηση.

Στην παρούσα εργασία η ResNet18 αποτελεί την πρώτη από τις τρεις αρχιτεκτονικές που εξετάζονται. Η ύπαρξη σαφώς διαχωρισμένου feature extractor και τελικού classifier επιτρέπει την προσαρμογή της σε νέο πρόβλημα ταξινόμησης μέσω transfer learning. Εξετάζεται τόσο ως **feature extractor** όσο και μέσω **partial fine-tuning**, χωρίς οι συγκεκριμένες επιλογές παγώματος και αποδέσμευσης των επιπέδων να αναλύονται στο παρόν θεωρητικό κεφάλαιο.

2.9.2 EfficientNet και Compound Scaling

Η οικογένεια **EfficientNet** αναπτύχθηκε με στόχο την αποτελεσματική εξισορρόπηση μεταξύ της ικανότητας αναπαράστασης ενός νευρωνικού δικτύου και του υπολογιστικού κόστους που απαιτείται για τη λειτουργία του. Κεντρική ιδέα της είναι ότι η αύξηση της κλίμακας ενός CNN δεν πρέπει απαραίτητα να πραγματοποιείται μεταβάλλοντας μόνο μία διάσταση της αρχιτεκτονικής, αλλά με συντονισμένη μεταβολή διαφορετικών διαστάσεών της.

Τρεις βασικές διαστάσεις κλιμάκωσης ενός CNN είναι το **βάθος (depth)**, το **πλάτος (width)** και η **ανάλυση εισόδου (resolution)**. Το depth σχετίζεται με τον αριθμό των επιπέδων του δικτύου, το width με το πλήθος των καναλιών που χρησιμοποιούνται στα επίπεδα και το resolution με τις χωρικές διαστάσεις των εικόνων εισόδου. Η αύξηση καθεμιάς από αυτές μπορεί να ενισχύσει την ικανότητα του μοντέλου, αλλά ταυτόχρονα αυξάνει και τις υπολογιστικές απαιτήσεις.

Η EfficientNet εισάγει τη μέθοδο **compound scaling**, σύμφωνα με την οποία οι τρεις παραπάνω διαστάσεις κλιμακώνονται με συντονισμένο τρόπο. Η γενική λογική της μπορεί να αποδοθεί ως:

$$depth = \alpha^{\varphi}$$

$$width = \beta^{\varphi}$$

$$resolution = \gamma^{\varphi}$$

όπου α, β και γ είναι σταθεροί συντελεστές που καθορίζουν τον τρόπο κλιμάκωσης και ϕ αποτελεί τον συντελεστή με τον οποίο ελέγχεται συνολικά η αύξηση της κλίμακας του μοντέλου. Η βασική ιδέα είναι ότι depth, width και resolution δεν μεταβάλλονται ανεξάρτητα, αλλά ισορροπημένα.

Στο εσωτερικό της αρχιτεκτονικής χρησιμοποιούνται **Mobile Inverted Bottleneck Convolution (MBConv) blocks**, τα οποία αξιοποιούν μεταξύ άλλων depthwise convolutions για αποδοτικότερη επεξεργασία των χαρακτηριστικών. Στη depthwise convolution η χωρική συνέλιξη πραγματοποιείται χωριστά ανά κανάλι, γεγονός που διαφοροποιείται από μία συμβατική συνέλιξη στην οποία τα κανάλια εισόδου συνδυάζονται κατά τον ίδιο συνελικτικό μετασχηματισμό. Τα MBConv blocks συνδυάζουν την προσέγγιση αυτή με επιπλέον μετασχηματισμούς των καναλιών.

Σημαντικό στοιχείο των MBConv blocks της EfficientNet αποτελεί επίσης ο μηχανισμός **Squeeze-and-Excitation (SE)**. Μέσω αυτού το δίκτυο μπορεί να προσαρμόζει τη σχετική βαρύτητα των διαφορετικών καναλιών χαρακτηριστικών, χρησιμοποιώντας πληροφορία που προκύπτει από τη συνολική απόκριση κάθε καναλιού. Με τον τρόπο αυτό ενισχύονται ή περιορίζονται δυναμικά διαφορετικά κανάλια ανάλογα με την πληροφορία που μεταφέρει η είσοδος.

Η **EfficientNet-Bo** αποτελεί τη βασική αρχιτεκτονική της οικογένειας, από την οποία προέκυψαν μεγαλύτερες παραλλαγές μέσω της μεθόδου compound scaling. Η σχεδίασή της αποσκοπεί στην επίτευξη καλής σχέσης μεταξύ απόδοσης και υπολογιστικού κόστους και αντιπροσωπεύει διαφορετική αρχιτεκτονική προσέγγιση σε σχέση με τη ResNet18.

Στην παρούσα εργασία η EfficientNet-Bo αποτελεί τη δεύτερη αρχιτεκτονική που αξιολογείται. Όπως και στην περίπτωση της ResNet18, εξετάζεται με τις δύο στρατηγικές transfer learning που παρουσιάστηκαν στην Ενότητα 2.6, δηλαδή **feature extraction** και **partial fine-tuning**. Η συγκεκριμένη προσαρμογή των επιπέδων της στο πρόβλημα των εννέα γεωγραφικών κλάσεων παρουσιάζεται στο αντίστοιχο κεφάλαιο της μεθοδολογίας.

2.9.3 ConvNeXt

Η **ConvNeXt** αποτελεί μία νεότερη οικογένεια συνελικτικών νευρωνικών δικτύων, η οποία αναπτύχθηκε με στόχο τον εκσυγχρονισμό του σχεδιασμού των κλασικών CNNs. Η αρχιτεκτονική έχει ιδιαίτερο θεωρητικό ενδιαφέρον επειδή διατηρεί τη συνέλιξη ως βασικό μηχανισμό επεξεργασίας, αλλά επανεξετάζει αρκετές σχεδιαστικές επιλογές των προηγούμενων CNN αρχιτεκτονικών, αξιοποιώντας ιδέες που είχαν επικρατήσει σε νεότερες αρχιτεκτονικές Υπολογιστικής Όρασης.

Η ConvNeXt διατηρεί μία **ιεραρχική πολυσταδιακή δομή**, στην οποία η εικόνα μετασχηματίζεται διαδοχικά σε feature maps διαφορετικών χωρικών διαστάσεων και αριθμού καναλιών. Όπως και σε άλλα CNNs, καθώς η πληροφορία προχωρά προς τα βαθύτερα στάδια, οι χωρικές διαστάσεις μειώνονται και οι αναπαραστάσεις γίνονται περισσότερο αφηρημένες και σύνθετες.

Μία από τις χαρακτηριστικές σχεδιαστικές επιλογές της ConvNeXt είναι η εκτεταμένη χρήση **depthwise convolutions**. Σε μία τυπική convolution, κάθε φίλτρο μπορεί να συνδυάζει πληροφορία από όλα τα κανάλια εισόδου. Αντίθετα, στη depthwise convolution η χωρική επεξεργασία πραγματοποιείται ξεχωριστά για κάθε κανάλι. Στην ConvNeXt η λειτουργία αυτή συνδυάζεται με μετασχηματισμούς που αναλαμβάνουν την επεξεργασία και ανάμειξη της πληροφορίας μεταξύ των καναλιών.

Παράλληλα, η ConvNeXt χρησιμοποιεί **μεγαλύτερα συνελικτικά kernels**, χαρακτηριστικά kernels μεγέθους 7×7 στα ConvNeXt blocks. Μεγαλύτερα kernels επιτρέπουν σε μία συνελικτική λειτουργία να λαμβάνει υπόψη πληροφορία από μεγαλύτερη χωρική περιοχή του feature map, διαφοροποιώντας τη σχεδιαστική προσέγγιση από αρχιτεκτονικές που βασίζονται κυρίως σε μικρότερα kernels.

Μία ακόμη διαφοροποίηση αφορά τη χρήση **Layer Normalization** αντί της παραδοσιακής εκτεταμένης χρήσης Batch Normalization που συναντάται σε πολλές CNN αρχιτεκτονικές. Η κανονικοποίηση αποτελεί σημαντικό στοιχείο των βαθιών δικτύων, καθώς επηρεάζει τη συμπεριφορά των ενδιάμεσων ενεργοποιήσεων και τη διαδικασία εκπαίδευσης. Η ConvNeXt υιοθετεί επίσης τη συνάρτηση ενεργοποίησης **Gaussian Error Linear Unit (GELU)**, αντί της ReLU που χρησιμοποιείται ευρέως σε παλαιότερες CNN αρχιτεκτονικές.

Ο σχεδιασμός των ConvNeXt blocks περιλαμβάνει επομένως έναν συνδυασμό κλασικών συνελικτικών αρχών και νεότερων σχεδιαστικών επιλογών. Η σημασία της αρχιτεκτονικής βρίσκεται στο γεγονός ότι οι βελτιώσεις δεν βασίζονται στην εγκατάλειψη της συνέλιξης, αλλά στη συστηματική αναθεώρηση του τρόπου με τον οποίο οργανώνεται ένα σύγχρονο CNN.

Η οικογένεια ConvNeXt περιλαμβάνει διαφορετικές εκδόσεις ανάλογα με το μέγεθος και την υπολογιστική πολυπλοκότητα. Η **ConvNeXt-Tiny** αποτελεί τη μικρότερη από τις βασικές αρχικές παραλλαγές της οικογένειας. Η ονομασία *Tiny* δηλώνει τη σχετική θέση της μέσα στην οικογένεια ConvNeXt και δεν σημαίνει ότι πρόκειται για ένα απλό ή πολύ μικρό νευρωνικό δίκτυο.

Στην παρούσα εργασία η ConvNeXt-Tiny αποτελεί την τρίτη αρχιτεκτονική που εξετάζεται. Η χρήση της προσθέτει στην πειραματική σύγκριση μία πιο σύγχρονη σχεδιαστική προσέγγιση CNN σε σχέση με τις ResNet18 και EfficientNet-B0. Όπως και στις δύο προηγούμενες περιπτώσεις, αξιολογείται τόσο μέσω **feature extraction** όσο και μέσω **partial fine-tuning**.

2.9.4 Σύνδεση των αρχιτεκτονικών με την πειραματική διαδικασία

Οι τρεις αρχιτεκτονικές που παρουσιάστηκαν βασίζονται στη συνελικτική επεξεργασία εικόνων, αλλά έχουν διαφορετική σχεδιαστική φιλοσοφία. Η **ResNet18** χαρακτηρίζεται από τη χρήση residual connections, η **EfficientNet-B0** από την έμφαση στην αποδοτική σχεδίαση και το compound scaling, ενώ η **ConvNeXt-Tiny** αντιπροσωπεύει μία νεότερη προσέγγιση εκσυγχρονισμού των CNNs.

Η χρήση περισσότερων της μίας αρχιτεκτονικών επιτρέπει η αξιολόγηση να μην περιορίζεται στη συμπεριφορά ενός μόνο τύπου μοντέλου. Παράλληλα, η εφαρμογή των ίδιων δύο βασικών στρατηγικών transfer learning δημιουργεί ένα κοινό πλαίσιο πειραματικής σύγκρισης. Για κάθε αρχιτεκτονική εξετάζεται μία διαμόρφωση **Feature Extraction** και μία διαμόρφωση **Partial Fine-Tuning**.

Από τον συνδυασμό των τριών αρχιτεκτονικών και των δύο στρατηγικών προκύπτουν οι **έξι βασικές πειραματικές διαμορφώσεις** της εργασίας: ResNet18 Feature Extraction, ResNet18 Partial Fine-Tuning, EfficientNet-B0 Feature Extraction, EfficientNet-B0 Partial Fine-Tuning, ConvNeXt-Tiny Feature Extraction και ConvNeXt-Tiny Partial Fine-Tuning.

Η θεωρητική παρουσίαση της παρούσας ενότητας περιορίζεται στις βασικές αρχές λειτουργίας και στα χαρακτηριστικά των οικογενειών των μοντέλων. Οι ακριβείς επιλογές που αφορούν την προσαρμογή του τελικού classifier, το πάγωμα ή την αποδέσμευση συγκεκριμένων τμημάτων, τον αριθμό των εκπαιδεύσιμων παραμέτρων και τις υπόλοιπες ρυθμίσεις των έξι πειραμάτων παρουσιάζονται στα επόμενα κεφάλαια, όπου περιγράφεται η συγκεκριμένη μεθοδολογία που εφαρμόστηκε στην εργασία.

2.10 ΜΕΤΡΙΚΕΣ ΑΞΙΟΛΟΓΗΣΗΣ ΜΟΝΤΕΛΩΝ ΤΑΞΙΝΟΜΗΣΗΣ

Η αξιολόγηση ενός μοντέλου ταξινόμησης αποτελεί απαραίτητο στάδιο για την εκτίμηση της ικανότητάς του να πραγματοποιεί σωστές προβλέψεις σε δεδομένα που δεν χρησιμοποιήθηκαν κατά την εκπαίδευσή του. Η απόδοση ενός μοντέλου δεν είναι πάντοτε δυνατό να περιγραφεί επαρκώς από μία μόνο αριθμητική τιμή. Για τον λόγο αυτό χρησιμοποιούνται διαφορετικές μετρικές, οι οποίες εξετάζουν συμπληρωματικές πλευρές της συμπεριφοράς του.

Στην πολυταξική ταξινόμηση είναι ιδιαίτερα σημαντικό να εξετάζεται τόσο η συνολική απόδοση όσο και η συμπεριφορά του μοντέλου σε κάθε επιμέρους κλάση. Ένα μοντέλο μπορεί, για παράδειγμα, να επιτυγχάνει ικανοποιημένη συνολική Accuracy, αλλά να εμφανίζει σημαντικά χαμηλότερη απόδοση σε συγκεκριμένες κατηγορίες. Η ανάλυση ανά κλάση επιτρέπει επομένως τον εντοπισμό τέτοιων διαφορών και παρέχει πληρέστερη εικόνα της λειτουργίας του μοντέλου.

Για την αξιολόγηση των μοντέλων της παρούσας εργασίας χρησιμοποιούνται μετρικές όπως **Accuracy, Precision, Recall και F1-score**, καθώς και ο **πίνακας σύγχυσης (Confusion Matrix)** για την αναλυτικότερη μελέτη των προβλέψεων. Παράλληλα, η εξέταση της κατάταξης των πιθανότερων κλάσεων επιτρέπει τη χρήση μετρικών **Top-k**, οι οποίες παρουσιάζουν ιδιαίτερο ενδιαφέρον για το σύστημα αναγνώρισης παραδοσιακών φορεσιών.

2.10.1 Confusion Matrix

Ο **πίνακας σύγχυσης (Confusion Matrix)** αποτελεί ένα από τα βασικότερα εργαλεία ανάλυσης ενός μοντέλου ταξινόμησης. Πρόκειται για έναν πίνακα στον οποίο αντιπαραβάλλονται οι πραγματικές κλάσεις των δειγμάτων με τις κλάσεις που προέβλεψε το μοντέλο. Με τον τρόπο αυτό δεν καταγράφεται μόνο ο συνολικός αριθμός των σωστών και λανθασμένων προβλέψεων, αλλά και το είδος των σφαλμάτων που πραγματοποιούνται.

Σε ένα πρόβλημα με C κλάσεις, ο πίνακας σύγχυσης έχει διαστάσεις:

$$C \times C$$

Στην παρούσα εργασία, δεδομένου ότι το πρόβλημα περιλαμβάνει εννέα γεωγραφικές κλάσεις, ο αντίστοιχος πίνακας σύγχυσης έχει διαστάσεις:

$$9 \times 9$$

Κάθε στοιχείο του πίνακα εκφράζει τον αριθμό των δειγμάτων μιας πραγματικής κλάσης που ταξινομήθηκαν σε μία συγκεκριμένη προβλεπόμενη κλάση. Τα στοιχεία της κύριας διαγωνίου αντιστοιχούν στις σωστές ταξινομήσεις, ενώ τα στοιχεία εκτός της κύριας διαγωνίου αντιστοιχούν σε λανθασμένες προβλέψεις. Η εξέταση των τελευταίων επιτρέπει να εντοπιστούν οι κατηγορίες μεταξύ των οποίων το μοντέλο παρουσιάζει συχνότερα σύγχυση.

Για τον υπολογισμό μετρικών ανά κλάση χρησιμοποιούνται οι έννοιες **True Positive (TP)**, **False Positive (FP)** και **False Negative (FN)**. Για μία συγκεκριμένη κλάση, True Positive είναι ένα δείγμα που ανήκει πραγματικά στην κλάση και προβλέφθηκε σωστά ως αυτή, False Positive είναι ένα δείγμα άλλης κλάσης που προβλέφθηκε λανθασμένα ως αυτή, ενώ False Negative είναι ένα δείγμα της συγκεκριμένης κλάσης που ταξινομήθηκε λανθασμένα σε κάποια άλλη.

Η πληροφορία του Confusion Matrix είναι ιδιαίτερα χρήσιμη στο πρόβλημα της παρούσας εργασίας, καθώς επιτρέπει να εξεταστεί εάν συγκεκριμένες γεωγραφικές περιοχές συγχέονται συστηματικά μεταξύ τους. Με αυτόν τον τρόπο η αξιολόγηση δεν περιορίζεται στο εάν μία πρόβλεψη είναι σωστή ή λανθασμένη, αλλά μπορεί να αποκαλύψει συγκεκριμένα πρότυπα σφαλμάτων του μοντέλου.

2.10.2 Accuracy

Η **Accuracy** αποτελεί μία από τις απλούστερες και περισσότερο κατανοητές μετρικές ταξινόμησης. Εκφράζει το ποσοστό των δειγμάτων για τα οποία η προβλεπόμενη κλάση του μοντέλου συμφωνεί με την πραγματική κλάση.

Για ένα σύνολο NN δειγμάτων, η Accuracy μπορεί να υπολογιστεί ως:

$$\boldsymbol{Accuracy = \frac{Αριθμός\ σωστών\ προβλέψεων}{Συνολικός\ αριθμός\ προβλέψεων}}$$

ή ισοδύναμα:

$$\boldsymbol{Accuracy = \frac{N_{correct}}{N}}$$

Για παράδειγμα, εάν ένα μοντέλο ταξινομήσει σωστά 50 από 62 εικόνες του test set, η Accuracy του θα είναι:

$$\boldsymbol{Accuracy = \frac{50}{62} \approx 0,8065}$$

Το παράδειγμα είναι ενδεικτικό και δεν αντιστοιχεί απαραίτητα σε αποτέλεσμα κάποιου από τα μοντέλα της εργασίας.

Η Accuracy παρέχει μία συνοπτική εικόνα της συνολικής απόδοσης, αλλά δεν περιγράφει τον τρόπο με τον οποίο κατανέμονται τα λάθη μεταξύ των διαφορετικών κλάσεων. Για τον λόγο αυτό δεν χρησιμοποιείται απομονωμένα, αλλά εξετάζεται σε συνδυασμό με τις Precision, Recall, F1-score και τον Confusion Matrix.

2.10.3 Precision

Η **Precision** μιας κλάσης εκφράζει πόσες από τις προβλέψεις που πραγματοποίησε το μοντέλο για τη συγκεκριμένη κλάση ήταν πράγματι σωστές. Υπολογίζεται σύμφωνα με τη σχέση:

$$\boldsymbol{Precission = \frac{TP}{TP + FP}}$$

όπου *TP* είναι τα True Positives και *FP* τα False Positives της εξεταζόμενης κλάσης.

Υψηλή τιμή Precision σημαίνει ότι όταν το μοντέλο επιλέγει μία συγκεκριμένη κλάση, η επιλογή αυτή είναι συχνά σωστή. Αντίθετα, χαμηλότερη Precision υποδηλώνει ότι αρκετά δείγματα άλλων κατηγοριών ταξινομούνται λανθασμένα στη συγκεκριμένη κλάση.

Για παράδειγμα, εάν το μοντέλο προβλέπει συχνά την κλάση «Μακεδονία» για εικόνες που στην πραγματικότητα ανήκουν σε άλλες γεωγραφικές περιοχές, τα False Positives της Μακεδονίας αυξάνονται και η Precision της συγκεκριμένης κλάσης μειώνεται. Επομένως, η Precision είναι χρήσιμη για την εξέταση της αξιοπιστίας των θετικών προβλέψεων κάθε κατηγορίας.

2.10.4 Recall

Η **Recall**, η οποία αναφέρεται και ως ευαισθησία σε ορισμένα πλαίσια, εξετάζει το πρόβλημα από διαφορετική οπτική. Εκφράζει το ποσοστό των πραγματικών δειγμάτων μιας συγκεκριμένης κλάσης που αναγνωρίστηκαν σωστά από το μοντέλο.

Η Recall υπολογίζεται ως:

$$\boldsymbol{Recall = \frac{TP}{TP + FN}}$$

όπου *TP* είναι τα True Positives και *FN* τα False Negatives της συγκεκριμένης κλάσης.

Υψηλή Recall σημαίνει ότι το μοντέλο αναγνωρίζει σωστά μεγάλο ποσοστό των πραγματικών δειγμάτων της συγκεκριμένης κατηγορίας. Αντίθετα, χαμηλή Recall σημαίνει ότι αρκετές εικόνες που ανήκουν πραγματικά στην κλάση ταξινομούνται σε διαφορετικές κατηγορίες.

Στο πρόβλημα των παραδοσιακών φορεσιών, για παράδειγμα, εάν αρκετές πραγματικές εικόνες της Θεσσαλίας ταξινομούνται ως άλλες γεωγραφικές περιοχές, αυξάνονται τα False Negatives της Θεσσαλίας και μειώνεται η Recall της. Η συγκεκριμένη μετρική επιτρέπει επομένως να εξεταστεί πόσο αποτελεσματικά το μοντέλο αναγνωρίζει τα πραγματικά μέλη κάθε γεωγραφικής κλάσης.

2.10.5 F1-score

Η Precision και η Recall εξετάζουν διαφορετικές πλευρές της απόδοσης ενός μοντέλου. Για τη συνδυαστική αξιολόγησή τους χρησιμοποιείται συχνά το **F1-score**, το οποίο ορίζεται ως ο αρμονικός μέσος των δύο μετρικών:

$$\boldsymbol{F1 = 2 \bullet \frac{Precision \bullet Recall}{Precision + Recall}}$$

Η χρήση του αρμονικού αντί του αριθμητικού μέσου έχει ως αποτέλεσμα το F1-score να επηρεάζεται έντονα όταν μία από τις δύο επιμέρους μετρικές έχει χαμηλή τιμή. Για να επιτευχθεί υψηλό F1-score πρέπει επομένως τόσο η Precision όσο και η Recall να βρίσκονται σε σχετικά υψηλά επίπεδα.

Το F1-score είναι ιδιαίτερα χρήσιμο για τη σύγκριση της απόδοσης μεταξύ διαφορετικών κλάσεων, καθώς συνδυάζει σε μία τιμή τόσο τις λανθασμένες προβλέψεις προς μία κλάση όσο και τα πραγματικά δείγματα της κλάσης που δεν αναγνωρίστηκαν σωστά. Στην παρούσα εργασία η εξέταση του F1-score ανά γεωγραφική κλάση μπορεί να βοηθήσει στον εντοπισμό περιοχών στις οποίες ένα μοντέλο παρουσιάζει καλύτερη ή χειρότερη συμπεριφορά.

2.9.6 Macro και Weighted Average

Σε ένα πρόβλημα πολυταξικής ταξινόμησης, οι Precision, Recall και F1-score μπορούν να υπολογιστούν ξεχωριστά για καθεμία από τις κλάσεις. Για τη συνοπτική παρουσίαση της απόδοσης σε όλες τις κατηγορίες μπορούν στη συνέχεια να χρησιμοποιηθούν διαφορετικοί τρόποι υπολογισμού μέσου όρου. Δύο σημαντικές προσεγγίσεις είναι το **macro average** και το **weighted average**.

Στο **macro average**, η αντίστοιχη μετρική υπολογίζεται αρχικά ξεχωριστά για κάθε κλάση και στη συνέχεια λαμβάνεται ο απλός αριθμητικός μέσος των τιμών. Για το F1-score:

$$F1_{macro} = \frac{1}{C}\sum_{i=1}^{C} F1_i$$

όπου *C* είναι ο αριθμός των κλάσεων και $F1_i$ το F1-score της κλάσης *i*.

Το βασικό χαρακτηριστικό του macro average είναι ότι **κάθε κλάση συνεισφέρει με την ίδια βαρύτητα**, ανεξάρτητα από το πόσα δείγματα περιλαμβάνει. Στο πρόβλημα της παρούσας εργασίας αυτό σημαίνει ότι καθεμία από τις εννέα γεωγραφικές περιοχές επηρεάζει εξίσου το τελικό macro F1-score.

Η ιδιότητα αυτή καθιστά το macro average ιδιαίτερα χρήσιμο όταν ενδιαφέρει η ισορροπημένη συμπεριφορά του μοντέλου μεταξύ των διαφορετικών κλάσεων. Ένα μοντέλο που παρουσιάζει πολύ υψηλή απόδοση σε ορισμένες περιοχές αλλά σημαντικά χαμηλότερη σε άλλες μπορεί να εμφανίσει αυτή τη διαφοροποίηση σαφέστερα μέσω των per-class μετρικών και του macro average.

Στο **weighted average**, αντίθετα, η τιμή κάθε κλάσης σταθμίζεται σύμφωνα με τον αριθμό των πραγματικών δειγμάτων της. Για το weighted F1-score:

$$F1_{weighted} = \sum_{i=1}^{C} \frac{n_i}{N} F1_i$$

όπου n_i είναι ο αριθμός των πραγματικών δειγμάτων της κλάσης *i* και *N* ο συνολικός αριθμός δειγμάτων.

Με τον τρόπο αυτό, οι κλάσεις που περιλαμβάνουν περισσότερα δείγματα έχουν μεγαλύτερη επίδραση στην τελική τιμή. Επομένως, macro και weighted averages παρέχουν διαφορετική αλλά συμπληρωματική πληροφορία. Το macro average αντιμετωπίζει όλες τις κλάσεις ισότιμα, ενώ το weighted average λαμβάνει υπόψη τη σχετική συχνότητα εμφάνισής τους.

Στο dataset της παρούσας εργασίας οι αριθμητικές διαφορές μεταξύ των γεωγραφικών κλάσεων δεν είναι ακραίες, ωστόσο δεν είναι και απόλυτα ίσες. Για τον λόγο αυτό η παρουσίαση τόσο των επιμέρους μετρικών ανά κλάση όσο και κατάλληλων συνολικών μέσων όρων επιτρέπει πληρέστερη αξιολόγηση των έξι πειραματικών διαμορφώσεων.

2.10.7 Top-1 και Top-3 Accuracy

Στην κλασική αξιολόγηση ενός μοντέλου ταξινόμησης ως τελική πρόβλεψη επιλέγεται η κλάση με την υψηλότερη βαθμολογία. Όταν η κλάση αυτή συμπίπτει με την πραγματική κατηγορία του δείγματος, η πρόβλεψη θεωρείται σωστή. Η αντίστοιχη μετρική ονομάζεται **Top-1 Accuracy** και ουσιαστικά αντιστοιχεί στην τυπική Accuracy ενός προβλήματος single-label classification.

Ωστόσο, η έξοδος ενός μοντέλου πολυταξικής ταξινόμησης παρέχει πληροφορία και για τις υπόλοιπες κλάσεις. Οι κανονικοποιημένες βαθμολογίες μπορούν να ταξινομηθούν από τη μεγαλύτερη προς τη μικρότερη και έτσι να δημιουργηθεί μία κατάταξη των πιθανότερων κατηγοριών.

Η **Top-k Accuracy** εξετάζει εάν η πραγματική κλάση βρίσκεται μεταξύ των *k* υψηλότερα καταταγμένων προβλέψεων. Μπορεί να εκφραστεί ως:

$$\textbf{Top} - \textbf{k Accuracy} = \frac{\textbf{Αριθμός δειγμάτων με την πραγματική κλάση στις Top} - \textbf{k προβλέψεις}}{\textbf{Συνολικός αριθμός δειγμάτων}}$$

Για *k*=3, προκύπτει η **Top-3 Accuracy**. Μία πρόβλεψη θεωρείται επιτυχής ως προς τη συγκεκριμένη μετρική όταν η πραγματική γεωγραφική περιοχή βρίσκεται μεταξύ των τριών υψηλότερα καταταγμένων κλάσεων, ακόμη και αν δεν βρίσκεται στην πρώτη θέση.

Η Top-3 Accuracy δεν αντικαθιστά την Top-1 Accuracy. Η πρώτη θέση εξακολουθεί να αποτελεί την κύρια πρόβλεψη του μοντέλου και χρησιμοποιείται για την τυπική αξιολόγηση της ταξινόμησης. Η Top-3 παρέχει **συμπληρωματική πληροφορία** σχετικά με το κατά πόσο η πραγματική κλάση βρίσκεται μεταξύ των επικρατέστερων επιλογών του μοντέλου.

Η συγκεκριμένη πληροφορία έχει ιδιαίτερη πρακτική σημασία για την εφαρμογή που αναπτύσσεται στην παρούσα εργασία. Αντί το σύστημα να εμφανίζει αποκλειστικά μία γεωγραφική περιοχή, παρουσιάζει τις **τρεις πιθανότερες περιοχές** σύμφωνα με την έξοδο του μοντέλου. Με αυτόν τον τρόπο ο χρήστης αποκτά περισσότερη πληροφορία σχετικά με την κατάταξη των προβλέψεων, κάτι που είναι ιδιαίτερα χρήσιμο όταν διαφορετικές γεωγραφικές κλάσεις παρουσιάζουν οπτικές ομοιότητες.

Κατά συνέπεια, η συνδυαστική χρήση **Accuracy, Precision, Recall, F1-score, macro και weighted averages, Confusion Matrix και Top-3 Accuracy** επιτρέπει μία περισσότερο ολοκληρωμένη αξιολόγηση των μοντέλων. Οι μετρικές αυτές θα χρησιμοποιηθούν στα επόμενα κεφάλαια για την παρουσίαση και σύγκριση των αποτελεσμάτων των διαφορετικών πειραματικών διαμορφώσεων.

2.11 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο παρουσιάστηκε το θεωρητικό υπόβαθρο που είναι απαραίτητο για την κατανόηση της μεθοδολογίας και της πειραματικής διαδικασίας της εργασίας. Αρχικά αναλύθηκαν οι βασικές έννοιες της **Μηχανικής Μάθησης** και της **Βαθιάς Μάθησης**, με έμφαση στην επιβλεπόμενη μάθηση και στην πολυταξική ταξινόμηση, στην οποία εντάσσεται το πρόβλημα αναγνώρισης της γεωγραφικής προέλευσης ελληνικών παραδοσιακών φορεσιών.

Στη συνέχεια παρουσιάστηκαν οι βασικές αρχές λειτουργίας των **Τεχνητών Νευρωνικών Δικτύων** και η αριθμητική αναπαράσταση των ψηφιακών εικόνων. Ιδιαίτερη έμφαση δόθηκε στα **Συνελικτικά Νευρωνικά Δίκτυα (CNNs)**, καθώς αποτελούν τη βάση των αρχιτεκτονικών που χρησιμοποιούνται στην εργασία. Αναλύθηκαν έννοιες όπως η συνέλιξη, τα φίλτρα και οι χάρτες χαρακτηριστικών, το stride, το padding και η ιεραρχική εξαγωγή οπτικών χαρακτηριστικών.

Παρουσιάστηκαν επίσης οι βασικές αρχές εκπαίδευσης ενός νευρωνικού δικτύου ταξινόμησης, όπως τα logits, η Softmax, η συνάρτηση απώλειας, το backpropagation, η βελτιστοποίηση, το learning rate, τα batches και τα epochs. Οι έννοιες αυτές αποτελούν τη θεωρητική βάση για την κατανόηση των συγκεκριμένων επιλογών εκπαίδευσης και των υπερπαραμέτρων που χρησιμοποιούνται αργότερα στην πειραματική διαδικασία.

Ιδιαίτερη σημασία δόθηκε στο **transfer learning**, καθώς το διαθέσιμο σύνολο δεδομένων είναι σχετικά περιορισμένο σε σχέση με τις απαιτήσεις εκπαίδευσης σύγχρονων βαθιών νευρωνικών δικτύων. Στο πλαίσιο αυτό παρουσιάστηκαν οι στρατηγικές **feature extraction** και **partial fine-tuning**, οι οποίες αποτελούν βασικό άξονα της πειραματικής σύγκρισης που πραγματοποιείται στην εργασία.

Παράλληλα, παρουσιάστηκε ο ρόλος του **ImageNet** και των προεκπαιδευμένων βαρών, καθώς και οι βασικές αρχές των τριών αρχιτεκτονικών που χρησιμοποιούνται: **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny**. Οι αρχιτεκτονικές αυτές αντιπροσωπεύουν διαφορετικές σχεδιαστικές προσεγγίσεις στα συνελικτικά νευρωνικά δίκτυα και εξετάζονται καθεμία με feature extraction και partial fine-tuning, σχηματίζοντας συνολικά τις **έξι βασικές πειραματικές διαμορφώσεις** της εργασίας.

Τέλος, αναλύθηκαν οι βασικές μετρικές που χρησιμοποιούνται για την αξιολόγηση μοντέλων ταξινόμησης, όπως **Accuracy, Precision, Recall και F1-score**, καθώς και ο **Confusion Matrix**, οι macro και weighted μέσοι όροι και η **Top-3 Accuracy**. Η συνδυαστική αξιοποίησή τους επιτρέπει την αξιολόγηση τόσο της συνολικής απόδοσης όσο και της συμπεριφοράς των μοντέλων στις επιμέρους γεωγραφικές κλάσεις.

Με την ολοκλήρωση του θεωρητικού υποβάθρου, το επόμενο κεφάλαιο επικεντρώνεται στο **σύνολο δεδομένων και στην προεπεξεργασία του**, παρουσιάζοντας τη διαδικασία συλλογής και οργάνωσης των εικόνων, τον έλεγχο ποιότητας και ομοιότητας, τον διαχωρισμό σε training, validation και test sets και την τελική προετοιμασία των δεδομένων για την εκπαίδευση των μοντέλων.

ΚΕΦΑΛΑΙΟ 3 - ΣΥΝΟΛΟ ΔΕΔΟΜΕΝΩΝ ΚΑΙ ΠΡΟΕΠΕΞΕΡΓΑΣΙΑ

3.1 ΕΙΣΑΓΩΓΗ

Η ποιότητα και η καταλληλόλητα των δεδομένων αποτελούν καθοριστικούς παράγοντες για την ανάπτυξη ενός συστήματος Μηχανικής Μάθησης. Ακόμη και ένα εξελιγμένο μοντέλο δεν μπορεί να μάθει αποτελεσματικά ένα πρόβλημα όταν τα δεδομένα που χρησιμοποιούνται για την εκπαίδευσή του είναι ανεπαρκή, λανθασμένα οργανωμένα ή δεν αντιπροσωπεύουν ικανοποιητικά το πρόβλημα που καλείται να επιλύσει.

Η παρατήρηση αυτή αποκτά ιδιαίτερη σημασία στα προβλήματα Υπολογιστικής Όρασης (Computer Vision), όπου τα δεδομένα αποτελούνται από εικόνες. Για έναν άνθρωπο, μία φωτογραφία παραδοσιακής φορεσιάς αποτελεί άμεσα αντιληπτό οπτικό περιεχόμενο. Για ένα υπολογιστικό μοντέλο, όμως, η ίδια εικόνα αναπαρίσταται ως ένα σύνολο αριθμητικών τιμών που αντιστοιχούν στα pixels της. Μέσω της διαδικασίας εκπαίδευσης, το μοντέλο επιχειρεί να εντοπίσει επαναλαμβανόμενα οπτικά χαρακτηριστικά και να τα συσχετίσει με τις κατηγορίες που έχουν οριστεί.

Στην παρούσα εργασία, το πρόβλημα αφορά την αναγνώριση της γεωγραφικής προέλευσης ελληνικών παραδοσιακών φορεσιών από εικόνες. Για τον σκοπό αυτό δημιουργήθηκε ένα εξειδικευμένο σύνολο δεδομένων (dataset), αποτελούμενο από εικόνες παραδοσιακών ενδυμασιών οργανωμένες σε εννέα γεωγραφικές κατηγορίες.

Η δημιουργία του dataset δεν περιορίστηκε στη συλλογή εικόνων. Ακολούθησε μία ολοκληρωμένη διαδικασία, η οποία περιλάμβανε την κατηγοριοποίηση του οπτικού υλικού, την αρχική τυποποίηση των αρχείων, τον προγραμματιστικό έλεγχο εγκυρότητας, τον εντοπισμό ακριβών και οπτικά παρόμοιων εικόνων, τον ελεγχόμενο διαχωρισμό των δεδομένων σε σύνολα εκπαίδευσης, επικύρωσης και ελέγχου και, τέλος, την προετοιμασία τους για την είσοδο στα μοντέλα βαθιάς μάθησης.

Ιδιαίτερη προσοχή δόθηκε στην αποφυγή της **διαρροής δεδομένων (data leakage)**. Ο όρος περιγράφει καταστάσεις στις οποίες πληροφορία από τα δεδομένα εκπαίδευσης εμφανίζεται, άμεσα ή έμμεσα, στα δεδομένα που χρησιμοποιούνται για την αξιολόγηση του μοντέλου. Στην περίπτωση εικόνων, ένα χαρακτηριστικό παράδειγμα θα ήταν η τοποθέτηση δύο σχεδόν πανομοιότυπων εικόνων, της μίας στο training set και της άλλης στο test set. Μία τέτοια κατάσταση θα μπορούσε να οδηγήσει σε υπερβολικά αισιόδοξη εκτίμηση της πραγματικής απόδοσης του μοντέλου.

Στις ενότητες που ακολουθούν παρουσιάζεται ολόκληρη η διαδικασία, από τη συλλογή των αρχικών εικόνων μέχρι τη δημιουργία της τελικής ροής δεδομένων που χρησιμοποιήθηκε για την εκπαίδευση και αξιολόγηση των μοντέλων.

3.2 ΣΥΛΛΟΓΗ ΚΑΙ ΟΡΓΑΝΩΣΗ

3.2.1 Ορισμός του προβλήματος και των γεωγραφικών κλάσεων

Το πρόβλημα της παρούσας εργασίας διατυπώθηκε ως πρόβλημα **επιβλεπόμενης πολυταξικής ταξινόμησης εικόνων** (supervised multiclass image classification). Με τον όρο επιβλεπόμενη μάθηση περιγράφεται μία διαδικασία κατά την οποία για κάθε δείγμα που χρησιμοποιείται στην εκπαίδευση είναι γνωστή η σωστή κατηγορία στην οποία ανήκει. Η πληροφορία αυτή ονομάζεται **ετικέτα (label)** και αποτελεί την αναμενόμενη απάντηση που χρησιμοποιείται ώστε το μοντέλο να μάθει να συσχετίζει το περιεχόμενο μιας εικόνας με τη σωστή κατηγορία. Ο όρος *πολυταξική ταξινόμηση* δηλώνει ότι υπάρχουν περισσότερες από δύο πιθανές κατηγορίες και κάθε εικόνα πρέπει να αντιστοιχιστεί σε μία από αυτές.

Στην παρούσα εργασία ορίστηκαν εννέα γεωγραφικές κλάσεις:

1. Ήπειρος
2. Θεσσαλία
3. Θράκη
4. Κρήτη
5. Μακεδονία
6. Νησιά Αιγαίου
7. Νησιά Ιονίου
8. Πελοπόννησος
9. Στερεά Ελλάδα

Στον κώδικα οι αντίστοιχες κλάσεις αναπαρίστανται με τις ονομασίες ipeiros, thessalia, thraki, kriti, makedonia, nisia_aigaiou, nisia_ioniou, peloponnisos και sterea_ellada.

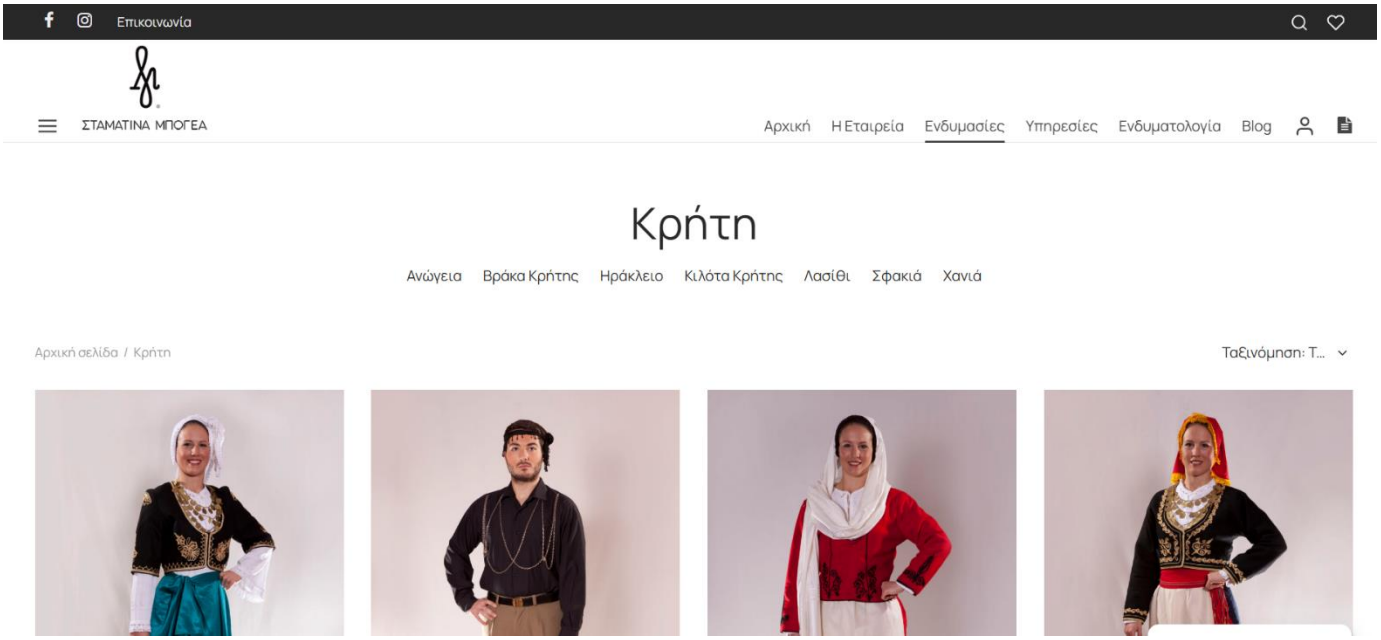
Είναι σημαντικό να διευκρινιστεί ότι **κάθε γεωγραφική περιοχή αποτελεί μία κλάση**. Οι διαφορετικές τοπικές φορεσιές που μπορεί να συναντώνται μέσα στην ίδια περιοχή δεν αντιμετωπίζονται ως ξεχωριστές κατηγορίες. Για παράδειγμα, διαφορετικές θεσσαλικές φορεσιές εξακολουθούν να αντιστοιχούν στην ίδια κλάση thessalia. Συνεπώς, το μοντέλο δεν καλείται να προσδιορίσει την ακριβή τοπική ονομασία μιας φορεσιάς, αλλά να εκτιμήσει σε ποια από τις εννέα ευρύτερες γεωγραφικές περιοχές ανήκει. Η επιλογή αυτή καθόρισε τόσο τον τρόπο οργάνωσης του dataset όσο και τη μορφή της τελικής εξόδου του συστήματος.

3.2.2 Πηγή και διαδικασία συλλογής των εικόνων

Το αρχικό οπτικό υλικό συλλέχθηκε από τον διαδικτυακό τόπο «**Σταματίνα Μπογέα – Παραδοσιακές Ενδυμασίες**» [1], στον οποίο παρουσιάζονται ελληνικές παραδοσιακές φορεσιές οργανωμένες ανά γεωγραφική περιοχή. Η συγκεκριμένη δομή διευκόλυνε τη διαδικασία δημιουργίας του dataset, καθώς υπήρχε σαφής σύνδεση μεταξύ της παρουσιαζόμενης φορεσιάς και της γεωγραφικής περιοχής στην οποία αυτή ανήκει.

Οι εικόνες του αρχικού dataset δημιουργήθηκαν μέσω στιγμιοτύπων οθόνης (screenshots) από τις αντίστοιχες σελίδες. Για κάθε γεωγραφική περιοχή εξετάστηκε το διαθέσιμο υλικό, συλλέχθηκαν οι επιλεγμένες εικόνες και στη συνέχεια τοποθετήθηκαν χειροκίνητα στον αντίστοιχο φάκελο.

Η σωστή αρχική κατηγοριοποίηση είναι ιδιαίτερα σημαντική σε ένα πρόβλημα επιβλεπόμενης μάθησης, επειδή η κατηγορία στην οποία τοποθετείται κάθε εικόνα αποτελεί το ground truth, δηλαδή την πληροφορία που θεωρείται σωστή και χρησιμοποιείται ως σημείο αναφοράς κατά την εκπαίδευση και την αξιολόγηση. Εάν, για παράδειγμα, μία εικόνα φορεσιάς της Θεσσαλίας τοποθετούνταν λανθασμένα στον φάκελο της Μακεδονίας, το μοντέλο θα λάμβανε λανθασμένη πληροφορία κατά την εκπαίδευσή του. Το ίδιο το μοντέλο δεν έχει τη δυνατότητα να γνωρίζει ότι το label που του δόθηκε είναι εσφαλμένο. Για τον λόγο αυτό η συλλογή και η χειροκίνητη οργάνωση των εικόνων αποτέλεσαν ουσιαστικό μέρος της κατασκευής του dataset και όχι απλώς ένα προκαταρκτικό βήμα.



Εικόνα 3.1: *Παράδειγμα της διαδικτυακής πηγής συλλογής του οπτικού υλικού – Πηγή: Σταματίνα Μπογέα – Παραδοσιακές Ενδυμασίες [1]*

3.2.3 Σύνθεση και κατανομή του αρχικού dataset

Μετά την ολοκλήρωση της συλλογής, το dataset περιλάμβανε συνολικά **411 εικόνες**, κατανεμημένες στις εννέα γεωγραφικές κλάσεις.

Η πραγματική κατανομή παρουσιάζεται στον Πίνακα 3.1.

ΚΛΑΣΗ	ΓΕΩΓΡΑΦΙΚΗ ΠΕΡΙΟΧΗ	ΕΙΚΟΝΕΣ
ipeiros	Ήπειρος	44
thessalia	Θεσσαλία	40
thraki	Θράκη	53
kriti	Κρήτη	41
makedonia	Μακεδονία	54
nisia_aigaiou	Νησιά Αιγαίου	52
nisia_ioniou	Νησιά Ιονίου	39
peloponnisos	Πελοπόννησος	48
sterea_ellada	Στερεά Ελλάδα	40
ΣΥΝΟΛΟ	9 ΚΛΑΣΕΙΣ	411

Πίνακας 3.1: *Κατανομή των αρχικών εικόνων ανά γεωγραφική κλάση*

Η μικρότερη κλάση ήταν τα Νησιά Ιονίου με 39 εικόνες, ενώ η μεγαλύτερη ήταν η Μακεδονία με 54. Επομένως, το dataset δεν ήταν απόλυτα αριθμητικά ισορροπημένο, χωρίς όμως να παρουσιάζει ακραίες διαφορές μεταξύ των κλάσεων.

Στη Μηχανική Μάθηση χρησιμοποιείται ο όρος **class imbalance** όταν ορισμένες κλάσεις εκπροσωπούνται από σημαντικά περισσότερα δείγματα σε σχέση με άλλες. Σε ακραίες περιπτώσεις, ένα μοντέλο μπορεί να επηρεαστεί από τη συχνότερη παρουσία συγκεκριμένων κατηγοριών. Στο συγκεκριμένο dataset υπάρχει μία ήπια αριθμητική ανομοιομορφία, καθώς το εύρος κυμαίνεται από 39 έως 54 εικόνες ανά κλάση. Η διαφορά αυτή πρέπει να λαμβάνεται υπόψη στην αξιολόγηση, αλλά δεν είναι αρκετή από μόνη της

για να εξηγήσει οποιαδήποτε μελλοντική διαφορά στην απόδοση των μοντέλων.

Ένας εξίσου σημαντικός παράγοντας είναι ότι οι εννέα γεωγραφικές περιοχές δεν εκπροσωπούνται από τον ίδιο αριθμό διαφορετικών τοπικών φορεσιών. Μία περιοχή μπορεί να περιλαμβάνει, για παράδειγμα, τρεις διαφορετικές φορεσιές, ενώ κάποια άλλη περισσότερες. Αυτό δημιουργεί διαφορετικό βαθμό **ενδοταξικής μεταβλητότητας (intra-class variability)**. Ο όρος περιγράφει το πόσο διαφορετικά μπορούν να είναι μεταξύ τους τα δείγματα που ανήκουν στην ίδια κλάση. Δύο κλάσεις μπορεί, επομένως, να διαθέτουν παρόμοιο αριθμό εικόνων αλλά η μία να πρέπει να αντιπροσωπεύσει πολύ μεγαλύτερη οπτική ποικιλία.

Παράλληλα, φορεσιές διαφορετικών γεωγραφικών περιοχών μπορούν να παρουσιάζουν κοινά στοιχεία ως προς το χρώμα, το είδος των ενδυμάτων, τα κεντήματα ή άλλα χαρακτηριστικά. Το φαινόμενο αυτό μπορεί να περιγραφεί ως **ομοιότητα μεταξύ διαφορετικών κλάσεων (inter-class similarity)**.

Η συνύπαρξη ενδοταξικής μεταβλητότητας και διαταξικής ομοιότητας καθιστά το πρόβλημα περισσότερο απαιτητικό από μία ταξινόμηση κατηγοριών με σαφώς διαφορετική οπτική εμφάνιση.



Σχήμα 3.1: *Ραβδόγραμμα του αριθμού αρχικών εικόνων ανά γεωγραφική κλάση*

3.3 ΠΡΟΕΠΕΞΕΡΓΑΣΙΑ ΚΑΙ ΈΛΕΓΧΟΣ ΠΟΙΟΤΗΤΑΣ ΤΟΥ ΣΥΝΟΛΟΥ ΔΕΔΟΜΕΝΩΝ

3.3.1 Αρχική τυποποίηση των εικόνων

Μετά τη συλλογή των screenshots πραγματοποιήθηκε ένα πρώτο στάδιο τυποποίησης με χρήση του λογισμικού **XnConvert**. Η συγκεκριμένη διαδικασία πρέπει να διακρίνεται από την προεπεξεργασία που πραγματοποιήθηκε αργότερα μέσω της PyTorch. Το XnConvert χρησιμοποιήθηκε για τη μόνιμη τυποποίηση των αρχείων του dataset, ενώ οι μετασχηματισμοί της PyTorch εφαρμόζονται κατά τη φόρτωση των δεδομένων, ώστε οι εικόνες να αποκτούν την κατάλληλη μορφή για την εκπαίδευση και την αξιολόγηση των μοντέλων. Οι εικόνες μετατράπηκαν σε κοινή μορφή: **JPG – JPEG/JFIF** και εφαρμόστηκε λειτουργία Resize με τις ακόλουθες ρυθμίσεις:

- Mode: Fit
- Width: 800 pixels
- Height: 1200 pixels
- Keep ratio: ενεργοποιημένο
- Reduce only: ενεργοποιημένο
- Resampling: Lanczos

Η επιλογή Keep ratio διατηρεί την αρχική αναλογία μεταξύ πλάτους και ύψους. Με αυτόν τον τρόπο αποφεύγεται η γεωμετρική παραμόρφωση της φορεσιάς, η οποία θα μπορούσε να προκύψει εάν το πλάτος και το ύψος μεταβάλλονταν ανεξάρτητα.

Η επιλογή Reduce only επιτρέπει τη μείωση μιας εικόνας όταν αυτή υπερβαίνει τα καθορισμένα όρια, αλλά δεν επιβάλλει τεχνητή μεγέθυνση μικρότερων εικόνων.

Η μέθοδος Lanczos χρησιμοποιήθηκε ως αλγόριθμος **επαναδειγματοληψίας (resampling)**. Ο όρος περιγράφει τον τρόπο με τον οποίο υπολογίζονται οι νέες τιμές των pixels όταν αλλάζουν οι διαστάσεις μιας ψηφιακής εικόνας.

Είναι σημαντικό να διευκρινιστεί ότι οι τιμές 800 × 1200 δεν σημαίνουν ότι όλες οι εικόνες μετατράπηκαν υποχρεωτικά σε αυτές ακριβώς τις διαστάσεις. Η λειτουργία Fit, σε συνδυασμό με το Keep ratio, προσαρμόζει την εικόνα ώστε να χωρά μέσα στα συγκεκριμένα όρια διατηρώντας τις αρχικές της αναλογίες.

Αυτό επιβεβαιώνεται και από την ανάλυση του dataset, στην οποία καταγράφηκαν διαφορετικές τελικές διαστάσεις. Οι συχνότερες βρίσκονταν περίπου στην περιοχή των 560–570 pixels σε πλάτος και 850–860 pixels σε ύψος.

3.3.2 Αυτοματοποιημένος έλεγχος εγκυρότητας

Μετά την αρχική τυποποίηση πραγματοποιήθηκε αυτοματοποιημένος έλεγχος του dataset μέσω ειδικού Python script, του analyze_dataset.py. Η χρήση αυτοματοποιημένου ελέγχου επιτρέπει την εφαρμογή των ίδιων ακριβώς κανόνων σε όλες τις εικόνες και παρέχει τη δυνατότητα επανάληψης της διαδικασίας εάν το dataset τροποποιηθεί. Το script διατρέχει τους φακέλους των γεωγραφικών κλάσεων και, για κάθε εικόνα, καταγράφει πληροφορίες όπως η κλάση, το όνομα και η διαδρομή του αρχείου, η επέκταση, το μέγεθος αρχείου, το format, το χρωματικό mode, το πλάτος, το ύψος και η αναλογία διαστάσεων. Παράλληλα, κάθε αρχείο ανοίγεται μέσω της βιβλιοθήκης Pillow και ελέγχεται η ακεραιότητά του.

Τα αποτελέσματα έδειξαν:

- **411 συνολικές εικόνες**
- **411 έγκυρες εικόνες**
- **0 μη έγκυρες ή κατεστραμμένες εικόνες**
- **9 γεωγραφικές κλάσεις**
- **411 αρχεία JPG**

Επομένως, όλες οι εικόνες του αρχικού dataset πέρασαν επιτυχώς τον προγραμματιστικό έλεγχο εγκυρότητας.

3.3.3 Έλεγχος διπλοτύπων και οπτικής ομοιότητας

Πέρα από την εγκυρότητα των αρχείων, εξετάστηκε η πιθανότητα ύπαρξης ακριβώς ίδιων ή ιδιαίτερα όμοιων εικόνων. Για τον εντοπισμό **ακριβών διπλοτύπων** χρησιμοποιήθηκε ο αλγόριθμος **SHA-256**. Ο SHA-256 είναι μία συνάρτηση κατακερματισμού (hash function), η οποία δέχεται τα δεδομένα ενός αρχείου και παράγει μία χαρακτηριστική τιμή σταθερού μήκους. Η τιμή αυτή μπορεί να θεωρηθεί ως ένα είδος ψηφιακού αποτυπώματος του αρχείου. Δύο αρχεία που είναι ακριβώς ίδια σε επίπεδο bytes παράγουν το ίδιο SHA-256 hash. Με αυτόν τον τρόπο μπορούν να εντοπιστούν ακριβή αντίγραφα χωρίς να απαιτείται χειροκίνητη σύγκριση των εικόνων. Ο έλεγχος των 411 εικόνων δεν εντόπισε καμία ομάδα ακριβών διπλοτύπων. Ο SHA-256, ωστόσο, δεν μπορεί να εντοπίσει εικόνες που είναι οπτικά σχεδόν ίδιες αλλά αποτελούν διαφορετικά ψηφιακά αρχεία. Ακόμη και μία μικρή αλλαγή στις διαστάσεις ή στη συμπίεση μιας εικόνας αρκεί ώστε να παραχθεί διαφορετικό SHA-256.

Για τον λόγο αυτό εφαρμόστηκε και **perceptual hashing**, συγκεκριμένα ο αλγόριθμος **difference hash (dHash)**. Σε αντίθεση με το SHA-256, το perceptual hashing επιχειρεί να δημιουργήσει μία σύντομη αναπαράσταση που σχετίζεται με το οπτικό περιεχόμενο της εικόνας. Στην υλοποίηση του dHash η εικόνα μετατρέπεται σε αποχρώσεις του γκρι, μειώνεται σε πολύ μικρή ανάλυση και συγκρίνονται οι φωτεινότητες γειτονικών pixels.

Για να εκτιμηθεί η διαφορά δύο dHashes χρησιμοποιήθηκε η **Hamming distance**, η οποία μετρά τον αριθμό των bits στα οποία διαφέρουν δύο δυαδικές τιμές. Όσο μικρότερη είναι η Hamming distance μεταξύ δύο dHashes, τόσο μεγαλύτερη είναι η ομοιότητα των αντίστοιχων ψηφιακών αναπαραστάσεων και, κατά συνέπεια, τόσο ισχυρότερη είναι η ένδειξη ότι οι δύο εικόνες παρουσιάζουν οπτική ομοιότητα.

Ως όριο για τον χαρακτηρισμό ενός ζεύγους ως πιθανώς οπτικά όμοιου χρησιμοποιήθηκε:

$$d_{Hamming} \leq 6$$

Ο έλεγχος πραγματοποιήθηκε μεταξύ εικόνων της ίδιας γεωγραφικής κλάσης και εντόπισε **110 ζεύγη πιθανώς πολύ όμοιων εικόνων**.

Τα 110 ζεύγη δεν χαρακτηρίστηκαν αυτομάτως ως διπλότυπα και οι εικόνες δεν διαγράφηκαν. Δύο διαφορετικές λήψεις της ίδιας φορεσιάς μπορούν να παρουσιάζουν μεγάλη ομοιότητα και παρ' όλα αυτά να αποτελούν διαφορετικά και χρήσιμα δείγματα. Αντί της αυτόματης διαγραφής, η πληροφορία της ομοιότητας αξιοποιήθηκε στο επόμενο στάδιο, κατά τον διαχωρισμό του dataset.

3.4 ΔΙΑΧΩΡΙΣΜΟΣ ΤΟΥ DATASET ΚΑΙ ΑΠΟΦΥΓΗ ΔΙΑΡΡΟΗΣ ΔΕΔΟΜΕΝΩΝ

3.4.1 Δημιουργία των Training, Validation και Test Sets

Για την ανάπτυξη και την αντικειμενική αξιολόγηση ενός μοντέλου Μηχανικής Μάθησης δεν είναι σωστό να χρησιμοποιούνται τα ίδια δεδομένα τόσο για την εκπαίδευση όσο και για την τελική μέτρηση της επίδοσής του. Για τον λόγο αυτό το dataset χωρίστηκε σε τρία ανεξάρτητα υποσύνολα: **training, validation και test**.

Το **training set** είναι το σύνολο από το οποίο το μοντέλο μαθαίνει. Οι εικόνες του περνούν επανειλημμένα από το νευρωνικό δίκτυο και οι εσωτερικές παράμετροι του μοντέλου προσαρμόζονται ώστε να μειώνεται το σφάλμα μεταξύ προβλεπόμενης και πραγματικής κατηγορίας.

Το **validation set** δεν χρησιμοποιείται για την άμεση ενημέρωση των βαρών. Αποτελεί ανεξάρτητο σύνολο που χρησιμοποιείται κατά τη διαδικασία ανάπτυξης και εκπαίδευσης ώστε να παρακολουθείται η συμπεριφορά του μοντέλου σε δεδομένα που δεν χρησιμοποιεί άμεσα για μάθηση.

Το **test set** προορίζεται για την τελική αξιολόγηση του εκπαιδευμένου μοντέλου. Για τον λόγο αυτό πρέπει να παραμένει ανεξάρτητο από τη διαδικασία εκπαίδευσης.

Οι επιθυμητές αναλογίες διαχωρισμού του συνόλου δεδομένων ορίστηκαν σε:

70% Training, 15% Validation, 15% Test

Για την αναπαραγωγιμότητα της διαδικασίας χρησιμοποιήθηκε σταθερό **seed=42**. Το **random seed** είναι μία αρχική τιμή που χρησιμοποιείται από μία γεννήτρια ψευδοτυχαίων αριθμών. Η χρήση σταθερού seed επιτρέπει την επανάληψη μιας τυχαίας διαδικασίας με συνεπή τρόπο, γεγονός ιδιαίτερα σημαντικό όταν συγκρίνονται διαφορετικά πειράματα.

3.4.2 Ομαδοποίηση παρόμοιων εικόνων

Ένας απλός τυχαίος διαχωρισμός των 411 εικόνων θα δημιουργούσε έναν σημαντικό κίνδυνο: δύο από τις εικόνες που είχαν χαρακτηριστεί ως ιδιαίτερα όμοιες θα μπορούσαν να καταλήξουν σε διαφορετικά subsets. Για παράδειγμα, μία εικόνα θα μπορούσε να τοποθετηθεί στο training set και μία σχεδόν ίδια εικόνα στο test set. Σε μία τέτοια περίπτωση το μοντέλο θα αξιολογούνταν πάνω σε οπτικό περιεχόμενο πολύ κοντινό σε αυτό που είχε ήδη παρατηρήσει κατά την εκπαίδευσή του. Αυτό αποτελεί μορφή **data leakage** και θα μπορούσε να οδηγήσει σε τεχνητά αυξημένη επίδοση.

Για την αντιμετώπιση του προβλήματος, τα 110 ζεύγη που είχαν εντοπιστεί μέσω dHash χρησιμοποιήθηκαν για τη δημιουργία ομάδων οπτικά συνδεδεμένων εικόνων.

Η ομαδοποίηση πραγματοποιήθηκε με τη δομή δεδομένων **Union-Find**, γνωστή και ως Disjoint Set Union. Η λειτουργία της μπορεί να γίνει κατανοητή με ένα απλό παράδειγμα. Εάν η εικόνα Α είναι παρόμοια με την εικόνα Β και η εικόνα Β είναι παρόμοια με την εικόνα Γ, είναι επιθυμητό οι Α, Β και Γ να αντιμετωπιστούν ως μία κοινή ομάδα, ακόμη και αν δεν υπάρχει άμεση καταγεγραμμένη σχέση μεταξύ Α και Γ. Η Union-Find επιτρέπει την αποδοτική δημιουργία τέτοιων ομάδων συνδεδεμένων στοιχείων. Από τη διαδικασία προέκυψαν **25 ομάδες που περιείχαν περισσότερες από μία εικόνες**, στις οποίες συμμετείχαν συνολικά **86 εικόνες**.

3.4.3 Group-aware Split και τελική κατανομή

Μετά τη δημιουργία των ομάδων εφαρμόστηκε **group-aware split**, δηλαδή διαχωρισμός του dataset με βάση τις ομάδες ομοιότητας που είχαν δημιουργηθεί στο προηγούμενο στάδιο. Η βασική αρχή της διαδικασίας ήταν ότι κάθε ομάδα έπρεπε να παραμένει ακέραιη. Εάν δύο ή περισσότερες εικόνες ανήκαν στην ίδια ομάδα ομοιότητας, τοποθετούνταν υποχρεωτικά στο ίδιο υποσύνολο δεδομένων. Για παράδειγμα, εάν τρεις παρόμοιες εικόνες είχαν ενταχθεί στην ίδια ομάδα, μπορούσαν να τοποθετηθούν όλες στο training set, αλλά δεν επιτρεπόταν η κατανομή τους μεταξύ training, validation και test. Με αυτόν τον τρόπο περιορίστηκε ο κίνδυνος εμφάνισης πολύ παρόμοιων εικόνων τόσο στα δεδομένα εκπαίδευσης όσο και στα δεδομένα αξιολόγησης.

Η κατανομή πραγματοποιήθηκε ξεχωριστά για κάθε γεωγραφική κλάση μέσω **greedy στρατηγικής**. Ένας greedy αλγόριθμος επιλέγει σε κάθε βήμα την καταλληλότερη διαθέσιμη επιλογή με βάση την τρέχουσα κατάσταση. Στη συγκεκριμένη περίπτωση, στόχος ήταν η τελική κατανομή να προσεγγίζει όσο το δυνατόν περισσότερο τις επιθυμητές αναλογίες **70% training, 15% validation και 15% test**, χωρίς να διασπώνται οι ομάδες παρόμοιων εικόνων.

Επειδή κάθε ομάδα έπρεπε να τοποθετείται ολόκληρη σε ένα μόνο υποσύνολο, δεν ήταν πάντοτε δυνατό να επιτευχθούν ακριβώς οι επιθυμητές αναλογίες. Η διατήρηση της ακεραιότητας των ομάδων είχε προτεραιότητα έναντι της απόλυτης αριθμητικής αντιστοίχισης στις αναλογίες 70/15/15.
Η τελική πραγματική κατανομή του συνόλου δεδομένων ήταν:

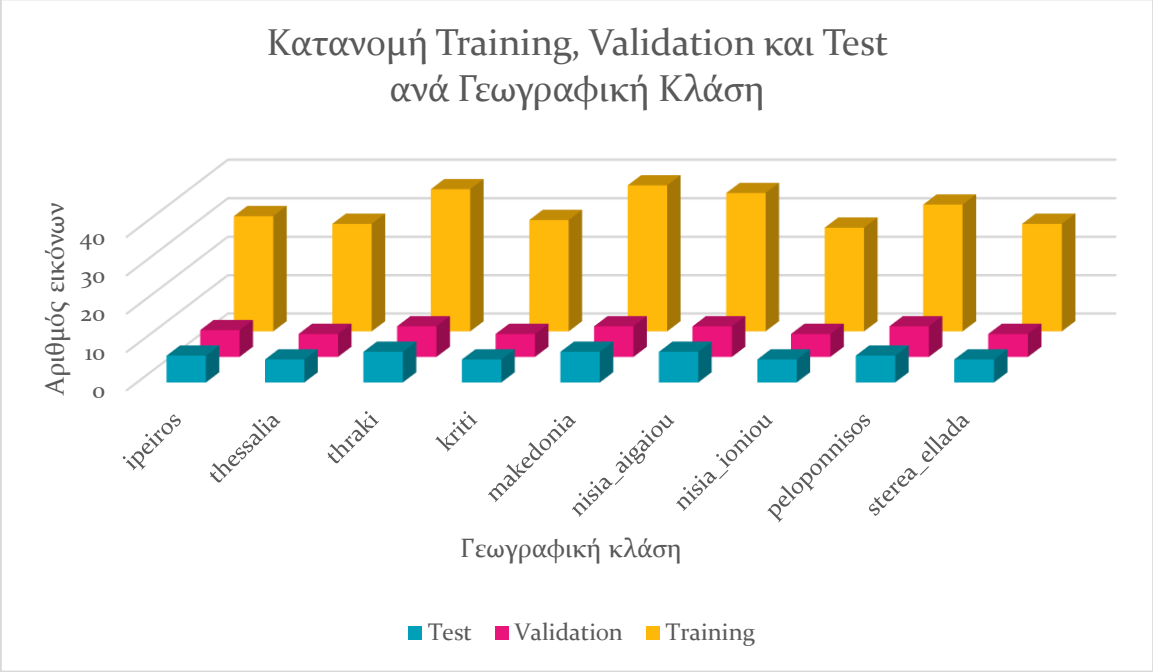
- **Training:** 286 εικόνες (69,59%)
- **Validation:** 63 εικόνες (15,33%)
- **Test:** 62 εικόνες (15,09%)

ΠΕΡΙΟΧΗ	TRAINING	VALIDATION	TEST	ΣΥΝΟΛΟ
Ήπειρος	30	7	7	44
Θεσσαλία	28	6	6	40
Θράκη	37	8	8	53
Κρήτη	29	6	6	41
Μακεδονία	38	8	8	54
Νησιά Αιγαίου	36	8	8	52
Νησιά Ιονίου	27	6	6	39
Πελοπόννησος	33	8	7	48
Στερεά Ελλάδα	28	6	6	40
ΣΥΝΟΛΟ	286	63	62	411

Πίνακας 3.2: *Κατανομή των εικόνων ανά γεωγραφική κλάση στα σύνολα training, validation και test*

Μετά την ολοκλήρωση του split πραγματοποιήθηκε αυτοματοποιημένος έλεγχος μέσω του group_id κάθε ομάδας. Επιβεβαιώθηκε ότι κανένα group_id δεν εμφανιζόταν σε περισσότερα από ένα subsets. Παράλληλα ελέγχθηκε ότι το συνολικό πλήθος των εικόνων μετά την αντιγραφή παρέμενε ακριβώς 411. Το αρχικό dataset δεν τροποποιήθηκε, καθώς το script δημιούργησε νέο φάκελο dataset_split και αντέγραψε σε αυτόν τα αρχεία.

Η διαδικασία δημιούργησε επίσης αρχεία αναφοράς, όπως split_manifest.csv, split_class_distribution.csv και split_summary.txt, τα οποία επιτρέπουν την αναδρομική επαλήθευση του τρόπου με τον οποίο πραγματοποιήθηκε ο διαχωρισμός. Η δυνατότητα αυτή σχετίζεται με την **ιχνηλασιμότητα (traceability)**, δηλαδή τη δυνατότητα να εξακριβωθεί αργότερα τι ακριβώς συνέβη σε κάθε δεδομένο κατά τη διάρκεια μιας υπολογιστικής διαδικασίας.



Σχήμα 3.2: *Κατανομή των εικόνων των συνόλων Training, Validation και Test ανά γεωγραφική κλάση*

3.5 ΠΡΟΕΤΟΙΜΑΣΙΑ ΤΩΝ ΔΕΔΟΜΕΝΩΝ ΓΙΑ ΤΑ ΜΟΝΤΕΛΑ ΒΑΘΙΑΣ ΜΑΘΗΣΗΣ

3.5.1 Φόρτωση και αριθμητική αναπαράσταση των εικόνων

Μετά τη δημιουργία των συνόλων training, validation και test, τα δεδομένα φορτώθηκαν στο περιβάλλον της **PyTorch** μέσω της κλάσης torchvision.datasets.ImageFolder. Η συγκεκριμένη κλάση είναι σχεδιασμένη για σύνολα εικόνων στα οποία κάθε κατηγορία αντιστοιχεί σε ξεχωριστό υποφάκελο. Η τελική δομή του dataset είχε επομένως τη γενική μορφή:

```
dataset_split/
 ├── train/
 │    ├── ipeiros/
 │    ├── kriti/
 │    ├── makedonia/
 │    └── ...
```

```
├─── validation/
│    ├─── ipeiros/
│    ├─── kriti/
│    ├─── makedonia/
│    └─── ...
└─── test/
   ├─── ipeiros/
   ├─── kriti/
   ├─── makedonia/
   └─── ...
```

Η ImageFolder αναγνωρίζει αυτόματα τις ονομασίες των υποφακέλων ως κλάσεις και δημιουργεί μία αριθμητική αντιστοίχιση (class_to_idx) μεταξύ κάθε κλάσης και ενός αριθμητικού δείκτη. Η μετατροπή αυτή είναι απαραίτητη, καθώς κατά την εκπαίδευση του μοντέλου οι πραγματικές κατηγορίες των εικόνων χρησιμοποιούνται με τη μορφή αριθμητικών ετικετών και όχι ως λεκτικές ονομασίες, όπως thessalia ή kriti.

Για καθένα από τα σύνολα training, validation και test δημιουργήθηκε το αντίστοιχο dataset μέσω της ImageFolder. Πριν συνεχιστεί η διαδικασία, ελέγχθηκε ότι κανένα από τα τρία datasets δεν ήταν κενό και ότι τόσο οι διαθέσιμες κλάσεις όσο και η αντιστοίχιση class_to_idx ήταν ακριβώς ίδιες στα training, validation και test sets.

Ο έλεγχος αυτός είναι σημαντικός για την ορθότητα της διαδικασίας εκπαίδευσης και αξιολόγησης. Εάν, για παράδειγμα, ένας αριθμητικός δείκτης αντιστοιχούσε στη Μακεδονία στο training set αλλά στην Κρήτη στο test set, οι ετικέτες των εικόνων θα ερμηνεύονταν διαφορετικά μεταξύ των συνόλων, με αποτέλεσμα η αξιολόγηση των προβλέψεων να είναι λανθασμένη.

3.5.2 Resize, Tensors και ImageNet Normalization

Πριν εισαχθεί μία εικόνα στο μοντέλο, πρέπει να μετατραπεί σε κατάλληλη αριθμητική μορφή και σε συγκεκριμένες διαστάσεις. Για τον σκοπό αυτό εφαρμόστηκε αρχικά μετασχηματισμός αλλαγής μεγέθους (**Resize**), με τελικές διαστάσεις **224 × 224 pixels**.

Επομένως, ανεξάρτητα από τις διαστάσεις του αποθηκευμένου αρχείου JPG, κάθε εικόνα που χρησιμοποιείται ως είσοδος στο μοντέλο έχει διαστάσεις 224 × 224 pixels. Το στάδιο αυτό είναι διαφορετικό από την προηγούμενη επεξεργασία με το **XnConvert**. Το XnConvert χρησιμοποιήθηκε για την αρχική και μόνιμη τυποποίηση των αρχείων του dataset, ενώ το Resize της PyTorch εφαρμόζεται κατά τη φόρτωση των δεδομένων και αποτελεί μέρος της προετοιμασίας των εικόνων για την εκπαίδευση και την αξιολόγηση των μοντέλων.

Στη συνέχεια, οι εικόνες μετατράπηκαν σε **tensors** μέσω της λειτουργίας transforms.ToTensor(). Ένα tensor είναι μία πολυδιάστατη αριθμητική δομή που χρησιμοποιείται για την αναπαράσταση και επεξεργασία δεδομένων στο περιβάλλον της PyTorch. Στην περίπτωση μιας έγχρωμης εικόνας RGB, το tensor περιλαμβάνει τις αριθμητικές τιμές των τριών χρωματικών καναλιών: κόκκινου (Red), πράσινου (Green) και μπλε (Blue).

Η μορφή μιας μεμονωμένης εικόνας στην PyTorch εκφράζεται ως:

$$[C,H,W]$$

όπου CC είναι ο αριθμός των χρωματικών καναλιών, HH το ύψος και WW το πλάτος της εικόνας.

Στην παρούσα εφαρμογή, μετά την αλλαγή μεγέθους, κάθε εικόνα έχει επομένως τη μορφή:

$$[3,224,224]$$

Μετά τη μετατροπή σε tensor εφαρμόστηκε **κανονικοποίηση (normalization)** με βάση τις τιμές που χρησιμοποιούνται για μοντέλα προεκπαιδευμένα στο **ImageNet**. Για τα τρία κανάλια RGB χρησιμοποιήθηκαν οι ακόλουθες τιμές μέσης τιμής και τυπικής απόκλισης:

$$Mean=[0.485,\ 0.456,\ 0.406]$$

$$Std=[0.229,\ 0.224,\ 0.225]$$

Η κανονικοποίηση κάθε τιμής πραγματοποιείται σύμφωνα με τη σχέση:

$$x' = \frac{x - \mu}{\sigma}$$

όπου x είναι η αρχική τιμή, μ η μέση τιμή και σ η τυπική απόκλιση του αντίστοιχου χρωματικού καναλιού.

Η χρήση των συγκεκριμένων τιμών κανονικοποίησης συνδέεται με το γεγονός ότι τα μοντέλα που αξιοποιήθηκαν στην παρούσα εργασία βασίζονται σε **προεκπαίδευση στο ImageNet**. Το ImageNet είναι ένα μεγάλης κλίμακας σύνολο δεδομένων εικόνων, το οποίο χρησιμοποιείται ευρέως για την προεκπαίδευση μοντέλων υπολογιστικής όρασης.

Η αξιοποίηση ενός ήδη προπαιδευμένου μοντέλου ως αφετηρία για την επίλυση ενός νέου και περισσότερο εξειδικευμένου προβλήματος αποτελεί τη βασική ιδέα του **transfer learning**. Στην παρούσα εργασία, η τεχνική αυτή αξιοποιήθηκε για την προσαρμογή των μοντέλων στο πρόβλημα αναγνώρισης της γεωγραφικής προέλευσης των ελληνικών παραδοσιακών φορεσιών. Η αρχιτεκτονική των μοντέλων και η διαδικασία του transfer learning παρουσιάζονται αναλυτικότερα στο επόμενο κεφάλαιο. Για τον λόγο αυτό, η προετοιμασία των νέων εικόνων πραγματοποιήθηκε με τρόπο συμβατό με την προεπεξεργασία που αντιστοιχεί στα προεκπαιδευμένα βάρη των μοντέλων.

3.6 ΕΜΠΛΟΥΤΙΣΜΟΣ ΚΑΙ ΦΟΡΤΩΣΗ ΤΩΝ ΔΕΔΟΜΕΝΩΝ ΚΑΤΑ ΤΗΝ ΕΚΠΑΙΔΕΥΣΗ

3.6.1 Data Augmentation

Οι **411 αρχικές εικόνες** αποτελούν ένα σχετικά περιορισμένο σύνολο δεδομένων για την εκπαίδευση μοντέλων βαθιάς μάθησης, ενώ μετά τον διαχωρισμό του dataset το training set περιλαμβάνει **286 εικόνες**. Για τον λόγο αυτό εφαρμόστηκε **data augmentation** αποκλειστικά στο training set.

Το data augmentation αποτελεί τεχνική εμπλουτισμού των δεδομένων, κατά την οποία εφαρμόζονται ελεγχόμενες και τυχαίες μεταβολές στις εικόνες κατά τη διάρκεια της εκπαίδευσης. Με αυτόν τον τρόπο, το μοντέλο δεν επεξεργάζεται κάθε εικόνα εκπαίδευσης συνεχώς με ακριβώς την ίδια μορφή, αλλά μπορεί να συναντά ελαφρώς διαφορετικές εκδοχές της. Στόχος είναι η αύξηση της ποικιλίας των δεδομένων που παρατηρεί το μοντέλο και η βελτίωση της ικανότητάς του να γενικεύει σε εικόνες που δεν έχει συναντήσει κατά την εκπαίδευση.

Οι μετασχηματισμοί που χρησιμοποιήθηκαν επιλέχθηκαν σκόπιμα ώστε να είναι ήπιοι. Συγκεκριμένα, εφαρμόστηκε **τυχαία περιστροφή έως ±5°**, καθώς και τυχαίος affine μετασχηματισμός που επέτρεπε **μετατόπιση έως 5%** ως προς τους δύο άξονες και μικρή μεταβολή της κλίμακας της εικόνας μεταξύ **95% και 105%**. Επιπλέον, χρησιμοποιήθηκε ColorJitter για την εφαρμογή ήπιων μεταβολών στη **φωτεινότητα και την αντίθεση κατά ±10%**.

```
train_transforms = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomRotation(degrees=5),
    transforms.RandomAffine(
        degrees=0,
        translate=(0.05, 0.05),
        scale=(0.95, 1.05)),
    transforms.ColorJitter(
        brightness=0.10,
        contrast=0.10 ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=IMAGENET_MEAN,
        std=IMAGENET_STD)
])
```

Κώδικας 3.1: *Ορισμός των μετασχηματισμών προεπεξεργασίας και data augmentation για το training set.*

Η επιλογή περιορισμένων μετασχηματισμών ήταν σκόπιμη, ώστε να αυξάνεται η ποικιλία των δεδομένων εκπαίδευσης χωρίς να δημιουργούνται υπερβολικά αλλοιωμένες ή μη ρεαλιστικές εκδοχές των παραδοσιακών φορεσιών. Δεν χρησιμοποιήθηκε **οριζόντια αναστροφή (RandomHorizontalFlip)**, παρότι αποτελεί συνηθισμένη τεχνική data augmentation σε εφαρμογές Υπολογιστικής Όρασης. Η οριζόντια αναστροφή μπορεί να μεταβάλει την πλευρά στην οποία εμφανίζονται ασύμμετρα ενδυματολογικά ή διακοσμητικά χαρακτηριστικά. Επειδή τέτοιου είδους λεπτομέρειες ενδέχεται να αποτελούν χρήσιμα οπτικά χαρακτηριστικά των παραδοσιακών φορεσιών, επιλέχθηκε να μη δημιουργούνται τεχνητά κατοπτρικές εκδοχές των εικόνων.

Η επιλογή αυτή αναδεικνύει ότι το data augmentation δεν πρέπει να εφαρμόζεται με τον ίδιο τρόπο σε κάθε πρόβλημα ταξινόμησης εικόνων. Οι κατάλληλοι μετασχηματισμοί πρέπει να επιλέγονται λαμβάνοντας υπόψη τη φύση του οπτικού υλικού και τα χαρακτηριστικά που ενδέχεται να είναι σημαντικά για τη διάκριση μεταξύ των κλάσεων.

Στα **validation και test sets δεν εφαρμόστηκε τυχαίο data augmentation**, καθώς τα συγκεκριμένα σύνολα χρησιμοποιούνται για την αξιολόγηση του μοντέλου και πρέπει να παραμένουν σταθερά. Η προεπεξεργασία τους περιορίστηκε στην αλλαγή μεγέθους (**Resize**), τη μετατροπή σε tensor (**ToTensor**) και την κανονικοποίηση (**Normalize**):

Resize → ToTensor → Normalize

Με αυτόν τον τρόπο, οι εικόνες των validation και test sets υποβάλλονται κάθε φορά στην ίδια διαδικασία προετοιμασίας, επιτρέποντας συνεπή και συγκρίσιμη αξιολόγηση των μοντέλων.

3.6.2 On-the-fly Augmentation και DataLoaders

Το data augmentation εφαρμόστηκε με τη λογική **on-the-fly**, δηλαδή οι μετασχηματισμένες εικόνες δεν δημιουργήθηκαν εκ των προτέρων ούτε αποθηκεύτηκαν ως νέα αρχεία στον δίσκο. Αντίθετα, οι τυχαίοι μετασχηματισμοί εφαρμόζονται κατά τη φόρτωση μιας εικόνας από το training dataset.

Ως αποτέλεσμα, η ίδια αρχική εικόνα μπορεί να παρουσιαστεί στο μοντέλο με διαφορετικές μικρές μεταβολές κατά τη διάρκεια της εκπαίδευσης. Για παράδειγμα, σε μία φόρτωση μπορεί να εφαρμοστεί μικρή περιστροφή, μετατόπιση ή μεταβολή φωτεινότητας διαφορετική από εκείνη που εφαρμόστηκε σε προηγούμενη φόρτωση της ίδιας εικόνας.

Η προσέγγιση αυτή αυξάνει τη μεταβλητότητα των παραδειγμάτων που παρατηρεί το μοντέλο, χωρίς να αυξάνει τον αριθμό των αρχείων που είναι αποθηκευμένα στο dataset. Είναι σημαντικό, ωστόσο, να διευκρινιστεί ότι το data augmentation δεν δημιουργεί νέα ανεξάρτητη πληροφορία. Οι μετασχηματισμένες εικόνες εξακολουθούν να προέρχονται από τις **286 πραγματικές εικόνες του training set**.

Για την οργανωμένη παροχή των δεδομένων στα μοντέλα χρησιμοποιήθηκαν αντικείμενα **DataLoader** της PyTorch. Ο DataLoader αναλαμβάνει, μεταξύ άλλων, την ομαδοποίηση των εικόνων σε μικρότερες ομάδες, οι οποίες ονομάζονται **batches**. Με τον τρόπο αυτό, αντί να επεξεργάζεται ολόκληρο το training set ταυτόχρονα, το μοντέλο λαμβάνει διαδοχικά μικρότερες ομάδες εικόνων. Η διαδικασία αυτή επιτρέπει αποτελεσματικότερη διαχείριση της διαθέσιμης μνήμης και αποτελεί βασικό μέρος της διαδικασίας εκπαίδευσης μοντέλων βαθιάς μάθησης.

Για το training set ενεργοποιήθηκε η τυχαία αναδιάταξη των δεδομένων (shuffle=True), ώστε οι εικόνες να μη δίνονται στο μοντέλο με την ίδια σειρά σε κάθε εποχή εκπαίδευσης. Αντίθετα, στα validation και test sets χρησιμοποιήθηκε shuffle=False, καθώς δεν απαιτείται τυχαία αναδιάταξη των δεδομένων κατά την αξιολόγηση.

Ιδιαίτερη προσοχή δόθηκε επίσης στην **αναπαραγωγιμότητα (reproducibility)** της διαδικασίας. Χρησιμοποιήθηκε αντικείμενο torch.Generator() με σταθερή τιμή RANDOM_SEED, ενώ ορίστηκε και συνάρτηση seed_worker() για την ελεγχόμενη αρχικοποίηση των workers που συμμετέχουν στη φόρτωση των δεδομένων. Μέσω της διαδικασίας αυτής αρχικοποιούνται με ελεγχόμενο τρόπο οι αντίστοιχες πηγές τυχαιότητας της **NumPy** και της **Python** για κάθε worker.

Η αναπαραγωγιμότητα είναι ιδιαίτερα σημαντική στα πειράματα Μηχανικής Μάθησης, καθώς κατά την εκπαίδευση εμπλέκονται πολλαπλές τυχαίες διαδικασίες. Η χρήση σταθερών seeds περιορίζει τις ανεξέλεγκτες διαφοροποιήσεις μεταξύ επαναλήψεων του ίδιου πειράματος και διευκολύνει την αξιόπιστη σύγκριση διαφορετικών μοντέλων ή πειραματικών ρυθμίσεων.

3.7 ΈΛΕΓΧΟΣ ΚΑΙ ΕΠΑΛΗΘΕΥΣΗ ΤΗΣ ΤΕΛΙΚΗΣ ΡΟΗΣ ΔΕΔΟΜΕΝΩΝ

3.7.1 Προγραμματιστικός και οπτικός έλεγχος

Πριν από την έναρξη της εκπαίδευσης πραγματοποιήθηκαν επιπλέον έλεγχοι, ώστε να επιβεβαιωθεί ότι η τελική ροή των δεδομένων λειτουργεί όπως αναμένεται και ότι οι εικόνες και οι αντίστοιχες ετικέτες τους έχουν την κατάλληλη μορφή. Αρχικά ελέγχθηκε ότι καθένας από τους DataLoaders των training, validation και test sets περιλαμβάνει τουλάχιστον ένα batch. Στη συνέχεια ανακτήθηκε ένα batch από το training set και εξετάστηκε η μορφή του tensor που περιέχει τις εικόνες.

Για ένα batch εικόνων, η αναμενόμενη μορφή του tensor στην PyTorch είναι:

$$[B, C, H, W]$$

όπου B είναι ο αριθμός των εικόνων του batch, C ο αριθμός των χρωματικών καναλιών, H το ύψος και W το πλάτος της εικόνας. Στην παρούσα εργασία, επομένως, η αναμενόμενη μορφή είναι:

$$[B, 3, 224, 224]$$

Πραγματοποιήθηκαν έλεγχοι ώστε να επιβεβαιωθεί ότι το tensor διαθέτει τέσσερις διαστάσεις, ότι κάθε εικόνα αποτελείται από ακριβώς τρία χρωματικά κανάλια RGB, ότι το ύψος και το πλάτος είναι **224 pixels** και ότι ο αριθμός των εικόνων του batch δεν υπερβαίνει το καθορισμένο BATCH_SIZE.

Αντίστοιχοι έλεγχοι πραγματοποιήθηκαν και στις αριθμητικές ετικέτες (**labels**) των εικόνων. Ελέγχθηκε ότι ο αριθμός των labels αντιστοιχεί στον αριθμό των εικόνων του batch, ότι οι ετικέτες έχουν τύπο torch.int64 και ότι κάθε αριθμητική τιμή βρίσκεται εντός του επιτρεπτού εύρους:

$$\mathbf{0} \leq \boldsymbol{label} < \boldsymbol{NUM_CLASSES}$$

Οι παραπάνω έλεγχοι επιτρέπουν τον έγκαιρο εντοπισμό πιθανών προβλημάτων στη μορφή ή στην αντιστοίχιση των δεδομένων πριν από την έναρξη της υπολογιστικά απαιτητικότερης διαδικασίας εκπαίδευσης.

Πέρα από τους προγραμματιστικούς ελέγχους πραγματοποιήθηκε και **οπτική επιθεώρηση των δεδομένων μετά την εφαρμογή του data augmentation**. Ο έλεγχος αυτός είναι σημαντικός, καθώς ένας μετασχηματισμός μπορεί να είναι προγραμματιστικά έγκυρος, αλλά το οπτικό αποτέλεσμά του να μην είναι κατάλληλο για το συγκεκριμένο πρόβλημα. Για τον σκοπό αυτό επιλέχθηκε ένα batch από τον training DataLoader και εμφανίστηκαν έως **16 εικόνες** σε μορφή πλέγματος. Επειδή οι εικόνες είχαν ήδη υποστεί ImageNet normalization, πριν από την απεικόνισή τους εφαρμόστηκε αντίστροφη κανονικοποίηση μέσω της συνάρτησης denormalize_image(). Η αντίστροφη διαδικασία περιγράφεται από τη σχέση:

$$\boldsymbol{x_{original}} = \boldsymbol{x_{normalized}} \cdot \boldsymbol{\sigma} + \boldsymbol{\mu}$$

όπου $x_{normalized}$ είναι η κανονικοποιημένη τιμή, μ η μέση τιμή και σ η τυπική απόκλιση του αντίστοιχου χρωματικού καναλιού. Με αυτόν τον τρόπο οι εικόνες μπορούσαν να παρουσιαστούν ξανά με κατάλληλες χρωματικές τιμές για οπτική εξέταση.

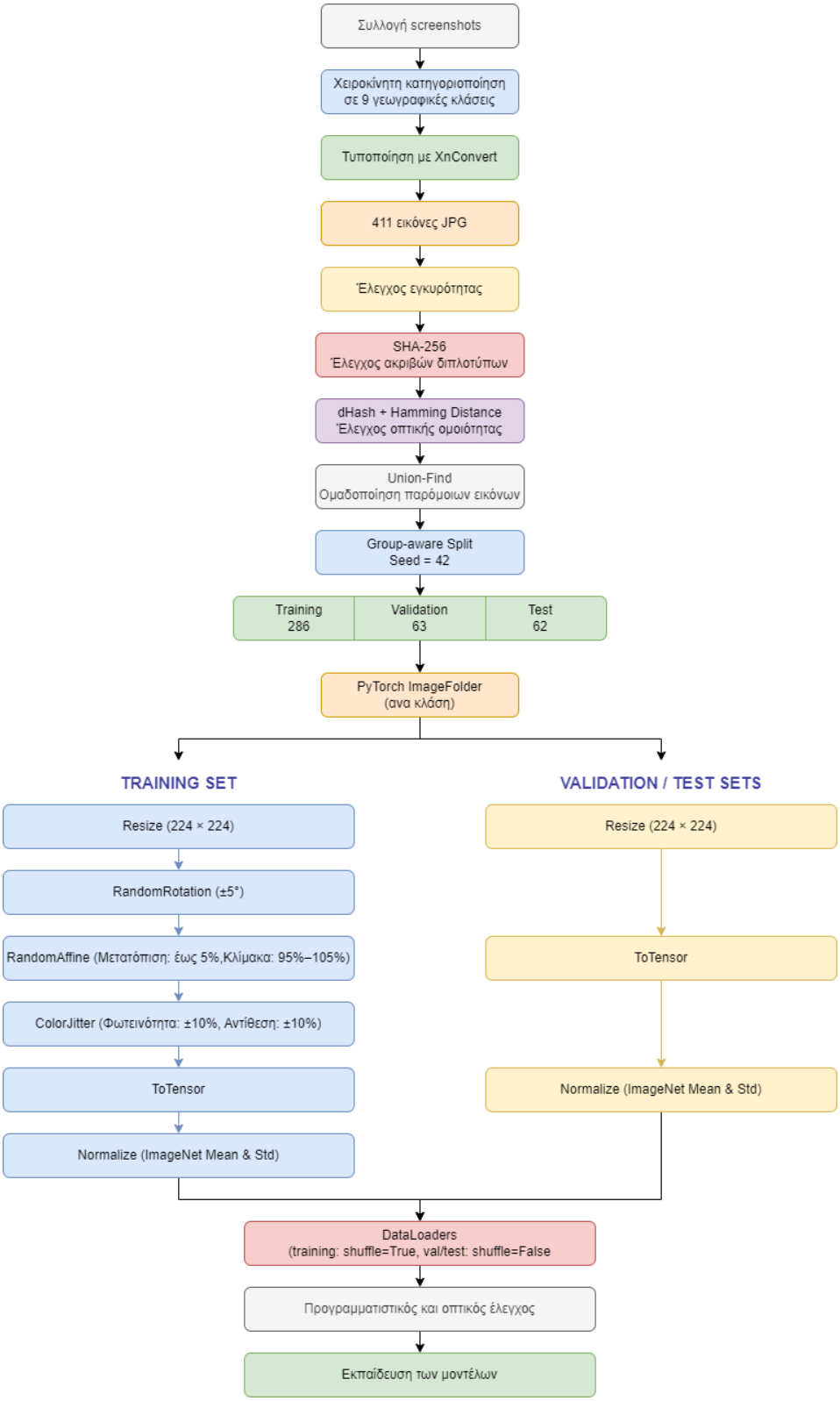
Μέσω της οπτικής επιθεώρησης μπορούσε να επιβεβαιωθεί ότι οι περιστροφές, οι μετατοπίσεις και οι μεταβολές φωτεινότητας και αντίθεσης παρέμεναν ήπιες και δεν αλλοίωναν σε μη ρεαλιστικό βαθμό τα χαρακτηριστικά των παραδοσιακών φορεσιών. Παράλληλα, εμφανίζονταν οι πραγματικές αριθμητικές ετικέτες και οι αντίστοιχες ονομασίες των γεωγραφικών κλάσεων, παρέχοντας ένα επιπλέον επίπεδο ελέγχου της ορθής αντιστοίχισης εικόνων και κατηγοριών.



Εικόνα 3.2: Δείγμα εικόνων του training set μετά την εφαρμογή data augmentation

3.7.2 Συνολική ροή προετοιμασίας του dataset

Με βάση τα στάδια που παρουσιάστηκαν στις προηγούμενες ενότητες, η συνολική διαδικασία δημιουργίας, ελέγχου και προετοιμασίας του συνόλου δεδομένων παρουσιάζεται σχηματικά στο Σχήμα 3.3.



Σχήμα 3.3: Συνολική ροή δημιουργίας, ελέγχου και προετοιμασίας του dataset

Η παραπάνω ροή αποτυπώνει διαδοχικά τη διαδικασία από τη συλλογή και οργάνωση του αρχικού οπτικού υλικού έως την τελική προετοιμασία και φόρτωση των δεδομένων που χρησιμοποιούνται για την εκπαίδευση και την αξιολόγηση των μοντέλων.

3.8 ΠΕΡΙΟΡΙΣΜΟΙ ΤΟΥ DATASET ΚΑΙ ΣΥΜΠΕΡΑΣΜΑΤΑ

3.8.1 Περιορισμοί και δυνατότητες επέκτασης

Παρά τη συστηματική διαδικασία δημιουργίας, ελέγχου και προετοιμασίας του συνόλου δεδομένων, υπάρχουν ορισμένοι περιορισμοί που πρέπει να λαμβάνονται υπόψη κατά την ερμηνεία των αποτελεσμάτων της παρούσας εργασίας.

Ένας βασικός περιορισμός αφορά το **μέγεθος του dataset**. Οι 411 αρχικές εικόνες αποτελούν σχετικά περιορισμένο αριθμό για ένα πρόβλημα ταξινόμησης εικόνων με τεχνικές βαθιάς μάθησης, ιδιαίτερα από τη στιγμή που κατανέμονται σε εννέα διαφορετικές γεωγραφικές κλάσεις. Η αξιοποίηση του transfer learning και του data augmentation επιτρέπει την αποτελεσματικότερη χρήση των διαθέσιμων δεδομένων, χωρίς όμως να μπορεί να υποκαταστήσει την πληροφορία που θα παρείχε ένας μεγαλύτερος αριθμός πραγματικών και ανεξάρτητων εικόνων.

Ένας δεύτερος περιορισμός αφορά την **αριθμητική κατανομή των εικόνων μεταξύ των κλάσεων**. Στο αρχικό dataset ο αριθμός των εικόνων ανά γεωγραφική περιοχή κυμαίνεται από **39 έως 54 εικόνες**. Επομένως, υπάρχει μία σχετικά μικρή διαφοροποίηση ως προς το πλήθος των δειγμάτων που εκπροσωπούν κάθε κλάση. Η διαφορά αυτή δεν είναι ιδιαίτερα μεγάλη, ωστόσο αποτελεί χαρακτηριστικό του dataset που πρέπει να συνεκτιμάται κατά την ανάλυση της απόδοσης του μοντέλου ανά γεωγραφική περιοχή.

Διαφορετικό ζήτημα αποτελεί η **εσωτερική ποικιλομορφία των κλάσεων**. Κάθε γεωγραφική περιοχή μπορεί να περιλαμβάνει διαφορετικές τοπικές φορεσιές, οι οποίες στην παρούσα εργασία αντιμετωπίζονται ως δείγματα της ίδιας γεωγραφικής κλάσης. Ωστόσο, οι εννέα περιοχές δεν εκπροσωπούνται από τον ίδιο αριθμό διαφορετικών τοπικών φορεσιών. Κατά συνέπεια, δύο κλάσεις με παρόμοιο συνολικό αριθμό εικόνων μπορεί να διαφέρουν σημαντικά ως προς την ποικιλία των οπτικών χαρακτηριστικών που περιλαμβάνουν και τα οποία καλείται να μάθει το μοντέλο.

Παράλληλα, παραδοσιακές φορεσιές διαφορετικών γεωγραφικών περιοχών μπορεί να παρουσιάζουν **οπτικές ομοιότητες** ως προς επιμέρους χαρακτηριστικά, όπως το είδος των ενδυμάτων, τα χρώματα, τα διακοσμητικά στοιχεία ή τη γενικότερη μορφολογία τους. Η ομοιότητα αυτή μπορεί να καθιστά δυσκολότερη τη διάκριση ορισμένων γεωγραφικών κλάσεων και αποτελεί εγγενή δυσκολία του συγκεκριμένου προβλήματος ταξινόμησης.

Ένας ακόμη περιορισμός προκύπτει από το γεγονός ότι το αρχικό οπτικό υλικό συλλέχθηκε από **μία συγκεκριμένη διαδικτυακή πηγή**. Ως αποτέλεσμα, οι εικόνες είναι πιθανό να παρουσιάζουν κοινά χαρακτηριστικά ως προς τον τρόπο φωτογράφισης και παρουσίασης, όπως το υπόβαθρο, το κάδρο, τον φωτισμό ή τη στάση των εικονιζόμενων προσώπων. Ένα μοντέλο ενδέχεται να αξιοποιήσει όχι μόνο χαρακτηριστικά της ίδιας της φορεσιάς, αλλά και τέτοιες επαναλαμβανόμενες οπτικές πληροφορίες. Για τον λόγο αυτό, η απόδοση σε εικόνες που προέρχονται από διαφορετικές πηγές και συνθήκες λήψης αποτελεί σημαντικό ζήτημα γενίκευσης.

Σε μελλοντική επέκταση της εργασίας θα ήταν επομένως ιδιαίτερα χρήσιμη η δημιουργία ενός μεγαλύτερου και περισσότερο ποικιλόμορφου dataset, με εικόνες από **περισσότερες ανεξάρτητες πηγές**, διαφορετικά υπόβαθρα και συνθήκες λήψης, καθώς και μεγαλύτερη κάλυψη των επιμέρους τοπικών φορεσιών κάθε γεωγραφικής περιοχής. Μια τέτοια επέκταση θα μπορούσε να ενισχύσει την αντιπροσωπευτικότητα του συνόλου δεδομένων και να επιτρέψει πιο αξιόπιστη αξιολόγηση της ικανότητας γενίκευσης των μοντέλων.

Τέλος, ένα σημαντικά μεγαλύτερο και λεπτομερέστερα επισημασμένο σύνολο δεδομένων θα μπορούσε να επιτρέψει την εξέταση ενός πιο εξειδικευμένου προβλήματος ταξινόμησης. Αντί της πρόβλεψης μόνο της γεωγραφικής περιοχής, ένα μελλοντικό σύστημα θα μπορούσε να επιχειρεί την αναγνώριση της **συγκεκριμένης τοπικής φορεσιάς**. Μια τέτοια προσέγγιση θα απαιτούσε σημαντικά περισσότερα δεδομένα ανά τοπική φορεσιά και αντίστοιχη λεπτομερέστερη επισήμανση των εικόνων.

3.9 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο παρουσιάστηκε η διαδικασία **δημιουργίας, ελέγχου και προετοιμασίας του συνόλου δεδομένων** που χρησιμοποιήθηκε ως βάση για την εκπαίδευση και την αξιολόγηση των μοντέλων. Το αρχικό dataset περιλάμβανε **411 εικόνες ελληνικών παραδοσιακών φορεσιών**, οργανωμένες σε **εννέα γεωγραφικές κλάσεις**. Όλες οι εικόνες πέρασαν επιτυχώς τον έλεγχο εγκυρότητας, ενώ μέσω του SHA-256 δεν εντοπίστηκαν ακριβή διπλότυπα. Ο συμπληρωματικός έλεγχος οπτικής ομοιότητας με **dHash και Hamming distance** εντόπισε **110 ζεύγη πιθανώς όμοιων εικόνων**. Η πληροφορία αυτή αξιοποιήθηκε για την ομαδοποίηση των σχετικών εικόνων μέσω της δομής Union-Find, με αποτέλεσμα **86 εικόνες να συμμετέχουν σε 25 ομάδες με περισσότερα από ένα μέλη**.

Για τον διαχωρισμό του dataset εφαρμόστηκε **group-aware split**, ώστε όλες οι εικόνες που ανήκαν στην ίδια ομάδα ομοιότητας να παραμένουν στο ίδιο υποσύνολο δεδομένων. Με επιθυμητή αναλογία **70% training, 15% validation και 15% test** και σταθερό seed ίσο με **42**, προέκυψαν τελικά **286 εικόνες για το training set, 63 για το validation set και 62 για το test set**. Ο έλεγχος της τελικής κατανομής επιβεβαίωσε ότι καμία ομάδα δεν διαμοιράστηκε μεταξύ των τριών συνόλων, περιορίζοντας με αυτόν τον τρόπο τον κίνδυνο **διαρροής δεδομένων (data leakage)** λόγω της παρουσίας πολύ όμοιων εικόνων σε διαφορετικά σύνολα.

Στη συνέχεια, τα δεδομένα φορτώθηκαν μέσω της **PyTorch**, μετατράπηκαν σε tensors κατάλληλων διαστάσεων και κανονικοποιήθηκαν με βάση τις τιμές του **ImageNet**. Στο training set εφαρμόστηκε ήπιο **on-the-fly data augmentation**, ενώ στα validation και test sets χρησιμοποιήθηκε σταθερή διαδικασία προεπεξεργασίας χωρίς τυχαίους μετασχηματισμούς.

Τέλος, η ορθότητα της συνολικής ροής δεδομένων επαληθεύτηκε τόσο προγραμματιστικά, μέσω ελέγχων στα datasets, στα labels, στα tensors και στους DataLoaders, όσο και οπτικά μέσω της απεικόνισης εικόνων του training set μετά την εφαρμογή του data augmentation.

Με την ολοκλήρωση των παραπάνω σταδίων διαμορφώθηκε ένα **ελεγχόμενο και αναπαραγώγιμο σύνολο δεδομένων**, κατάλληλα οργανωμένο και προετοιμασμένο για την πειραματική διαδικασία. Στο επόμενο κεφάλαιο παρουσιάζονται οι αρχιτεκτονικές των μοντέλων που αξιοποιήθηκαν, καθώς και η μεθοδολογία προσαρμογής τους στο πρόβλημα αναγνώρισης της γεωγραφικής προέλευσης των ελληνικών παραδοσιακών φορεσιών.

ΚΕΦΑΛΑΙΟ 4 - ΜΕΘΟΔΟΛΟΓΙΑ ΚΑΙ ΠΡΟΣΑΡΜΟΓΗ ΤΩΝ ΜΟΝΤΕΛΩΝ

4.1 ΕΙΣΑΓΩΓΗ

Μετά την προετοιμασία και τον έλεγχο του συνόλου δεδομένων, το επόμενο στάδιο της εργασίας αφορά την επιλογή και την προσαρμογή κατάλληλων μοντέλων Βαθιάς Μάθησης για την ταξινόμηση των εικόνων ελληνικών παραδοσιακών φορεσιών στις εννέα γεωγραφικές κλάσεις που έχουν οριστεί. Στο παρόν κεφάλαιο παρουσιάζεται η μεθοδολογία που ακολουθήθηκε για τη διαμόρφωση των μοντέλων και τη δημιουργία των διαφορετικών πειραματικών διαμορφώσεων που χρησιμοποιούνται στη συνέχεια κατά την εκπαίδευση και την αξιολόγηση.

Για την πειραματική διαδικασία επιλέχθηκαν τρεις προεκπαιδευμένες αρχιτεκτονικές Συνελικτικών Νευρωνικών Δικτύων, οι **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny**. Οι αρχιτεκτονικές αυτές παρουσιάστηκαν θεωρητικά στο προηγούμενο κεφάλαιο και αντιπροσωπεύουν διαφορετικές σχεδιαστικές προσεγγίσεις στην επεξεργασία και εξαγωγή χαρακτηριστικών από ψηφιακές εικόνες. Στο παρόν κεφάλαιο το ενδιαφέρον μετατοπίζεται από τη θεωρητική τους λειτουργία στον τρόπο με τον οποίο χρησιμοποιήθηκαν και προσαρμόστηκαν στο συγκεκριμένο πρόβλημα ταξινόμησης.

Δεδομένου του σχετικά περιορισμένου μεγέθους του διαθέσιμου συνόλου δεδομένων, η ανάπτυξη των μοντέλων δεν βασίστηκε σε εκπαίδευση από τυχαία αρχικοποιημένα βάρη. Αντίθετα, αξιοποιήθηκε η τεχνική του **transfer learning**, με χρήση προεκπαιδευμένων βαρών και προσαρμογή των μοντέλων στη νέα εργασία ταξινόμησης. Με αυτόν τον τρόπο αξιοποιούνται οπτικά χαρακτηριστικά που έχουν ήδη αποκτηθεί κατά την προεκπαίδευση, ενώ η τελική έξοδος κάθε αρχιτεκτονικής προσαρμόζεται στις εννέα κλάσεις του παρόντος προβλήματος.

Για καθεμία από τις τρεις αρχιτεκτονικές εξετάζονται δύο διαφορετικές στρατηγικές μεταφοράς μάθησης. Η πρώτη είναι το **Feature Extraction**, κατά το οποίο ο κύριος κορμός του προεκπαιδευμένου μοντέλου χρησιμοποιείται ως σταθερός εξαγωγέας χαρακτηριστικών και εκπαιδεύεται κυρίως το νέο τμήμα ταξινόμησης. Η δεύτερη είναι το **Partial Fine-Tuning**, κατά το οποίο επιτρέπεται η περαιτέρω προσαρμογή επιλεγμένου μέρους των προεκπαιδευμένων παραμέτρων στο νέο σύνολο δεδομένων.

Ο συνδυασμός των τριών αρχιτεκτονικών με τις δύο στρατηγικές οδηγεί σε **έξι πειραματικές διαμορφώσεις**, γεγονός που επιτρέπει την εξέταση όχι μόνο της επίδρασης της επιλογής αρχιτεκτονικής αλλά και της στρατηγικής προσαρμογής των προεκπαιδευμένων μοντέλων. Με αυτόν τον τρόπο δημιουργείται ένα κοινό πλαίσιο σύγκρισης, το οποίο θα χρησιμοποιηθεί στα επόμενα κεφάλαια για τη μελέτη της εκπαιδευτικής συμπεριφοράς και της τελικής απόδοσης των μοντέλων.

Στις επόμενες ενότητες παρουσιάζονται το υπολογιστικό και προγραμματιστικό περιβάλλον της υλοποίησης, η αρχικοποίηση και προσαρμογή των τριών μοντέλων, η διαμόρφωση των στρατηγικών Feature Extraction και Partial Fine-Tuning και οι έλεγχοι που πραγματοποιήθηκαν ώστε να επιβεβαιωθεί η σωστή προετοιμασία τους πριν από την έναρξη της εκπαίδευσης.

4.2 ΥΠΟΛΟΓΙΣΤΙΚΟ ΠΕΡΙΒΑΛΛΟΝ ΚΑΙ ΒΙΒΛΙΟΘΗΚΕΣ

Η ανάπτυξη και η εκτέλεση των πειραμάτων πραγματοποιήθηκαν σε περιβάλλον **Google Colab**, το οποίο επιτρέπει την εκτέλεση κώδικα Python μέσω διαδικτυακών notebooks και παρέχει πρόσβαση σε υπολογιστικούς επιταχυντές. Η δυνατότητα αυτή είναι ιδιαίτερα χρήσιμη σε εφαρμογές Βαθιάς Μάθησης, καθώς η εκπαίδευση νευρωνικών δικτύων απαιτεί την εκτέλεση μεγάλου αριθμού αριθμητικών πράξεων και μπορεί να είναι ιδιαίτερα χρονοβόρα όταν πραγματοποιείται αποκλειστικά από την κεντρική μονάδα επεξεργασίας (CPU).

Για τη μόνιμη αποθήκευση των δεδομένων και των αρχείων που παράγονται κατά την πειραματική διαδικασία χρησιμοποιήθηκε το **Google Drive**, το οποίο συνδέθηκε με το περιβάλλον του Google Colab. Με αυτόν τον τρόπο, τα δεδομένα δεν περιορίζονται στον προσωρινό αποθηκευτικό χώρο μιας συνεδρίας του Colab, αλλά μπορούν να διατηρούνται και να επαναχρησιμοποιούνται σε επόμενες εκτελέσεις.

Ο βασικός φάκελος του έργου ορίστηκε στη διαδρομή */content/drive/MyDrive/ptyxiaki*, κάτω από την οποία οργανώθηκαν τα δεδομένα και τα επιμέρους αρχεία της εργασίας.

Ως βασικό πλαίσιο ανάπτυξης των μοντέλων χρησιμοποιήθηκε η βιβλιοθήκη **PyTorch**. Πρόκειται για πλαίσιο Βαθιάς Μάθησης που παρέχει τις απαραίτητες δομές και λειτουργίες για την αναπαράσταση δεδομένων με tensors, τη δημιουργία νευρωνικών δικτύων, τον υπολογισμό των προβλέψεων και των gradients και την ενημέρωση των παραμέτρων κατά την εκπαίδευση. Στην πειραματική εκτέλεση που καταγράφεται στο notebook χρησιμοποιήθηκε η έκδοση **PyTorch 2.11.0+cu128**, δηλαδή έκδοση με υποστήριξη του αντίστοιχου περιβάλλοντος CUDA.

Η εκπαίδευση των μοντέλων πραγματοποιήθηκε με αξιοποίηση **μονάδας επεξεργασίας γραφικών (Graphics Processing Unit – GPU)**. Σε αντίθεση με μία CPU, η οποία είναι σχεδιασμένη για την εκτέλεση ενός ευρέος φάσματος γενικών υπολογιστικών εργασιών, μία GPU διαθέτει αρχιτεκτονική ιδιαίτερα κατάλληλη για την παράλληλη εκτέλεση μεγάλου αριθμού αριθμητικών πράξεων. Το χαρακτηριστικό αυτό είναι σημαντικό στη Βαθιά Μάθηση, όπου λειτουργίες όπως οι πράξεις μεταξύ tensors και οι συνελίξεις επαναλαμβάνονται σε μεγάλο αριθμό δεδομένων και παραμέτρων.

Η επικοινωνία του PyTorch με τη GPU πραγματοποιήθηκε μέσω της πλατφόρμας **CUDA**, η οποία επιτρέπει την αξιοποίηση συμβατών GPU της NVIDIA για υπολογιστικές εργασίες. Ο κώδικας ελέγχει κατά την εκτέλεση εάν υπάρχει διαθέσιμη υποστήριξη CUDA και επιλέγει αντίστοιχα την κατάλληλη υπολογιστική συσκευή. Στη συγκεκριμένη εκτέλεση η CUDA ήταν διαθέσιμη και χρησιμοποιήθηκε **NVIDIA Tesla T4**, με την υπολογιστική συσκευή να ορίζεται ως cuda. Με αυτόν τον τρόπο οι απαιτητικές πράξεις της εκπαίδευσης μπορούσαν να εκτελούνται στη GPU αντί αποκλειστικά στη CPU.

Συμπληρωματικά προς το PyTorch χρησιμοποιήθηκε η βιβλιοθήκη **Torchvision**, η οποία παρέχει λειτουργίες ειδικά σχεδιασμένες για εφαρμογές Υπολογιστικής Όρασης. Στην παρούσα εργασία αξιοποιήθηκε τόσο για τους μετασχηματισμούς των εικόνων όσο και για τη φόρτωση των δεδομένων και την πρόσβαση στις προεκπαιδευμένες αρχιτεκτονικές νευρωνικών δικτύων που χρησιμοποιήθηκαν στα πειράματα. Με αυτόν τον τρόπο η διαδικασία προετοιμασίας των εικόνων και η ανάπτυξη των μοντέλων ενσωματώθηκαν στο ίδιο οικοσύστημα βιβλιοθηκών.

Η φόρτωση των εικόνων πραγματοποιήθηκε με την κλάση **ImageFolder** της Torchvision. Η ImageFolder αξιοποιεί την οργάνωση του dataset σε φακέλους, όπου κάθε υποφάκελος αντιστοιχεί σε διαφορετική κλάση, και δημιουργεί την απαραίτητη αντιστοίχιση μεταξύ των ονομάτων των κλάσεων και αριθμητικών ετικετών. Στην εργασία δημιουργήθηκαν ανεξάρτητα αντικείμενα dataset για τα **training, validation και test sets**, ενώ πραγματοποιήθηκαν έλεγχοι ώστε και τα τρία να περιλαμβάνουν τις ίδιες γεωγραφικές κλάσεις και την ίδια αντιστοίχιση μεταξύ ονομάτων και αριθμητικών labels.

Κατά την εκπαίδευση δεν είναι πρακτικό ούτε απαραίτητο να επεξεργάζεται το μοντέλο ολόκληρο το σύνολο δεδομένων ταυτόχρονα. Οι εικόνες οργανώνονται επομένως σε μικρότερες ομάδες, οι οποίες ονομάζονται **batches**. Στο PyTorch η διαδικασία αυτή πραγματοποιείται μέσω της κλάσης **DataLoader**, η οποία αναλαμβάνει να τροφοδοτεί διαδοχικά τα δεδομένα στο μοντέλο. Για το training set χρησιμοποιήθηκε τυχαία αναδιάταξη των δειγμάτων (shuffle=True), ώστε η σειρά παρουσίασης των εικόνων να μεταβάλλεται κατά την εκπαίδευση, ενώ για τα validation και test sets χρησιμοποιήθηκε shuffle=False, καθώς κατά την αξιολόγηση δεν απαιτείται τυχαία αναδιάταξη.

Σημαντικό στοιχείο μιας πειραματικής διαδικασίας αποτελεί η **αναπαραγωγιμότητα (reproducibility)**, δηλαδή η δυνατότητα επανάληψης ενός πειράματος κάτω από όσο το δυνατόν πιο ελεγχόμενες συνθήκες. Στη Μηχανική Μάθηση υπάρχουν διαδικασίες που περιλαμβάνουν τυχαιότητα, όπως η αναδιάταξη των δεδομένων και οι τυχαίοι μετασχηματισμοί που εφαρμόζονται κατά το data augmentation. Για τον έλεγχο αυτής της τυχαιότητας χρησιμοποιήθηκε σταθερός **random seed** με τιμή:

seed=42

Η συγκεκριμένη τιμή δεν έχει ιδιαίτερη μαθηματική σημασία· λειτουργεί ως σταθερό σημείο αρχικοποίησης των γεννητριών ψευδοτυχαίων αριθμών. Στην υλοποίηση η διαχείριση της τυχαιότητας υποστηρίζεται από το PyTorch, τη **NumPy** και την ενσωματωμένη βιβλιοθήκη random της Python. Επιπλέον, πραγματοποιείται κατάλληλη αρχικοποίηση των workers του DataLoader, ώστε η παράλληλη φόρτωση και οι τυχαίες διαδικασίες που σχετίζονται με τα δεδομένα να πραγματοποιούνται με ελεγχόμενο τρόπο. Η χρήση σταθερού seed δεν εξαλείφει κατ' ανάγκη κάθε πιθανή πηγή μη ντετερμινισμού, συμβάλλει όμως στη δημιουργία περισσότερο αναπαραγώγιμων πειραματικών συνθηκών.

Η υλοποίηση λαμβάνει επίσης υπόψη τον τρόπο μεταφοράς των δεδομένων προς τη GPU. Όταν εντοπίζεται διαθέσιμη CUDA συσκευή, ενεργοποιείται η επιλογή **pin_memory** στους DataLoaders. Η χρήση pinned μνήμης μπορεί να διευκολύνει την αποδοτικότερη μεταφορά των batches από τη μνήμη του συστήματος προς τη GPU. Η ρύθμιση αυτή πραγματοποιείται δυναμικά, ώστε η συμπεριφορά του προγράμματος να προσαρμόζεται ανάλογα με το αν υπάρχει διαθέσιμη GPU.

Πριν από την εκπαίδευση πραγματοποιούνται επιπλέον **προγραμματιστικοί έλεγχοι εγκυρότητας**, ώστε να επιβεβαιωθεί ότι η ροή δεδομένων έχει διαμορφωθεί σωστά. Μεταξύ άλλων, ελέγχεται ότι τα training, validation και test sets δεν είναι κενά, ότι περιλαμβάνουν τις ίδιες κλάσεις και ότι η αντιστοίχιση των κλάσεων σε αριθμητικές ετικέτες παραμένει κοινή. Ελέγχεται επίσης ένα πραγματικό training batch ως προς τις διαστάσεις των tensors, τον αριθμό των χρωματικών καναλιών, τις χωρικές διαστάσεις των εικόνων και την εγκυρότητα των αριθμητικών labels.

Συνολικά, το παραπάνω υπολογιστικό και προγραμματιστικό περιβάλλον διαμορφώθηκε ώστε να παρέχει μία ενιαία και ελεγχόμενη βάση για την εκτέλεση των πειραμάτων. Το **Google Colab και η NVIDIA Tesla T4** παρέχουν τους απαιτούμενους υπολογιστικούς πόρους, ενώ οι **PyTorch και Torchvision** υποστηρίζουν τη φόρτωση των δεδομένων, την ανάπτυξη και προσαρμογή των νευρωνικών δικτύων και τη διαδικασία εκπαίδευσης. Πάνω σε αυτό το περιβάλλον υλοποιήθηκαν οι πειραματικές διαμορφώσεις των **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny**, οι οποίες αναλύονται στις επόμενες ενότητες.

4.3 ΕΠΙΛΟΓΗ ΚΑΙ ΑΡΧΙΚΟΠΟΙΗΣΗ ΜΟΝΤΕΛΩΝ

Για την πειραματική διαδικασία χρησιμοποιήθηκαν τρεις αρχιτεκτονικές Συνελικτικών Νευρωνικών Δικτύων: **ResNet18, EfficientNet-B0 και ConvNeXt-Tiny**. Οι βασικές αρχές λειτουργίας και τα ιδιαίτερα χαρακτηριστικά των τριών αρχιτεκτονικών παρουσιάστηκαν στο Κεφάλαιο 2. Στο παρόν κεφάλαιο το ενδιαφέρον μετατοπίζεται από τη θεωρητική τους περιγραφή στον τρόπο με τον οποίο χρησιμοποιήθηκαν και προσαρμόστηκαν στο συγκεκριμένο πρόβλημα ταξινόμησης εικόνων.

Για κάθε αρχιτεκτονική δημιουργήθηκαν δύο διαφορετικές πειραματικές διαμορφώσεις, μία με τη στρατηγική **Feature Extraction** και μία με τη στρατηγική **Partial Fine-Tuning**. Με αυτόν τον τρόπο προέκυψαν συνολικά έξι πειράματα. Η χρήση των ίδιων τριών αρχιτεκτονικών με δύο διαφορετικούς τρόπους μεταφοράς μάθησης επιτρέπει στη συνέχεια να εξεταστεί όχι μόνο η επίδραση της επιλογής του μοντέλου, αλλά και η επίδραση του τρόπου με τον οποίο αυτό προσαρμόζεται στο νέο πρόβλημα.

Η δημιουργία των μοντέλων πραγματοποιήθηκε μέσω του υποπακέτου torchvision.models. Και στις τρεις περιπτώσεις χρησιμοποιήθηκαν **προεκπαιδευμένα βάρη**, αντί τα μοντέλα να αρχικοποιηθούν με τυχαίες τιμές. Συγκεκριμένα, στο notebook χρησιμοποιούνται οι επιλογές ResNet18_Weights.DEFAULT, EfficientNet_B0_Weights.DEFAULT και ConvNeXt_Tiny_Weights.DEFAULT για τις ResNet18, EfficientNet-B0 και ConvNeXt-Tiny αντίστοιχα. Με την αρχικοποίηση αυτή, τα δίκτυα ξεκινούν από ήδη διαθέσιμα προεκπαιδευμένα βάρη και πάνω σε αυτή τη βάση πραγματοποιείται η προσαρμογή τους στο dataset της εργασίας.

Η βασική λογική της αρχικοποίησης μπορεί να φανεί στο ακόλουθο αντιπροσωπευτικό απόσπασμα του notebook. Οι αντίστοιχες εντολές χρησιμοποιούνται στις επιμέρους πειραματικές ενότητες για τη φόρτωση των τριών αρχιτεκτονικών:

```
resnet18_weights = ResNet18_Weights.DEFAULT
model = resnet18(
    weights=resnet18_weights
)
efficientnet_weights = EfficientNet_B0_Weights.DEFAULT
model = efficientnet_b0(
    weights=efficientnet_weights
)
convnext_weights = ConvNeXt_Tiny_Weights.DEFAULT
model = convnext_tiny(
    weights=convnext_weights
)
```

Κώδικας 4.1: *Αρχικοποίηση των τριών αρχιτεκτονικών με τα προεκπαιδευμένα βάρη που χρησιμοποιούνται στο notebook.*

Η φόρτωση ενός προεκπαιδευμένου μοντέλου, ωστόσο, δεν αρκεί από μόνη της για τη χρήση του στο συγκεκριμένο πρόβλημα. Κάθε μοντέλο διαθέτει στο τέλος της αρχιτεκτονικής του ένα επίπεδο ταξινόμησης, το οποίο μετατρέπει τα χαρακτηριστικά που έχουν εξαχθεί από την εικόνα στις τελικές εξόδους του δικτύου. Επειδή στην παρούσα εργασία το ζητούμενο είναι η ταξινόμηση κάθε εικόνας σε μία από **εννέα γεωγραφικές κλάσεις**, το τελικό επίπεδο κάθε αρχιτεκτονικής πρέπει να προσαρμοστεί ώστε να παράγει ακριβώς εννέα εξόδους.

Ο αριθμός αυτός δεν καταχωρίζεται ανεξάρτητα σε κάθε μοντέλο. Στο notebook χρησιμοποιείται η μεταβλητή NUM_CLASSES, η οποία προκύπτει από τον αριθμό των κλάσεων που έχουν αναγνωριστεί κατά τη φόρτωση του training dataset. Πριν από τη δημιουργία των μοντέλων πραγματοποιούνται επίσης έλεγχοι ώστε οι κλάσεις και η αντιστοίχισή τους σε αριθμητικούς δείκτες να είναι ίδιες στα training, validation και test sets. Έτσι, η έξοδος κάθε μοντέλου συνδέεται με το ίδιο σύνολο εννέα κατηγοριών σε ολόκληρη την πειραματική διαδικασία.

Παρότι η βασική λογική είναι κοινή, το τελικό επίπεδο δεν βρίσκεται στο ίδιο σημείο και στις τρεις αρχιτεκτονικές. Στη **ResNet18** το επίπεδο ταξινόμησης προσπελαύνεται στο notebook μέσω του model.fc. Στην **EfficientNet-B0** βρίσκεται στη δεύτερη θέση του τμήματος classifier και προσπελαύνεται μέσω του model.classifier[1], ενώ στην **ConvNeXt-Tiny** το αντίστοιχο γραμμικό επίπεδο βρίσκεται στη θέση model.classifier[2]. Οι εκφράσεις αυτές αποτελούν τον τρόπο με τον οποίο η υλοποίηση της torchvision επιτρέπει την πρόσβαση στο κατάλληλο τελικό επίπεδο κάθε αρχιτεκτονικής· δεν αλλάζουν τη βασική μεθοδολογική αρχή που εφαρμόζεται.

Πριν από την αντικατάσταση του τελικού επιπέδου, το notebook ανακτά τον αριθμό των χαρακτηριστικών που δέχεται ως είσοδο το υπάρχον επίπεδο μέσω της ιδιότητας in_features. Ο αριθμός των εισόδων διατηρείται, επειδή πρέπει να εξακολουθεί να είναι συμβατός με το προηγούμενο τμήμα του δικτύου. Αντίθετα, ο αριθμός των εξόδων αλλάζει και ορίζεται ίσος με NUM_CLASSES.

Επομένως, δεν τροποποιείται αυθαίρετα η σύνδεση με το προηγούμενο τμήμα της αρχιτεκτονικής· τροποποιείται η έξοδος ώστε να ανταποκρίνεται στις εννέα κατηγορίες του νέου προβλήματος.

Η ίδια αρχή εφαρμόζεται και στις άλλες δύο αρχιτεκτονικές, με τη διαφορά ότι χρησιμοποιείται η κατάλληλη θέση του classifier για καθεμία. Μετά την αντικατάσταση του τελικού επιπέδου, κάθε μοντέλο παράγει μία έξοδο για καθεμία από τις εννέα κλάσεις. Με αυτόν τον τρόπο, τρεις αρχιτεκτονικές με διαφορετική εσωτερική δομή καταλήγουν να επιλύουν ακριβώς το ίδιο πρόβλημα ταξινόμησης και μπορούν επομένως να συγκριθούν στο πλαίσιο της ίδιας πειραματικής διαδικασίας.

Μετά τη δημιουργία και την προσαρμογή του, κάθε μοντέλο μεταφέρεται στη διαθέσιμη υπολογιστική συσκευή μέσω της εντολής model.to(DEVICE). Η μεταβλητή DEVICE έχει ήδη καθοριστεί στο κοινό τμήμα προετοιμασίας του notebook, ώστε η εκτέλεση να πραγματοποιείται στην επιλεγμένη συσκευή. Με τον τρόπο αυτό η επιλογή του υπολογιστικού πόρου διατηρείται κοινή για τις διαφορετικές αρχιτεκτονικές.

Το notebook περιλαμβάνει επίσης ελέγχους μετά τη δημιουργία των μοντέλων. Με τη συνάρτηση print_model_summary εμφανίζονται πληροφορίες για το μοντέλο και τις εκπαιδεύσιμες παραμέτρους του, ενώ με τη validate_model_output πραγματοποιείται έλεγχος της εξόδου του μοντέλου πριν από την κανονική διαδικασία εκπαίδευσης. Οι έλεγχοι αυτοί λειτουργούν ως στάδιο επαλήθευσης της διαμόρφωσης, ώστε να εντοπιστεί πιθανή ασυμβατότητα πριν ξεκινήσει ένα πλήρες πείραμα.

Η αρχικοποίηση με προεκπαιδευμένα βάρη και η προσαρμογή της τελικής εξόδου αποτελούν κοινά στοιχεία και για τις έξι πειραματικές διαμορφώσεις. Αυτό που διαφοροποιείται στη συνέχεια είναι **ποιο μέρος των παραμέτρων κάθε δικτύου επιτρέπεται να μεταβληθεί κατά την εκπαίδευση**. Η διαφοροποίηση αυτή αποτελεί τη βάση των δύο στρατηγικών μεταφοράς μάθησης που εφαρμόζονται στο notebook, δηλαδή του Feature Extraction και του Partial Fine-Tuning, και αναλύεται στις επόμενες ενότητες.

4.4 ΠΡΟΣΑΡΜΟΓΗ ΤΩΝ ΜΟΝΤΕΛΩΝ ΣΤΟ ΠΡΟΒΛΗΜΑ ΤΑΞΙΝΟΜΗΣΗΣ

Μετά τη φόρτωση των προεκπαιδευμένων αρχιτεκτονικών, ήταν απαραίτητη η προσαρμογή τους στο συγκεκριμένο πρόβλημα της εργασίας. Το σύνολο δεδομένων περιλαμβάνει εννέα γεωγραφικές κλάσεις και επομένως κάθε μοντέλο πρέπει, για κάθε εικόνα εισόδου, να παράγει εννέα αντίστοιχες τιμές εξόδου. Οι τιμές αυτές αποτελούν την έξοδο του τελικού επιπέδου ταξινόμησης και χρησιμοποιούνται στη συνέχεια για την επιλογή της προβλεπόμενης κλάσης.

Ο αριθμός των κλάσεων προσδιορίζεται στο notebook από το ίδιο το training dataset. Μετά τη φόρτωσή του μέσω της ImageFolder, αποθηκεύονται τα ονόματα των κλάσεων στη μεταβλητή CLASS_NAMES, η αντιστοίχιση μεταξύ ονόματος και αριθμητικού δείκτη στη CLASS_TO_IDX και το συνολικό πλήθος των κλάσεων στη NUM_CLASSES. Συγκεκριμένα, η τελευταία υπολογίζεται ως το μήκος της λίστας CLASS_NAMES. Στη συνέχεια πραγματοποιείται έλεγχος ότι η τιμή της είναι ίση με την αναμενόμενη τιμή EXPECTED_NUM_CLASSES, η οποία στο notebook έχει οριστεί σε 9. Με αυτόν τον τρόπο η διαμόρφωση των μοντέλων συνδέεται άμεσα με τη δομή του dataset και παράλληλα ελέγχεται ότι έχει φορτωθεί ο αναμενόμενος αριθμός κατηγοριών.

Πριν από αυτό το στάδιο, το notebook ελέγχει επίσης ότι τα training, validation και test sets περιέχουν ακριβώς τις ίδιες κλάσεις και, ακόμη σημαντικότερα, ότι χρησιμοποιούν την ίδια αντιστοίχιση class_to_idx. Η αντιστοίχιση αυτή μετατρέπει κάθε λεκτική ονομασία κλάσης σε έναν αριθμητικό δείκτη που μπορεί να χρησιμοποιηθεί κατά την εκπαίδευση και την αξιολόγηση. Ο έλεγχος είναι απαραίτητος, επειδή διαφορετική αρίθμηση μιας κλάσης μεταξύ των τριών υποσυνόλων θα οδηγούσε σε ασυνεπή ερμηνεία των προβλέψεων.

Η πραγματική προσαρμογή των αρχιτεκτονικών πραγματοποιείται στο τελευταίο επίπεδο ταξινόμησης. Το τμήμα αυτό δέχεται τα χαρακτηριστικά που έχουν εξαχθεί από τα προηγούμενα επίπεδα του δικτύου και τα μετατρέπει στις τελικές τιμές εξόδου. Επειδή οι τρεις αρχιτεκτονικές έχουν διαφορετική εσωτερική οργάνωση, το επίπεδο που πρέπει να αντικατασταθεί δεν προσπελαύνεται με τον ίδιο τρόπο σε όλες.

Στη **ResNet18**, το τελικό πλήρως συνδεδεμένο επίπεδο βρίσκεται στο model.fc. Το notebook αρχικά διαβάζει μέσω της ιδιότητας in_features πόσα χαρακτηριστικά δέχεται το υπάρχον επίπεδο.

Η πληροφορία αυτή πρέπει να διατηρηθεί, ώστε το νέο επίπεδο να εξακολουθεί να συνδέεται σωστά με το προηγούμενο τμήμα του δικτύου. Στη συνέχεια το αρχικό fc αντικαθίσταται από ένα νέο γραμμικό επίπεδο nn.Linear, του οποίου ο αριθμός εξόδων ορίζεται σε NUM_CLASSES.

Στην **EfficientNet-Bo**, η ίδια λειτουργία πραγματοποιείται στο model.classifier[1]. Αντίστοιχα, στην **ConvNeXt-Tiny** το γραμμικό επίπεδο που χρησιμοποιείται για την τελική ταξινόμηση βρίσκεται στο model.classifier[2]. Και στις δύο περιπτώσεις το notebook διατηρεί τον υφιστάμενο αριθμό χαρακτηριστικών εισόδου και αντικαθιστά το τελικό επίπεδο με νέο nn.Linear που διαθέτει εννέα εξόδους. Επομένως, η διαφορετική σύνταξη του κώδικα οφείλεται στη διαφορετική δομή των τριών αρχιτεκτονικών, ενώ ο στόχος της επέμβασης παραμένει κοινός.

```
# ResNet18
classifier_input_features = model.fc.in_features
model.fc = nn.Linear(
    in_features=classifier_input_features,
    out_features=NUM_CLASSES
)

# EfficientNet-Bo
classifier_input_features = model.classifier[1].in_features
model.classifier[1] = nn.Linear(
    in_features=classifier_input_features,
    out_features=NUM_CLASSES
)

# ConvNeXt-Tiny
classifier_input_features = model.classifier[2].in_features
model.classifier[2] = nn.Linear(
    in_features=classifier_input_features,
    out_features=NUM_CLASSES
)
```

Κώδικας 4.3: *Προσαρμογή του τελικού επιπέδου ταξινόμησης των τριών αρχιτεκτονικών στις εννέα κλάσεις του προβλήματος.*

Στο παραπάνω απόσπασμα, η παράμετρος in_features εκφράζει το πλήθος των χαρακτηριστικών που φτάνουν στο τελικό επίπεδο από το προηγούμενο τμήμα του δικτύου. Η τιμή αυτή λαμβάνεται απευθείας από την εκάστοτε προεκπαιδευμένη αρχιτεκτονική και δεν ορίζεται αυθαίρετα. Αντίθετα, η παράμετρος out_features καθορίζει πόσες τιμές θα παράγει το νέο επίπεδο και ορίζεται ίση με NUM_CLASSES. Εφόσον NUM_CLASSES = 9, κάθε εικόνα οδηγεί τελικά σε εννέα τιμές εξόδου.

Η αντιστοίχιση αυτή είναι ιδιαίτερα σημαντική για τη συνοχή ολόκληρης της διαδικασίας. Η θέση κάθε τιμής στην έξοδο του μοντέλου συνδέεται με τον αντίστοιχο αριθμητικό δείκτη της κλάσης που έχει δημιουργηθεί από την ImageFolder. Έτσι, το ίδιο σύστημα αντιστοίχισης που χρησιμοποιείται κατά τη φόρτωση των πραγματικών ετικετών χρησιμοποιείται και κατά την ερμηνεία των προβλέψεων του μοντέλου.

Μετά την προσαρμογή του τελικού επιπέδου, το notebook πραγματοποιεί επιπλέον **προγραμματιστικό έλεγχο της εξόδου** μέσω της συνάρτησης validate_model_output. Η συνάρτηση χρησιμοποιείται μετά τη δημιουργία των μοντέλων, πριν από την κανονική διαδικασία εκπαίδευσης. Σκοπός της είναι να επιβεβαιωθεί ότι η διαμορφωμένη αρχιτεκτονική μπορεί να δεχθεί δεδομένα με τις αναμενόμενες διαστάσεις και ότι η έξοδός της είναι συμβατή με τον αριθμό των κλάσεων του προβλήματος. Με αυτόν τον τρόπο ένα πιθανό σφάλμα στη διαμόρφωση του classifier μπορεί να εντοπιστεί πριν ξεκινήσει η πολύ πιο χρονοβόρα διαδικασία εκπαίδευσης.

Παράλληλα, στα πειραματικά τμήματα χρησιμοποιείται η συνάρτηση print_model_summary, η οποία παρέχει σύνοψη της εκάστοτε διαμόρφωσης του μοντέλου.

Σε συνδυασμό με τους ελέγχους των εκπαιδεύσιμων παραμέτρων που πραγματοποιούνται στις αντίστοιχες πειραματικές ενότητες, το notebook δεν περιορίζεται απλώς στη δημιουργία ενός μοντέλου, αλλά επαληθεύει προγραμματιστικά βασικά χαρακτηριστικά της διαμόρφωσής του πριν από την εκπαίδευση.

Με την παραπάνω διαδικασία, οι ResNet18, EfficientNet-B0 και ConvNeXt-Tiny αποκτούν κοινή έξοδο εννέα κατηγοριών, παρά τις διαφορές στην εσωτερική τους αρχιτεκτονική. Η κοινή αυτή προσαρμογή αποτελεί απαραίτητη προϋπόθεση για τη συγκριτική μελέτη τους, καθώς εξασφαλίζει ότι και τα τρία μοντέλα καλούνται να επιλύσουν το ίδιο ακριβώς πρόβλημα ταξινόμησης. Η διαφοροποίηση μεταξύ των πειραμάτων αφορά στη συνέχεια το ποια τμήματα κάθε μοντέλου παραμένουν σταθερά και ποια επιτρέπεται να προσαρμοστούν κατά την εκπαίδευση, γεγονός που οδηγεί στις στρατηγικές **Feature Extraction** και **Partial Fine-Tuning** των επόμενων ενοτήτων.

4.5 ΣΤΡΑΤΙΓΙΚΗ FEATURE EXTRACTION

Η πρώτη στρατηγική μεταφοράς μάθησης που εφαρμόστηκε στα τρία μοντέλα ήταν το **Feature Extraction**. Στη συγκεκριμένη προσέγγιση αξιοποιούνται τα χαρακτηριστικά που έχουν ήδη μάθει τα προεκπαιδευμένα δίκτυα, χωρίς να μεταβάλλονται οι παράμετροι των επιπέδων που είναι υπεύθυνα για την εξαγωγή αυτών των χαρακτηριστικών. Η εκπαίδευση περιορίζεται στο νέο τελικό επίπεδο ταξινόμησης, το οποίο έχει προσαρμοστεί στις εννέα κλάσεις του προβλήματος.

Στην υλοποίηση του notebook, η διαδικασία ξεκινά με το πάγωμα όλων των παραμέτρων του προεκπαιδευμένου μοντέλου. Αυτό πραγματοποιείται με επανάληψη πάνω στις παραμέτρους του δικτύου και με ορισμό της ιδιότητας requires_grad σε False. Στο PyTorch, η ιδιότητα requires_grad καθορίζει αν για μια παράμετρο θα υπολογίζονται gradients κατά τη διαδικασία της οπισθοδιάδοσης. Όταν η τιμή της είναι False, η συγκεκριμένη παράμετρος δεν ενημερώνεται από τη διαδικασία βελτιστοποίησης και επομένως παραμένει σταθερή κατά την εκπαίδευση.

```
for parameter in model.parameters():
    parameter.requires_grad = False
```

Κώδικας 4.3: *Πάγωμα των προεκπαιδευμένων παραμέτρων κατά την εφαρμογή της στρατηγικής Feature Extraction.*

Η ίδια βασική εντολή εφαρμόζεται στις υλοποιήσεις Feature Extraction των ResNet18, EfficientNet-B0 και ConvNeXt-Tiny. Με τον τρόπο αυτό παγώνει αρχικά ολόκληρο το προεκπαιδευμένο δίκτυο. Στη συνέχεια αντικαθίσταται το υπάρχον τελικό επίπεδο ταξινόμησης από το νέο γραμμικό επίπεδο που αντιστοιχεί στις εννέα κλάσεις, όπως περιγράφηκε στην προηγούμενη ενότητα. Επειδή το νέο επίπεδο δημιουργείται μετά το πάγωμα των ήδη υπαρχουσών παραμέτρων, οι δικές του παράμετροι διατηρούν την προεπιλεγμένη κατάσταση requires_grad=True και είναι αυτές που μπορούν να μεταβληθούν κατά την εκπαίδευση.

Στη **ResNet18**, το εκπαιδεύσιμο τμήμα είναι το νέο τελικό επίπεδο fc. Οι υπόλοιπες παράμετροι του δικτύου παραμένουν παγωμένες και χρησιμοποιούνται για την εξαγωγή των οπτικών χαρακτηριστικών από τις εικόνες. Το notebook περιλαμβάνει επιπλέον προγραμματιστικό έλεγχο της συγκεκριμένης διαμόρφωσης. Δημιουργείται λίστα με τα ονόματα όλων των παραμέτρων για τις οποίες η ιδιότητα requires_grad είναι True και η λίστα αυτή συγκρίνεται με τις δύο αναμενόμενες εκπαιδεύσιμες παραμέτρους του νέου τελικού επιπέδου, fc.weight και fc.bias.

```
trainable_parameter_names = [
    parameter_name
    for parameter_name, parameter in model.named_parameters()
    if parameter.requires_grad
]

expected_trainable_parameter_names = [
    "fc.weight",
    "fc.bias",
]

if trainable_parameter_names != expected_trainable_parameter_names:
    raise ValueError(
        "Στο Feature Extraction πρέπει να είναι εκπαιδεύσιμες "
        "μόνο οι παράμετροι fc.weight και fc.bias."
    )
```

Κώδικας 4.4: *Έλεγχος των εκπαιδεύσιμων παραμέτρων της ResNet18 κατά την εφαρμογή Feature Extraction.*

Η παράμετρος fc.weight περιλαμβάνει τα βάρη του νέου γραμμικού επιπέδου, ενώ η fc.bias περιλαμβάνει τις αντίστοιχες τιμές πόλωσης. Ο παραπάνω έλεγχος επιβεβαιώνει ότι δεν έχει παραμείνει κατά λάθος εκπαιδεύσιμη κάποια παράμετρος από το προεκπαιδευμένο τμήμα της ResNet18. Σε περίπτωση που η πραγματική λίστα των εκπαιδεύσιμων παραμέτρων διαφέρει από την αναμενόμενη, η εκτέλεση διακόπτεται μέσω εξαίρεσης ValueError, ώστε να μη συνεχιστεί το πείραμα με διαφορετική διαμόρφωση από την προβλεπόμενη.

Στην **EfficientNet-B0** εφαρμόζεται η ίδια στρατηγική ως προς το πάγωμα του προεκπαιδευμένου δικτύου. Αρχικά όλες οι παράμετροι ορίζονται ως μη εκπαιδεύσιμες και στη συνέχεια το model.classifier[1] αντικαθίσταται από νέο γραμμικό επίπεδο με εννέα εξόδους. Κατά συνέπεια, το προεκπαιδευμένο τμήμα της EfficientNet-B0 παραμένει σταθερό και η προσαρμογή στο νέο πρόβλημα πραγματοποιείται μέσω του νέου τελικού επιπέδου ταξινόμησης.

Στο αντίστοιχο τμήμα του notebook πραγματοποιείται επίσης καταγραφή των εκπαιδεύσιμων παραμέτρων της EfficientNet-B0 μέσω των model.named_parameters(). Εμφανίζονται οι παράμετροι για τις οποίες το requires_grad είναι True, καθώς και το πλήθος των αντίστοιχων tensors. Σε αντίθεση με τον έλεγχο που εφαρμόζεται στη ResNet18, στο συγκεκριμένο σημείο της υλοποίησης της EfficientNet-B0 δεν πραγματοποιείται σύγκριση της λίστας με προκαθορισμένη αναμενόμενη λίστα ονομάτων.

Στην **ConvNeXt-Tiny** ακολουθείται επίσης η ίδια βασική διαδικασία. Όλες οι αρχικές παράμετροι του προεκπαιδευμένου δικτύου παγώνουν και στη συνέχεια αντικαθίσταται το model.classifier[2] από νέο γραμμικό επίπεδο κατάλληλο για τις εννέα κλάσεις. Το notebook πραγματοποιεί ρητό έλεγχο των εκπαιδεύσιμων παραμέτρων και επιβεβαιώνει ότι αυτές περιορίζονται στις classifier.2.weight και classifier.2.bias.

Επομένως, όπως και στη ResNet18, ελέγχεται προγραμματιστικά ότι δεν έχει παραμείνει εκπαιδεύσιμο κάποιο επιπλέον τμήμα του προεκπαιδευμένου δικτύου.

Μετά τη διαμόρφωση των μοντέλων χρησιμοποιείται η συνάρτηση print_model_summary, μέσω της οποίας παρουσιάζονται πληροφορίες σχετικά με τις συνολικές, τις εκπαιδεύσιμες και τις παγωμένες παραμέτρους. Επιπλέον, η συνάρτηση validate_model_output χρησιμοποιείται για την πραγματοποίηση δοκιμαστικής προώθησης δεδομένων μέσα από το μοντέλο και για τον έλεγχο της παραγόμενης εξόδου πριν από την έναρξη της κανονικής εκπαίδευσης. Οι έλεγχοι αυτοί επιτρέπουν την επιβεβαίωση ότι η διαμόρφωση του μοντέλου είναι συμβατή με το πρόβλημα ταξινόμησης πριν ξεκινήσει το αντίστοιχο πείραμα.

Συνολικά, στις τρεις διαμορφώσεις Feature Extraction οι ResNet18, EfficientNet-B0 και ConvNeXt-Tiny αξιοποιούνται ως προεκπαιδευμένοι και σταθεροί εξαγωγείς χαρακτηριστικών, ενώ η προσαρμογή στις εννέα γεωγραφικές κλάσεις πραγματοποιείται μέσω της εκπαίδευσης του νέου τελικού επιπέδου ταξινόμησης. Η κοινή αυτή μεθοδολογία δημιουργεί μία αντίστοιχη πειραματική διαμόρφωση για καθεμία από τις τρεις αρχιτεκτονικές και επιτρέπει στη συνέχεια τη σύγκρισή τους με τις διαμορφώσεις **Partial Fine-Tuning**, στις οποίες επιτρέπεται η προσαρμογή και επιλεγμένων τμημάτων του προεκπαιδευμένου δικτύου.

4.6 ΣΤΡΑΤΙΓΙΚΗ PARTIAL FINE-TUNING

Η δεύτερη στρατηγική μεταφοράς μάθησης που εφαρμόστηκε στα τρία μοντέλα ήταν το **Partial Fine-Tuning**. Σε αντίθεση με το Feature Extraction, όπου το προεκπαιδευμένο τμήμα του δικτύου παραμένει πλήρως παγωμένο και εκπαιδεύεται μόνο ο νέος ταξινομητής, στο Partial Fine-Tuning επιτρέπεται να ενημερωθούν και επιλεγμένα τμήματα της προεκπαιδευμένης αρχιτεκτονικής. Με αυτόν τον τρόπο διατηρείται μεγάλο μέρος της γνώσης που έχει αποκτηθεί κατά την προεκπαίδευση, ενώ παράλληλα δίνεται στο μοντέλο η δυνατότητα να προσαρμόσει μέρος των εσωτερικών αναπαραστάσεών του στο νέο πρόβλημα ταξινόμησης.

Η επιλογή του Partial Fine-Tuning συνδέεται με την ιεραρχική φύση των χαρακτηριστικών που μαθαίνουν τα Συνελικτικά Νευρωνικά Δίκτυα. Τα πρώτα επίπεδα επεξεργάζονται την εικόνα σε σχετικά χαμηλό επίπεδο και μαθαίνουν γενικά οπτικά χαρακτηριστικά, όπως ακμές, κατευθύνσεις, χρωματικές μεταβολές και απλές υφές. Τέτοιου είδους χαρακτηριστικά είναι χρήσιμα σε ένα μεγάλο εύρος προβλημάτων Υπολογιστικής Όρασης και δεν εξαρτώνται έντονα από τη συγκεκριμένη κατηγορία αντικειμένων που πρόκειται να αναγνωριστεί.

Καθώς η πληροφορία προχωρά προς τα βαθύτερα επίπεδα του δικτύου, τα απλούστερα χαρακτηριστικά συνδυάζονται σε περισσότερο σύνθετες αναπαραστάσεις. Τα βαθύτερα επίπεδα βρίσκονται πιο κοντά στην τελική απόφαση ταξινόμησης και οι αναπαραστάσεις τους είναι περισσότερο εξειδικευμένες ως προς το πρόβλημα για το οποίο είχε αρχικά εκπαιδευτεί το μοντέλο. Στην παρούσα εργασία, όμως, το νέο πρόβλημα αφορά τη διάκριση εικόνων ελληνικών παραδοσιακών φορεσιών σε εννέα γεωγραφικές κλάσεις. Για τον λόγο αυτό επιλέχθηκε να επιτραπεί η προσαρμογή βαθύτερων τμημάτων των δικτύων, ενώ τα πρώτα επίπεδα διατηρήθηκαν παγωμένα.

Η επιλογή των τελευταίων και όχι των πρώτων τμημάτων έχει επομένως συγκεκριμένη λειτουργική σημασία. Ένα πρώιμο επίπεδο μπορεί να παραμένει χρήσιμο επειδή ανιχνεύει, για παράδειγμα, μια ακμή ή μια βασική υφή ανεξάρτητα από το περιεχόμενο της εικόνας. Αντίθετα, ένα βαθύτερο επίπεδο συνδυάζει τέτοιες απλές πληροφορίες σε πιο σύνθετα οπτικά πρότυπα. Στην περίπτωση των παραδοσιακών φορεσιών, οι υψηλότερου επιπέδου αναπαραστάσεις μπορούν να σχετίζονται με συνδυασμούς σχημάτων, υφών, χρωμάτων, μοτίβων και διακοσμητικών στοιχείων που συμβάλλουν στη διάκριση μεταξύ των γεωγραφικών κατηγοριών. Η δυνατότητα μεταβολής των βαθύτερων παραμέτρων επιτρέπει επομένως στο μοντέλο να προσαρμόσει μέρος των πιο εξειδικευμένων χαρακτηριστικών του στο νέο σύνολο δεδομένων.

Η συγκεκριμένη επιλογή σχετίζεται επίσης με το περιορισμένο μέγεθος του διαθέσιμου dataset. Το σύνολο δεδομένων της εργασίας περιλαμβάνει συνολικά 411 εικόνες και η πλήρης επανεκπαίδευση όλων των παραμέτρων των προεκπαιδευμένων δικτύων θα απαιτούσε την προσαρμογή πολύ μεγαλύτερου αριθμού παραμέτρων στα συγκεκριμένα δεδομένα. Με το πάγωμα των πρώτων τμημάτων διατηρείται μεγάλο μέρος της ήδη αποκτημένης γνώσης, ενώ η εκπαίδευση περιορίζεται σε ένα μικρότερο τμήμα του μοντέλου.

Το Partial Fine-Tuning αποτελεί έτσι μια ενδιάμεση στρατηγική μεταξύ του Feature Extraction, όπου παραμένει σταθερός ολόκληρος ο προεκπαιδευμένος κορμός, και του πλήρους Fine-Tuning, όπου θα ήταν εκπαιδεύσιμο το σύνολο του δικτύου.

Η υλοποίηση ακολουθεί αρχικά κοινή λογική και στις τρεις αρχιτεκτονικές. Μετά τη φόρτωση των προεκπαιδευμένων βαρών, όλες οι υπάρχουσες παράμετροι του μοντέλου παγώνουν μέσω της ιδιότητας requires_grad=False. Στη συνέχεια συγκεκριμένα βαθύτερα τμήματα επαναφέρονται σε εκπαιδεύσιμη κατάσταση μέσω requires_grad=True. Το ακριβές τμήμα που ξεπαγώνει διαφέρει μεταξύ των ResNet18, EfficientNet-B0 και ConvNeXt-Tiny, καθώς οι τρεις αρχιτεκτονικές έχουν διαφορετική εσωτερική οργάνωση.

Στη **ResNet18**, το τμήμα που ξεπαγώνει είναι το layer4. Η αρχιτεκτονική οργανώνει ένα σημαντικό μέρος του συνελικτικού της κορμού σε διαδοχικά στάδια, τα οποία στη συγκεκριμένη υλοποίηση εμφανίζονται ως layer1, layer2, layer3 και layer4. Το layer4 αποτελεί το τελευταίο από αυτά τα κύρια residual στάδια πριν από το τελικό τμήμα ταξινόμησης. Επομένως, η επιλογή του επιτρέπει την προσαρμογή του βαθύτερου κύριου σταδίου του δικτύου, ενώ τα προηγούμενα layer1, layer2 και layer3 εξακολουθούν να αξιοποιούν τα προεκπαιδευμένα βάρη τους χωρίς να μεταβάλλονται.

```
for parameter in model.parameters():
    parameter.requires_grad = False


for parameter in model.layer4.parameters():
    parameter.requires_grad = True
```

Κώδικας 4.5: *Πάγωμα του μεγαλύτερου μέρους της ResNet18 και ενεργοποίηση της εκπαίδευσης του layer4 κατά το Partial Fine-Tuning.*

Ο Κώδικας 4.5 αποτυπώνει τη λογική της μερικής προσαρμογής. Η πρώτη επανάληψη θέτει όλες τις υπάρχουσες παραμέτρους της ResNet18 σε μη εκπαιδεύσιμη κατάσταση. Η δεύτερη επανάληψη εφαρμόζεται αποκλειστικά στις παραμέτρους του layer4 και επαναφέρει την ιδιότητα requires_grad σε True. Οι παράμετροι του συγκεκριμένου σταδίου συμμετέχουν επομένως στον υπολογισμό των gradients και μπορούν να ενημερωθούν κατά την εκπαίδευση, σε αντίθεση με τις παραμέτρους των προηγούμενων σταδίων.

Παράλληλα, το αρχικό fc αντικαθίσταται από το νέο γραμμικό επίπεδο των εννέα εξόδων που παρουσιάστηκε στην Ενότητα 4.4. Επειδή το επίπεδο αυτό δημιουργείται εκ νέου, οι παράμετροί του είναι επίσης εκπαιδεύσιμες. Συνεπώς, στη διαμόρφωση Partial Fine-Tuning της ResNet18 προσαρμόζονται τόσο το layer4 όσο και το νέο fc, ενώ το προηγούμενο τμήμα του προεκπαιδευμένου δικτύου παραμένει παγωμένο.

Στην **EfficientNet-B0**, η αντίστοιχη επιλογή πραγματοποιείται στο model.features, δηλαδή στο κύριο τμήμα της αρχιτεκτονικής που είναι υπεύθυνο για την εξαγωγή χαρακτηριστικών. Στο notebook ξεπαγώνουν συγκεκριμένα τα δύο τελευταία στοιχεία του features μέσω της έκφρασης model.features[-2:]. Στην Python η σύνταξη [-2:] επιλέγει τα δύο τελευταία στοιχεία μιας ακολουθίας. Κατά συνέπεια, τα προηγούμενα στοιχεία του features παραμένουν παγωμένα, ενώ τα δύο τελευταία μπορούν να μεταβάλουν τις παραμέτρους τους κατά την εκπαίδευση.

```
for parameter in model.parameters():
    parameter.requires_grad = False
for parameter in model.features[-2:].parameters():
    parameter.requires_grad = True
for parameter in model.classifier.parameters():
    parameter.requires_grad = True
```

Κώδικας 4.6: *Ενεργοποίηση των δύο τελευταίων τμημάτων features και του classifier της EfficientNet-B0 κατά το Partial Fine-Tuning.*

Στην περίπτωση της EfficientNet-B0, το notebook ενεργοποιεί ρητά και όλες τις παραμέτρους του model.classifier. Ο classifier αποτελεί το τελικό τμήμα που μετατρέπει τα χαρακτηριστικά του δικτύου στις προβλέψεις των κλάσεων. Επομένως, στη συγκεκριμένη πειραματική διαμόρφωση εκπαιδεύονται τα δύο τελευταία στοιχεία του features και ο classifier, ενώ το υπόλοιπο προεκπαιδευμένο δίκτυο διατηρείται σταθερό.

Η επιλογή των τελευταίων στοιχείων του features ακολουθεί την ίδια γενική λογική με την επιλογή του layer4 στη ResNet18: επιτρέπεται η προσαρμογή τμημάτων που βρίσκονται προς το τέλος της διαδικασίας εξαγωγής χαρακτηριστικών και επομένως παράγουν περισσότερο σύνθετες αναπαραστάσεις, ενώ διατηρούνται σταθερά τα πρώιμα τμήματα που παρέχουν γενικότερα οπτικά χαρακτηριστικά. Στη συγκεκριμένη υλοποίηση επιλέχθηκαν τα δύο τελευταία στοιχεία του features. Δεν πραγματοποιήθηκε ξεχωριστή συγκριτική διερεύνηση διαφορετικού αριθμού ξεπαγωμένων τμημάτων και, επομένως, η επιλογή αυτή αποτελεί τη διαμόρφωση που χρησιμοποιήθηκε στο πείραμα και όχι αποτέλεσμα αναζήτησης ενός αποδεδειγμένα βέλτιστου βάθους Fine-Tuning.

Στην **ConvNeXt-Tiny**, η ίδια μεθοδολογική αρχή εφαρμόζεται σύμφωνα με τη διαφορετική δομή της αρχιτεκτονικής. Αφού παγώσουν όλες οι παράμετροι, το notebook ενεργοποιεί την εκπαίδευση του model.features[-1]. Η έκφραση [-1] αναφέρεται στο τελευταίο στοιχείο του features. Με αυτόν τον τρόπο το τελευταίο τμήμα του μηχανισμού εξαγωγής χαρακτηριστικών μπορεί να προσαρμοστεί στα δεδομένα των παραδοσιακών φορεσιών, ενώ τα προηγούμενα τμήματα εξακολουθούν να χρησιμοποιούν σταθερά τα προεκπαιδευμένα βάρη τους.

```
for parameter in model.parameters():
    parameter.requires_grad = False


for parameter in model.features[-1].parameters():
    parameter.requires_grad = True


for parameter in model.classifier.parameters():
    parameter.requires_grad = True
```

Κώδικας 4.7: *Ενεργοποίηση του τελευταίου τμήματος features και του classifier της ConvNeXt-Tiny κατά το Partial Fine-Tuning.*

Όπως και στην EfficientNet-B0, το notebook ενεργοποιεί ρητά την εκπαίδευση ολόκληρου του model.classifier. Στην ConvNeXt-Tiny το γραμμικό επίπεδο που αντικαταστάθηκε για την παραγωγή των εννέα εξόδων βρίσκεται στη θέση classifier[2], ωστόσο κατά το Partial Fine-Tuning η εντολή εφαρμόζεται σε ολόκληρο το model.classifier. Επομένως, η συγκεκριμένη πειραματική διαμόρφωση περιλαμβάνει ως εκπαιδεύσιμα τόσο το features[-1] όσο και ολόκληρο το τμήμα classifier.

Η επιλογή του τελευταίου στοιχείου features της ConvNeXt-Tiny ακολουθεί και εδώ την αρχή της προσαρμογής των βαθύτερων αναπαραστάσεων αντί των πρώιμων γενικών χαρακτηριστικών. Ωστόσο, όπως και στην EfficientNet-B0, το notebook δεν περιλαμβάνει πειράματα στα οποία συγκρίνονται διαφορετικά σημεία ξεπαγώματος της ίδιας αρχιτεκτονικής. Συνεπώς, δεν μπορεί να υποστηριχθεί ότι το features[-1] αποτελεί το βέλτιστο δυνατό σημείο Fine-Tuning. Αποτελεί τη συγκεκριμένη επιλογή με την οποία υλοποιήθηκε και αξιολογήθηκε η στρατηγική Partial Fine-Tuning στην παρούσα εργασία.

Οι επιλογές layer4, features[-2:] και features[-1] δεν αντιστοιχούν σε απόλυτα ισοδύναμα δομικά τμήματα. Οι τρεις αρχιτεκτονικές οργανώνουν διαφορετικά τον κορμό εξαγωγής χαρακτηριστικών και για τον λόγο αυτό η ίδια γενική στρατηγική εκφράζεται διαφορετικά στον κώδικα. Στη ResNet18 η επιλογή πραγματοποιείται σε επίπεδο residual stage μέσω του layer4, ενώ στις EfficientNet-B0 και ConvNeXt-Tiny πραγματοποιείται μέσω των τελευταίων στοιχείων της δομής features. Κοινός στόχος είναι η διατήρηση του μεγαλύτερου μέρους του προεκπαιδευμένου δικτύου σε παγωμένη κατάσταση και η προσαρμογή μόνο ενός βαθύτερου τμήματος μαζί με το τμήμα ταξινόμησης.

Η συγκεκριμένη διαφορά είναι σημαντική και για την ερμηνεία των μεταγενέστερων πειραματικών αποτελεσμάτων. Το Partial Fine-Tuning δεν μεταβάλλει απλώς τον αριθμό των εκπαιδεύσιμων παραμέτρων σε σχέση με το Feature Extraction, αλλά επιτρέπει σε μέρος του ίδιου του μηχανισμού εξαγωγής χαρακτηριστικών να προσαρμοστεί στο νέο πρόβλημα. Επομένως, η σύγκριση των δύο στρατηγικών μπορεί να δείξει αν η διατήρηση των προεκπαιδευμένων χαρακτηριστικών χωρίς μεταβολή είναι επαρκής ή αν η μερική προσαρμογή των βαθύτερων αναπαραστάσεων συνδέεται με καλύτερη απόδοση στο συγκεκριμένο σύνολο δεδομένων. Η πραγματική επίδραση της επιλογής αυτής εξετάζεται μέσα από τα αποτελέσματα των πειραμάτων και δεν θεωρείται εκ των προτέρων δεδομένη.

Μετά τον καθορισμό των εκπαιδεύσιμων τμημάτων, τα μοντέλα μεταφέρονται στην επιλεγμένη υπολογιστική συσκευή και πραγματοποιούνται οι αντίστοιχοι προγραμματιστικοί έλεγχοι της διαμόρφωσής τους. Η print_model_summary χρησιμοποιείται για την παρουσίαση πληροφοριών σχετικά με τις συνολικές, εκπαιδεύσιμες και παγωμένες παραμέτρους, ενώ η validate_model_output ελέγχει ότι το προσαρμοσμένο μοντέλο μπορεί να πραγματοποιήσει επιτυχή forward pass και να παράγει έξοδο συμβατή με τις εννέα κλάσεις του προβλήματος. Με τον τρόπο αυτό ελέγχεται η διαμόρφωση πριν από την έναρξη της κανονικής διαδικασίας εκπαίδευσης.

Συνολικά, το Partial Fine-Tuning υλοποιήθηκε με διαφορετικό βάθος προσαρμογής ανάλογα με τη δομή κάθε αρχιτεκτονικής: στη ResNet18 ξεπαγώνει το layer4 μαζί με το νέο τελικό επίπεδο fc, στην EfficientNet-B0 τα δύο τελευταία στοιχεία του features μαζί με τον classifier και στην ConvNeXt-Tiny το τελευταίο στοιχείο του features μαζί με τον classifier. Οι επιλογές αυτές επιτρέπουν τη διατήρηση των γενικότερων προεκπαιδευμένων χαρακτηριστικών στα πρώτα τμήματα των δικτύων και ταυτόχρονα την προσαρμογή βαθύτερων αναπαραστάσεων στο νέο πρόβλημα. Με τις τρεις αυτές διαμορφώσεις και τις τρεις αντίστοιχες διαμορφώσεις Feature Extraction προκύπτουν συνολικά οι έξι πειραματικές διαμορφώσεις που χρησιμοποιούνται στην εργασία.

4.7 ΟΙ ΕΞΙ ΠΕΙΡΑΜΑΤΙΚΕΣ ΔΙΑΜΟΡΦΩΣΕΙΣ

Με βάση τις τρεις αρχιτεκτονικές που επιλέχθηκαν και τις δύο στρατηγικές μεταφοράς μάθησης που παρουσιάστηκαν στις προηγούμενες ενότητες, διαμορφώθηκαν συνολικά **έξι διαφορετικά πειράματα**. Για καθεμία από τις ResNet18, EfficientNet-B0 και ConvNeXt-Tiny δημιουργήθηκε μία διαμόρφωση Feature Extraction και μία διαμόρφωση Partial Fine-Tuning. Με αυτόν τον σχεδιασμό είναι δυνατό να εξεταστούν δύο διαφορετικοί παράγοντες: η επιλογή της αρχιτεκτονικής και ο βαθμός στον οποίο επιτρέπεται η προσαρμογή των προεκπαιδευμένων παραμέτρων στο νέο πρόβλημα ταξινόμησης.

Οι έξι πειραματικές διαμορφώσεις που υλοποιήθηκαν είναι οι ακόλουθες:

I. **ResNet18 – Feature Extraction:** ο προεκπαιδευμένος κορμός της ResNet18 παραμένει παγωμένος και εκπαιδεύεται το νέο τελικό επίπεδο fc, το οποίο παράγει τις εννέα εξόδους του προβλήματος.

II. **ResNet18 – Partial Fine-Tuning:** τα προηγούμενα τμήματα του προεκπαιδευμένου κορμού παραμένουν παγωμένα, ενώ εκπαιδεύονται το layer4 και το νέο τελικό επίπεδο fc.

III. **EfficientNet-Bo – Feature Extraction:** οι προεκπαιδευμένες παράμετροι της αρχιτεκτονικής παγώνουν και η εκπαίδευση περιορίζεται στο νέο τμήμα ταξινόμησης που έχει προσαρμοστεί στις εννέα κλάσεις.

IV. **EfficientNet-Bo – Partial Fine-Tuning:** παραμένει παγωμένο το μεγαλύτερο μέρος του προεκπαιδευμένου δικτύου, ενώ εκπαιδεύονται τα δύο τελευταία στοιχεία του features και ο classifier.

V. **ConvNeXt-Tiny – Feature Extraction:** ο προεκπαιδευμένος κορμός παραμένει παγωμένος και εκπαιδεύεται το νέο τμήμα ταξινόμησης για τις εννέα γεωγραφικές κλάσεις.

VI. **ConvNeXt-Tiny – Partial Fine-Tuning:** παραμένουν παγωμένα τα προηγούμενα τμήματα του δικτύου, ενώ εκπαιδεύονται το τελευταίο στοιχείο του features και ο classifier.
↳ Ο παραπάνω πειραματικός σχεδιασμός μπορεί να θεωρηθεί ως συνδυασμός **τριών αρχιτεκτονικών** × **δύο στρατηγικών μεταφοράς μάθησης**. Η ύπαρξη δύο διαφορετικών διαμορφώσεων για κάθε αρχιτεκτονική είναι σημαντική, επειδή επιτρέπει την άμεση σύγκριση του Feature Extraction με το Partial Fine-Tuning χωρίς να αλλάζει ταυτόχρονα η βασική αρχιτεκτονική του μοντέλου. Για παράδειγμα, η σύγκριση της ResNet18 – Feature Extraction με τη ResNet18 – Partial Fine-Tuning επιτρέπει να εξεταστεί η επίδραση που έχει η εκπαίδευση του layer4 σε σχέση με την εκπαίδευση μόνο του νέου ταξινομητή.

Αντίστοιχα, η σύγκριση μεταξύ διαφορετικών αρχιτεκτονικών που χρησιμοποιούν την ίδια στρατηγική επιτρέπει να εξεταστεί η συμπεριφορά διαφορετικών σχεδιασμών Συνελικτικών Νευρωνικών Δικτύων στο ίδιο πρόβλημα. Για παράδειγμα, οι τρεις διαμορφώσεις Feature Extraction χρησιμοποιούν την ίδια γενική φιλοσοφία μεταφοράς μάθησης, αλλά βασίζονται σε διαφορετικές αρχιτεκτονικές. Το ίδιο ισχύει για τις τρεις διαμορφώσεις Partial Fine-Tuning. Με αυτόν τον τρόπο η πειραματική διαδικασία δεν περιορίζεται στην αξιολόγηση ενός μόνο μοντέλου, αλλά επιτρέπει συστηματικότερη σύγκριση τόσο ως προς την αρχιτεκτονική όσο και ως προς τη στρατηγική προσαρμογής.

Για να είναι η σύγκριση των έξι διαμορφώσεων ουσιαστική, το πρόβλημα ταξινόμησης παραμένει κοινό σε όλες τις περιπτώσεις. Όλα τα μοντέλα προσαρμόζονται ώστε να παράγουν εννέα εξόδους, οι οποίες αντιστοιχούν στις ίδιες εννέα γεωγραφικές κλάσεις, ενώ χρησιμοποιούνται τα ίδια προκαθορισμένα υποσύνολα training, validation και test. Επομένως, οι εικόνες του test set που χρησιμοποιούνται για την τελική αξιολόγηση δεν αλλάζουν ανάλογα με το μοντέλο ή τη στρατηγική μεταφοράς μάθησης. Αυτό είναι απαραίτητο ώστε οι μεταγενέστερες διαφορές στις μετρικές αξιολόγησης να μπορούν να συγκριθούν πάνω σε κοινή βάση.

Παράλληλα, οι έξι διαμορφώσεις δεν έχουν τον ίδιο αριθμό εκπαιδεύσιμων παραμέτρων. Στο Feature Extraction η εκπαίδευση περιορίζεται ουσιαστικά στο νέο τμήμα ταξινόμησης, με αποτέλεσμα το μεγαλύτερο μέρος κάθε δικτύου να λειτουργεί ως σταθερός εξαγωγέας χαρακτηριστικών. Στο Partial Fine-Tuning, αντίθετα, προστίθενται στις εκπαιδεύσιμες παραμέτρους και επιλεγμένα βαθύτερα τμήματα του αντίστοιχου μοντέλου. Η διαφορά αυτή είναι θεμελιώδης για τον πειραματικό σχεδιασμό, καθώς μεταβάλλει το πόσο έντονα μπορεί κάθε προεκπαιδευμένο δίκτυο να προσαρμοστεί στο συγκεκριμένο dataset.

Η μεγαλύτερη δυνατότητα προσαρμογής του Partial Fine-Tuning δεν συνεπάγεται εκ των προτέρων και καλύτερη απόδοση. Η εκπαίδευση περισσότερων παραμέτρων μπορεί να επιτρέψει στο μοντέλο να δημιουργήσει αναπαραστάσεις περισσότερο κατάλληλες για τις ελληνικές παραδοσιακές φορεσιές, αλλά η πραγματική επίδραση της επιλογής αυτής πρέπει να προσδιοριστεί πειραματικά.

Για τον λόγο αυτό οι δύο στρατηγικές διατηρούνται ως ξεχωριστές διαμορφώσεις και αξιολογούνται με κοινές μετρικές, αντί να θεωρείται εξαρχής ότι μία από τις δύο είναι ανώτερη.

Αντίστοιχα, ούτε η επιλογή μιας νεότερης ή περισσότερο σύνθετης αρχιτεκτονικής θεωρείται από μόνη της εγγύηση καλύτερης επίδοσης στο συγκεκριμένο πρόβλημα. Οι ResNet18, EfficientNet-B0 και ConvNeXt-Tiny διαφέρουν ως προς τον σχεδιασμό και τον τρόπο με τον οποίο πραγματοποιούν την εξαγωγή χαρακτηριστικών, όπως αναλύθηκε στο Κεφάλαιο 2. Η τελική τους καταλληλότητα για το συγκεκριμένο dataset προσδιορίζεται από τα πραγματικά αποτελέσματα της πειραματικής διαδικασίας και όχι αποκλειστικά από τα γενικά χαρακτηριστικά της κάθε αρχιτεκτονικής.

Η οργάνωση των πειραμάτων αποτυπώνεται και στο notebook, όπου οι έξι διαμορφώσεις αντιστοιχίζονται σε διαφορετικά ονόματα πειραμάτων. Η συγκεκριμένη αντιστοίχιση χρησιμοποιείται για τη συγκέντρωση και τη μεταγενέστερη σύγκριση των αποτελεσμάτων.

```
experiments = {
    "ResNet18\nFeature Extraction": "resnet18_feature_extraction",
    "ResNet18\nPartial Fine-Tuning": "resnet18_partial_fine_tuning",
    "EfficientNet-B0\nFeature Extraction": "efficientnet_b0_feature_extraction",
    "EfficientNet-B0\nPartial Fine-Tuning": "efficientnet_b0_partial_fine_tuning",
    "ConvNeXt-Tiny\nFeature Extraction": "convnext_tiny_feature_extraction",
    "ConvNeXt-Tiny\nPartial Fine-Tuning": "convnext_tiny_partial_fine_tuning",
}
```

Κώδικας 4.8: *Αντιστοίχιση των έξι πειραματικών διαμορφώσεων που χρησιμοποιούνται για τη συγκέντρωση και σύγκριση των αποτελεσμάτων.*

Η συγκεκριμένη δομή κάνει σαφή τη συμμετρική οργάνωση της πειραματικής διαδικασίας: κάθε μία από τις τρεις αρχιτεκτονικές εμφανίζεται ακριβώς δύο φορές, μία για Feature Extraction και μία για Partial Fine-Tuning. Τα επιμέρους αποτελέσματα μπορούν έτσι να οργανωθούν με κοινή ονοματολογία και στη συνέχεια να συγκριθούν μεταξύ τους.

Ο καθορισμός των έξι διαμορφώσεων ολοκληρώνει τον βασικό σχεδιασμό των μοντέλων πριν από την εκπαίδευσή τους. Οι παράμετροι που αφορούν την ίδια τη διαδικασία εκπαίδευσης, όπως ο optimizer, ο ρυθμός μάθησης, ο αριθμός των epochs, το early stopping και η αποθήκευση checkpoints, παρουσιάζονται στο επόμενο κεφάλαιο. Πριν από την εκπαίδευση, ωστόσο, είναι απαραίτητο να επιβεβαιωθεί προγραμματιστικά ότι κάθε μοντέλο έχει προσαρμοστεί σωστά και ότι οι εκπαιδεύσιμες και παγωμένες παράμετροι συμφωνούν με τη στρατηγική που έχει οριστεί. Οι σχετικοί έλεγχοι παρουσιάζονται στην επόμενη ενότητα.

4.8 ΈΛΕΓΧΟΣ ΤΗΣ ΟΡΘΗΣ ΠΡΟΣΑΡΜΟΓΗΣ ΤΩΝ ΜΟΝΤΕΛΩΝ

Πριν από την έναρξη της εκπαίδευσης, πραγματοποιήθηκαν προγραμματιστικοί έλεγχοι με σκοπό να επιβεβαιωθεί ότι κάθε μοντέλο είχε διαμορφωθεί σύμφωνα με την αντίστοιχη πειραματική στρατηγική και ότι η έξοδός του ήταν συμβατή με το πρόβλημα ταξινόμησης των εννέα γεωγραφικών κλάσεων. Οι έλεγχοι αυτοί είναι σημαντικοί, επειδή η επιτυχής δημιουργία ενός αντικειμένου μοντέλου στη PyTorch δεν αρκεί από μόνη της για να αποδείξει ότι έχουν παγώσει τα σωστά επίπεδα, ότι παραμένουν εκπαιδεύσιμες οι επιθυμητές παράμετροι ή ότι το τελικό επίπεδο παράγει τον απαιτούμενο αριθμό εξόδων.

Ένας από τους βασικούς ελέγχους αφορά τις **εκπαιδεύσιμες και παγωμένες παραμέτρους**. Όπως αναλύθηκε στις προηγούμενες ενότητες, η ιδιότητα requires_grad καθορίζει αν για μια παράμετρο θα υπολογίζονται gradients και, επομένως, αν αυτή μπορεί να ενημερώνεται κατά την εκπαίδευση. Η απλή εκτέλεση των εντολών παγώματος και ξεπαγώματος δεν θεωρήθηκε επαρκής· το notebook περιλαμβάνει επιπλέον έλεγχο και παρουσίαση της τελικής κατάστασης των παραμέτρων, ώστε να επιβεβαιώνεται ότι η πραγματική διαμόρφωση του μοντέλου συμφωνεί με τη στρατηγική που έχει οριστεί.

Για τον σκοπό αυτό χρησιμοποιείται η συνάρτηση print_model_summary. Η συνάρτηση υπολογίζει τον συνολικό αριθμό παραμέτρων του μοντέλου, τον αριθμό των παραμέτρων για τις οποίες requires_grad=True και, από τη διαφορά τους, τον αριθμό των παγωμένων παραμέτρων. Παράλληλα εμφανίζει τα αντίστοιχα ποσοστά, παρέχοντας μια συνοπτική εικόνα του βαθμού στον οποίο κάθε δίκτυο παραμένει σταθερό ή επιτρέπεται να προσαρμοστεί.

```
def print_model_summary(
    model,
    model_name,
    training_strategy,
):
    total_parameters = sum(
        parameter.numel()
        for parameter in model.parameters()
    )
    trainable_parameters = sum(
        parameter.numel()
        for parameter in model.parameters()
        if parameter.requires_grad
    )
    frozen_parameters = (
        total_parameters
        - trainable_parameters
    )
    trainable_percentage = (
        100.0
        * trainable_parameters
        / total_parameters
    )
```

```
    frozen_percentage = (
        100.0
        * frozen_parameters
        / total_parameters
    )
```

Κώδικας 4.9: *Υπολογισμός των συνολικών, εκπαιδεύσιμων και παγωμένων παραμέτρων ενός μοντέλου.*

Η μέθοδος parameter.numel() επιστρέφει το πλήθος των αριθμητικών στοιχείων που περιέχει κάθε παράμετρος του δικτύου. Με την άθροιση των τιμών αυτών για όλες τις παραμέτρους προκύπτει το συνολικό μέγεθος του μοντέλου ως προς τον αριθμό παραμέτρων. Όταν στην ίδια διαδικασία συμπεριλαμβάνονται μόνο οι παράμετροι για τις οποίες ισχύει parameter.requires_grad, προκύπτει ο αριθμός των εκπαιδεύσιμων παραμέτρων. Η αφαίρεση των εκπαιδεύσιμων από τις συνολικές δίνει αντίστοιχα τον αριθμό των παγωμένων παραμέτρων.

Ο συγκεκριμένος έλεγχος έχει ιδιαίτερη σημασία για τη διάκριση μεταξύ Feature Extraction και Partial Fine-Tuning. Στο Feature Extraction αναμένεται να είναι εκπαιδεύσιμο ένα πολύ περιορισμένο τμήμα του μοντέλου, καθώς ο προεκπαιδευμένος κορμός παραμένει παγωμένος. Στο Partial Fine-Tuning, αντίθετα, ο αριθμός των εκπαιδεύσιμων παραμέτρων αυξάνεται επειδή ενεργοποιούνται και επιλεγμένα βαθύτερα τμήματα. Επομένως, η καταμέτρηση λειτουργεί ως πρακτικός έλεγχος ότι η διαφορετική στρατηγική μεταφοράς μάθησης έχει πράγματι αποτυπωθεί στη διαμόρφωση του μοντέλου και δεν παραμένει μόνο ως πρόθεση στον κώδικα.

Εκτός από τη συνολική καταμέτρηση, σε επιμέρους πειραματικές διαμορφώσεις πραγματοποιείται και έλεγχος των **ονομάτων των εκπαιδεύσιμων παραμέτρων**. Η διαδικασία αυτή είναι αυστηρότερη, επειδή δεν ελέγχει μόνο πόσες παράμετροι είναι εκπαιδεύσιμες αλλά και σε ποια ακριβώς τμήματα του δικτύου ανήκουν. Χαρακτηριστικό παράδειγμα αποτελεί η διαμόρφωση Feature Extraction της ResNet18, όπου αναμένεται να είναι εκπαιδεύσιμες αποκλειστικά οι παράμετροι fc.weight και fc.bias του νέου τελικού γραμμικού επιπέδου. Ο αντίστοιχος έλεγχος παρουσιάστηκε στην Ενότητα 4.5 και εξασφαλίζει ότι δεν έχει παραμείνει κατά λάθος εκπαιδεύσιμο κάποιο τμήμα του προεκπαιδευμένου κορμού.

Ένας δεύτερος βασικός έλεγχος αφορά τη **μορφή της εξόδου του μοντέλου**. Μετά την αντικατάσταση του αρχικού classifier, δεν αρκεί να θεωρηθεί ότι η αλλαγή πραγματοποιήθηκε σωστά μόνο επειδή ο κώδικας εκτελέστηκε χωρίς σφάλμα. Το μοντέλο πρέπει να μπορεί να δεχθεί ένα batch εικόνων με τις αναμενόμενες διαστάσεις και να επιστρέψει έναν πίνακα εξόδου στον οποίο κάθε εικόνα αντιστοιχεί σε ακριβώς εννέα τιμές, μία για κάθε κλάση.

Για τον έλεγχο αυτό χρησιμοποιείται στο notebook η συνάρτηση validate_model_output. Η συνάρτηση δημιουργεί ένα δοκιμαστικό tensor εικόνων με τρία κανάλια και χωρικές διαστάσεις ίσες με το IMAGE_SIZE, το μεταφέρει στην ίδια υπολογιστική συσκευή με το μοντέλο και πραγματοποιεί ένα δοκιμαστικό forward pass. Η αναμενόμενη μορφή της εξόδου προσδιορίζεται από το μέγεθος του batch και τη μεταβλητή NUM_CLASSES.

```
expected_output_shape = (
    batch_size,
    NUM_CLASSES,
)
```

```
if outputs.shape != expected_output_shape:
    raise ValueError(
        "Μη αναμενόμενο σχήμα εξόδου. "
        f"Αναμενόταν {expected_output_shape}, "
        f"αλλά βρέθηκε {tuple(outputs.shape)}."
    )
```

Κώδικας 4.10: *Έλεγχος ότι η έξοδος του προσαρμοσμένου μοντέλου αντιστοιχεί στο μέγεθος του batch και στις εννέα κλάσεις.*

Αν, για παράδειγμα, το batch_size είναι 16 και το NUM_CLASSES ισούται με 9, η αναμενόμενη έξοδος έχει διαστάσεις **[16, 9]**. Οι 16 γραμμές αντιστοιχούν στις 16 εικόνες του batch, ενώ οι 9 τιμές κάθε γραμμής αποτελούν τις εξόδους του μοντέλου για τις εννέα πιθανές γεωγραφικές κλάσεις. Στο στάδιο αυτό οι τιμές αυτές αποτελούν τις ακατέργαστες εξόδους του δικτύου, δηλαδή τα logits, και όχι απευθείας πιθανότητες.

Η χρήση του ValueError έχει επίσης συγκεκριμένο σκοπό. Αν η πραγματική έξοδος δεν συμφωνεί με την αναμενόμενη μορφή, η εκτέλεση διακόπτεται και εμφανίζεται μήνυμα που περιλαμβάνει τόσο το αναμενόμενο όσο και το πραγματικό σχήμα. Με αυτόν τον τρόπο ένα σφάλμα στην προσαρμογή του classifier μπορεί να εντοπιστεί πριν ξεκινήσει η πλήρης διαδικασία εκπαίδευσης.

Ο έλεγχος ενός forward pass εξυπηρετεί και έναν ευρύτερο σκοπό. Επιβεβαιώνει ότι τα επιμέρους τμήματα του προσαρμοσμένου δικτύου μπορούν να συνεργαστούν υπολογιστικά μετά τις αλλαγές που πραγματοποιήθηκαν στην αρχική αρχιτεκτονική. Η αντικατάσταση ενός classifier απαιτεί σωστή αντιστοίχιση μεταξύ του αριθμού χαρακτηριστικών που παράγει ο προεκπαιδευμένος κορμός και του αριθμού εισόδων του νέου γραμμικού επιπέδου. Η ανάκτηση του in_features από το αρχικό επίπεδο, όπως παρουσιάστηκε στην Ενότητα 4.4, διατηρεί αυτή τη συμβατότητα, ενώ το δοκιμαστικό forward pass παρέχει έναν επιπλέον πρακτικό έλεγχο ότι το προσαρμοσμένο δίκτυο μπορεί πράγματι να επεξεργαστεί δεδομένα και να παράγει την απαιτούμενη έξοδο.

Οι έλεγχοι αυτοί δεν αποτελούν αξιολόγηση της ικανότητας ταξινόμησης των μοντέλων. Ένα μοντέλο μπορεί να έχει τη σωστή δομή και να πραγματοποιεί επιτυχώς forward pass χωρίς αυτό να σημαίνει ότι έχει μάθει να ταξινομεί σωστά τις εικόνες. Ο σκοπός τους είναι διαφορετικός: να επιβεβαιώσουν την **τεχνική ορθότητα της διαμόρφωσης** πριν αρχίσει η εκπαίδευση. Η πραγματική απόδοση των μοντέλων προσδιορίζεται αργότερα μέσω της εκπαίδευσης και των μετρικών αξιολόγησης.

Συνεπώς, πριν από την πειραματική διαδικασία ελέγχεται ότι κάθε αρχιτεκτονική έχει προσαρμοστεί στις εννέα κλάσεις, ότι οι παράμετροι που πρέπει να παραμένουν παγωμένες ή εκπαιδεύσιμες βρίσκονται στην αναμενόμενη κατάσταση και ότι το μοντέλο μπορεί να παράγει έξοδο κατάλληλων διαστάσεων. Οι έλεγχοι αυτοί λειτουργούν ως ενδιάμεσο στάδιο επαλήθευσης μεταξύ της προσαρμογής των αρχιτεκτονικών και της πραγματικής εκπαίδευσής τους, μειώνοντας την πιθανότητα να εκτελεστεί ένα πλήρες πείραμα με λανθασμένη διαμόρφωση.

4.9 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο παρουσιάστηκε η μεθοδολογία επιλογής και προσαρμογής των μοντέλων Βαθιάς Μάθησης που χρησιμοποιήθηκαν για την ταξινόμηση των ελληνικών παραδοσιακών φορεσιών στις εννέα γεωγραφικές κλάσεις. Η διαδικασία βασίστηκε στη μεταφορά μάθησης και στην αξιοποίηση προεκπαιδευμένων μοντέλων, ώστε η ήδη αποκτημένη γνώση από ένα εκτεταμένο σύνολο εικόνων να αποτελέσει το αρχικό σημείο για την αντιμετώπιση του νέου προβλήματος, αντί τα δίκτυα να εκπαιδευτούν από τυχαία αρχικοποίηση.

Για την πειραματική διαδικασία επιλέχθηκαν οι αρχιτεκτονικές ResNet18, EfficientNet-B0 και ConvNeXt-Tiny, οι οποίες αντιπροσωπεύουν διαφορετικές προσεγγίσεις στον σχεδιασμό Συνελικτικών Νευρωνικών Δικτύων. Και στις τρεις περιπτώσεις χρησιμοποιήθηκαν προεκπαιδευμένα βάρη και το αρχικό τμήμα ταξινόμησης προσαρμόστηκε έτσι ώστε η έξοδος του δικτύου να αποτελείται από εννέα τιμές, μία για κάθε γεωγραφική κλάση του συνόλου δεδομένων.

Η προσαρμογή των μοντέλων πραγματοποιήθηκε με δύο διαφορετικές στρατηγικές μεταφοράς μάθησης. Στο Feature Extraction ο προεκπαιδευμένος κορμός διατηρήθηκε παγωμένος και η εκπαίδευση περιορίστηκε στο νέο τμήμα ταξινόμησης. Με αυτόν τον τρόπο τα χαρακτηριστικά που είχαν ήδη μάθει τα μοντέλα χρησιμοποιήθηκαν χωρίς μεταβολή για την αναπαράσταση των νέων εικόνων.

Στο Partial Fine-Tuning ακολουθήθηκε διαφορετική προσέγγιση, καθώς εκτός από το τμήμα ταξινόμησης ενεργοποιήθηκαν για εκπαίδευση και επιλεγμένα βαθύτερα τμήματα των προεκπαιδευμένων δικτύων. Συγκεκριμένα, στη ResNet18 ενεργοποιήθηκε το layer4, στην EfficientNet-B0 τα δύο τελευταία στοιχεία του features και στην ConvNeXt-Tiny το τελευταίο στοιχείο του features. Η επιλογή βαθύτερων αντί πρώιμων τμημάτων βασίζεται στην ιεραρχική οργάνωση των χαρακτηριστικών στα Συνελικτικά Νευρωνικά Δίκτυα: τα πρώτα επίπεδα εξάγουν γενικότερα οπτικά χαρακτηριστικά, ενώ τα βαθύτερα σχηματίζουν περισσότερο σύνθετες και εξειδικευμένες αναπαραστάσεις. Με τον τρόπο αυτό επιδιώκεται η διατήρηση της γενικής προεκπαιδευμένης γνώσης και ταυτόχρονα η δυνατότητα μερικής προσαρμογής της στα χαρακτηριστικά του νέου προβλήματος.

Ο συνδυασμός των τριών αρχιτεκτονικών με τις δύο στρατηγικές οδήγησε στη δημιουργία συνολικά έξι πειραματικών διαμορφώσεων. Η οργάνωση αυτή επιτρέπει αφενός τη σύγκριση διαφορετικών αρχιτεκτονικών υπό την ίδια γενική στρατηγική μεταφοράς μάθησης και αφετέρου τη σύγκριση Feature Extraction και Partial Fine-Tuning για την ίδια αρχιτεκτονική. Η πειραματική διαδικασία μπορεί επομένως να εξετάσει όχι μόνο ποιο μοντέλο επιτυγχάνει καλύτερη απόδοση, αλλά και κατά πόσο η δυνατότητα προσαρμογής μέρους του προεκπαιδευμένου κορμού επηρεάζει την απόδοση στο συγκεκριμένο πρόβλημα.

Ιδιαίτερη σημασία δόθηκε και στον προγραμματιστικό έλεγχο της ορθότητας των διαμορφώσεων πριν από την εκπαίδευση. Ελέγχθηκε η κατάσταση των εκπαιδεύσιμων και παγωμένων παραμέτρων, υπολογίστηκε ο αριθμός τους και πραγματοποιήθηκε δοκιμαστικό forward pass, ώστε να επιβεβαιωθεί ότι κάθε προσαρμοσμένο μοντέλο παράγει έξοδο συμβατή με τις εννέα κλάσεις. Οι έλεγχοι αυτοί δεν αποτελούν αξιολόγηση της ικανότητας ταξινόμησης, αλλά επαλήθευση ότι η υλοποίηση ανταποκρίνεται στον πειραματικό σχεδιασμό πριν από την εκτέλεση της εκπαίδευσης.

Με την ολοκλήρωση του κεφαλαίου έχουν επομένως καθοριστεί οι αρχιτεκτονικές, τα προεκπαιδευμένα βάρη, ο τρόπος προσαρμογής των classifiers, οι δύο στρατηγικές μεταφοράς μάθησης και οι έξι τελικές πειραματικές διαμορφώσεις. Το επόμενο στάδιο αφορά την πραγματική εκπαίδευση των μοντέλων. Στο Κεφάλαιο 5 παρουσιάζεται η πειραματική διαδικασία που εφαρμόστηκε στις έξι διαμορφώσεις, συμπεριλαμβανομένων των υπερπαραμέτρων, της συνάρτησης απώλειας και του optimizer, του μηχανισμού προσαρμογής του ρυθμού μάθησης, του early stopping, της αποθήκευσης checkpoints και της συνολικής ροής εκπαίδευσης και επικύρωσης.

ΚΕΦΑΛΑΙΟ 5 - ΕΚΠΑΙΔΕΥΣΗ ΚΑΙ ΠΕΙΡΑΜΑΤΙΚΗ ΔΙΑΔΙΚΑΣΙΑ

5.1 ΕΙΣΑΓΩΓΗ

Μετά την προετοιμασία του συνόλου δεδομένων και την προσαρμογή των προεκπαιδευμένων αρχιτεκτονικών στο πρόβλημα ταξινόμησης των εννέα γεωγραφικών κατηγοριών, το επόμενο στάδιο αφορά την εκπαίδευση και την οργανωμένη εκτέλεση των πειραμάτων. Σκοπός του παρόντος κεφαλαίου είναι η παρουσίαση της πειραματικής διαδικασίας που ακολουθήθηκε για τις έξι διαμορφώσεις μοντέλων που ορίστηκαν στο προηγούμενο κεφάλαιο, με έμφαση στις κοινές συνθήκες εκπαίδευσης, στη διαδικασία βελτιστοποίησης και επικύρωσης και στον τρόπο επιλογής και αποθήκευσης του καλύτερου μοντέλου κάθε πειράματος.

Όπως παρουσιάστηκε στο Κεφάλαιο 4, η πειραματική μελέτη βασίζεται στις αρχιτεκτονικές ResNet18, EfficientNet-B0 και ConvNeXt-Tiny, καθεμία από τις οποίες εξετάζεται με δύο στρατηγικές Μεταφοράς Μάθησης: Feature Extraction και Partial Fine-Tuning. Προκύπτουν επομένως συνολικά έξι πειραματικές διαμορφώσεις. Η εκτέλεσή τους υπό κοινό πειραματικό πλαίσιο επιτρέπει τη μετέπειτα σύγκριση της συμπεριφοράς τους, περιορίζοντας όσο είναι δυνατόν την επίδραση διαφορετικών συνθηκών εκπαίδευσης.

Η διαδικασία εκπαίδευσης βασίζεται στα υποσύνολα training και validation που δημιουργήθηκαν στο Κεφάλαιο 3. Το training set χρησιμοποιείται για την ενημέρωση των εκπαιδεύσιμων παραμέτρων του εκάστοτε μοντέλου, ενώ το validation set χρησιμοποιείται κατά τη διάρκεια της εκπαίδευσης για την παρακολούθηση της συμπεριφοράς του μοντέλου σε δεδομένα που δεν συμμετέχουν στην ενημέρωση των βαρών. Αντίθετα, το test set διατηρείται εκτός της διαδικασίας εκπαίδευσης και επιλογής του καλύτερου checkpoint και χρησιμοποιείται για την τελική αξιολόγηση των εκπαιδευμένων μοντέλων, η οποία παρουσιάζεται στο Κεφάλαιο 6. Ο διαχωρισμός αυτός είναι απαραίτητος ώστε η τελική αξιολόγηση να πραγματοποιείται σε δεδομένα που δεν έχουν χρησιμοποιηθεί για τη λήψη αποφάσεων κατά την εκπαίδευση.

Για κάθε πείραμα εφαρμόζεται επαναληπτική διαδικασία εκπαίδευσης σε epochs. Σε κάθε epoch πραγματοποιείται αρχικά η φάση training, κατά την οποία το μοντέλο επεξεργάζεται τα batches του συνόλου εκπαίδευσης, υπολογίζεται η συνάρτηση απώλειας και ενημερώνονται οι εκπαιδεύσιμες παράμετροι μέσω του optimizer. Ακολουθεί η φάση validation, στην οποία το μοντέλο αξιολογείται χωρίς ενημέρωση των βαρών. Με αυτόν τον τρόπο είναι δυνατή η παρακολούθηση της εξέλιξης της εκπαίδευσης και της ικανότητας γενίκευσης του μοντέλου κατά τη διάρκεια των διαδοχικών epochs.

Η πειραματική διαδικασία δεν περιορίζεται στην εκτέλεση ενός προκαθορισμένου αριθμού epochs. Η απόδοση στο validation set παρακολουθείται κατά την εκπαίδευση και χρησιμοποιείται για τη διατήρηση του καλύτερου checkpoint, ενώ εφαρμόζεται και μηχανισμός Early Stopping ώστε η διαδικασία να μπορεί να ολοκληρωθεί πριν από το μέγιστο επιτρεπόμενο πλήθος epochs όταν δεν παρατηρείται περαιτέρω βελτίωση σύμφωνα με το κριτήριο που έχει οριστεί. Παράλληλα, καταγράφεται το ιστορικό της εκπαίδευσης, ώστε να είναι δυνατή η μετέπειτα μελέτη της εξέλιξης των μετρικών και της συνάρτησης απώλειας.

Η χρήση κοινής διαδικασίας για τα έξι πειράματα είναι σημαντική για τη μεθοδολογική συνέπεια της σύγκρισης. Οι διαφορές μεταξύ των πειραμάτων αφορούν κυρίως την αρχιτεκτονική και τη στρατηγική προσαρμογής των προεκπαιδευμένων παραμέτρων που έχουν ήδη οριστεί, ενώ η βασική λογική εκπαίδευσης και παρακολούθησης παραμένει κοινή. Έτσι, τα αποτελέσματα που θα παρουσιαστούν στο επόμενο κεφάλαιο μπορούν να εξεταστούν στο πλαίσιο μιας ενιαίας πειραματικής διαδικασίας.

Στις επόμενες ενότητες παρουσιάζονται αναλυτικότερα το περιβάλλον και οι κοινές ρυθμίσεις εκπαίδευσης, η συνάρτηση απώλειας και η διαδικασία βελτιστοποίησης, ο κύκλος training και validation, ο μηχανισμός Early Stopping και η αποθήκευση των checkpoints, καθώς και η εκτέλεση και παρακολούθηση των έξι πειραμάτων. Η παρουσίαση επικεντρώνεται στις επιλογές που αφορούν την πραγματική διαδικασία εκπαίδευσης, αποφεύγοντας την επανάληψη της θεωρητικής περιγραφής των αρχιτεκτονικών και της προετοιμασίας των δεδομένων που αναπτύχθηκαν στα προηγούμενα κεφάλαια.

5.2 ΚΟΙΝΕΣ ΡΥΘΜΙΣΕΙΣ ΤΗΣ ΠΕΙΡΑΜΑΤΙΚΗΣ ΕΚΠΑΙΔΕΥΣΗΣ

Για την εκτέλεση των έξι πειραμάτων καθορίστηκε ένα κοινό σύνολο ρυθμίσεων εκπαίδευσης, ώστε οι διαφορετικές αρχιτεκτονικές και στρατηγικές Μεταφοράς Μάθησης να εξεταστούν μέσα σε όσο το δυνατόν πιο συνεπές πειραματικό πλαίσιο. Οι λεπτομέρειες που αφορούν την προετοιμασία των εικόνων, τη δημιουργία των DataLoaders και την προσαρμογή των αρχιτεκτονικών έχουν ήδη παρουσιαστεί στα Κεφάλαια 3 και 4 και, επομένως, στην παρούσα ενότητα εξετάζονται αποκλειστικά οι παράμετροι που σχετίζονται με την εκτέλεση της εκπαίδευσης.

Ως ανώτατο όριο εκπαίδευσης ορίστηκαν 25 epochs. Το όριο αυτό προσδιορίζει τη μέγιστη διάρκεια ενός πειράματος και δεν συνεπάγεται ότι κάθε μοντέλο εκπαιδεύεται υποχρεωτικά για το σύνολο των 25 epochs, καθώς η πραγματική διάρκεια εξαρτάται και από τον μηχανισμό Early Stopping που εφαρμόζεται κατά την εκπαίδευση. Για τον συγκεκριμένο μηχανισμό ορίστηκε patience ίσο με 5 epochs, επιτρέποντας τη διακοπή της διαδικασίας όταν η επίδοση επικύρωσης δεν παρουσιάζει την απαιτούμενη βελτίωση για πέντε διαδοχικά epochs. Η ακριβής λειτουργία του κριτηρίου επιλογής και διακοπής παρουσιάζεται στην Ενότητα 5.5.

Στο κοινό πειραματικό πλαίσιο ορίστηκε προεπιλεγμένο learning rate ίσο με 10^{-3} και weight decay ίσο με 10^{-4}. Το learning rate καθορίζει το μέγεθος των ενημερώσεων που πραγματοποιούνται στις εκπαιδεύσιμες παραμέτρους κατά τη βελτιστοποίηση, ενώ το weight decay χρησιμοποιείται ως παράμετρος κανονικοποίησης του optimizer. Οι συγκεκριμένες τιμές αποτελούν τις βασικές ρυθμίσεις της υλοποίησης. Στα πειράματα Partial Fine-Tuning εφαρμόζονται οι αντίστοιχες ειδικές ρυθμίσεις που έχουν οριστεί για τη συγκεκριμένη στρατηγική και παρουσιάζονται μαζί με την εκτέλεση των επιμέρους πειραμάτων, ώστε να μην συγχέονται με τις κοινές προεπιλεγμένες τιμές.

Για τον έλεγχο των στοχαστικών διαδικασιών χρησιμοποιήθηκε σταθερό random seed με τιμή 42. Η τιμή εφαρμόστηκε στις γεννήτριες τυχαίων αριθμών της Python, της NumPy και της PyTorch, καθώς και στη CUDA όταν αυτή ήταν διαθέσιμη. Επιπλέον, χρησιμοποιήθηκαν οι ρυθμίσεις torch.backends.cudnn.deterministic = True και torch.backends.cudnn.benchmark = False. Στόχος των επιλογών αυτών είναι ο περιορισμός ανεπιθύμητων διαφορών μεταξύ επαναλαμβανόμενων εκτελέσεων και η δημιουργία ενός περισσότερο ελεγχόμενου πειραματικού περιβάλλοντος.

Παράμετρος	**Τιμή**
Μέγιστος αριθμός epochs	25
Early Stopping patience	5 epochs
Προεπιλεγμένο learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Random seed	42

Πίνακας 5.1: Βασικές ρυθμίσεις της πειραματικής εκπαίδευσης

Οι παραπάνω ρυθμίσεις αποτελούν το βασικό πλαίσιο εκτέλεσης των πειραμάτων και όχι αποτέλεσμα ξεχωριστής διαδικασίας αναζήτησης της βέλτιστης τιμής κάθε υπερπαραμέτρου. Η διάκριση αυτή είναι σημαντική, καθώς οι συγκεκριμένες τιμές αντιμετωπίζονται στην παρούσα εργασία ως προκαθορισμένες πειραματικές επιλογές πάνω στις οποίες πραγματοποιείται η σύγκριση των μοντέλων και όχι ως αποδεδειγμένα βέλτιστες υπερπαράμετροι για το συγκεκριμένο σύνολο δεδομένων.

5.3 ΣΥΝΑΡΤΗΣΗ ΑΠΩΛΕΙΑΣ ΚΑΙ ΒΕΛΤΙΣΤΟΠΟΙΗΣΗ

Για την εκπαίδευση και των έξι πειραματικών διαμορφώσεων χρησιμοποιήθηκε η CrossEntropyLoss της PyTorch ως συνάρτηση απώλειας. Η θεωρητική λειτουργία της Cross-Entropy Loss στο πρόβλημα πολυταξικής ταξινόμησης έχει ήδη παρουσιαστεί στο Κεφάλαιο 2 και, επομένως, στην παρούσα ενότητα εξετάζεται αποκλειστικά ο τρόπος με τον οποίο ενσωματώθηκε στην πειραματική διαδικασία.

Σε κάθε batch, η έξοδος του μοντέλου και οι πραγματικές ετικέτες των εικόνων χρησιμοποιούνται για τον υπολογισμό της απώλειας, η οποία στη συνέχεια αξιοποιείται για την ενημέρωση των παραμέτρων που έχουν οριστεί ως εκπαιδεύσιμες.

Ως αλγόριθμος βελτιστοποίησης χρησιμοποιήθηκε ο Adam σε όλα τα πειράματα. Κατά τη δημιουργία του optimizer δεν εισάγεται το σύνολο των παραμέτρων του μοντέλου χωρίς διάκριση. Ο κώδικας επιλέγει αποκλειστικά εκείνες για τις οποίες η ιδιότητα requires_grad έχει τιμή True. Η επιλογή αυτή συνδέει άμεσα τη διαδικασία βελτιστοποίησης με τη στρατηγική Μεταφοράς Μάθησης που έχει οριστεί για κάθε πείραμα: στο Feature Extraction ενημερώνονται μόνο οι παράμετροι του νέου ταξινομητή, ενώ στο Partial Fine-Tuning ο optimizer περιλαμβάνει επιπλέον τις παραμέτρους των τμημάτων της αρχιτεκτονικής που έχουν αποδεσμευτεί για περαιτέρω εκπαίδευση, όπως καθορίστηκε στο Κεφάλαιο 4.

Η συγκεκριμένη επιλογή υλοποιείται με τον έλεγχο της ιδιότητας requires_grad, όπως παρουσιάζεται στον Κώδικα 5.1. Μετά τη συλλογή των εκπαιδεύσιμων παραμέτρων πραγματοποιείται επιπλέον έλεγχος ότι η λίστα δεν είναι κενή, πριν αυτές δοθούν στον optimizer.

```
 criterion = nn.CrossEntropyLoss()
trainable_parameters = [
    parameter
    for parameter in model.parameters()
    if parameter.requires_grad
]
if len(trainable_parameters) == 0:
    raise ValueError(
        "Δεν βρέθηκαν εκπαιδεύσιμες παράμετροι."
    )
optimizer = optim.Adam(
    params=trainable_parameters,
    lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY
)
```

Κώδικας 5.1: Επιλογή εκπαιδεύσιμων παραμέτρων και δημιουργία του optimizer

Μετά τη δημιουργία του optimizer πραγματοποιείται και προγραμματιστικός έλεγχος συνέπειας. Υπολογίζεται αφενός το πλήθος των παραμέτρων που έχουν εισαχθεί στις ομάδες παραμέτρων του Adam και αφετέρου το πλήθος των παραμέτρων του μοντέλου για τις οποίες ισχύει requires_grad=True. Αν τα δύο μεγέθη διαφέρουν, η εκτέλεση διακόπτεται με μήνυμα σφάλματος. Ο έλεγχος αυτός είναι ιδιαίτερα χρήσιμος στο συγκεκριμένο πειραματικό σχήμα, επειδή τα τμήματα του δικτύου που εκπαιδεύονται μεταβάλλονται μεταξύ Feature Extraction και Partial Fine-Tuning. Με αυτόν τον τρόπο επιβεβαιώνεται πριν από την εκπαίδευση ότι ο optimizer ενημερώνει ακριβώς το σύνολο παραμέτρων που προβλέπει η αντίστοιχη διαμόρφωση.

Η βασική διαφοροποίηση στις ρυθμίσεις βελτιστοποίησης μεταξύ των δύο στρατηγικών αφορά το learning rate. Στα τρία πειράματα Feature Extraction χρησιμοποιήθηκε learning rate 10–310^{-3}, ενώ στα τρία πειράματα Partial Fine-Tuning χρησιμοποιήθηκε μικρότερη τιμή 10–410^{-4}. Το weight decay παρέμεινε σταθερό σε 10–410^{-4} και στις έξι διαμορφώσεις. Στην περίπτωση του Partial Fine-Tuning, το μικρότερο learning rate επιτρέπει η προσαρμογή των ήδη προεκπαιδευμένων παραμέτρων που έχουν αποδεσμευτεί να πραγματοποιείται με μικρότερα βήματα, σε αντίθεση με το Feature Extraction όπου το προεκπαιδευμένο τμήμα παραμένει παγωμένο και εκπαιδεύεται ο νέος ταξινομητής.

Ρύθμιση	**Feature Extraction**	**Partial Fine-Tuning**
Συνάρτηση απώλειας	CrossEntropyLoss	CrossEntropyLoss
Optimizer	Adam	Adam
Learning rate	1×10^{-3}	1×10^{-4}
Weight decay	1×10^{-4}	1×10^{-4}

Πίνακας 5.2: Ρυθμίσεις βελτιστοποίησης ανά στρατηγική Μεταφοράς Μάθησης

Η διαφοροποίηση του learning rate αποτελεί μέρος της προκαθορισμένης πειραματικής διαμόρφωσης και όχι αποτέλεσμα ξεχωριστής διαδικασίας αναζήτησης υπερπαραμέτρων. Συνεπώς, η σύγκριση που πραγματοποιείται στην εργασία αφορά τις έξι συγκεκριμένες διαμορφώσεις όπως υλοποιήθηκαν και δεν αποσκοπεί στην απόδειξη ότι οι χρησιμοποιούμενες τιμές αποτελούν τις βέλτιστες δυνατές για κάθε αρχιτεκτονική.

5.4 ΔΙΑΔΙΚΑΣΙΑ ΕΚΠΑΙΔΕΥΣΗΣ ΚΑΙ ΕΠΙΚΥΡΩΣΗΣ

Η εκπαίδευση κάθε πειραματικής διαμόρφωσης πραγματοποιήθηκε μέσω κοινής διαδικασίας, η οποία οργανώθηκε σε δύο διαδοχικές φάσεις ανά epoch: τη φάση εκπαίδευσης και τη φάση επικύρωσης. Με τον τρόπο αυτό, η ενημέρωση των εκπαιδεύσιμων παραμέτρων διαχωρίζεται από την παρακολούθηση της συμπεριφοράς του μοντέλου στο validation set. Η θεωρητική λειτουργία των epochs, του backpropagation και της επικύρωσης έχει ήδη παρουσιαστεί στο Κεφάλαιο 2· εδώ εξετάζεται ο τρόπος με τον οποίο τα στοιχεία αυτά υλοποιήθηκαν στην πειραματική διαδικασία.

Η φάση εκπαίδευσης υλοποιήθηκε μέσω της συνάρτησης train_one_epoch. Στην αρχή κάθε epoch το μοντέλο τίθεται σε κατάσταση εκπαίδευσης με την κλήση model.train(). Για κάθε batch, οι εικόνες και οι αντίστοιχες ετικέτες μεταφέρονται στη διαθέσιμη υπολογιστική συσκευή και μηδενίζονται τα gradients που έχουν αποθηκευτεί από το προηγούμενο βήμα. Στη συνέχεια πραγματοποιείται η εμπρόσθια διέλευση των εικόνων από το μοντέλο, υπολογίζεται η απώλεια και εκτελείται η οπισθοδιάδοση. Τέλος, ο optimizer ενημερώνει τις παραμέτρους που έχουν οριστεί ως εκπαιδεύσιμες για τη συγκεκριμένη πειραματική διαμόρφωση.

Κατά την ίδια διαδικασία συσσωρεύονται η απώλεια και ο αριθμός των σωστών προβλέψεων για το σύνολο των δειγμάτων που επεξεργάστηκε το μοντέλο. Με την ολοκλήρωση του epoch υπολογίζονται η μέση training loss και η training accuracy. Οι δύο αυτές τιμές καταγράφονται στο ιστορικό εκπαίδευσης και χρησιμοποιούνται για την παρακολούθηση της εξέλιξης του μοντέλου από epoch σε epoch.

Η φάση επικύρωσης υλοποιήθηκε ξεχωριστά μέσω της συνάρτησης evaluate_one_epoch. Το μοντέλο τίθεται σε κατάσταση αξιολόγησης με model.eval() και η επεξεργασία πραγματοποιείται χωρίς υπολογισμό gradients. Στη συγκεκριμένη φάση δεν εκτελούνται backward() και optimizer.step(), επομένως οι παράμετροι του μοντέλου παραμένουν αμετάβλητες. Για ολόκληρο το validation set υπολογίζονται η μέση validation loss και η validation accuracy, οι οποίες καταγράφονται μετά από κάθε epoch.

Ο διαχωρισμός των δύο φάσεων επιτρέπει την ταυτόχρονη παρακολούθηση της συμπεριφοράς του μοντέλου στα δεδομένα που χρησιμοποιούνται για την ενημέρωση των παραμέτρων και σε ανεξάρτητα δεδομένα επικύρωσης. Στην παρούσα υλοποίηση, η validation loss αποκτά επιπλέον λειτουργικό ρόλο, καθώς

αποτελεί το κριτήριο με βάση το οποίο προσδιορίζεται το καλύτερο checkpoint κατά την εκπαίδευση. Η διαδικασία αυτή αναλύεται ειδικότερα στην Ενότητα 5.5.

Η συνολική εκπαίδευση οργανώνεται από τη συνάρτηση train_model, η οποία επαναλαμβάνει τις δύο παραπάνω φάσεις μέχρι να συμπληρωθεί το μέγιστο πλήθος των 25 epochs ή μέχρι να ενεργοποιηθεί ο μηχανισμός Early Stopping. Για κάθε epoch αποθηκεύονται στο ιστορικό οι τιμές train_loss, validation_loss, train_accuracy, validation_accuracy και learning_rate. Έτσι, μετά την ολοκλήρωση κάθε πειράματος διατηρείται μια αναλυτική καταγραφή της πορείας του, η οποία επιτρέπει τη δημιουργία των αντίστοιχων καμπυλών εκπαίδευσης.

Η βασική ακολουθία ενεργειών της διαδικασίας παρουσιάζεται στον Κώδικα 5.2. Το απόσπασμα επικεντρώνεται στον κύκλο εκπαίδευσης και επικύρωσης, χωρίς να επαναλαμβάνει τις επιμέρους λειτουργίες που έχουν ήδη περιγραφεί.

```
for epoch in range(1, max_epochs + 1):
    train_loss, train_accuracy = train_one_epoch(
        model=model,
        data_loader=train_loader,
        criterion=criterion,
        optimizer=optimizer,
        device=device,
    )
    validation_loss, validation_accuracy = evaluate_one_epoch(
        model=model,
        data_loader=validation_loader,
        criterion=criterion,
        device=device,
    )
    history["train_loss"].append(train_loss)
    history["validation_loss"].append(validation_loss)
    history["train_accuracy"].append(train_accuracy)
    history["validation_accuracy"].append(validation_accuracy)
```

Κώδικας 5.2: Βασικός κύκλος εκπαίδευσης και επικύρωσης

Η ίδια διαδικασία εφαρμόζεται στις έξι πειραματικές διαμορφώσεις, ανεξάρτητα από την αρχιτεκτονική ή τη στρατηγική Μεταφοράς Μάθησης. Με αυτόν τον τρόπο διατηρείται κοινός μηχανισμός εκπαίδευσης και επικύρωσης, ενώ οι διαφοροποιήσεις μεταξύ των πειραμάτων περιορίζονται στις παραμέτρους και στα τμήματα των δικτύων που έχουν οριστεί ως εκπαιδεύσιμα, καθώς και στις αντίστοιχες ρυθμίσεις βελτιστοποίησης.

5.5 EARLY STOPPING ΚΑΙ ΑΠΟΘΗΚΕΥΣΗ ΚΑΛΥΤΕΡΟΥ ΜΟΝΤΕΛΟΥ

Για τον έλεγχο της διάρκειας της εκπαίδευσης εφαρμόστηκε μηχανισμός Early Stopping, με κριτήριο τη μεταβολή του validation loss. Σε αντίθεση με το ανώτατο όριο των 25 epochs, το οποίο καθορίζει τη μέγιστη δυνατή διάρκεια ενός πειράματος, το Early Stopping επιτρέπει την ολοκλήρωση της εκπαίδευσης νωρίτερα όταν το μοντέλο παύει να παρουσιάζει βελτίωση στο σύνολο επικύρωσης.

Στην αρχή κάθε πειράματος η καλύτερη τιμή του validation loss αρχικοποιείται στο άπειρο (float("inf")), ενώ ο μετρητής του Early Stopping αρχικοποιείται στο μηδέν. Μετά από κάθε epoch, η νέα τιμή του validation loss συγκρίνεται με την καλύτερη τιμή που έχει καταγραφεί έως εκείνη τη χρονική στιγμή. Ως βελτίωση θεωρείται αποκλειστικά η περίπτωση κατά την οποία το νέο validation loss είναι μικρότερο από το προηγούμενο καλύτερο.

Όταν εντοπίζεται βελτίωση, αποθηκεύονται στη μνήμη τα βάρη του μοντέλου που αντιστοιχούν στο συγκεκριμένο epoch και ο μετρητής Early Stopping μηδενίζεται. Παράλληλα, δημιουργείται νέο checkpoint στον αντίστοιχο φάκελο του πειράματος. Το checkpoint περιλαμβάνει, μεταξύ άλλων, το όνομα του πειράματος, το καλύτερο epoch, το model_state_dict, την κατάσταση του optimizer, το validation loss και validation accuracy, καθώς και πληροφορίες για τις κλάσεις και το μέγεθος εισόδου των εικόνων.

Αντίθετα, όταν το validation loss δεν παρουσιάζει βελτίωση, ο μετρητής αυξάνεται κατά μία μονάδα. Η τιμή του early_stopping_patience έχει οριστεί σε 5, επομένως η εκπαίδευση διακόπτεται όταν συμπληρωθούν πέντε διαδοχικά epochs χωρίς νέα χαμηλότερη τιμή validation loss. Η επιλογή αυτή επιτρέπει στο μοντέλο να συνεχίσει προσωρινά την εκπαίδευση ακόμη και αν ένα μεμονωμένο epoch δεν παρουσιάσει βελτίωση, χωρίς όμως να συνεχίζεται απεριόριστα η διαδικασία όταν η επίδοση επικύρωσης δεν βελτιώνεται.

Η βασική λογική επιλογής και αποθήκευσης του καλύτερου μοντέλου παρουσιάζεται στον Κώδικα 5.3.

```
if validation_loss < best_validation_loss:
    best_validation_loss = validation_loss
    best_validation_accuracy = validation_accuracy
    best_epoch = epoch + 1
    best_model_weights = copy.deepcopy(
        model.state_dict()
    )
    early_stopping_counter = 0
    checkpoint = {
        "experiment_name": experiment_name,
        "epoch": best_epoch,
        "model_state_dict": best_model_weights,
        "optimizer_state_dict": optimizer.state_dict(),
        "validation_loss": best_validation_loss,
        "validation_accuracy": best_validation_accuracy,
        "class_names": CLASS_NAMES,
        "class_to_idx": CLASS_TO_IDX,
```

```
        "image_size": IMAGE_SIZE
    }
    torch.save(
        checkpoint,
        checkpoint_path
    )
else:
    early_stopping_counter += 1
```

Κώδικας 5.3: Έλεγχος validation loss και αποθήκευση του καλύτερου checkpoint

Η χρήση του validation loss ως κριτηρίου σημαίνει ότι το «καλύτερο μοντέλο» της εκπαίδευσης δεν ορίζεται από το τελευταίο epoch ούτε από τη μεγαλύτερη training accuracy. Αντίθετα, αντιστοιχεί στην κατάσταση του μοντέλου που πέτυχε το μικρότερο validation loss κατά τη διάρκεια του συγκεκριμένου πειράματος. Η validation accuracy καταγράφεται επίσης στο checkpoint, αλλά δεν χρησιμοποιείται ως το κριτήριο που αποφασίζει αν ένα νέο checkpoint είναι καλύτερο από το προηγούμενο.

Με την ολοκλήρωση του κύκλου εκπαίδευσης, είτε λόγω Early Stopping είτε λόγω συμπλήρωσης του μέγιστου αριθμού epochs, η συνάρτηση επαναφέρει στο μοντέλο τα βάρη που αντιστοιχούν στο καλύτερο validation loss. Επιπλέον, πριν από την τελική αξιολόγηση στο test set, το αποθηκευμένο best_checkpoint.pth φορτώνεται εκ νέου και πραγματοποιούνται έλεγχοι ως προς το πείραμα και τις κλάσεις στις οποίες αντιστοιχεί. Με αυτόν τον τρόπο, η τελική αξιολόγηση δεν πραγματοποιείται αυθαίρετα με την κατάσταση του τελευταίου epoch, αλλά με το checkpoint που είχε επιλεγεί προηγουμένως αποκλειστικά μέσω του validation set.

Η διαδικασία αυτή διατηρεί διακριτούς τους ρόλους των validation και test δεδομένων. Το validation set συμμετέχει στην επιλογή του καλύτερου checkpoint και στην απόφαση πρόωρης διακοπής, ενώ το test set παραμένει εκτός της διαδικασίας εκπαίδευσης και επιλογής του μοντέλου και χρησιμοποιείται στη συνέχεια για την τελική αξιολόγηση των έξι πειραματικών διαμορφώσεων, η οποία παρουσιάζεται στο Κεφάλαιο 6.

5.6 ΕΚΤΕΛΕΣΗ ΚΑΙ ΠΑΡΑΚΟΛΟΥΘΗΣΗ ΤΩΝ ΈΞΙ ΠΕΙΡΑΜΑΤΩΝ

Η πειραματική διαδικασία οργανώθηκε σε έξι ανεξάρτητες διαμορφώσεις, οι οποίες προέκυψαν από τον συνδυασμό των τριών αρχιτεκτονικών ResNet18, EfficientNet-B0 και ConvNeXt-Tiny με τις στρατηγικές Feature Extraction και Partial Fine-Tuning. Η προσαρμογή των αρχιτεκτονικών και ο καθορισμός των εκπαιδεύσιμων τμημάτων τους έχουν ήδη παρουσιαστεί στο Κεφάλαιο 4. Στην παρούσα ενότητα παρουσιάζεται η εκτέλεση των συγκεκριμένων διαμορφώσεων και ο τρόπος με τον οποίο καταγράφηκε η πορεία της εκπαίδευσής τους.

Κάθε πείραμα εκτελέστηκε ως ξεχωριστή διαμόρφωση και συνδέθηκε με δικό του αναγνωριστικό και φάκελο αποτελεσμάτων. Με αυτόν τον τρόπο, τα checkpoints, το ιστορικό εκπαίδευσης, οι γραφικές παραστάσεις και τα υπόλοιπα παραγόμενα αρχεία διατηρούνται χωριστά για κάθε μοντέλο και στρατηγική. Η οργάνωση αυτή διευκολύνει τόσο τον έλεγχο της εκτέλεσης όσο και τη μεταγενέστερη συγκριτική αξιολόγηση των αποτελεσμάτων.

Πείραμα	Αρχιτεκτονική	Στρατηγική	Εκπαιδεύσιμα τμήματα	Learning rate
1	ResNet18	Feature Extraction	Νέος ταξινομητής	1×10^{-3}
2	ResNet18	Partial Fine-Tuning	layer4 + fc	1×10^{-4}
3	EfficientNet-Bo	Feature Extraction	Νέος ταξινομητής	1×10^{-3}
4	EfficientNet-Bo	Partial Fine-Tuning	Τελευταία δύο τμήματα των features + classifier	1×10^{-4}
5	ConvNeXt-Tiny	Feature Extraction	Νέος ταξινομητής	1×10^{-3}
6	ConvNeXt-Tiny	Partial Fine-Tuning	Τελευταίο τμήμα των features + classifier	1×10^{-4}

Πίνακας 5.3: Πειραματικές διαμορφώσεις που εκτελέστηκαν

Για καθεμία από τις έξι διαμορφώσεις ακολουθήθηκε η κοινή διαδικασία εκπαίδευσης και επικύρωσης που παρουσιάστηκε στις προηγούμενες ενότητες. Κατά την εκτέλεση κάθε epoch καταγράφονταν οι τιμές train_loss, validation_loss, train_accuracy και validation_accuracy. Καταγραφόταν επίσης το learning rate, ώστε το ιστορικό κάθε πειράματος να περιλαμβάνει τις βασικές πληροφορίες που περιγράφουν την εξέλιξη της εκπαίδευσης.

Οι καταγεγραμμένες τιμές χρησιμοποιήθηκαν για τη δημιουργία δύο γραφικών παραστάσεων ανά πείραμα. Η πρώτη απεικονίζει την εξέλιξη του training loss και του validation loss ως προς τα epochs, ενώ η δεύτερη παρουσιάζει αντίστοιχα την training accuracy και τη validation accuracy. Οι καμπύλες αυτές επιτρέπουν την οπτική εξέταση της πορείας της εκπαίδευσης και της σχέσης μεταξύ της συμπεριφοράς του μοντέλου στο training και στο validation set.

Η δημιουργία των διαγραμμάτων πραγματοποιείται μετά την ολοκλήρωση της εκπαίδευσης μέσω της συνάρτησης plot_training_history. Τα διαγράμματα αποθηκεύονται στον φάκελο του αντίστοιχου πειράματος ως loss_curve.png και accuracy_curve.png, ώστε να διατηρείται μόνιμη οπτική καταγραφή της πορείας κάθε διαμόρφωσης. Η αποθήκευσή τους σε ξεχωριστά αρχεία επιτρέπει επίσης την αξιοποίησή τους κατά τη μεταγενέστερη ανάλυση των αποτελεσμάτων.

Παράλληλα με τις γραφικές παραστάσεις, αποθηκεύεται το αριθμητικό ιστορικό της εκπαίδευσης στο αρχείο training_history.json. Επιπλέον, στο training_information.json καταγράφονται πληροφορίες που χαρακτηρίζουν την εκτέλεση του πειράματος, όπως ο αριθμός των epochs που πραγματοποιήθηκαν, το epoch στο οποίο εντοπίστηκε το καλύτερο validation loss και οι αντίστοιχες τιμές επικύρωσης. Με αυτόν τον τρόπο, η πορεία ενός πειράματος δεν διατηρείται μόνο οπτικά αλλά και σε δομημένη μορφή κατάλληλη για μεταγενέστερη επεξεργασία.

Η παρακολούθηση των καμπυλών loss και accuracy δεν χρησιμοποιήθηκε ως μηχανισμός χειροκίνητης επιλογής του καλύτερου checkpoint. Όπως παρουσιάστηκε στην Ενότητα 5.5, η επιλογή αυτή πραγματοποιείται προγραμματιστικά με βάση το ελάχιστο validation loss. Οι καμπύλες λειτουργούν συμπληρωματικά, καθώς παρέχουν μια συνολική εικόνα της εξέλιξης της εκπαίδευσης και επιτρέπουν να εξεταστεί εάν οι μεταβολές των μετρικών του training set συνοδεύονται από αντίστοιχη συμπεριφορά στο validation set.

Μετά την ολοκλήρωση της εκπαίδευσης κάθε διαμόρφωσης, διατηρούνται επομένως τόσο το καλύτερο checkpoint όσο και τα στοιχεία που περιγράφουν την πορεία της εκπαίδευσης. Στη συνέχεια, το καλύτερο checkpoint χρησιμοποιείται για την αξιολόγηση στο κοινό test set και παράγονται τα αντίστοιχα αρχεία τελικών μετρικών, προβλέψεων και πίνακα σύγχυσης. Η αριθμητική σύγκριση αυτών των αποτελεσμάτων δεν πραγματοποιείται στο παρόν κεφάλαιο, αλλά αποτελεί το κύριο αντικείμενο του Κεφαλαίου 6.

Η κοινή διαδικασία καταγραφής είναι ιδιαίτερα σημαντική για τη συγκριτική φύση της εργασίας, καθώς παρέχει το ίδιο είδος πληροφοριών για όλες τις πειραματικές διαμορφώσεις. Έτσι, η επόμενη φάση της μελέτης μπορεί να βασιστεί τόσο στις τελικές μετρικές του test set όσο και στην πορεία που ακολούθησε κάθε μοντέλο κατά την εκπαίδευση, διατηρώντας σαφή διάκριση μεταξύ της εκτέλεσης των πειραμάτων και της ερμηνείας των αποτελεσμάτων τους.

5.7 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο παρουσιάστηκε η πειραματική διαδικασία που εφαρμόστηκε για την εκπαίδευση των έξι διαμορφώσεων της εργασίας. Καθορίστηκε ένα κοινό πλαίσιο εκπαίδευσης και επικύρωσης, ώστε οι διαφορετικές αρχιτεκτονικές και στρατηγικές Μεταφοράς Μάθησης να μπορούν να εξεταστούν υπό συνεπείς πειραματικές συνθήκες.

Η εκπαίδευση βασίστηκε στη συνάρτηση απώλειας Cross-Entropy και στον optimizer Adam, ενώ χρησιμοποιήθηκαν διαφορετικά learning rates για τις διαμορφώσεις Feature Extraction και Partial Fine-Tuning. Παράλληλα, η διαδικασία οργανώθηκε έτσι ώστε ο optimizer να ενημερώνει αποκλειστικά τις παραμέτρους που είχαν οριστεί ως εκπαιδεύσιμες για κάθε πειραματική διαμόρφωση.

Κατά τη διάρκεια της εκπαίδευσης πραγματοποιούνταν διαδοχικά οι φάσεις training και validation, ενώ η εξέλιξη των αντίστοιχων loss και accuracy καταγραφόταν ανά epoch. Η χρήση Early Stopping περιόριζε τη συνέχιση της εκπαίδευσης όταν το validation loss δεν παρουσίαζε περαιτέρω βελτίωση, ενώ η αποθήκευση του καλύτερου checkpoint εξασφάλιζε ότι για την επόμενη φάση διατηρούνταν η κατάσταση του μοντέλου που αντιστοιχούσε στο χαμηλότερο validation loss και όχι απλώς εκείνη του τελευταίου epoch.

Με την ολοκλήρωση των έξι πειραμάτων δημιουργήθηκε για κάθε διαμόρφωση ένα οργανωμένο σύνολο αρχείων που περιλαμβάνει το καλύτερο checkpoint, το ιστορικό και τις πληροφορίες εκπαίδευσης, καθώς και τις γραφικές παραστάσεις της πορείας των βασικών μεγεθών. Η κοινή αυτή διαδικασία παρέχει τη βάση για την τελική και συγκριτική αξιολόγηση των μοντέλων.

Το επόμενο κεφάλαιο επικεντρώνεται στα αποτελέσματα που προέκυψαν από την εφαρμογή των εκπαιδευμένων μοντέλων στο ανεξάρτητο test set. Η αξιολόγηση πραγματοποιείται τόσο συνολικά όσο και ανά γεωγραφική κατηγορία, με χρήση των κατάλληλων μετρικών ταξινόμησης, των Top-1 και Top-3 προβλέψεων και των πινάκων σύγχυσης, με στόχο να προσδιοριστούν οι διαφορές μεταξύ των έξι πειραματικών διαμορφώσεων και να επιλεγεί το μοντέλο που παρουσιάζει την καταλληλότερη συμπεριφορά για την εφαρμογή αναγνώρισης παραδοσιακών φορεσιών.

ΚΕΦΑΛΑΙΟ 6 - ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ

6.1 ΕΙΣΑΓΩΓΗ

Μετά την ολοκλήρωση της εκπαίδευσης των έξι πειραματικών διαμορφώσεων, ακολουθεί η αξιολόγηση της απόδοσής τους στο ανεξάρτητο test set. Το στάδιο αυτό αποτελεί βασικό μέρος της πειραματικής μελέτης, καθώς επιτρέπει να εξεταστεί η ικανότητα των εκπαιδευμένων μοντέλων να ταξινομούν εικόνες που δεν χρησιμοποιήθηκαν ούτε για την ενημέρωση των παραμέτρων τους κατά την εκπαίδευση ούτε για την επιλογή του καλύτερου checkpoint κατά την επικύρωση.

Η αξιολόγηση αφορά τις έξι διαμορφώσεις που προέκυψαν από τις αρχιτεκτονικές ResNet18, EfficientNet-B0 και ConvNeXt-Tiny σε συνδυασμό με τις στρατηγικές Feature Extraction και Partial Fine-Tuning. Για όλες τις διαμορφώσεις χρησιμοποιείται το ίδιο test set, το οποίο αποτελείται από 62 εικόνες κατανεμημένες στις εννέα γεωγραφικές κατηγορίες του προβλήματος. Η χρήση του ίδιου συνόλου εικόνων παρέχει κοινή βάση για τη σύγκριση των μοντέλων.

Η απόδοση δεν εξετάζεται αποκλειστικά μέσω μιας συνολικής τιμής accuracy. Για πληρέστερη αποτίμηση αξιοποιούνται οι μετρικές precision, recall και F1-score, τόσο σε συνολικό επίπεδο όσο και ανά γεωγραφική κατηγορία. Παράλληλα, οι πίνακες σύγχυσης χρησιμοποιούνται για την εξέταση των σωστών και λανθασμένων ταξινομήσεων και, κυρίως, για τον εντοπισμό συγκεκριμένων γεωγραφικών κατηγοριών μεταξύ των οποίων εμφανίζονται συχνότερες συγχύσεις.

Ιδιαίτερη σημασία έχει επίσης η εξέταση των Top-1 και Top-3 προβλέψεων. Στην περίπτωση του Top-1, μία πρόβλεψη θεωρείται επιτυχής μόνο όταν η πραγματική κατηγορία αποτελεί την πρώτη επιλογή του μοντέλου. Αντίθετα, στο Top-3 εξετάζεται εάν η πραγματική περιοχή περιλαμβάνεται μεταξύ των τριών κατηγοριών με τις υψηλότερες προβλεπόμενες πιθανότητες. Η δεύτερη προσέγγιση έχει ιδιαίτερη πρακτική σημασία για τη διαδικτυακή εφαρμογή της εργασίας, καθώς η διεπαφή αναγνώρισης παρουσιάζει στον χρήστη τις τρεις πιθανότερες γεωγραφικές περιοχές.

Πέρα από τη συνολική απόδοση, η ανάλυση εξετάζει και τη συμπεριφορά των μοντέλων ανά κατηγορία. Η διάκριση αυτή είναι απαραίτητη, επειδή ένα μοντέλο μπορεί να εμφανίζει ικανοποιητική συνολική επίδοση, αλλά ταυτόχρονα να παρουσιάζει σημαντικές δυσκολίες στην αναγνώριση συγκεκριμένων περιοχών. Η εξέταση των επιμέρους αποτελεσμάτων επιτρέπει επομένως να εντοπιστούν τόσο οι κατηγορίες που αναγνωρίζονται με μεγαλύτερη συνέπεια όσο και εκείνες στις οποίες εμφανίζονται περισσότερα σφάλματα ταξινόμησης.

Στη συνέχεια πραγματοποιείται συγκριτική αποτίμηση των δύο στρατηγικών Μεταφοράς Μάθησης για καθεμία από τις τρεις αρχιτεκτονικές. Με τον τρόπο αυτό εξετάζεται κατά πόσο η προσαρμογή ενός μέρους των προεκπαιδευμένων επιπέδων μέσω Partial Fine-Tuning συνδέεται, στο συγκεκριμένο σύνολο δεδομένων και στις συγκεκριμένες πειραματικές ρυθμίσεις, με διαφορετική απόδοση σε σχέση με την εκπαίδευση μόνο του νέου ταξινομητή μέσω Feature Extraction. Η σύγκριση περιορίζεται στα αποτελέσματα των πειραμάτων της παρούσας εργασίας και δεν θεωρείται γενικό συμπέρασμα για τις συγκεκριμένες αρχιτεκτονικές ή στρατηγικές.

Τελικός στόχος του κεφαλαίου είναι η συνολική σύγκριση των έξι πειραματικών διαμορφώσεων και η τεκμηριωμένη επιλογή του μοντέλου που θα χρησιμοποιηθεί ως τελικό μοντέλο αναγνώρισης στη διαδικτυακή εφαρμογή. Για τον λόγο αυτό, η επιλογή δεν βασίζεται μόνο σε μία μεμονωμένη μετρική, αλλά λαμβάνει υπόψη τη συνολική επίδοση, τη συμπεριφορά ανά κατηγορία, τα πρότυπα σφαλμάτων και την αποτελεσματικότητα των Top-3 προβλέψεων.

6.2 ΠΛΑΙΣΙΟ ΚΑΙ ΜΕΤΡΙΚΕΣ ΑΞΙΟΛΟΓΗΣΗΣ

Η τελική αξιολόγηση των έξι πειραματικών διαμορφώσεων πραγματοποιείται στο test set, το οποίο παρέμεινε ανεξάρτητο από τη διαδικασία εκπαίδευσης και επικύρωσης. Όπως παρουσιάστηκε στο Κεφάλαιο 3, το συγκεκριμένο σύνολο αποτελείται από 62 εικόνες και περιλαμβάνει δείγματα και από τις εννέα γεωγραφικές κατηγορίες.

Για κάθε πείραμα χρησιμοποιείται το καλύτερο checkpoint που επιλέχθηκε με βάση το validation loss κατά τη διαδικασία που περιγράφηκε στην Ενότητα 5.5.

Η αξιολόγηση βασίζεται αρχικά στην Top-1 πρόβλεψη, δηλαδή στην κατηγορία στην οποία το μοντέλο αποδίδει τη μεγαλύτερη πιθανότητα. Οι προβλέψεις αυτές συγκρίνονται με τις πραγματικές ετικέτες του test set και χρησιμοποιούνται για τον υπολογισμό των βασικών μετρικών ταξινόμησης. Συγκεκριμένα, εξετάζονται οι Accuracy, Precision, Recall και F1-score, οι οποίες έχουν παρουσιαστεί θεωρητικά στην Ενότητα 2.9 και επομένως δεν αναλύονται εκ νέου ως προς τους μαθηματικούς ορισμούς τους.

Η Accuracy εκφράζει το ποσοστό των εικόνων του test set που ταξινομήθηκαν σωστά στην πρώτη επιλογή του μοντέλου. Ωστόσο, επειδή η συνολική Accuracy δεν αποκαλύπτει τον τρόπο με τον οποίο κατανέμεται η απόδοση μεταξύ των εννέα κατηγοριών, η αξιολόγηση συμπληρώνεται με Precision, Recall και F1-score. Οι μετρικές αυτές επιτρέπουν να εξεταστεί αναλυτικότερα εάν ένα μοντέλο εμφανίζει διαφορετική συμπεριφορά μεταξύ των γεωγραφικών περιοχών.

Για τη συνοπτική αποτίμηση της απόδοσης σε επίπεδο πολλαπλών κλάσεων χρησιμοποιούνται επίσης οι macro και weighted εκδοχές των μετρικών. Στο macro average κάθε κατηγορία συνεισφέρει ισότιμα στον τελικό μέσο όρο, ανεξάρτητα από το πλήθος των δειγμάτων της, γεγονός που το καθιστά χρήσιμο για την εξέταση της ισορροπίας της απόδοσης μεταξύ των εννέα περιοχών. Αντίθετα, στο weighted average η συνεισφορά κάθε κατηγορίας σταθμίζεται σύμφωνα με τον αριθμό των πραγματικών δειγμάτων της στο test set.

Εκτός από τις αριθμητικές μετρικές, για κάθε πειραματική διαμόρφωση χρησιμοποιείται πίνακας σύγχυσης (confusion matrix). Οι γραμμές του πίνακα αντιστοιχούν στις πραγματικές κατηγορίες και οι στήλες στις κατηγορίες που προβλέπει το μοντέλο. Οι τιμές της κύριας διαγωνίου αντιπροσωπεύουν τις σωστές ταξινομήσεις, ενώ οι τιμές εκτός αυτής αποτυπώνουν τα σφάλματα. Η συγκεκριμένη αναπαράσταση είναι ιδιαίτερα χρήσιμη στην παρούσα εργασία, επειδή επιτρέπει να προσδιοριστεί όχι μόνο πόσες εικόνες ταξινομήθηκαν λανθασμένα, αλλά και προς ποια γεωγραφική περιοχή κατευθύνονται συχνότερα οι λανθασμένες προβλέψεις.

Παράλληλα με την Top-1 αξιολόγηση εξετάζεται και η Top-3 Accuracy. Για κάθε εικόνα ταξινομούνται οι πιθανότητες εξόδου του μοντέλου και επιλέγονται οι τρεις κατηγορίες με τις υψηλότερες τιμές. Μία πρόβλεψη θεωρείται επιτυχής ως προς το Top-3 όταν η πραγματική γεωγραφική κατηγορία βρίσκεται σε οποιαδήποτε από αυτές τις τρεις θέσεις. Επομένως, η Top-3 Accuracy υπολογίζεται ως:

$$Top-3\ Accuracy = \frac{Πλήθος\ δειγμάτων\ όπου\ η\ πραγματική\ κλάση\ βρίσκεται\ στις\ 3\ πρώτες\ προβλέψεις}{Συνολικό\ πλήθος\ δειγμάτων}$$

Η Top-3 αξιολόγηση δεν αντικαθιστά την Top-1 Accuracy και οι δύο μετρικές αποτυπώνουν διαφορετικές πλευρές της συμπεριφοράς του μοντέλου. Η Top-1 αποτελεί αυστηρότερο κριτήριο ταξινόμησης, καθώς απαιτεί η πραγματική περιοχή να αποτελεί την πρώτη επιλογή. Η Top-3 εξετάζει επιπλέον εάν το μοντέλο έχει τοποθετήσει τη σωστή περιοχή μεταξύ των επικρατέστερων υποψηφίων, ακόμη και όταν κάποια άλλη κατηγορία έχει λάβει την υψηλότερη πιθανότητα.

Η διάκριση αυτή συνδέεται άμεσα με τον τρόπο λειτουργίας της διαδικτυακής εφαρμογής. Αντί να εμφανίζεται αποκλειστικά μία περιοχή ως μοναδικό αποτέλεσμα, παρουσιάζονται οι τρεις πιθανότερες γεωγραφικές περιοχές με σειρά κατάταξης. Για τον λόγο αυτό, η Top-3 Accuracy αποτελεί συμπληρωματική μετρική με πρακτική σημασία για το συγκεκριμένο σύστημα, χωρίς όμως να αναιρεί τη σημασία των υπόλοιπων μετρικών και της αναλυτικής εξέτασης των σφαλμάτων.

Το κοινό πλαίσιο αξιολόγησης εφαρμόζεται με τον ίδιο τρόπο και στις έξι πειραματικές διαμορφώσεις. Έτσι, τα αποτελέσματα που παρουσιάζονται στις επόμενες ενότητες μπορούν να συγκριθούν στην ίδια βάση. Αρχικά εξετάζονται οι συνολικές επιδόσεις των έξι πειραμάτων και στη συνέχεια η ανάλυση επεκτείνεται στις επιμέρους αρχιτεκτονικές, στις στρατηγικές Μεταφοράς Μάθησης, στην απόδοση ανά γεωγραφική κατηγορία και στα συγκεκριμένα πρότυπα σφαλμάτων που εμφανίζονται κατά την ταξινόμηση.

6.3 ΣΥΝΟΛΙΚΑ ΑΠΟΤΕΛΕΣΜΑΤΑ ΤΩΝ ΈΞΙ ΠΕΙΡΑΜΑΤΩΝ

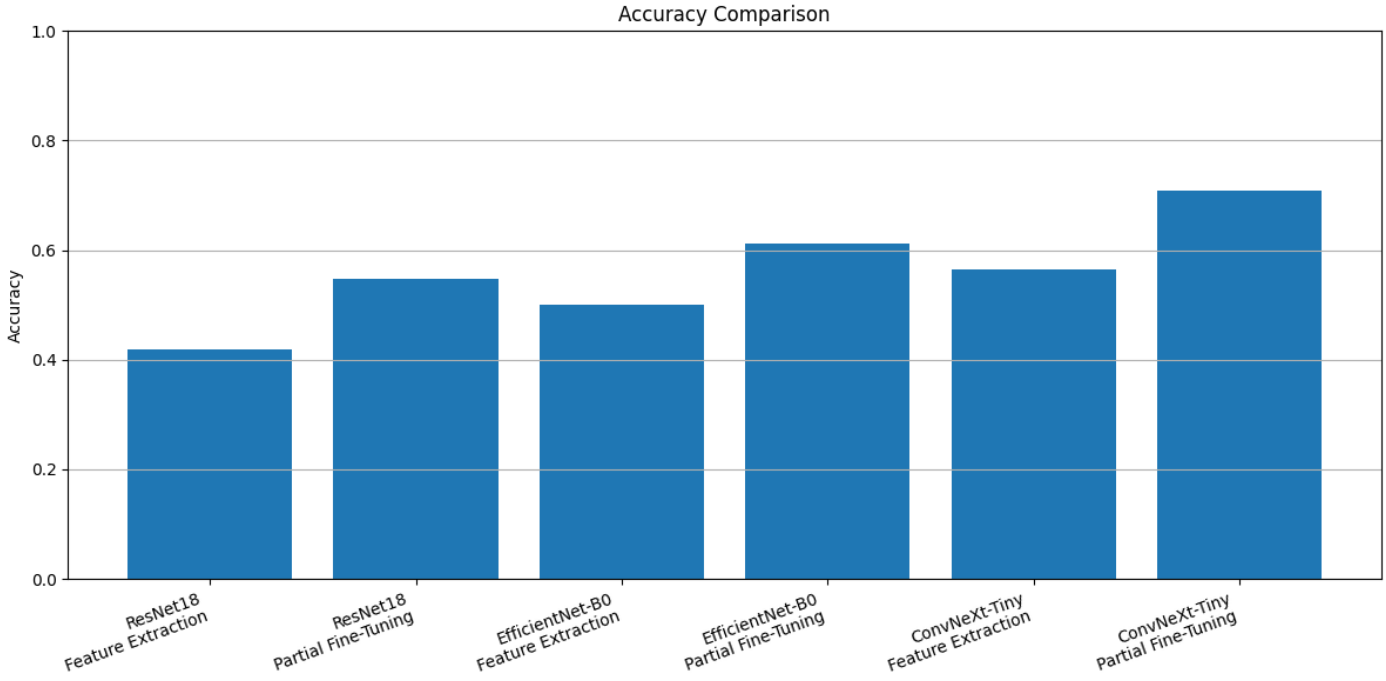
Η εφαρμογή του κοινού πλαισίου αξιολόγησης στο test set παρήγαγε τα συνολικά αποτελέσματα των έξι πειραματικών διαμορφώσεων. Στον Πίνακα 6.1 παρουσιάζονται συγκεντρωτικά οι τιμές Accuracy, Top-3 Accuracy, Macro Precision, Macro Recall, Macro F1-score και Weighted F1-score. Οι τιμές προκύπτουν από τις 62 εικόνες του ίδιου test set, γεγονός που επιτρέπει την άμεση σύγκριση των πειραμάτων.

Αρχιτεκτονική	Στρατηγική	Accuracy	Top-3 Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
ResNet18	Feature Extraction	41,94%	87,10%	49,44%	41,67%	40,92%	41,20%
ResNet18	Partial Fine-Tuning	54,84%	87,10%	57,65%	53,97%	54,02%	54,62%
EfficientNet-Bo	Feature Extraction	50,00%	85,48%	58,52%	48,15%	48,57%	49,38%
EfficientNet-Bo	Partial Fine-Tuning	61,29%	91,94%	67,14%	60,38%	60,72%	61,30%
ConvNeXt-Tiny	Feature Extraction	56,45%	91,94%	59,63%	55,56%	56,59%	57,17%
ConvNeXt-Tiny	Partial Fine-Tuning	**70,97%**	**91,94%**	**73,06%**	**70,83%**	**70,64%**	**70,56%**

Πίνακας 6.1: Συγκεντρωτικά αποτελέσματα των έξι πειραματικών διαμορφώσεων στο test set

Τα αποτελέσματα παρουσιάζουν σαφή διαφοροποίηση μεταξύ των έξι πειραματικών διαμορφώσεων. Η χαμηλότερη Top-1 Accuracy καταγράφεται από το ResNet18 με Feature Extraction, με 41,94%, ενώ η υψηλότερη από το ConvNeXt-Tiny με Partial Fine-Tuning, με 70,97%. Η διαφορά μεταξύ των δύο ακραίων περιπτώσεων ανέρχεται σε 29,03 ποσοστιαίες μονάδες, γεγονός που δείχνει ότι η επιλογή τόσο της αρχιτεκτονικής όσο και της στρατηγικής προσαρμογής συνδέεται με ουσιαστικές διαφοροποιήσεις στην απόδοση για το συγκεκριμένο πρόβλημα.

Για την οπτικότερη παρουσίαση της συγκεκριμένης διαφοράς, στο Σχήμα 6.1 απεικονίζεται η Accuracy των έξι πειραμάτων. Η απεικόνιση καθιστά εμφανή τόσο τη διαφοροποίηση μεταξύ των τριών αρχιτεκτονικών όσο και τη μεταβολή της επίδοσης όταν το Feature Extraction αντικαθίσταται από Partial Fine-Tuning.



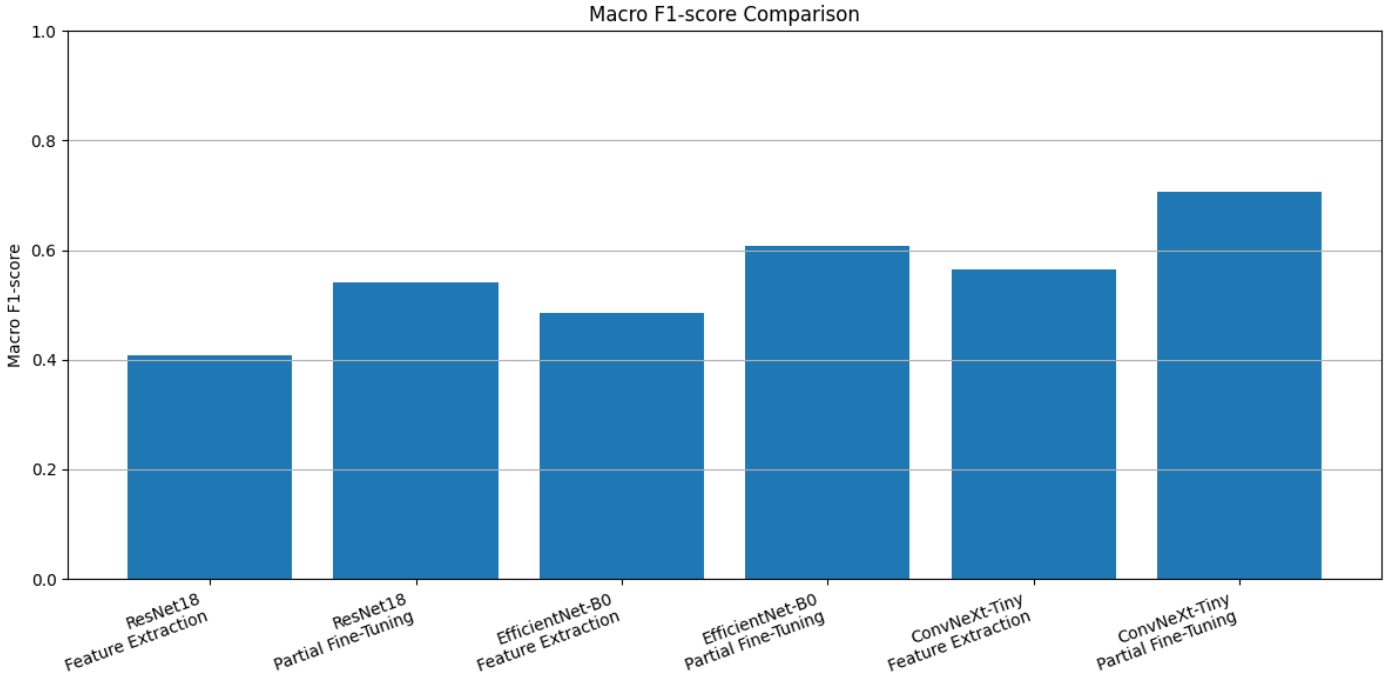
Εικόνα 6.1: Σύγκριση της Accuracy των έξι πειραματικών διαμορφώσεων στο test set.

Σε καθεμία από τις τρεις αρχιτεκτονικές, η διαμόρφωση Partial Fine-Tuning επιτυγχάνει υψηλότερη Accuracy από την αντίστοιχη διαμόρφωση Feature Extraction. Στο ResNet18 η Accuracy αυξάνεται από 41,94% σε 54,84%, δηλαδή κατά 12,90 ποσοστιαίες μονάδες. Στο EfficientNet-Bo αυξάνεται από 50,00% σε 61,29%, με διαφορά 11,29 μονάδων, ενώ στο ConvNeXt-Tiny αυξάνεται από 56,45% σε 70,97%, με διαφορά 14,52 μονάδων.

Στις συνθήκες των συγκεκριμένων πειραμάτων, επομένως, η μερική προσαρμογή των προεκπαιδευμένων παραμέτρων συνδέθηκε με υψηλότερη Top-1 επίδοση και στις τρεις αρχιτεκτονικές.

Η Accuracy, ωστόσο, δεν αποτελεί τη μοναδική βάση σύγκρισης. Ιδιαίτερη σημασία έχει το Macro F1-score, καθώς υπολογίζει την επίδοση κάθε κατηγορίας ισότιμα πριν από τον υπολογισμό του μέσου όρου. Οι τιμές του ακολουθούν παρόμοια τάση με την Accuracy. Το ResNet18 με Feature Extraction παρουσιάζει Macro F1 40,92%, το EfficientNet-B0 με Feature Extraction 48,57% και το ConvNeXt-Tiny με Feature Extraction 56,59%. Στις αντίστοιχες διαμορφώσεις Partial Fine-Tuning οι τιμές αυξάνονται σε 54,02%, 60,72% και 70,64%.

Η σύγκριση του Macro F1-score παρουσιάζεται γραφικά στο Σχήμα 6.2. Το διάγραμμα συμπληρώνει την εικόνα που παρέχει η Accuracy, καθώς επιτρέπει να εξεταστεί εάν η κατάταξη των μοντέλων διατηρείται και όταν οι εννέα κατηγορίες συμμετέχουν ισότιμα στη συνολική μετρική.



Εικόνα 6.2: Σύγκριση του Macro F1-score των έξι πειραματικών διαμορφώσεων στο test set.

Η κατάταξη που προκύπτει από το Macro F1 είναι συμβατή με τη γενικότερη εικόνα της Accuracy. Το ConvNeXt-Tiny με Partial Fine-Tuning καταγράφει την υψηλότερη τιμή, 70,64%, ενώ η αντίστοιχη Accuracy είναι 70,97%. Επιπλέον, το ίδιο πείραμα παρουσιάζει Macro Precision 73,06% και Macro Recall 70,83%. Συνεπώς, η υψηλότερη Accuracy της συγκεκριμένης διαμόρφωσης συνοδεύεται και από την υψηλότερη συνολική επίδοση των βασικών macro μετρικών.

Οι τιμές Weighted F1 ακολουθούν επίσης σε γενικές γραμμές την ίδια κατάταξη. Το ResNet18 με Feature Extraction καταγράφει 41,20%, το EfficientNet-B0 με Feature Extraction 49,38% και το ConvNeXt-Tiny με Feature Extraction 57,17%. Με Partial Fine-Tuning οι αντίστοιχες τιμές διαμορφώνονται σε 54,62%, 61,30% και 70,56%. Η σχετικά μικρή απόσταση μεταξύ Macro F1 και Weighted F1 σε κάθε πείραμα δείχνει ότι η βασική εικόνα της συγκριτικής επίδοσης δεν μεταβάλλεται σημαντικά όταν η συνεισφορά κάθε κατηγορίας σταθμίζεται με βάση το πλήθος των δειγμάτων της.

Ιδιαίτερο ενδιαφέρον παρουσιάζουν και οι τιμές Top-3 Accuracy, οι οποίες είναι σημαντικά υψηλότερες από τις αντίστοιχες Top-1 τιμές. Το ResNet18 επιτυγχάνει Top-3 Accuracy 87,10% τόσο με Feature Extraction όσο και με Partial Fine-Tuning. Το EfficientNet-B0 επιτυγχάνει 85,48% με Feature Extraction και 91,94% με Partial Fine-Tuning, ενώ το ConvNeXt-Tiny καταγράφει 91,94% τόσο με Feature Extraction όσο και με Partial Fine-Tuning. Επομένως, η υψηλότερη Top-3 Accuracy μεταξύ των έξι πειραμάτων είναι 91,94% και επιτυγχάνεται από τρεις πειραματικές διαμορφώσεις: EfficientNet-B0 με Partial Fine-Tuning, ConvNeXt-Tiny με Feature Extraction και ConvNeXt-Tiny με Partial Fine-Tuning.

Η διαφορά μεταξύ Top-1 και Top-3 δείχνει ότι ένα μέρος των λανθασμένων πρώτων επιλογών δεν αντιστοιχεί απαραίτητα σε πλήρη αποτυχία εντοπισμού της σωστής περιοχής. Στο ConvNeXt-Tiny με Partial Fine-Tuning, για παράδειγμα, η Top-1 Accuracy είναι 70,97%, ενώ η Top-3 Accuracy φτάνει το 91,94%.

Επομένως, σε αρκετές από τις περιπτώσεις όπου η πραγματική περιοχή δεν κατατάσσεται πρώτη, εξακολουθεί να περιλαμβάνεται στη δεύτερη ή στην τρίτη θέση. Η συγκεκριμένη συμπεριφορά θα εξεταστεί αναλυτικότερα στην Ενότητα 6.7, καθώς συνδέεται άμεσα με την επιλογή παρουσίασης τριών πιθανότερων περιοχών στη διαδικτυακή εφαρμογή.

Συνολικά, το ConvNeXt-Tiny με Partial Fine-Tuning καταγράφει την υψηλότερη Accuracy, Top-3 Accuracy, Macro Precision, Macro Recall, Macro F1-score και Weighted F1-score μεταξύ των έξι πειραματικών διαμορφώσεων. Παράλληλα, και στις τρεις αρχιτεκτονικές το Partial Fine-Tuning παρουσιάζει υψηλότερη Top-1 Accuracy και Macro F1 από το αντίστοιχο Feature Extraction. Τα συγκεντρωτικά αυτά αποτελέσματα αποτελούν την πρώτη ένδειξη για την καταλληλότερη πειραματική διαμόρφωση, χωρίς όμως να αρκούν για την τελική επιλογή. Για τον λόγο αυτό, στη συνέχεια εξετάζονται αναλυτικότερα οι διαφορές μεταξύ των αρχιτεκτονικών και των δύο στρατηγικών Μεταφοράς Μάθησης, πριν η αξιολόγηση προχωρήσει στη συμπεριφορά ανά γεωγραφική κατηγορία και στα επιμέρους σφάλματα ταξινόμησης.

6.4 ΑΠΟΤΕΛΕΣΜΑΤΑ ΑΝΑ ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΣΤΡΑΤΗΓΙΚΗ ΜΕΤΑΦΟΡΑΣ ΜΑΘΗΣΗΣ

Η συγκεντρωτική αξιολόγηση της προηγούμενης ενότητας παρουσίασε τη συνολική κατάταξη των έξι πειραματικών διαμορφώσεων. Για την πληρέστερη ερμηνεία των αποτελεσμάτων είναι χρήσιμο κάθε αρχιτεκτονική να εξεταστεί ως ζεύγος πειραμάτων, συγκρίνοντας το Feature Extraction με το Partial Fine-Tuning. Με αυτόν τον τρόπο η σύγκριση δεν περιορίζεται αποκλειστικά στις τελικές μετρικές του test set, αλλά λαμβάνει υπόψη και τη συμπεριφορά που παρουσίασε κάθε διαμόρφωση κατά τη διαδικασία επικύρωσης.

Μία βασική διαφορά μεταξύ των δύο στρατηγικών αφορά το πλήθος των παραμέτρων που συμμετέχουν στην εκπαίδευση. Στο Feature Extraction το μεγαλύτερο μέρος του προεκπαιδευμένου δικτύου παραμένει παγωμένο και εκπαιδεύεται ο νέος ταξινομητής, ενώ στο Partial Fine-Tuning ενεργοποιούνται επιπλέον επιλεγμένα βαθύτερα τμήματα του δικτύου, όπως παρουσιάστηκε στο Κεφάλαιο 4. Η διαφοροποίηση αυτή αυξάνει σημαντικά το πλήθος των εκπαιδεύσιμων παραμέτρων και παρέχει στο μοντέλο μεγαλύτερη δυνατότητα προσαρμογής στα χαρακτηριστικά του συγκεκριμένου συνόλου δεδομένων.

Αρχιτεκτονική	Στρατηγική	Εκπαιδεύσιμες παράμετροι	Epochs που εκτελέστηκαν	Καλύτερο epoch	Καλύτερο Validation Loss	Καλύτερη Validation Accuracy
ResNet18	Feature Extraction	4.617	25	25	1,4273	47,62%
ResNet18	Partial Fine-Tuning	8.398.345	12	7	1,2738	60,32%
EfficientNet-B0	Feature Extraction	11.529	18	13	1,6637	36,51%
EfficientNet-B0	Partial Fine-Tuning	1.140.921	25	23	1,4847	46,03%
ConvNeXt-Tiny	Feature Extraction	6.921	25	24	1,1497	58,73%
ConvNeXt-Tiny	**Partial Fine-Tuning**	**14.297.865**	**21**	**16**	**1,0733**	**65,08%**

Πίνακας 6.2: Σύγκριση της διαδικασίας εκπαίδευσης και επικύρωσης των έξι πειραματικών διαμορφώσεων

Στην περίπτωση του ResNet18, η μετάβαση από Feature Extraction σε Partial Fine-Tuning συνοδεύεται από σημαντική αύξηση του αριθμού των εκπαιδεύσιμων παραμέτρων, από 4.617 σε 8.398.345. Παράλληλα, το καλύτερο validation loss μειώνεται από 1,4273 σε 1,2738, ενώ η καλύτερη validation accuracy αυξάνεται από 47,62% σε 60,32%. Το Feature Extraction έφτασε στο μέγιστο όριο των 25 epochs και το καλύτερο checkpoint καταγράφηκε στο τελευταίο epoch, ενώ στο Partial Fine-Tuning το καλύτερο checkpoint εμφανίστηκε στο 7ο epoch και η εκπαίδευση ολοκληρώθηκε μετά από 12 epochs μέσω του Early Stopping. Η βελτιωμένη συμπεριφορά στο validation set συνοδεύεται και από υψηλότερη τελική επίδοση στο test set, όπου η Accuracy αυξήθηκε από 41,94% σε 54,84%.

Στο EfficientNet-B0 παρατηρείται αντίστοιχη τάση. Το Feature Extraction περιλαμβάνει 11.529 εκπαιδεύσιμες παραμέτρους, ενώ με το Partial Fine-Tuning ο αριθμός αυξάνεται σε 1.140.921. Στην πρώτη περίπτωση το καλύτερο validation loss ήταν 1,6637 και η καλύτερη validation accuracy 36,51%, ενώ στη δεύτερη οι αντίστοιχες τιμές διαμορφώθηκαν σε 1,4847 και 46,03%. Το Feature Extraction ολοκλήρωσε 18 epochs και το καλύτερο checkpoint καταγράφηκε στο 130, ενώ το Partial Fine-Tuning εκτέλεσε 25 epochs και πέτυχε το καλύτερο αποτέλεσμα επικύρωσης στο 230 epoch. Η ίδια κατεύθυνση παρατηρείται και στο ανεξάρτητο test set, όπου η Accuracy αυξήθηκε από 50,00% σε 61,29%.

Ιδιαίτερο ενδιαφέρον παρουσιάζει το ConvNeXt-Tiny, καθώς ήδη με Feature Extraction είχε εμφανίσει την υψηλότερη τελική Accuracy μεταξύ των τριών αντίστοιχων διαμορφώσεων. Στην περίπτωση αυτή εκπαιδεύονται 6.921 παράμετροι και το καλύτερο checkpoint καταγράφηκε στο 240 epoch, με validation loss 1,1497 και validation accuracy 58,73%. Με την εφαρμογή Partial Fine-Tuning, οι εκπαιδεύσιμες παράμετροι αυξάνονται σε 14.297.865 και το καλύτερο checkpoint καταγράφεται στο 160 epoch. Το validation loss του συγκεκριμένου checkpoint είναι 1,0733 και η validation accuracy 65,08%, παρουσιάζοντας βελτίωση και στις δύο μετρικές σε σχέση με το Feature Extraction.

Η βελτίωση του ConvNeXt-Tiny κατά την επικύρωση συνοδεύεται και από την ισχυρότερη τελική Top-1 επίδοση μεταξύ των έξι πειραμάτων. Συγκεκριμένα, η Accuracy στο test set αυξάνεται από 56,45% με Feature Extraction σε 70,97% με Partial Fine-Tuning. Το Macro F1-score αυξάνεται αντίστοιχα από 56,59% σε 70,64%. Επομένως, για το συγκεκριμένο σύνολο δεδομένων, η δυνατότητα προσαρμογής του επιλεγμένου βαθύτερου τμήματος του ConvNeXt-Tiny συνδέθηκε με ουσιαστική βελτίωση της ικανότητας ταξινόμησης σε σχέση με τη χρήση του ίδιου δικτύου αποκλειστικά ως εξαγωγέα χαρακτηριστικών.

Εξετάζοντας συνολικά τα τρία ζεύγη πειραμάτων, προκύπτει μία συνεπής τάση υπέρ του Partial Fine-Tuning στις τελικές Top-1 επιδόσεις. Η βελτίωση της Accuracy είναι 12,90 ποσοστιαίες μονάδες για το ResNet18, 11,29 μονάδες για το EfficientNet-B0 και 14,52 μονάδες για το ConvNeXt-Tiny. Αντίστοιχη εικόνα παρουσιάζει και το Macro F1-score, γεγονός που δείχνει ότι η βελτίωση δεν περιορίζεται μόνο στο συνολικό ποσοστό σωστών προβλέψεων.

Η αύξηση των εκπαιδεύσιμων παραμέτρων στο Partial Fine-Tuning προσφέρει μεγαλύτερη ευελιξία προσαρμογής των προεκπαιδευμένων αναπαραστάσεων στα ιδιαίτερα οπτικά χαρακτηριστικά των ελληνικών παραδοσιακών φορεσιών. Ταυτόχρονα, όμως, αυξάνει το τμήμα του δικτύου που μεταβάλλεται κατά την εκπαίδευση και συνεπώς απαιτεί προσεκτικότερη βελτιστοποίηση. Για τον λόγο αυτό, στις συγκεκριμένες διαμορφώσεις χρησιμοποιήθηκε χαμηλότερος ρυθμός μάθησης, όπως παρουσιάστηκε στο Κεφάλαιο 5, ενώ η επιλογή του καλύτερου checkpoint βασίστηκε στο validation loss και εφαρμόστηκε Early Stopping για τον περιορισμό της συνέχισης της εκπαίδευσης χωρίς περαιτέρω βελτίωση.

Τα αποτελέσματα αυτά δεν υποδεικνύουν ότι το Partial Fine-Tuning αποτελεί καθολικά ανώτερη στρατηγική για οποιοδήποτε πρόβλημα ταξινόμησης εικόνων. Τεκμηριώνουν, ωστόσο, ότι στις πειραματικές συνθήκες της παρούσας εργασίας η συγκεκριμένη στρατηγική απέδωσε υψηλότερη Accuracy και Macro F1 και στις τρεις αρχιτεκτονικές. Παράλληλα, το ConvNeXt-Tiny παρουσίασε την ισχυρότερη συμπεριφορά τόσο στην εκδοχή Feature Extraction όσο και στην εκδοχή Partial Fine-Tuning, με τη δεύτερη να επιτυγχάνει την υψηλότερη συνολική Top-1 επίδοση.

Η συνολική σύγκριση των αρχιτεκτονικών και των στρατηγικών Μεταφοράς Μάθησης δεν αποκαλύπτει, ωστόσο, εάν η απόδοση κατανέμεται ομοιόμορφα στις εννέα γεωγραφικές κατηγορίες. Δύο μοντέλα με παρόμοια συνολική επίδοση μπορούν να παρουσιάζουν σημαντικά διαφορετική συμπεριφορά σε συγκεκριμένες περιοχές. Για τον λόγο αυτό, στην επόμενη ενότητα η αξιολόγηση μεταφέρεται από τις συνολικές μετρικές στην επίδοση ανά γεωγραφική κατηγορία.

6.5 ΑΞΙΟΛΟΓΗΣΗ ΑΝΑ ΓΕΩΓΡΑΦΙΚΗ ΚΑΤΗΓΟΡΙΑ

Η συνολική Accuracy παρέχει μία γενική εικόνα της απόδοσης ενός μοντέλου, χωρίς όμως να αποτυπώνει τον βαθμό στον οποίο η επίδοση κατανέμεται μεταξύ των επιμέρους κατηγοριών. Για τον λόγο αυτό, η αξιολόγηση του ConvNeXt-Tiny με Partial Fine-Tuning, το οποίο κατέγραψε την υψηλότερη Top-1 Accuracy μεταξύ των έξι πειραματικών διαμορφώσεων, εξετάζεται στη συνέχεια σε επίπεδο γεωγραφικής κατηγορίας. Η ανάλυση βασίζεται στις τιμές Precision, Recall και F1-score για καθεμία από τις εννέα περιοχές του test set.

Γεωγραφική κατηγορία	Precision	Recall	F1-score	Support
Ήπειρος	37,50%	42,86%	40,00%	7
Κρήτη	100,00%	83,33%	90,91%	6
Μακεδονία	66,67%	50,00%	57,14%	8
Νησιά Αιγαίου	77,78%	87,50%	82,35%	8
Νησιά Ιονίου	71,43%	83,33%	76,92%	6
Πελοπόννησος	100,00%	57,14%	72,73%	7
Στερεά Ελλάδα	60,00%	50,00%	54,55%	6
Θεσσαλία	71,43%	83,33%	76,92%	6
Θράκη	72,73%	100,00%	84,21%	8

Πίνακας 6.3: Αποτελέσματα του ConvNeXt-Tiny με Partial Fine-Tuning ανά γεωγραφική κατηγορία στο test set.

Η υψηλότερη τιμή F1-score καταγράφεται στην Κρήτη, με 90,91%. Η κατηγορία παρουσιάζει Precision 100,00% και Recall 83,33%. Η τιμή Precision 100% σημαίνει ότι όλες οι εικόνες που ταξινομήθηκαν από το μοντέλο ως Κρήτη ανήκαν πράγματι στη συγκεκριμένη κατηγορία. Το Recall 83,33% δείχνει παράλληλα ότι αναγνωρίστηκαν σωστά πέντε από τις έξι εικόνες της Κρήτης που περιλαμβάνονταν στο test set.

Ιδιαίτερα υψηλή επίδοση παρουσιάζει και η Θράκη, με F1-score 84,21%. Το Recall της κατηγορίας φτάνει το 100,00%, γεγονός που σημαίνει ότι και οι οκτώ εικόνες της Θράκης στο test set αναγνωρίστηκαν σωστά ως Θράκη. Το Precision είναι χαμηλότερο, 72,73%, γεγονός που υποδηλώνει ότι υπήρξαν επιπλέον εικόνες άλλων περιοχών οι οποίες ταξινομήθηκαν λανθασμένα στη συγκεκριμένη κατηγορία.

Τα Νησιά Αιγαίου αποτελούν επίσης μία από τις ισχυρότερες κατηγορίες, με Precision 77,78%, Recall 87,50% και F1-score 82,35%. Από τις οκτώ πραγματικές εικόνες της κατηγορίας, οι επτά αναγνωρίστηκαν σωστά. Αντίστοιχα θετική εικόνα παρουσιάζουν τα Νησιά Ιονίου και η Θεσσαλία, οι οποίες καταγράφουν το ίδιο F1-score, 76,92%, και Recall 83,33%. Επομένως, στις συγκεκριμένες κατηγορίες το μοντέλο κατορθώνει να αναγνωρίσει σωστά το μεγαλύτερο μέρος των πραγματικών δειγμάτων του test set.

Η Πελοπόννησος παρουσιάζει διαφορετική συμπεριφορά. Το Precision φτάνει το 100,00%, ενώ το Recall περιορίζεται στο 57,14%, με αποτέλεσμα F1-score 72,73%. Αυτό σημαίνει ότι όταν το μοντέλο προβλέπει την Πελοπόννησο η πρόβλεψη είναι σωστή στα συγκεκριμένα δεδομένα του test set, αλλά δεν εντοπίζει όλες τις πραγματικές εικόνες της κατηγορίας. Συγκεκριμένα, τέσσερις από τις επτά εικόνες της Πελοποννήσου αναγνωρίστηκαν σωστά, ενώ οι υπόλοιπες τρεις αποδόθηκαν σε διαφορετικές περιοχές.

Χαμηλότερη επίδοση εμφανίζουν η Μακεδονία και η Στερεά Ελλάδα. Η Μακεδονία παρουσιάζει Precision 66,67%, Recall 50,00% και F1-score 57,14%, γεγονός που αντιστοιχεί σε σωστή αναγνώριση τεσσάρων από τις οκτώ πραγματικές εικόνες της. Η Στερεά Ελλάδα παρουσιάζει Precision 60,00%, Recall 50,00% και F1-score 54,55%, με τρεις από τις έξι εικόνες της να αναγνωρίζονται σωστά. Οι τιμές αυτές υποδεικνύουν ότι οι δύο κατηγορίες αποτελούν δυσκολότερες περιπτώσεις για το τελικό μοντέλο σε σχέση με τις κατηγορίες που παρουσιάζουν υψηλότερο Recall και F1-score.

Η μεγαλύτερη δυσκολία παρατηρείται στην Ήπειρο. Η συγκεκριμένη κατηγορία εμφανίζει Precision 37,50%, Recall 42,86% και F1-score 40,00%, που αποτελεί τη χαμηλότερη τιμή F1-score μεταξύ των εννέα περιοχών. Από τις επτά εικόνες της Ηπείρου στο test set, μόνο τρεις ταξινομήθηκαν σωστά.

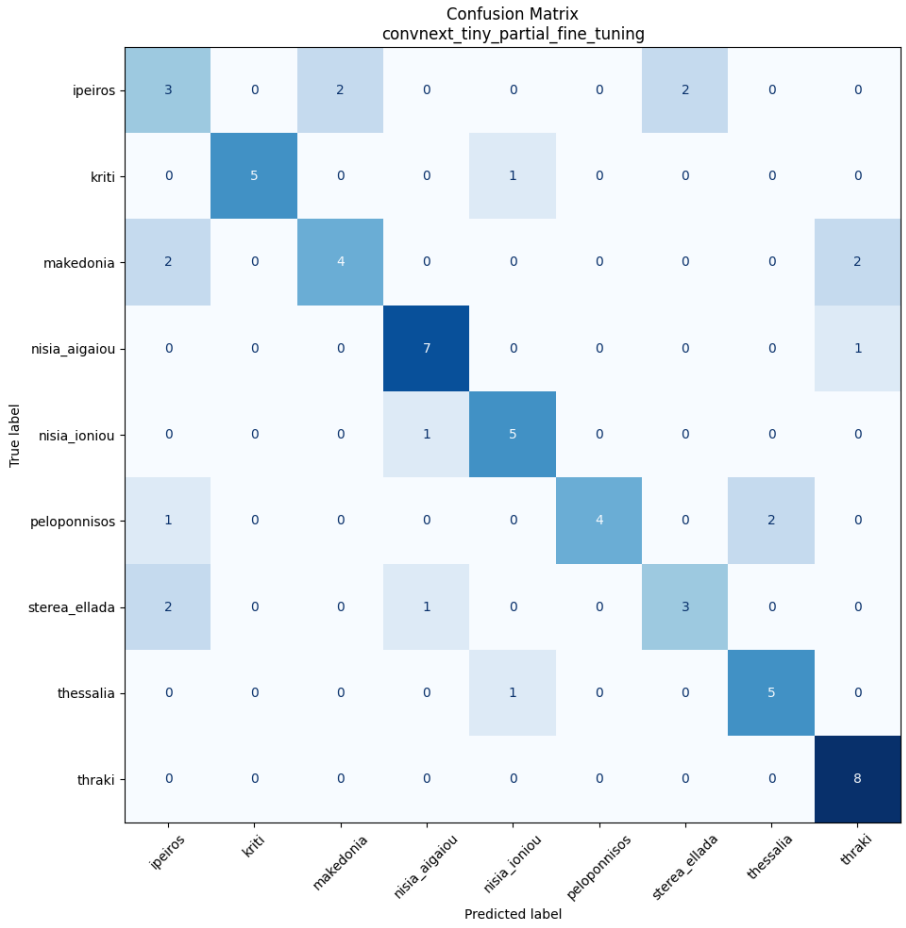
Παράλληλα, το χαμηλό Precision δείχνει ότι το μοντέλο απέδωσε στην Ήπειρο και εικόνες που στην πραγματικότητα ανήκαν σε άλλες γεωγραφικές κατηγορίες.

Η διαφορά μεταξύ της καλύτερης και της δυσκολότερης κατηγορίας είναι σημαντική. Το F1-score κυμαίνεται από 40,00% για την Ήπειρο έως 90,91% για την Κρήτη, μία απόσταση 50,91 ποσοστιαίων μονάδων. Επομένως, η συνολική Accuracy 70,97% δεν πρέπει να ερμηνεύεται ως ομοιόμορφη επίδοση σε όλες τις περιοχές. Το μοντέλο εμφανίζει πολύ ισχυρή συμπεριφορά σε ορισμένες κατηγορίες, όπως η Κρήτη, η Θράκη και τα Νησιά Αιγαίου, αλλά σημαντικά μεγαλύτερη δυσκολία στην Ήπειρο, στη Στερεά Ελλάδα και στη Μακεδονία.

Οι διαφοροποιήσεις αυτές είναι ιδιαίτερα σημαντικές για την αξιολόγηση της εφαρμογής, επειδή αποκαλύπτουν περιπτώσεις στις οποίες μία συνολική μετρική μπορεί να αποκρύπτει επιμέρους αδυναμίες. Ωστόσο, οι τιμές Precision, Recall και F1-score από μόνες τους δεν δείχνουν με ποιες συγκεκριμένες περιοχές συγχέεται κάθε κατηγορία. Για να προσδιοριστούν οι κατευθύνσεις αυτών των λανθασμένων ταξινομήσεων απαιτείται η εξέταση του πίνακα σύγχυσης και των επιμέρους σφαλμάτων, η οποία πραγματοποιείται στην επόμενη ενότητα.

6.6 ΑΝΑΛΥΣΗ ΠΙΝΑΚΑ ΣΥΓΧΥΣΗΣ ΚΑΙ ΣΦΑΛΜΑΤΩΝ ΤΑΞΙΝΟΜΗΣΗΣ

Η αξιολόγηση ανά γεωγραφική κατηγορία έδειξε ότι η απόδοση του τελικού ConvNeXt-Tiny με Partial Fine-Tuning διαφοροποιείται σημαντικά μεταξύ των εννέα περιοχών. Για την ακριβέστερη διερεύνηση των σφαλμάτων χρησιμοποιείται ο πίνακας σύγχυσης (confusion matrix), ο οποίος επιτρέπει να προσδιοριστεί όχι μόνο ο αριθμός των λανθασμένων ταξινομήσεων, αλλά και οι συγκεκριμένες κατηγορίες μεταξύ των οποίων πραγματοποιούνται οι συγχύσεις. Στον πίνακα οι γραμμές αντιστοιχούν στις πραγματικές κατηγορίες και οι στήλες στις προβλεπόμενες, επομένως οι τιμές της κύριας διαγωνίου αντιπροσωπεύουν τις σωστές ταξινομήσεις, ενώ οι υπόλοιπες τιμές τα σφάλματα του μοντέλου.



Εικόνα 6.3: Πίνακας σύγχυσης του ConvNeXt-Tiny με Partial Fine-Tuning στο test set.

Η κύρια διαγώνιος του πίνακα περιλαμβάνει συνολικά 44 σωστές ταξινομήσεις από τις 62 εικόνες του test set, τιμή που αντιστοιχεί στην τελική Accuracy 70,97%. Η κατανομή των σωστών προβλέψεων δεν είναι, ωστόσο, ομοιόμορφη. Η Θράκη παρουσιάζει την ισχυρότερη συμπεριφορά ως προς το Recall, καθώς και οι οκτώ εικόνες της αναγνωρίστηκαν σωστά. Ακολουθούν τα Νησιά Αιγαίου με επτά σωστές ταξινομήσεις στις οκτώ, ενώ η Κρήτη, τα Νησιά Ιονίου και η Θεσσαλία παρουσιάζουν από πέντε σωστές ταξινομήσεις στις έξι εικόνες κάθε κατηγορίας.

Στην Ήπειρο εντοπίζεται η μεγαλύτερη δυσκολία, σε συμφωνία με το χαμηλότερο F1-score που καταγράφηκε στην προηγούμενη ενότητα. Από τις επτά εικόνες της κατηγορίας, τρεις ταξινομήθηκαν σωστά, δύο αποδόθηκαν στη Μακεδονία και δύο στη Στερεά Ελλάδα. Επομένως, τα τέσσερα σφάλματα της Ηπείρου κατανέμονται ισόποσα μεταξύ αυτών των δύο κατηγοριών.

Αμφίδρομη σύγχυση παρατηρείται μεταξύ Ηπείρου και Μακεδονίας. Από τις οκτώ πραγματικές εικόνες της Μακεδονίας, τέσσερις αναγνωρίστηκαν σωστά, ενώ δύο ταξινομήθηκαν ως Ήπειρος και δύο ως Θράκη. Παράλληλα, όπως προηγήθηκε, δύο εικόνες της Ηπείρου ταξινομήθηκαν ως Μακεδονία. Η συγκεκριμένη αμφίδρομη σχέση δείχνει ότι το μοντέλο δυσκολεύεται να διαχωρίσει μέρος των δειγμάτων των δύο κατηγοριών, χωρίς όμως ο πίνακας σύγχυσης από μόνος του να επαρκεί για να προσδιοριστούν τα συγκεκριμένα οπτικά χαρακτηριστικά που προκαλούν αυτή τη συμπεριφορά.

Στη Στερεά Ελλάδα αναγνωρίστηκαν σωστά τρεις από τις έξι εικόνες. Από τις υπόλοιπες, δύο ταξινομήθηκαν ως Ήπειρος και μία ως Νησιά Αιγαίου. Σε συνδυασμό με τις δύο εικόνες της Ηπείρου που ταξινομήθηκαν ως Στερεά Ελλάδα, προκύπτει μία ακόμη αμφίδρομη σύγχυση μεταξύ των δύο κατηγοριών. Η συμπεριφορά αυτή συμβάλλει τόσο στο χαμηλό Recall της Στερεάς Ελλάδας όσο και στη χαμηλή Precision της Ηπείρου.

Η Πελοπόννησος παρουσιάζει τέσσερις σωστές ταξινομήσεις στις επτά εικόνες. Από τις τρεις λανθασμένες προβλέψεις, μία εικόνα αποδόθηκε στην Ήπειρο και δύο στη Θεσσαλία. Παρά το σχετικά περιορισμένο Recall της Πελοποννήσου, 57,14%, καμία εικόνα άλλης κατηγορίας δεν ταξινομήθηκε ως Πελοπόννησος. Το γεγονός αυτό εξηγεί την Precision 100,00% που καταγράφηκε για τη συγκεκριμένη περιοχή.

Στις υπόλοιπες κατηγορίες τα σφάλματα είναι περισσότερο περιορισμένα. Από τις έξι εικόνες της Κρήτης, πέντε αναγνωρίστηκαν σωστά και μία ταξινομήθηκε ως Νησιά Ιονίου. Στα Νησιά Ιονίου, πέντε από τις έξι εικόνες αναγνωρίστηκαν σωστά και μία αποδόθηκε στα Νησιά Αιγαίου. Στη Θεσσαλία, πέντε από τις έξι εικόνες ταξινομήθηκαν σωστά και μία ως Νησιά Ιονίου. Τέλος, στα Νησιά Αιγαίου επτά από τις οκτώ εικόνες αναγνωρίστηκαν σωστά, ενώ η μοναδική λανθασμένη πρόβλεψη αποδόθηκε στη Θράκη.

Ιδιαίτερη περίπτωση αποτελεί η Θράκη. Το Recall 100,00% προκύπτει επειδή καμία από τις οκτώ πραγματικές εικόνες της δεν ταξινομήθηκε σε διαφορετική περιοχή. Ωστόσο, στη Θράκη αποδόθηκαν επιπλέον δύο εικόνες της Μακεδονίας και μία των Νησιών Αιγαίου. Για τον λόγο αυτό, παρά την πλήρη ανάκληση των πραγματικών δειγμάτων της, η Precision της κατηγορίας περιορίζεται στο 72,73%.

Συνολικά, ο πίνακας σύγχυσης αποκαλύπτει ότι τα 18 λανθασμένα δείγματα δεν κατανέμονται τυχαία μεταξύ όλων των πιθανών συνδυασμών κατηγοριών. Ορισμένες σχέσεις εμφανίζονται περισσότερο έντονα, με χαρακτηριστικότερες τις συγχύσεις Ήπειρος–Μακεδονία και Ήπειρος–Στερεά Ελλάδα. Αντίθετα, κατηγορίες όπως η Κρήτη, τα Νησιά Αιγαίου και η Θράκη παρουσιάζουν υψηλότερο ποσοστό σωστών αναγνωρίσεων. Τα αποτελέσματα αυτά περιγράφουν με ακρίβεια τη συμπεριφορά του μοντέλου στο συγκεκριμένο test set, αλλά λόγω του περιορισμένου αριθμού δειγμάτων ανά κατηγορία δεν είναι επαρκή για γενίκευση ότι οι ίδιες συγχύσεις θα εμφανίζονται με την ίδια συχνότητα σε μεγαλύτερο και περισσότερο διαφοροποιημένο σύνολο εικόνων.

Η ανάλυση των σφαλμάτων εξηγεί τις διαφοροποιήσεις που παρατηρήθηκαν στις μετρικές ανά κατηγορία και παρέχει πληρέστερη εικόνα από την εξέταση της Accuracy μεμονωμένα. Ωστόσο, η λειτουργία της εφαρμογής δεν περιορίζεται στην πρώτη πρόβλεψη του μοντέλου, καθώς παρουσιάζονται στον χρήστη οι τρεις πιθανότερες γεωγραφικές κατηγορίες. Για τον λόγο αυτό, στην επόμενη ενότητα εξετάζεται η διαφορά μεταξύ Top-1 και Top-3 Accuracy και αξιολογείται κατά πόσο η πραγματική περιοχή περιλαμβάνεται στις τρεις επικρατέστερες προβλέψεις ακόμη και όταν δεν κατατάσσεται στην πρώτη θέση.

6.7 ΑΞΙΟΛΟΓΗΣΗ TOP-1 ΚΑΙ TOP-3 ΠΡΟΒΛΕΨΕΩΝ

Η αξιολόγηση που προηγήθηκε βασίστηκε κυρίως στην Top-1 Accuracy, δηλαδή στην περίπτωση κατά την οποία μία πρόβλεψη θεωρείται σωστή μόνο όταν η πραγματική γεωγραφική κατηγορία βρίσκεται στην πρώτη θέση της κατάταξης του μοντέλου. Στην παρούσα εφαρμογή, ωστόσο, παρουσιάζονται στον χρήστη οι τρεις πιθανότερες περιοχές. Για τον λόγο αυτό, η Top-3 Accuracy αποτελεί ιδιαίτερα χρήσιμη συμπληρωματική μετρική, καθώς μία εικόνα θεωρείται επιτυχώς αναγνωρισμένη όταν η πραγματική περιοχή περιλαμβάνεται σε οποιαδήποτε από τις τρεις πρώτες προβλέψεις.

Αρχιτεκτονική	**Στρατηγική**	**Top-1Accuracy**	**Top-3 Accuracy**	**Διαφορά**
ResNet18	Feature Extraction	41,94%	87,10%	+45,16 π.μ.
ResNet18	Partial Fine-Tuning	54,84%	87,10%	+32,26 π.μ.
EfficientNet-B0	Feature Extraction	50,00%	85,48%	+35,48 π.μ.
EfficientNet-B0	Partial Fine-Tuning	61,29%	91,94%	+30,65 π.μ.
ConvNeXt-Tiny	Feature Extraction	56,45%	91,94%	+35,49 π.μ.
ConvNeXt-Tiny	Partial Fine-Tuning	70,97%	91,94%	+20,97 π.μ.

Πίνακας 6.4: Σύγκριση Top-1 και Top-3 Accuracy των έξι πειραματικών διαμορφώσεων

Τα αποτελέσματα παρουσιάζουν σημαντική διαφορά μεταξύ Top-1 και Top-3 Accuracy σε όλες τις πειραματικές διαμορφώσεις. Ακόμη και το ResNet18 με Feature Extraction, το οποίο παρουσιάζει τη χαμηλότερη Top-1 Accuracy με 41,94%, επιτυγχάνει Top-3 Accuracy 87,10%. Αυτό σημαίνει ότι, παρόλο που η πραγματική περιοχή δεν κατατάσσεται συχνά πρώτη, βρίσκεται σε πολύ μεγαλύτερο ποσοστό περιπτώσεων μέσα στις τρεις πιθανότερες κατηγορίες που παράγει το μοντέλο.

Στο ResNet18 με Partial Fine-Tuning η Top-1 Accuracy αυξάνεται σε 54,84%, ενώ η Top-3 Accuracy παραμένει 87,10%. Επομένως, η βελτίωση που προέκυψε από το Partial Fine-Tuning αφορά κυρίως την ικανότητα του μοντέλου να τοποθετεί τη σωστή κατηγορία στην πρώτη θέση και όχι την παρουσία της μέσα στις τρεις πρώτες θέσεις. Η διαφορά μεταξύ Top-1 και Top-3 μειώνεται έτσι από 45,16 σε 32,26 ποσοστιαίες μονάδες.

Αντίστοιχη βελτίωση παρατηρείται στο EfficientNet-B0. Με Feature Extraction καταγράφεται Top-1 Accuracy 50,00% και Top-3 Accuracy 85,48%, ενώ με Partial Fine-Tuning οι τιμές αυξάνονται σε 61,29% και 91,94% αντίστοιχα. Στην περίπτωση αυτή, επομένως, το Partial Fine-Tuning βελτιώνει τόσο την πρώτη πρόβλεψη όσο και την πιθανότητα να βρίσκεται η πραγματική περιοχή μεταξύ των τριών επικρατέστερων επιλογών.

Το ConvNeXt-Tiny παρουσιάζει υψηλή Top-3 Accuracy και στις δύο στρατηγικές Μεταφοράς Μάθησης. Με Feature Extraction επιτυγχάνει Top-1 Accuracy 56,45% και Top-3 Accuracy 91,94%, ενώ με Partial Fine-Tuning η Top-1 Accuracy αυξάνεται σημαντικά σε 70,97%, με την Top-3 Accuracy να παραμένει στο 91,94%. Συνεπώς, η κύρια βελτίωση του Partial Fine-Tuning στο ConvNeXt-Tiny εντοπίζεται στην καλύτερη κατάταξη της σωστής κατηγορίας στην πρώτη θέση.

Η υψηλότερη Top-3 Accuracy είναι 91,94% και επιτυγχάνεται από τρεις διαφορετικές διαμορφώσεις: EfficientNet-B0 με Partial Fine-Tuning, ConvNeXt-Tiny με Feature Extraction και ConvNeXt-Tiny με Partial Fine-Tuning. Επομένως, το τελικό ConvNeXt-Tiny με Partial Fine-Tuning δεν υπερέχει αποκλειστικά ως προς την Top-3 Accuracy. Διαφοροποιείται, όμως, σημαντικά ως προς την Top-1 Accuracy, όπου η επίδοση 70,97% είναι η υψηλότερη μεταξύ των έξι πειραμάτων.

Στο τελικό ConvNeXt-Tiny με Partial Fine-Tuning, η διαφορά μεταξύ Top-1 και Top-3 Accuracy είναι 20,97 ποσοστιαίες μονάδες. Σε απόλυτους αριθμούς, η Top-1 Accuracy 70,97% αντιστοιχεί σε 44 σωστές πρώτες προβλέψεις στις 62 εικόνες του test set. Η Top-3 Accuracy 91,94% αντιστοιχεί σε 57 εικόνες για τις οποίες η πραγματική γεωγραφική κατηγορία περιλαμβάνεται στις τρεις πρώτες προβλέψεις. Επομένως, σε 13 από τις 18 περιπτώσεις όπου η πραγματική περιοχή δεν κατατάχθηκε πρώτη, εξακολουθούσε να βρίσκεται στη δεύτερη ή την τρίτη θέση. Μόνο σε 5 από τις 62 εικόνες η πραγματική κατηγορία απουσίαζε συνολικά από τις τρεις πρώτες επιλογές.

Η παρατήρηση αυτή είναι σημαντική για τον τρόπο με τον οποίο σχεδιάστηκε η διαδικτυακή εφαρμογή. Η παρουσίαση τριών πιθανών περιοχών δεν μετατρέπει μία λανθασμένη Top-1 πρόβλεψη σε βέβαιη σωστή αναγνώριση, αλλά παρέχει στον χρήστη περισσότερη πληροφορία σχετικά με την κατάταξη των πιθανότερων κατηγοριών. Στο συγκεκριμένο test set, η πραγματική περιοχή περιλαμβάνεται στις τρεις πρώτες επιλογές σε 91,94% των περιπτώσεων, έναντι 70,97% όταν λαμβάνεται υπόψη αποκλειστικά η πρώτη πρόβλεψη.

Η Top-3 Accuracy πρέπει επομένως να αντιμετωπίζεται συμπληρωματικά και όχι ως υποκατάστατο της Top-1 Accuracy. Η πρώτη θέση εξακολουθεί να αποτελεί την κύρια απόφαση ταξινόμησης του μοντέλου, ενώ οι επόμενες δύο θέσεις αποτυπώνουν πρόσθετη πληροφορία για τις εναλλακτικές κατηγορίες που θεωρούνται πιθανότερες. Ο συνδυασμός των δύο μετρικών παρέχει πληρέστερη εικόνα της πρακτικής συμπεριφοράς του συστήματος και αποτελεί σημαντικό κριτήριο για τη συνολική αποτίμηση και την επιλογή του τελικού μοντέλου που ακολουθεί στην επόμενη ενότητα.

6.8 ΣΥΓΚΡΙΤΙΚΗ ΑΠΟΤΙΜΗΣΗ ΚΑΙ ΕΠΙΛΟΓΗ ΤΕΛΙΚΟΥ ΜΟΝΤΕΛΟΥ

Η συνολική αξιολόγηση των έξι πειραματικών διαμορφώσεων επιτρέπει την επιλογή του μοντέλου που παρουσιάζει την καταλληλότερη συμπεριφορά για την τελική εφαρμογή. Η επιλογή δεν βασίζεται αποκλειστικά σε μία μετρική, αλλά στη συνδυαστική εξέταση της Top-1 Accuracy, της Top-3 Accuracy, των Macro και Weighted μετρικών, της επίδοσης ανά γεωγραφική κατηγορία και της κατανομής των σφαλμάτων που αναλύθηκαν στις προηγούμενες ενότητες.

Μεταξύ των έξι πειραμάτων, το ConvNeXt-Tiny με Partial Fine-Tuning επιτυγχάνει την υψηλότερη Top-1 Accuracy, με 70,97%. Παράλληλα, καταγράφει Macro Precision 73,06%, Macro Recall 70,83%, Macro F1-score 70,64% και Weighted F1-score 70,56%. Οι τιμές αυτές είναι οι υψηλότερες μεταξύ των έξι διαμορφώσεων για τις αντίστοιχες μετρικές και δείχνουν ότι η υπεροχή του μοντέλου δεν περιορίζεται μόνο στο συνολικό ποσοστό σωστών ταξινομήσεων.

Η σύγκριση με το ConvNeXt-Tiny που χρησιμοποιήθηκε ως Feature Extractor αναδεικνύει τη συμβολή της επιλεγμένης στρατηγικής προσαρμογής. Με Feature Extraction η αρχιτεκτονική πέτυχε Accuracy 56,45% και Macro F1-score 56,59%, ενώ με Partial Fine-Tuning οι αντίστοιχες τιμές αυξήθηκαν σε 70,97% και 70,64%. Πρόκειται για βελτίωση 14,52 ποσοστιαίων μονάδων στην Accuracy και 14,05 ποσοστιαίων μονάδων στο Macro F1-score. Η συγκεκριμένη διαφορά είναι ιδιαίτερα σημαντική, καθώς οι δύο διαμορφώσεις χρησιμοποιούν την ίδια βασική αρχιτεκτονική και διαφοροποιούνται ως προς τον βαθμό προσαρμογής των προεκπαιδευμένων παραμέτρων.

Η ίδια γενική τάση εμφανίστηκε και στις άλλες δύο αρχιτεκτονικές, καθώς το Partial Fine-Tuning βελτίωσε την Top-1 Accuracy τόσο στο ResNet18 όσο και στο EfficientNet-B0. Ωστόσο, οι τελικές επιδόσεις τους παρέμειναν χαμηλότερες από εκείνες του ConvNeXt-Tiny με Partial Fine-Tuning. Συγκεκριμένα, το ResNet18 με Partial Fine-Tuning πέτυχε Accuracy 54,84% και Macro F1-score 54,02%, ενώ το EfficientNet-B0 με Partial Fine-Tuning έφτασε σε Accuracy 61,29% και Macro F1-score 60,72%. Το ConvNeXt-Tiny με Partial Fine-Tuning υπερβαίνει επομένως την αμέσως επόμενη διαμόρφωση, το EfficientNet-B0 με Partial Fine-Tuning, κατά 9,68 ποσοστιαίες μονάδες στην Accuracy και 9,92 ποσοστιαίες μονάδες στο Macro F1-score.

Η Top-3 Accuracy ενισχύει περαιτέρω την πρακτική καταλληλότητα του μοντέλου, χωρίς όμως να αποτελεί μεμονωμένο κριτήριο υπεροχής. Το ConvNeXt-Tiny με Partial Fine-Tuning επιτυγχάνει Top-3 Accuracy 91,94%, τιμή που αποτελεί την υψηλότερη των πειραμάτων αλλά επιτυγχάνεται επίσης από το EfficientNet-B0 με Partial Fine-Tuning και το ConvNeXt-Tiny με Feature Extraction. Η διαφοροποίηση του

τελικού μοντέλου βρίσκεται στο γεγονός ότι συνδυάζει αυτή την υψηλή Top-3 επίδοση με σαφώς υψηλότερη Top-1 Accuracy. Στις 62 εικόνες του test set, η πραγματική περιοχή κατατάχθηκε πρώτη σε 44 περιπτώσεις και βρέθηκε μεταξύ των τριών πρώτων επιλογών σε 57 περιπτώσεις.

Η ανάλυση ανά γεωγραφική κατηγορία έδειξε παράλληλα ότι η επίδοση του τελικού μοντέλου δεν είναι ομοιόμορφη. Ιδιαίτερα υψηλά F1-scores καταγράφηκαν στην Κρήτη με 90,91%, στη Θράκη με 84,21% και στα Νησιά Αιγαίου με 82,35%, ενώ η Ήπειρος αποτέλεσε τη δυσκολότερη κατηγορία με F1-score 40,00%. Ο πίνακας σύγχυσης ανέδειξε συγκεκριμένες λανθασμένες ταξινομήσεις, ιδιαίτερα μεταξύ Ηπείρου και Μακεδονίας καθώς και μεταξύ Ηπείρου και Στερεάς Ελλάδας. Συνεπώς, η επιλογή του ConvNeXt-Tiny με Partial Fine-Tuning δεν συνεπάγεται ότι το μοντέλο επιλύει εξίσου αποτελεσματικά όλες τις επιμέρους περιπτώσεις ταξινόμησης.

Πέρα από την τελική επίδοση, η επιλογή πρέπει να εξεταστεί και σε σχέση με την πολυπλοκότητα της προσαρμογής. Στο ConvNeXt-Tiny με Feature Extraction εκπαιδεύονταν μόνο 6.921 παράμετροι, ενώ στο Partial Fine-Tuning ο αριθμός των εκπαιδεύσιμων παραμέτρων αυξήθηκε σε 14.297.865. Η σημαντική αυτή αύξηση συνεπάγεται μεγαλύτερη δυνατότητα προσαρμογής του προεκπαιδευμένου δικτύου στο συγκεκριμένο πρόβλημα, αλλά ταυτόχρονα αυξάνει τις απαιτήσεις της διαδικασίας εκπαίδευσης και τον κίνδυνο υπερπροσαρμογής σε ένα περιορισμένο σύνολο δεδομένων. Για τον έλεγχο της διαδικασίας χρησιμοποιήθηκαν οι μηχανισμοί που παρουσιάστηκαν στο Κεφάλαιο 5, με χαμηλότερο learning rate για το Partial Fine-Tuning, παρακολούθηση του validation loss, αποθήκευση του καλύτερου checkpoint και Early Stopping.

Το καλύτερο checkpoint της τελικής διαμόρφωσης καταγράφηκε στο 160 epoch, με validation loss 1,0733 και validation accuracy 65,08%. Η τελική αξιολόγηση πραγματοποιήθηκε στη συνέχεια στο ανεξάρτητο test set, το οποίο δεν χρησιμοποιήθηκε για την επιλογή του checkpoint. Με αυτόν τον τρόπο, η επιλογή του τελικού μοντέλου βασίζεται στην προκαθορισμένη διαδικασία επικύρωσης και η επίδοσή του στο test set χρησιμοποιείται για την τελική αποτίμηση της ικανότητας γενίκευσης στα διαθέσιμα δεδομένα.

Με βάση το σύνολο των παραπάνω αποτελεσμάτων, το **ConvNeXt-Tiny με Partial Fine-Tuning επιλέγεται ως το τελικό μοντέλο της παρούσας εργασίας**. Η επιλογή αιτιολογείται κυρίως από την υψηλότερη Top-1 Accuracy και τις υψηλότερες συνολικές Macro και Weighted μετρικές μεταξύ των έξι πειραμάτων, σε συνδυασμό με Top-3 Accuracy 91,94%. Παρά τις αδυναμίες που εξακολουθούν να εμφανίζονται σε συγκεκριμένες γεωγραφικές κατηγορίες, η συγκεκριμένη διαμόρφωση παρέχει την ισχυρότερη συνολική επίδοση στο πλαίσιο των πειραμάτων που πραγματοποιήθηκαν και αποτελεί συνεπώς την καταλληλότερη από τις εξεταζόμενες επιλογές για την ενσωμάτωση στο τελικό σύστημα αναγνώρισης.

6.9 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο πραγματοποιήθηκε η τελική αξιολόγηση των έξι πειραματικών διαμορφώσεων που προέκυψαν από τον συνδυασμό των αρχιτεκτονικών ResNet18, EfficientNet-B0 και ConvNeXt-Tiny με τις στρατηγικές Feature Extraction και Partial Fine-Tuning. Η αξιολόγηση πραγματοποιήθηκε στο ανεξάρτητο test set των 62 εικόνων και βασίστηκε σε πολλαπλές μετρικές, ώστε η σύγκριση να μην περιοριστεί αποκλειστικά στο συνολικό ποσοστό σωστών ταξινομήσεων.

Τα αποτελέσματα ανέδειξαν μία συνεπή τάση υπέρ του Partial Fine-Tuning ως προς την Top-1 Accuracy και το Macro F1-score και στις τρεις αρχιτεκτονικές. Η Accuracy αυξήθηκε από 41,94% σε 54,84% για το ResNet18, από 50,00% σε 61,29% για το EfficientNet-B0 και από 56,45% σε 70,97% για το ConvNeXt-Tiny. Η μεγαλύτερη βελτίωση στην Accuracy μεταξύ των αντίστοιχων ζευγών καταγράφηκε στο ConvNeXt-Tiny και ανήλθε σε 14,52 ποσοστιαίες μονάδες.

Η ισχυρότερη συνολική επίδοση καταγράφηκε από το ConvNeXt-Tiny με Partial Fine-Tuning. Το συγκεκριμένο μοντέλο πέτυχε Accuracy 70,97%, Macro Precision 73,06%, Macro Recall 70,83%, Macro F1-score 70,64% και Weighted F1-score 70,56%. Παράλληλα, η Top-3 Accuracy έφτασε το 91,94%, τιμή που αποτελεί την υψηλότερη μεταξύ των πειραμάτων από κοινού με το EfficientNet-B0 με Partial Fine-Tuning και το ConvNeXt-Tiny με Feature Extraction.

Η αξιολόγηση ανά γεωγραφική κατηγορία ανέδειξε σημαντικές διαφοροποιήσεις που δεν είναι εμφανείς από τη συνολική Accuracy. Η Κρήτη παρουσίασε το υψηλότερο F1-score με 90,91%, ενώ υψηλές επιδόσεις καταγράφηκαν επίσης στη Θράκη και στα Νησιά Αιγαίου, με F1-score 84,21% και 82,35% αντίστοιχα.

Αντίθετα, η Ήπειρος αποτέλεσε τη δυσκολότερη κατηγορία, με F1-score 40,00%, ενώ χαμηλότερες επιδόσεις παρατηρήθηκαν επίσης στη Στερεά Ελλάδα και στη Μακεδονία.

Η εξέταση του πίνακα σύγχυσης επέτρεψε την ακριβέστερη διερεύνηση των λανθασμένων ταξινομήσεων. Από τις 62 εικόνες του test set, οι 44 ταξινομήθηκαν σωστά στην πρώτη θέση και οι 18 λανθασμένα. Μεταξύ των εμφανέστερων συγχύσεων εντοπίστηκαν εκείνες μεταξύ Ηπείρου και Μακεδονίας καθώς και μεταξύ Ηπείρου και Στερεάς Ελλάδας. Τα αποτελέσματα αυτά δείχνουν ότι οι δυσκολίες του μοντέλου συγκεντρώνονται περισσότερο σε συγκεκριμένους συνδυασμούς κατηγοριών και όχι ομοιόμορφα σε ολόκληρο το πρόβλημα ταξινόμησης.

Ιδιαίτερη σημασία για την τελική εφαρμογή έχει η σύγκριση μεταξύ Top-1 και Top-3 Accuracy. Στο επιλεγμένο ConvNeXt-Tiny με Partial Fine-Tuning, η πραγματική περιοχή κατατάχθηκε πρώτη σε 44 από τις 62 εικόνες, ενώ συμπεριλήφθηκε στις τρεις πρώτες προβλέψεις σε 57 από τις 62. Επομένως, σε 13 από τις 18 περιπτώσεις όπου η πρώτη πρόβλεψη ήταν λανθασμένη, η πραγματική κατηγορία εξακολουθούσε να βρίσκεται στη δεύτερη ή την τρίτη θέση. Το αποτέλεσμα αυτό υποστηρίζει τη χρησιμότητα της παρουσίασης τριών πιθανότερων περιοχών στη διαδικτυακή εφαρμογή, χωρίς η Top-3 πρόβλεψη να αντιμετωπίζεται ως υποκατάστατο της κύριας Top-1 ταξινόμησης.

Με βάση τη συνδυαστική εξέταση των συνολικών μετρικών, της επίδοσης ανά κατηγορία, του πίνακα σύγχυσης και της Top-3 Accuracy, το ConvNeXt-Tiny με Partial Fine-Tuning επιλέχθηκε ως το τελικό μοντέλο μεταξύ των έξι διαμορφώσεων που αξιολογήθηκαν. Η επιλογή αφορά αποκλειστικά το συγκεκριμένο πειραματικό πλαίσιο και το διαθέσιμο σύνολο δεδομένων και δεν συνεπάγεται γενική υπεροχή της αρχιτεκτονικής ή της στρατηγικής Partial Fine-Tuning σε διαφορετικά προβλήματα ταξινόμησης εικόνων.

Με την ολοκλήρωση της πειραματικής αξιολόγησης διαμορφώνεται μία ολοκληρωμένη εικόνα της συμπεριφοράς του συστήματος αναγνώρισης. Τα αποτελέσματα παρέχουν τόσο την ποσοτική βάση για την επιλογή του τελικού μοντέλου όσο και σαφείς ενδείξεις για τις κατηγορίες στις οποίες υπάρχει περιθώριο περαιτέρω βελτίωσης. Η αξιοποίηση του επιλεγμένου μοντέλου στο πλαίσιο της διαδικτυακής εφαρμογής συνδέει με αυτόν τον τρόπο την πειραματική διαδικασία Μηχανικής Μάθησης με την πρακτική χρήση του συστήματος για την αναγνώριση ελληνικών παραδοσιακών φορεσιών.

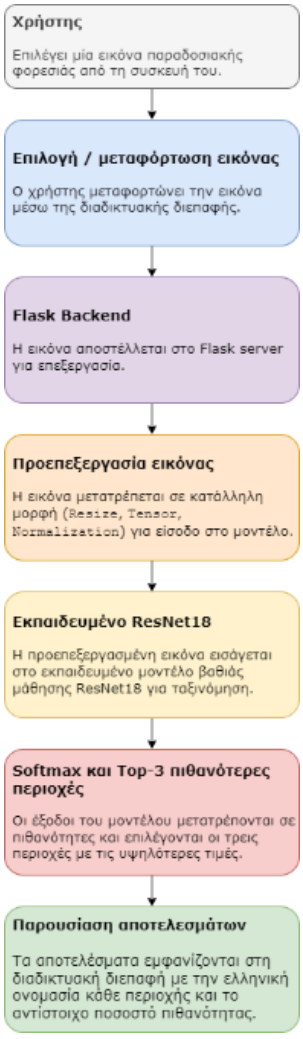
ΚΕΦΑΛΑΙΟ 7 - ΑΝΑΠΤΥΞΗ ΔΙΑΔΙΚΤΥΑΚΗΣ ΕΦΑΡΜΟΓΗΣ

7.1 ΕΙΣΑΓΩΓΗ

Η ολοκλήρωση της διαδικασίας εκπαίδευσης και αξιολόγησης των μοντέλων Μηχανικής Μάθησης επιτρέπει τη μετάβαση από το πειραματικό στάδιο σε ένα σύστημα που μπορεί να χρησιμοποιηθεί από τον τελικό χρήστη. Στο πλαίσιο της παρούσας εργασίας αναπτύχθηκε διαδικτυακή εφαρμογή για την αναγνώριση ελληνικών παραδοσιακών φορεσιών από εικόνες. Μέσω της εφαρμογής, ο χρήστης έχει τη δυνατότητα να επιλέξει μία εικόνα παραδοσιακής φορεσιάς και να λάβει ως αποτέλεσμα τις τρεις γεωγραφικές περιοχές στις οποίες το εκπαιδευμένο μοντέλο θεωρεί πιθανότερο ότι αυτή ανήκει.

Η εφαρμογή υλοποιήθηκε με τη γλώσσα προγραμματισμού Python και το διαδικτυακό πλαίσιο Flask, ενώ για την εκτέλεση του μοντέλου αξιοποιήθηκε η βιβλιοθήκη PyTorch. Η διεπαφή χρήστη αναπτύχθηκε με HTML και CSS, ενώ σε επιμέρους σημεία χρησιμοποιήθηκε JavaScript για λειτουργίες αλληλεπίδρασης, όπως η προεπισκόπηση της εικόνας πριν από την αποστολή της. Με τον τρόπο αυτό, η εφαρμογή συνδυάζει το μοντέλο βαθιάς μάθησης με ένα περιβάλλον που μπορεί να χρησιμοποιηθεί απευθείας μέσω ενός προγράμματος περιήγησης.

Η βασική ροή λειτουργίας ξεκινά από την επιλογή μιας εικόνας από τον χρήστη. Η εικόνα αποστέλλεται στο Flask backend και μετατρέπεται σε μορφή κατάλληλη για την είσοδό της στο εκπαιδευμένο μοντέλο. Στη συνέχεια πραγματοποιείται η πρόβλεψη, οι έξοδοι του δικτύου μετατρέπονται σε πιθανότητες και επιλέγονται οι τρεις κλάσεις με τις υψηλότερες τιμές. Τα αποτελέσματα επιστρέφονται στη διαδικτυακή διεπαφή, όπου παρουσιάζονται με την ελληνική ονομασία κάθε περιοχής με σειρά κατάταξης από την πιθανότερη προς την τρίτη πιθανότερη περιοχή.

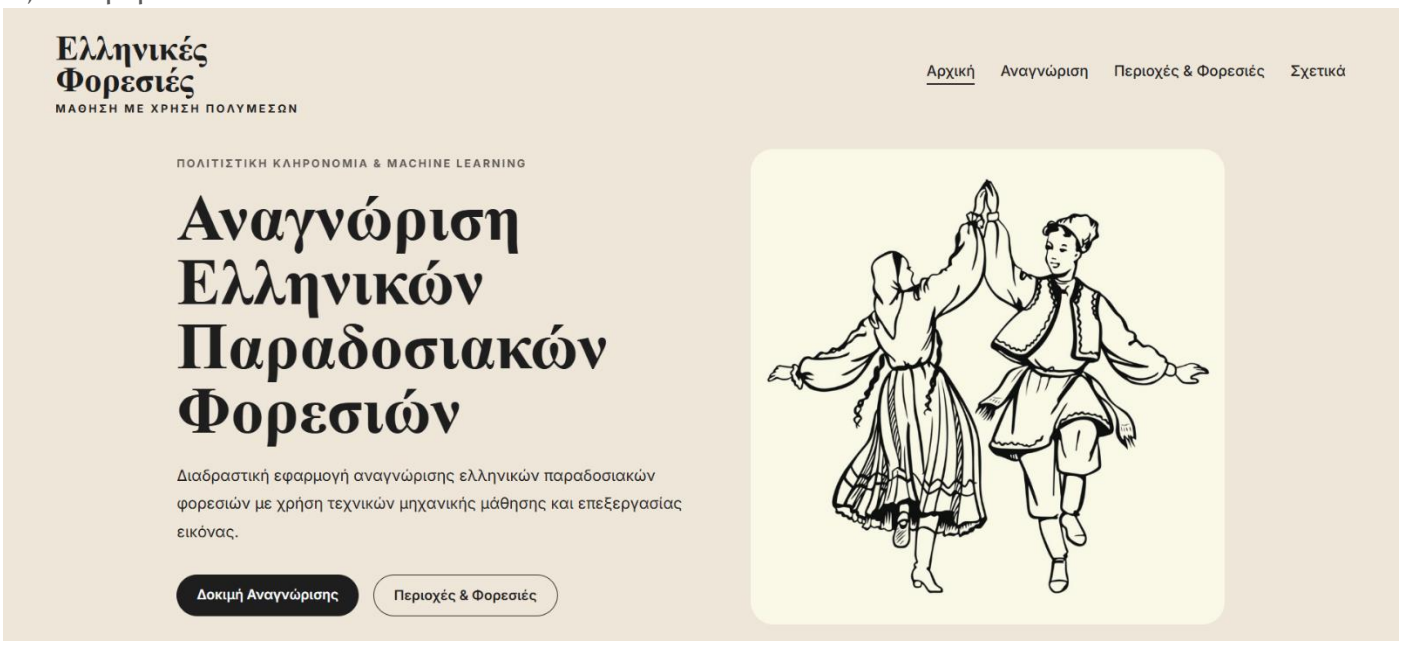


Σχήμα 7.1: Συνολική ροή από την εισαγωγή της εικόνας έως την παρουσίαση των Top-3 αποτελεσμάτων.

Η επιλογή παρουσίασης των τριών πιθανότερων περιοχών, αντί αποκλειστικά της κλάσης με τη μεγαλύτερη πιθανότητα, επιτρέπει την πληρέστερη παρουσίαση της εξόδου του μοντέλου. Η συγκεκριμένη επιλογή είναι ιδιαίτερα χρήσιμη στο υπό εξέταση πρόβλημα, καθώς διαφορετικές παραδοσιακές φορεσιές μπορούν να εμφανίζουν κοινά οπτικά χαρακτηριστικά, όπως παρόμοια χρώματα, υφάσματα, κεντήματα, ποδιές ή διακοσμητικά στοιχεία. Το πρώτο αποτέλεσμα εξακολουθεί να αποτελεί την κύρια πρόβλεψη του συστήματος, ενώ οι δύο επόμενες θέσεις παρέχουν πρόσθετη πληροφορία σχετικά με τις εναλλακτικές εκτιμήσεις του μοντέλου.

Η διαδικτυακή εφαρμογή δεν περιορίζεται αποκλειστικά στην εκτέλεση της ταξινόμησης. Οργανώνεται σε τέσσερις βασικές σελίδες: «Αρχική», «Αναγνώριση», «Περιοχές & Φορεσιές» και «Σχετικά». Η αρχική σελίδα παρουσιάζει τον σκοπό της εφαρμογής και λειτουργεί ως κεντρικό σημείο πλοήγησης. Η σελίδα «Αναγνώριση» επιτρέπει την εισαγωγή εικόνας και την εμφάνιση των αποτελεσμάτων του μοντέλου, ενώ η ενότητα «Περιοχές & Φορεσιές» περιλαμβάνει πληροφοριακό και οπτικό υλικό για τις εννέα γεωγραφικές κατηγορίες που χρησιμοποιήθηκαν στην ταξινόμηση.

Τέλος, η σελίδα «Σχετικά» συνοψίζει τον σκοπό της εφαρμογής και τις βασικές τεχνολογίες που αξιοποιήθηκαν.



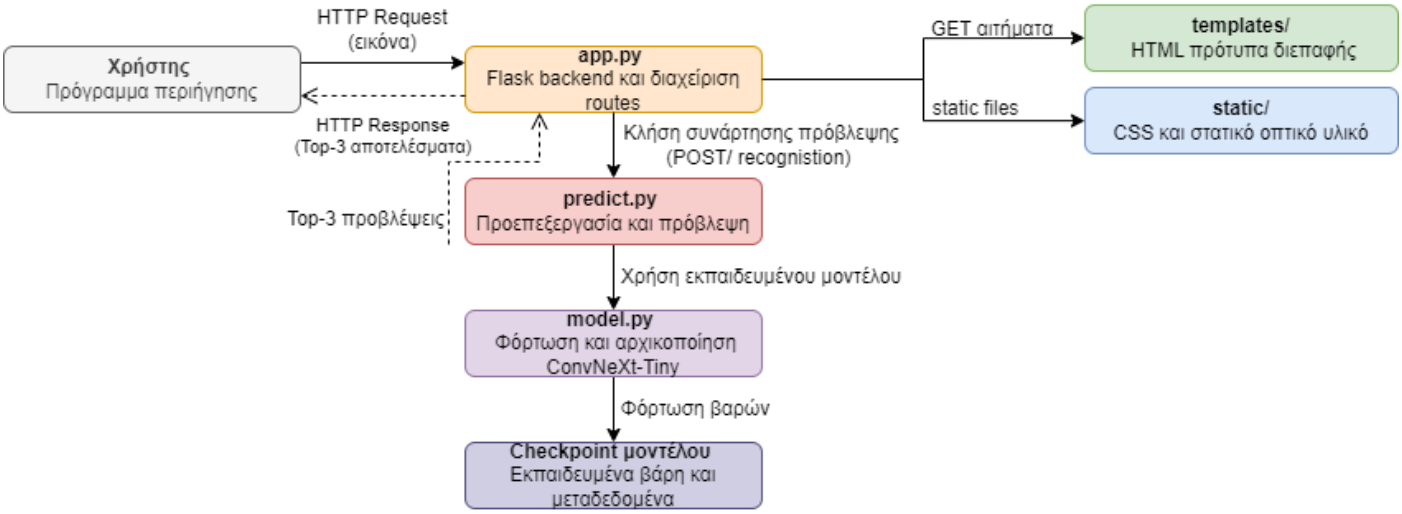
Εικόνα 7.1: *Αρχική σελίδα της διαδικτυακής εφαρμογής αναγνώρισης ελληνικών παραδοσιακών φορεσιών.*

Η ανάπτυξη της διαδικτυακής εφαρμογής αποτελεί το τελικό στάδιο ενοποίησης των επιμέρους τμημάτων της εργασίας. Το σύνολο δεδομένων, η προεπεξεργασία των εικόνων, η εκπαίδευση και αξιολόγηση των μοντέλων και η επιλογή του μοντέλου που χρησιμοποιείται για την αναγνώριση συνδέονται πλέον με ένα λειτουργικό περιβάλλον χρήστη. Στις επόμενες ενότητες παρουσιάζονται αναλυτικότερα η αρχιτεκτονική της εφαρμογής, η οργάνωση των αρχείων και των φακέλων, η λειτουργία του Flask backend, η φόρτωση του εκπαιδευμένου μοντέλου, η διαδικασία πρόβλεψης και η σχεδίαση της διεπαφής χρήστη.

7.2 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΟΡΓΑΝΩΣΗ ΤΗΣ ΕΦΑΡΜΟΓΗΣ

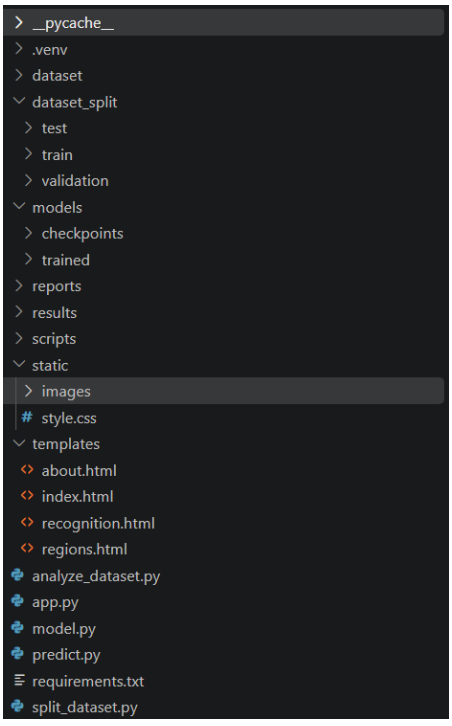
Η διαδικτυακή εφαρμογή οργανώθηκε με τρόπο ώστε η λειτουργία της διεπαφής χρήστη, η διαχείριση των διαδικτυακών αιτημάτων και η εκτέλεση του μοντέλου Μηχανικής Μάθησης να αποτελούν διακριτά τμήματα του συστήματος. Η συγκεκριμένη οργάνωση διευκολύνει την κατανόηση της ροής της εφαρμογής και επιτρέπει την ανεξάρτητη διαχείριση των βασικών λειτουργιών της. Στον πυρήνα της εφαρμογής βρίσκεται το Flask, το οποίο συνδέει τις σελίδες που εμφανίζονται στον χρήστη με τον κώδικα Python που είναι υπεύθυνος για την επεξεργασία της εικόνας και την εκτέλεση της πρόβλεψης.

Η υλοποίηση βασίζεται σε τρία κύρια αρχεία Python. Το app.py αποτελεί το κεντρικό σημείο εκκίνησης της διαδικτυακής εφαρμογής και είναι υπεύθυνο για τη δημιουργία του αντικειμένου Flask, τον καθορισμό των διαθέσιμων διαδρομών και τη διαχείριση των αιτημάτων του χρήστη. Το model.py αναλαμβάνει τη δημιουργία και τη φόρτωση του εκπαιδευμένου μοντέλου ταξινόμησης, ενώ το predict.py περιλαμβάνει τη διαδικασία προεπεξεργασίας της εισερχόμενης εικόνας και παραγωγής των Top-3 προβλέψεων. Με αυτόν τον διαχωρισμό, η λογική της διαδικτυακής εφαρμογής παραμένει διακριτή από τη λογική του μοντέλου και της πρόβλεψης.



Σχήμα 7.2: *Αρχιτεκτονική και αλληλεπίδραση των βασικών τμημάτων της διαδικτυακής εφαρμογής.*

Η δομή του project ακολουθεί την παραπάνω λογική διαχωρισμού. Τα αρχεία HTML αποθηκεύονται στον φάκελο templates, όπως προβλέπει η οργάνωση μιας εφαρμογής Flask, ενώ τα στατικά αρχεία βρίσκονται στον φάκελο static. Στον τελευταίο περιλαμβάνονται το αρχείο style.css, το οποίο καθορίζει την εμφάνιση της διεπαφής, καθώς και ο υποφάκελος images, στον οποίο βρίσκεται το οπτικό υλικό που χρησιμοποιείται στις επιμέρους σελίδες. Παράλληλα, στον φάκελο models αποθηκεύονται τα αρχεία που σχετίζονται με τα εκπαιδευμένα μοντέλα, ενώ οι φάκελοι reports και results περιλαμβάνουν αρχεία και αποτελέσματα που δημιουργήθηκαν κατά τα προηγούμενα στάδια της εργασίας.



Εικόνα 7.2: *Δομή φακέλων και βασικών αρχείων του project στο περιβάλλον ανάπτυξης Visual Studio Code.*

Στον φάκελο templates βρίσκονται τα αρχεία index.html, recognition.html, regions.html και about.html, τα οποία αντιστοιχούν στις τέσσερις κύριες σελίδες της εφαρμογής. Ο διαχωρισμός αυτός επιτρέπει στο Flask να αποδίδει διαφορετικό HTML template ανάλογα με τη διαδρομή που ζητά ο χρήστης. Αντίστοιχα, το κοινό αρχείο style.css χρησιμοποιείται για τη διαμόρφωση της οπτικής εμφάνισης των σελίδων, διατηρώντας ενιαία σχεδιαστική ταυτότητα σε ολόκληρη την εφαρμογή.

Η σύνδεση μεταξύ των επιμέρους τμημάτων πραγματοποιείται από το app.py. Κατά την εκκίνηση δημιουργείται η εφαρμογή Flask και ορίζεται ο φάκελος static για την εξυπηρέτηση των στατικών αρχείων. Το αντίστοιχο τμήμα του κώδικα παρουσιάζεται στον Κώδικα 7.1.

```
app = Flask(
    __name__,
    static_folder="static",
    static_url_path=""
)
```

Κώδικας 7.1: *Αρχικοποίηση της εφαρμογής Flask και ορισμός των στατικών αρχείων*

Με τη συγκεκριμένη αρχικοποίηση, το Flask χρησιμοποιείται ως το κεντρικό επίπεδο επικοινωνίας μεταξύ της διεπαφής και του υποσυστήματος πρόβλεψης. Όταν ο χρήστης επισκέπτεται μία σελίδα, το backend αποδίδει το αντίστοιχο HTML template, ενώ στην περίπτωση της αναγνώρισης παραλαμβάνει επιπλέον την εικόνα που έχει επιλεγεί και τη διαβιβάζει στη συνάρτηση πρόβλεψης. Η συγκεκριμένη ροή επιτρέπει στην εφαρμογή να συνδυάζει σε ένα ενιαίο σύστημα τη διαδικτυακή διεπαφή, την προεπεξεργασία της εικόνας και το εκπαιδευμένο μοντέλο ταξινόμησης.

Η παραπάνω αρχιτεκτονική αποτελεί τη βάση πάνω στην οποία υλοποιούνται οι επιμέρους λειτουργίες της εφαρμογής. Στην επόμενη ενότητα παρουσιάζεται αναλυτικότερα το Flask backend και ο τρόπος με τον οποίο οι διαδρομές της εφαρμογής συνδέονται με τις αντίστοιχες σελίδες και με τη διαδικασία αναγνώρισης.

7.3 BACKEND ΚΑΙ ΔΡΟΜΟΛΟΓΗΣΗ ΜΕ FLASK

Η λειτουργία της διαδικτυακής εφαρμογής ελέγχεται από το αρχείο app.py, το οποίο αποτελεί το βασικό σημείο σύνδεσης μεταξύ του προγράμματος περιήγησης, των HTML templates και της διαδικασίας αναγνώρισης. Για την υλοποίηση του backend χρησιμοποιήθηκε το Flask, ένα διαδικτυακό πλαίσιο της Python που επιτρέπει τον ορισμό διαδρομών (routes) και τη διαχείριση HTTP αιτημάτων. Στην εφαρμογή έχουν οριστεί τέσσερις βασικές διαδρομές, οι οποίες αντιστοιχούν στις σελίδες «Αρχική», «Αναγνώριση», «Περιοχές & Φορεσιές» και «Σχετικά».

Η αρχική σελίδα αντιστοιχεί στη διαδρομή /, ενώ οι υπόλοιπες σελίδες είναι διαθέσιμες μέσω των /recognition, /regions και /about. Για την απόδοση των αντίστοιχων HTML αρχείων χρησιμοποιείται η συνάρτηση render_template() του Flask. Οι τρεις σελίδες που έχουν αποκλειστικά πληροφοριακό ή πλοηγικό χαρακτήρα εξυπηρετούνται με απλά αιτήματα GET, ενώ η διαδρομή /recognition υποστηρίζει τόσο GET όσο και POST, καθώς πρέπει επιπλέον να παραλαμβάνει την εικόνα που επιλέγει ο χρήστης. Η βασική αντιστοίχιση των διαδρομών παρουσιάζεται στον Κώδικα 7.2.

```
@app.route("/")
def home():
    return render_template("index.html")
```

```
@app.route("/regions")
def regions():
    return render_template("regions.html")
@app.route("/about")
def about():
    return render_template("about.html")
```

Κώδικας 7.2: *Ορισμός των βασικών διαδρομών της διαδικτυακής εφαρμογής*

Η διαδρομή /recognition αποτελεί το σημαντικότερο σημείο του backend ως προς τη σύνδεση της διαδικτυακής εφαρμογής με το μοντέλο ταξινόμησης. Όταν ο χρήστης επισκέπτεται αρχικά τη συγκεκριμένη σελίδα, πραγματοποιείται αίτημα GET και αποδίδεται το recognition.html χωρίς αποτελέσματα πρόβλεψης. Όταν, αντίθετα, επιλέξει μια εικόνα και υποβάλει τη φόρμα αναγνώρισης, αποστέλλεται αίτημα POST. Το backend ελέγχει εάν το αίτημα περιλαμβάνει αρχείο με το αναμενόμενο όνομα και, εφόσον υπάρχει, παραλαμβάνει την εικόνα για περαιτέρω επεξεργασία.

```
 @app.route(
    "/recognition",
    methods=["GET", "POST"]
)
def recognition():
    results = None
    if request.method == "POST":
        if "image" not in request.files:
            return render_template(
                "recognition.html",
                results=None
            )
        file = request.files["image"]
```

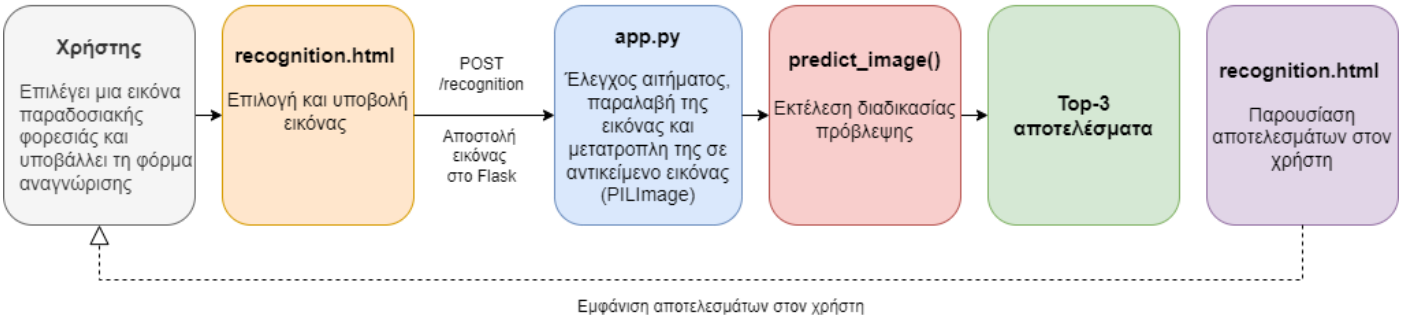
Κώδικας 7.3*: Διαχείριση GET και POST αιτημάτων στη σελίδα αναγνώρισης*

Το αρχείο που παραλαμβάνεται από το Flask δεν αποθηκεύεται ως απαραίτητο ενδιάμεσο αρχείο στον δίσκο για την πραγματοποίηση της πρόβλεψης. Αντίθετα, ανοίγεται ως εικόνα μέσω της βιβλιοθήκης Pillow και στη συνέχεια διαβιβάζεται απευθείας στη συνάρτηση predict_image(). Η συνάρτηση αυτή βρίσκεται στο predict.py και αποτελεί το επόμενο επίπεδο της διαδικασίας αναγνώρισης. Μετά την ολοκλήρωση της πρόβλεψης, επιστρέφει τις τρεις πιθανότερες περιοχές ταξινομημένες κατά σειρά πιθανότητας. Το app.py μεταβιβάζει τα αποτελέσματα στο recognition.html μέσω της μεταβλητής results.

```
image = Image.open(file)
results = predict_image(
    image
)
return render_template(
    "recognition.html",
    results=results
)
```

Κώδικας 7.4: *Σύνδεση του Flask backend με τη συνάρτηση πρόβλεψης*

Η συγκεκριμένη υλοποίηση δημιουργεί μια σαφή ακολουθία επικοινωνίας μεταξύ frontend και backend. Ο χρήστης δεν αλληλεπιδρά άμεσα με το μοντέλο Μηχανικής Μάθησης. Η εικόνα εισάγεται μέσω της φόρμας HTML, παραλαμβάνεται από το Flask, μεταφέρεται στη συνάρτηση πρόβλεψης και, αφού ολοκληρωθεί η επεξεργασία της, τα αποτελέσματα επιστρέφουν στο template για να παρουσιαστούν στον χρήστη. Η ροή ενός αιτήματος αναγνώρισης παρουσιάζεται στο Σχήμα 7.3.



Σχήμα 7.3: *Ροή επεξεργασίας ενός αιτήματος αναγνώρισης εικόνας από την υποβολή της φόρμας έως την παρουσίαση των αποτελεσμάτων.*

Ο παραπάνω διαχωρισμός είναι σημαντικός για την οργάνωση της εφαρμογής, καθώς το app.py περιορίζεται στη διαχείριση της διαδικτυακής επικοινωνίας και δεν περιλαμβάνει την εσωτερική λογική προεπεξεργασίας ή εκτέλεσης του νευρωνικού δικτύου. Η λειτουργία αυτή έχει μεταφερθεί στο predict.py, ενώ η δημιουργία και φόρτωση του μοντέλου πραγματοποιείται από το model.py. Στην επόμενη ενότητα αναλύεται η ενσωμάτωση του εκπαιδευμένου μοντέλου στην εφαρμογή και η διαδικασία μέσω της οποίας μια εισερχόμενη εικόνα μετατρέπεται στις τελικές προβλέψεις.

7.4 ΕΝΣΩΜΑΤΩΣΗ ΤΟΥ ΕΚΠΑΙΔΕΥΜΕΝΟΥ ΜΟΝΤΕΛΟΥ ΚΑΙ ΔΙΑΔΙΚΑΣΙΑ ΠΡΟΒΛΕΨΗΣ

Η λειτουργία αναγνώρισης της διαδικτυακής εφαρμογής προϋποθέτει τη φόρτωση του εκπαιδευμένου μοντέλου και την εφαρμογή στην εισερχόμενη εικόνα της ίδιας βασικής προεπεξεργασίας που χρησιμοποιείται κατά την αξιολόγηση του μοντέλου. Για τον σκοπό αυτό, η υλοποίηση διαχωρίζεται στα αρχεία model.py και predict.py. Το πρώτο είναι υπεύθυνο για τη φόρτωση του εκπαιδευμένου checkpoint και των απαραίτητων μεταδεδομένων, ενώ το δεύτερο αναλαμβάνει τη μετατροπή της εικόνας σε κατάλληλη μορφή, την εκτέλεση του μοντέλου και την επεξεργασία της εξόδου του.

Το εκπαιδευμένο μοντέλο αποθηκεύεται σε αρχείο checkpoint μορφής .pth μέσα στον φάκελο models/trained. Κατά την εκκίνηση της εφαρμογής, το model.py προσδιορίζει τη διαδρομή του checkpoint και επιλέγει αυτόματα τη διαθέσιμη υπολογιστική συσκευή. Εάν υπάρχει συμβατή GPU χρησιμοποιείται CUDA, ενώ σε διαφορετική περίπτωση η εκτέλεση πραγματοποιείται στην CPU. Με αυτόν τον τρόπο, η ίδια εφαρμογή μπορεί να λειτουργήσει σε διαφορετικά συστήματα χωρίς να απαιτείται χειροκίνητη αλλαγή της συσκευής εκτέλεσης.

```
BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = (
    BASE_DIR
    / "models"
    / "trained"
    / "..."
    / "trained_model.pth"
)
DEVICE = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
```

Κώδικας 7.5: *Ορισμός διαδρομής του checkpoint και επιλογή συσκευής εκτέλεσης*

Το checkpoint δεν περιλαμβάνει μόνο τις εκπαιδευμένες παραμέτρους του νευρωνικού δικτύου, αλλά και πληροφορίες που είναι απαραίτητες για τη σωστή εκτέλεση της πρόβλεψης. Από αυτό ανακτώνται οι ονομασίες των κλάσεων, ο αριθμός των κατηγοριών, το μέγεθος εισόδου των εικόνων και οι τιμές κανονικοποίησης που χρησιμοποιήθηκαν κατά την εκπαίδευση. Η αποθήκευση αυτών των πληροφοριών μαζί με το μοντέλο μειώνει τον κίνδυνο να χρησιμοποιηθούν κατά το inference διαφορετικές ρυθμίσεις από εκείνες με τις οποίες είχε εκπαιδευτεί και αξιολογηθεί το δίκτυο.

```
checkpoint = torch.load(
    MODEL_PATH,
    map_location=DEVICE,
    weights_only=False
)
class_names = checkpoint["class_names"]
number_of_classes = checkpoint["number_of_classes"]
metadata = {
    "class_names": class_names,
```

```
    "image_size": checkpoint["image_size"],
    "imagenet_mean": checkpoint["imagenet_mean"],
    "imagenet_std": checkpoint["imagenet_std"],
    "device": str(DEVICE)
}
```

Κώδικας 7.6: *Ανάκτηση μεταδεδομένων από το εκπαιδευμένο checkpoint*

Μετά τη δημιουργία της αρχιτεκτονικής του τελικού μοντέλου και τη φόρτωση των εκπαιδευμένων βαρών, το μοντέλο μεταφέρεται στην επιλεγμένη συσκευή και τίθεται σε κατάσταση αξιολόγησης με τη μέθοδο eval(). Η κατάσταση αυτή είναι απαραίτητη κατά το inference, καθώς το μοντέλο δεν πραγματοποιεί πλέον εκπαίδευση αλλά χρησιμοποιείται αποκλειστικά για παραγωγή προβλέψεων σε νέες εικόνες. Το τελικό αντικείμενο του μοντέλου και τα μεταδεδομένα του γίνονται στη συνέχεια διαθέσιμα στο predict.py.

```
model.load_state_dict(
    checkpoint["model_state_dict"]
)
model.to(DEVICE)
model.eval()
```

Κώδικας 7.7: *Προετοιμασία του εκπαιδευμένου μοντέλου για inference*

Η εικόνα που αποστέλλεται από τον χρήστη μέσω της διαδικτυακής φόρμας φτάνει στη συνάρτηση predict_image() ως αντικείμενο PIL.Image. Πριν δοθεί ως είσοδος στο νευρωνικό δίκτυο πραγματοποιείται έλεγχος του χρωματικού τρόπου της εικόνας και, εφόσον απαιτείται, μετατροπή σε RGB. Στη συνέχεια εφαρμόζεται ακολουθία μετασχηματισμών που περιλαμβάνει αλλαγή μεγέθους, μετατροπή σε tensor και κανονικοποίηση με τις τιμές που έχουν αποθηκευτεί στα μεταδεδομένα του μοντέλου.

```
image_transform = transforms.Compose([
    transforms.Resize((
        model_metadata["image_size"],
        model_metadata["image_size"]
    )),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=model_metadata["imagenet_mean"],
        std=model_metadata["imagenet_std"])
])
```

Κώδικας 7.8: *Προεπεξεργασία της εικόνας πριν από την πρόβλεψη*

Η χρήση των μεταδεδομένων του checkpoint για το μέγεθος της εικόνας και τις τιμές κανονικοποίησης εξασφαλίζει ότι η διαδικασία inference παραμένει συνεπής με τη διαδικασία προετοιμασίας των δεδομένων που χρησιμοποιήθηκε κατά την εκπαίδευση. Στη συγκεκριμένη εφαρμογή χρησιμοποιούνται οι τιμές κανονικοποίησης του ImageNet που είχαν αποθηκευτεί μαζί με το μοντέλο, ενώ η εικόνα μετατρέπεται στις απαιτούμενες χωρικές διαστάσεις πριν εισαχθεί στο δίκτυο.
Μετά την εφαρμογή των μετασχηματισμών, το tensor της εικόνας διαθέτει τις διαστάσεις που αντιστοιχούν σε μία μεμονωμένη εικόνα. Επειδή τα μοντέλα της PyTorch αναμένουν είσοδο σε μορφή batch, προστίθεται μία επιπλέον διάσταση με τη μέθοδο unsqueeze(0). Το tensor μεταφέρεται στη συνέχεια στην ίδια υπολογιστική συσκευή στην οποία βρίσκεται το μοντέλο.

```
if image.mode != "RGB":
    image = image.convert("RGB")
image_tensor = image_transform(image)
image_tensor = (
    image_tensor
    .unsqueeze(0)
    .to(DEVICE)
)
```

Κώδικας 7.9: *Προετοιμασία του tensor εισόδου*

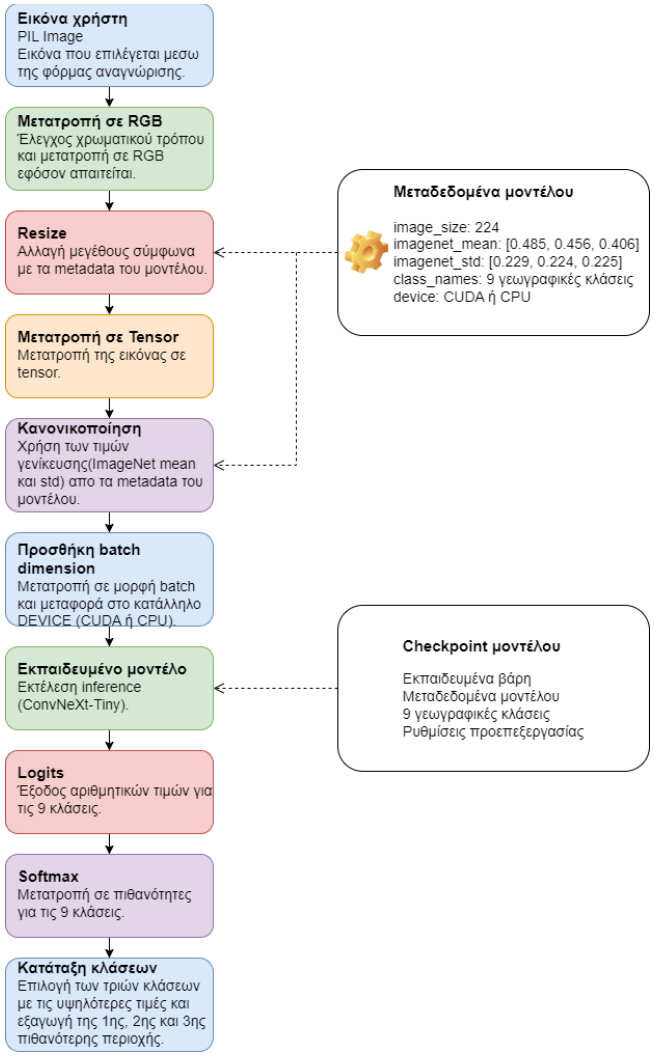
Η εκτέλεση της πρόβλεψης πραγματοποιείται μέσα σε περιβάλλον torch.inference_mode(). Με αυτόν τον τρόπο δεν υπολογίζονται gradients, καθώς δεν πραγματοποιείται ενημέρωση των παραμέτρων του δικτύου. Το μοντέλο δέχεται το προεπεξεργασμένο tensor και παράγει ως έξοδο logits, δηλαδή αριθμητικές τιμές για καθεμία από τις εννέα γεωγραφικές κλάσεις. Οι τιμές αυτές μετατρέπονται στη συνέχεια σε πιθανότητες μέσω της συνάρτησης Softmax.

```
with torch.inference_mode():
    logits = model(image_tensor)

    probabilities = torch.softmax(
        logits,
        dim=1
    )
```

Κώδικας 7.10: *Εκτέλεση inference και μετατροπή των logits σε πιθανότητες*

Η συνολική πορεία της εικόνας από τη μορφή που παρέχει ο χρήστης έως την αριθμητική έξοδο του μοντέλου παρουσιάζεται στο Σχήμα 7.4.



Σχήμα 7.4: Διαδοχικά στάδια προεπεξεργασίας της εισερχόμενης εικόνας και εκτέλεσης του εκπαιδευμένου μοντέλου.

Η έξοδος της Softmax περιλαμβάνει μία πιθανότητα για καθεμία από τις εννέα γεωγραφικές κατηγορίες του προβλήματος. Ωστόσο, η εφαρμογή δεν εμφανίζει στον χρήστη και τις εννέα τιμές. Από αυτές επιλέγονται οι τρεις υψηλότερες, οι οποίες αποτελούν τα Top-3 αποτελέσματα της αναγνώρισης. Η διαδικασία επιλογής, μετατροπής των εσωτερικών ονομασιών των κλάσεων σε ελληνικές ονομασίες και παρουσίασης των αντίστοιχων ποσοστών αναλύεται στην επόμενη ενότητα.

7.5 ΥΠΟΛΟΓΙΣΜΟΣ ΚΑΙ ΠΑΡΟΥΣΙΑΣΗ ΤΩΝ TOP-3 ΠΡΟΒΛΕΨΕΩΝ

Μετά την εκτέλεση του μοντέλου και τη μετατροπή των logits σε πιθανότητες μέσω της συνάρτησης Softmax, απαιτείται η επιλογή των κλάσεων που συγκεντρώνουν τις υψηλότερες τιμές. Στην εφαρμογή δεν χρησιμοποιείται αποκλειστικά η κλάση που κατατάσσεται πρώτη, αλλά εξάγονται οι τρεις επικρατέστερες γεωγραφικές περιοχές. Η συγκεκριμένη διαδικασία υλοποιείται στη συνάρτηση predict_image() του αρχείου predict.py με τη συνάρτηση torch.topk().

Η torch.topk() δέχεται τον πίνακα πιθανοτήτων που έχει προκύψει από τη Softmax και επιστρέφει δύο σύνολα τιμών: τις υψηλότερες πιθανότητες και τους δείκτες των κλάσεων στις οποίες αυτές αντιστοιχούν. Στην παρούσα εφαρμογή η προεπιλεγμένη τιμή της παραμέτρου top_k είναι ίση με 3. Παράλληλα, χρησιμοποιείται η συνάρτηση min() ώστε ο αριθμός των ζητούμενων αποτελεσμάτων να μην μπορεί να υπερβεί τον συνολικό αριθμό των διαθέσιμων κλάσεων.

```
top_probabilities, top_indices = torch.topk(
    probabilities,
    k=min(
        top_k,
        len(model_metadata["class_names"])
    ),
    dim=1
)
```

Κώδικας 7.11: *Επιλογή των τριών κλάσεων με τις υψηλότερες πιθανότητες*

Οι δείκτες που επιστρέφονται από τη torch.topk() δεν αποτελούν από μόνοι τους κατανοητό αποτέλεσμα για τον τελικό χρήστη. Για τον λόγο αυτό χρησιμοποιείται η λίστα class_names που έχει ανακτηθεί από τα μεταδεδομένα του checkpoint. Κάθε αριθμητικός δείκτης αντιστοιχίζεται στην εσωτερική ονομασία της αντίστοιχης γεωγραφικής κλάσης. Με αυτόν τον τρόπο διατηρείται η ίδια σειρά κλάσεων που χρησιμοποιήθηκε κατά την εκπαίδευση και την αξιολόγηση του μοντέλου.

```
class_name = (
    model_metadata["class_names"][
        class_index.item()
    ]
)
```

Κώδικας 7.12: *Αντιστοίχιση του δείκτη πρόβλεψης στην αντίστοιχη κλάση*

Οι εσωτερικές ονομασίες των κλάσεων έχουν μορφή κατάλληλη για χρήση στον κώδικα, όπως ipeiros, kriti, makedonia και sterea_ellada. Για την παρουσίασή τους στη διαδικτυακή διεπαφή χρησιμοποιείται αντιστοίχιση στις αντίστοιχες ελληνικές ονομασίες. Η αντιστοίχιση πραγματοποιείται μέσω του λεξικού GREEK_CLASS_NAMES, το οποίο καλύπτει και τις εννέα κατηγορίες του προβλήματος ταξινόμησης.

```
GREEK_CLASS_NAMES = {
    "ipeiros": "Ήπειρος",
    "kriti": "Κρήτη",
    "makedonia": "Μακεδονία",
    "nisia_aigaiou": "Νησιά Αιγαίου",
```

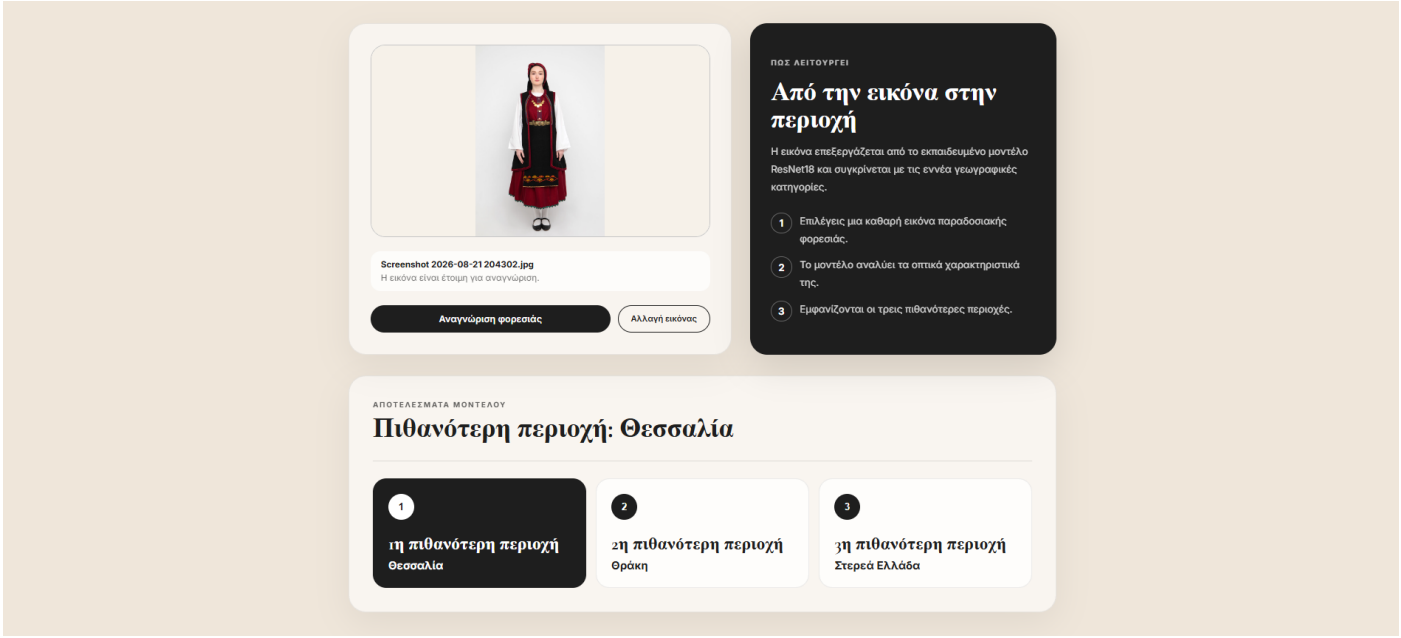
```
    "nisia_ioniou": "Νησιά Ιονίου",
    "peloponnisos": "Πελοπόννησος",
    "sterea_ellada": "Στερεά Ελλάδα",
    "thessalia": "Θεσσαλία",
    "thraki": "Θράκη",
}
```

Κώδικας 7.13: *Αντιστοίχιση των εσωτερικών ονομασιών στις ελληνικές γεωγραφικές περιοχές*

Οι αριθμητικές πιθανότητες εξακολουθούν να χρησιμοποιούνται εσωτερικά από την εφαρμογή, καθώς είναι απαραίτητες για τον προσδιορισμό της σειράς κατάταξης των κλάσεων. Ωστόσο, στη διεπαφή του τελικού χρήστη επιλέγεται να παρουσιάζεται η κατάταξη και όχι η αριθμητική τιμή κάθε πιθανότητας. Με αυτόν τον τρόπο, το αποτέλεσμα αποδίδεται ως «1η πιθανότερη περιοχή», «2η πιθανότερη περιοχή» και «3η πιθανότερη περιοχή». Η παρουσίαση αυτή διατηρεί την πληροφορία που είναι απαραίτητη για την ερμηνεία της πρόβλεψης χωρίς να προσδίδει στα αριθμητικά scores μεγαλύτερη βεβαιότητα από εκείνη που μπορούν να υποστηρίξουν.

Η διάκριση μεταξύ της εσωτερικής επεξεργασίας και της τελικής παρουσίασης είναι σημαντική. Η Softmax και οι αριθμητικές τιμές χρησιμοποιούνται από το λογισμικό για την ταξινόμηση των εννέα κλάσεων, ενώ ο χρήστης βλέπει μόνο το αποτέλεσμα της κατάταξης. Επομένως, η απουσία ποσοστών από τη διεπαφή δεν μεταβάλλει τη διαδικασία inference ούτε τον τρόπο με τον οποίο επιλέγονται οι Top-3 κλάσεις.

Τα αποτελέσματα της πρόβλεψης επιστρέφονται από το backend στο recognition.html, όπου η Jinja χρησιμοποιείται για τη δυναμική δημιουργία των αντίστοιχων στοιχείων της σελίδας. Η πρώτη θέση παρουσιάζεται ως η επικρατέστερη πρόβλεψη του μοντέλου, ενώ ακολουθούν η δεύτερη και η τρίτη πιθανότερη περιοχή. Η συγκεκριμένη σχεδίαση προσφέρει στον χρήστη μια άμεση και εύκολα κατανοητή εικόνα της κατάταξης που παρήγαγε το μοντέλο.



Εικόνα 7.3: *Παρουσίαση των τριών πιθανότερων γεωγραφικών περιοχών στη σελίδα αναγνώρισης της εφαρμογής.*

Με την ολοκλήρωση της παραπάνω διαδικασίας, η αριθμητική έξοδος του νευρωνικού δικτύου μετατρέπεται σε πληροφορία κατάλληλη για παρουσίαση στον τελικό χρήστη. Η επόμενη ενότητα επικεντρώνεται στη σχεδίαση της διεπαφής και στις τεχνολογίες που χρησιμοποιήθηκαν για την οργάνωση και την οπτική παρουσίαση των επιμέρους σελίδων της εφαρμογής.

7.6 ΣΧΕΔΙΑΣΗ ΤΗΣ ΔΙΕΠΑΦΗΣ ΧΡΗΣΤΗ

Η σχεδίαση της διεπαφής της διαδικτυακής εφαρμογής πραγματοποιήθηκε με χρήση HTML, CSS και JavaScript, με στόχο τη δημιουργία ενός απλού και ευανάγνωστου περιβάλλοντος στο οποίο ο χρήστης μπορεί να περιηγείται στις επιμέρους σελίδες και να εκτελεί τη διαδικασία αναγνώρισης χωρίς να απαιτείται γνώση της εσωτερικής λειτουργίας του μοντέλου. Η εφαρμογή οργανώνεται σε τέσσερις βασικές σελίδες: την αρχική σελίδα, τη σελίδα αναγνώρισης, τη σελίδα «Περιοχές & Φορεσιές» και τη σελίδα «Σχετικά». Η πρόσβαση στις σελίδες αυτές πραγματοποιείται μέσω κοινής γραμμής πλοήγησης, η οποία διατηρείται σε ολόκληρη την εφαρμογή.

Η γραμμή πλοήγησης περιλαμβάνει την ονομασία της εφαρμογής και συνδέσμους προς τις τέσσερις βασικές διαδρομές του Flask. Η ενεργή σελίδα επισημαίνεται μέσω της κλάσης is-active, ώστε ο χρήστης να γνωρίζει σε ποιο τμήμα της εφαρμογής βρίσκεται. Η επιλογή αυτή συμβάλλει στη διατήρηση σταθερής δομής πλοήγησης και οπτικής συνέπειας μεταξύ των διαφορετικών σελίδων.

```
<header class="nav">
  <div class="navInner">


    <div class="logo">
      Ελληνικές Φορεσιές
      <span>Μάθηση με χρήση πολυμέσων</span>
    </div>


    <nav class="menu">
      <a href="/">Αρχική</a>
      <a class="is-active" href="/recognition">Αναγνώριση</a>
      <a href="/regions">Περιοχές & Φορεσιές</a>
      <a href="/about">Σχετικά</a>
    </nav>


  </div>
</header>
```

Κώδικας 7.14: *Δομή της βασικής γραμμής πλοήγησης*

Η συνολική εμφάνιση της εφαρμογής καθορίζεται κεντρικά μέσω του αρχείου style.css. Ως βασική γραμματοσειρά του σώματος χρησιμοποιείται η Inter, ενώ για μεγαλύτερους τίτλους και επιλεγμένα στοιχεία χρησιμοποιείται η Cormorant Garamond. Το φόντο της εφαρμογής έχει ανοιχτή μπεζ απόχρωση (#efe6da), ενώ το βασικό χρώμα κειμένου είναι σκούρο (#1e1e1e). Με αυτόν τον τρόπο επιτυγχάνεται διακριτή διαφοροποίηση ανάμεσα στους τίτλους, στο κύριο περιεχόμενο και στα διαδραστικά στοιχεία χωρίς χρήση μεγάλου αριθμού διαφορετικών χρωμάτων.

```
@import url(

'https://fonts.googleapis.com/css2?family=Cormorant+Garamond:wght@500;600;700&family=Inter:wght@400;500;600;700&display=swap'

);


body{

  background:#efe6da;

  color:#1e1e1e;

  font-family:'Inter',sans-serif;

}


.pageTitle{

  font-family:'Cormorant Garamond',serif;

  font-size:58px;

  line-height:1.05;

}
```

Κώδικας 7.15: *Βασικές ρυθμίσεις εμφάνισης της εφαρμογής*

Η σελίδα αναγνώρισης αποτελεί το βασικό διαδραστικό τμήμα της εφαρμογής. Στο αριστερό μέρος βρίσκεται η περιοχή μεταφόρτωσης της εικόνας, ενώ δίπλα της εμφανίζεται σύντομη επεξήγηση των βημάτων που ακολουθεί η εφαρμογή. Η φόρμα χρησιμοποιεί τη μέθοδο POST και τον τύπο κωδικοποίησης multipart/form-data, ώστε το αρχείο εικόνας να μπορεί να αποσταλεί στο Flask backend. Το πεδίο επιλογής περιορίζεται σε αρχεία εικόνας των τύπων JPG, JPEG, PNG και WEBP.

```
<form

  class="uploadBox"

  method="POST"

  action="/recognition"

  enctype="multipart/form-data"

>

  <label

    class="uploadArea"

    id="uploadArea"
```

```
    for="imageInput"
  >
    <input
      type="file"
      id="imageInput"
      name="image"
      accept="image/png, image/jpeg, image/jpg, image/webp"
      required
    >
    <div class="uploadPlaceholder">
      <div class="uploadIcon">
        +
      </div>
      <h2>
        Ανέβασε μια εικόνα
      </h2>
    </div>
    <img
      class="imagePreview"
      id="imagePreview"
      alt="Προεπισκόπηση επιλεγμένης εικόνας"
    >
  </label>
</form>
```

Κώδικας 7.16: *Πεδίο επιλογής και μεταφόρτωσης εικόνας*

Η επιλογή μιας εικόνας δεν οδηγεί άμεσα στην αποστολή της στο backend. Πριν από την αναγνώριση, χρησιμοποιείται JavaScript ώστε να εμφανίζεται προεπισκόπηση του τοπικού αρχείου που επέλεξε ο χρήστης. Η μέθοδος URL.createObjectURL() δημιουργεί προσωρινό URL για το επιλεγμένο αρχείο και αυτό ανατίθεται στο στοιχείο imagePreview. Παράλληλα, προστίθενται οι CSS κλάσεις has-image και is-visible, οι οποίες αλλάζουν την εμφάνιση του πεδίου και εμφανίζουν πληροφορίες για το επιλεγμένο αρχείο. Η λειτουργία αυτή αφορά αποκλειστικά τη διεπαφή και προηγείται της αποστολής της εικόνας στο Flask backend.

```
imageInput.addEventListener("change", function () {
  const file = this.files[0];
  if (!file) {
    return;
  }
  const previewUrl = URL.createObjectURL(file);
  imagePreview.src = previewUrl;
  uploadArea.classList.add("has-image");
  fileDetails.classList.add("is-visible");
  fileName.textContent = file.name;
});
```

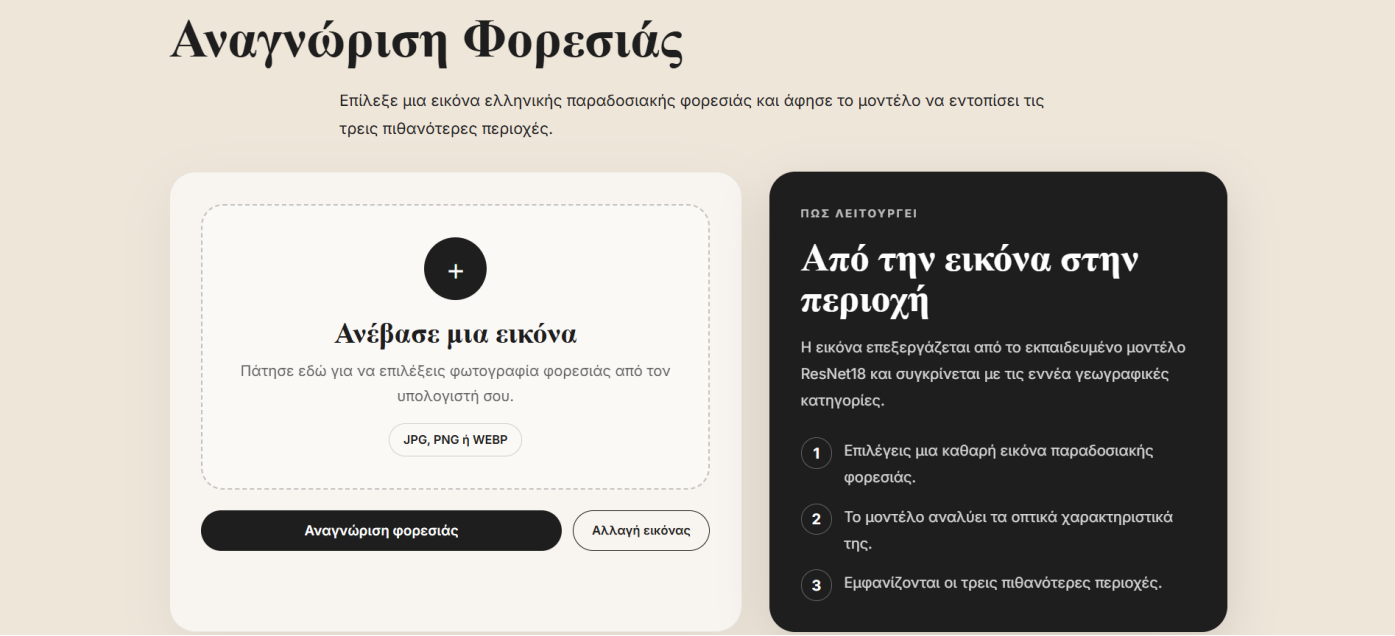
Κώδικας 7.17: *Δημιουργία προεπισκόπησης της επιλεγμένης εικόνας*

Η αντίστοιχη συμπεριφορά ελέγχεται από το CSS. Αρχικά το στοιχείο imagePreview παραμένει κρυφό. Μετά την προσθήκη της κλάσης has-image, το αρχικό μήνυμα μεταφόρτωσης αποκρύπτεται και στη θέση του εμφανίζεται η επιλεγμένη εικόνα. Η χρήση object-fit: contain επιτρέπει την προσαρμογή διαφορετικών αναλογιών εικόνας μέσα στη διαθέσιμη περιοχή χωρίς να απαιτείται περικοπή της εικόνας.

```
.imagePreview{
  display:none;
  width:100%;
  height:310px;
  object-fit:contain;
  background:#f6f1e9;
}
.uploadArea.has-image .uploadPlaceholder{
  display:none;
}
.uploadArea.has-image .imagePreview{
  display:block;
}
```

Κώδικας 7.18: *Εναλλαγή μεταξύ περιοχής μεταφόρτωσης και προεπισκόπησης*

Μετά την εκτέλεση της πρόβλεψης, τα αποτελέσματα ενσωματώνονται δυναμικά στην ίδια σελίδα μέσω του template recognition.html. Για την παρουσίαση των Top-3 αποτελεσμάτων χρησιμοποιείται διάταξη καρτών, στην οποία η πρώτη θέση διαφοροποιείται οπτικά από τις επόμενες. Η εμφάνιση της κατάταξης σε ξεχωριστές κάρτες επιτρέπει στον χρήστη να αντιληφθεί άμεσα τη σειρά των τριών επικρατέστερων περιοχών. Στην τελική μορφή της διεπαφής εμφανίζεται η 1η, η 2η και η 3η πιθανότερη περιοχή, ενώ οι αριθμητικές πιθανότητες που χρησιμοποιούνται εσωτερικά από το μοντέλο δεν είναι απαραίτητο να προβάλλονται στον χρήστη.



Εικόνα 7.4: *Σελίδα αναγνώρισης με την περιοχή μεταφόρτωσης εικόνας και το ενημερωτικό τμήμα της διαδικασίας αναγνώρισης.*

Η εφαρμογή έχει επίσης σχεδιαστεί ώστε να προσαρμόζει τη διάταξή της ανάλογα με το διαθέσιμο πλάτος της οθόνης. Η προσαρμογή αυτή υλοποιείται με CSS media queries. Για παράδειγμα, όταν το πλάτος της οθόνης είναι μικρότερο από 900 pixels, η βασική διάταξη της σελίδας αναγνώρισης μετατρέπεται από δύο στήλες σε μία, ενώ οι τρεις κάρτες αποτελεσμάτων εμφανίζονται επίσης κατακόρυφα. Αντίστοιχες προσαρμογές χρησιμοποιούνται στη γραμμή πλοήγησης, στις κάρτες των περιοχών και στα υπόλοιπα βασικά στοιχεία της εφαρμογής.

```
@media(max-width:900px){
  .recognitionLayout{
    grid-template-columns:1fr;}
  .recognitionInfo{
    min-height:300px;}
  .predictionGrid{
    grid-template-columns:1fr;}
}
```

Κώδικας 7.19: *Προσαρμογή της σελίδας αναγνώρισης σε μικρότερο πλάτος οθόνης*

Η επιλογή responsive σχεδίασης δεν μεταβάλλει τη λειτουργία του συστήματος αναγνώρισης, αλλά επηρεάζει τον τρόπο με τον οποίο παρουσιάζεται το περιεχόμενο στον χρήστη. Η ίδια HTML δομή μπορεί επομένως να προσαρμόζεται σε διαφορετικές διαστάσεις οθόνης χωρίς να απαιτείται ξεχωριστή έκδοση της εφαρμογής για κάθε συσκευή.

Πέρα από τη σελίδα αναγνώρισης, κοινή οπτική λογική εφαρμόζεται και στα υπόλοιπα τμήματα της εφαρμογής. Η σελίδα «Σχετικά» χρησιμοποιεί διάταξη καρτών για την παρουσίαση του σκοπού, των τεχνολογιών και των βασικών πληροφοριών της πτυχιακής εργασίας, ενώ η ίδια γραμμή πλοήγησης και οι ίδιες βασικές τυπογραφικές επιλογές επαναχρησιμοποιούνται ώστε να διατηρείται ενιαία αισθητική. Η σελίδα αναφέρει επίσης τις τεχνολογίες HTML, CSS, JavaScript, Python, Flask και PyTorch που χρησιμοποιούνται στην εφαρμογή.

Συνολικά, η διεπαφή λειτουργεί ως το επίπεδο επικοινωνίας μεταξύ του χρήστη και του συστήματος αναγνώρισης. Η HTML οργανώνει τη δομή των σελίδων, το CSS καθορίζει την οπτική εμφάνιση και την προσαρμογή της διάταξης, ενώ η JavaScript προσθέτει άμεση αλληλεπίδραση πριν από την αποστολή της εικόνας.

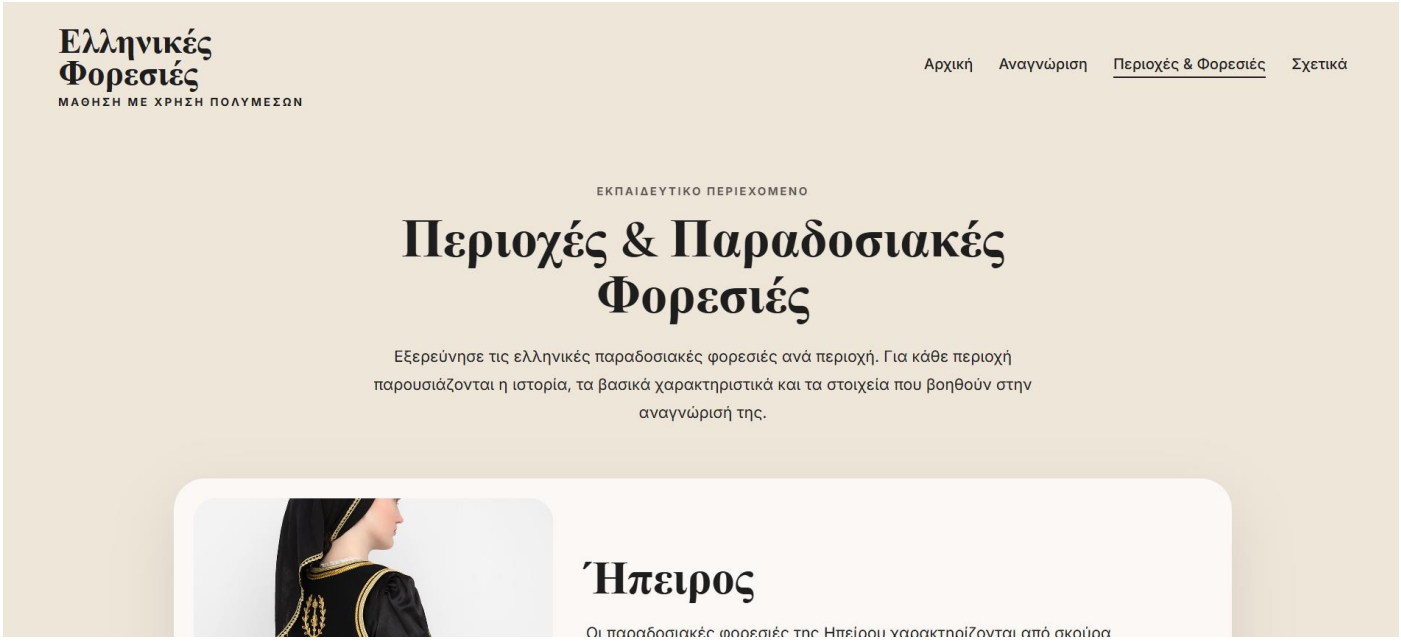
Με αυτόν τον τρόπο, η λειτουργικότητα του backend και του μοντέλου παρουσιάζεται μέσα από ένα συνεκτικό και εύχρηστο περιβάλλον, χωρίς να εκτίθενται στον τελικό χρήστη οι εσωτερικές τεχνικές λεπτομέρειες της διαδικασίας inference.

7.7 ΕΚΠΑΙΔΕΥΤΙΚΟ ΠΕΡΙΕΧΟΜΕΝΟ ΚΑΙ ΈΛΕΓΧΟΣ ΛΕΙΤΟΥΡΓΙΑΣ

Εκτός από τη λειτουργία αναγνώρισης μιας παραδοσιακής φορεσιάς, η διαδικτυακή εφαρμογή σχεδιάστηκε ώστε να παρέχει και συμπληρωματικό εκπαιδευτικό περιεχόμενο. Για τον σκοπό αυτό δημιουργήθηκε η σελίδα «Περιοχές & Φορεσιές», μέσω της οποίας ο χρήστης μπορεί να εξερευνήσει τις εννέα γεωγραφικές κατηγορίες που χρησιμοποιούνται από το σύστημα ταξινόμησης. Με αυτόν τον τρόπο, οι κλάσεις του μοντέλου δεν παρουσιάζονται μόνο ως εσωτερικές ετικέτες ενός προβλήματος Machine Learning, αλλά συνδέονται με πληροφορίες που μπορούν να προβληθούν και να εξερευνηθούν μέσα από τη διεπαφή.

Οι εννέα κατηγορίες που παρουσιάζονται αντιστοιχούν στις κλάσεις του συνόλου δεδομένων και είναι η Ήπειρος, η Κρήτη, η Μακεδονία, τα Νησιά Αιγαίου, τα Νησιά Ιονίου, η Πελοπόννησος, η Στερεά Ελλάδα, η Θεσσαλία και η Θράκη. Η αντιστοιχία μεταξύ των κατηγοριών του μοντέλου και των περιοχών που παρουσιάζονται στη διαδικτυακή εφαρμογή είναι σημαντική, καθώς διατηρεί κοινή οργάνωση μεταξύ του συστήματος ταξινόμησης και του εκπαιδευτικού περιεχομένου.

Η σελίδα regions.html οργανώνει τις περιοχές σε επιμέρους κάρτες, οι οποίες συνδυάζουν ονομασία, εικόνα και σχετικό περιεχόμενο. Η χρήση καρτών επιτρέπει τη διακριτή παρουσίαση κάθε γεωγραφικής κατηγορίας και διευκολύνει την περιήγηση χωρίς να συγκεντρώνεται όλο το πληροφοριακό υλικό σε ένα ενιαίο τμήμα της σελίδας.



Εικόνα 7.5: *Σελίδα «Περιοχές & Φορεσιές» της διαδικτυακής εφαρμογής με τις γεωγραφικές κατηγορίες των παραδοσιακών φορεσιών.*

Για την εμφάνιση περισσότερων πληροφοριών χωρίς μετάβαση σε ξεχωριστή σελίδα για κάθε περιοχή, η διεπαφή αξιοποιεί αναδυόμενα παράθυρα (modal windows). Ο χρήστης μπορεί να επιλέξει μια περιοχή και να εμφανίσει το αντίστοιχο περιεχόμενο μέσα στην ίδια σελίδα. Η συγκεκριμένη προσέγγιση διατηρεί την πλοήγηση συγκεντρωμένη και επιτρέπει την παρουσίαση πρόσθετου υλικού μόνο όταν αυτό ζητηθεί από τον χρήστη.



Εικόνα 7.6: *Παράδειγμα προβολής πληροφοριών για επιλεγμένη γεωγραφική περιοχή μέσω αναδυόμενου παραθύρου.*

Συμπληρωματικά, η σελίδα «Σχετικά» παρουσιάζει τον σκοπό της εφαρμογής και βασικές πληροφορίες σχετικά με την υλοποίησή της. Μέσω της συγκεκριμένης σελίδας ο χρήστης μπορεί να ενημερωθεί για τη σύνδεση της εφαρμογής με την παρούσα πτυχιακή εργασία και για τις βασικές τεχνολογίες που χρησιμοποιήθηκαν. Η ύπαρξη ξεχωριστής ενημερωτικής σελίδας επιτρέπει τον διαχωρισμό των πληροφοριών σχετικά με το έργο από την κύρια λειτουργία αναγνώρισης.

Μετά την ολοκλήρωση της σύνδεσης του frontend με το Flask backend και το εκπαιδευμένο μοντέλο, πραγματοποιήθηκε λειτουργικός έλεγχος της εφαρμογής σε τοπικό περιβάλλον. Η εφαρμογή εκκινείται μέσω του app.py και εξυπηρετείται από τον τοπικό Flask development server. Κατά την εκτέλεση είναι διαθέσιμη μέσω της διεύθυνσης http://127.0.0.1:5000/, γεγονός που επιτρέπει τον έλεγχο της ολοκληρωμένης ροής μέσα από έναν συμβατικό φυλλομετρητή.

Ο λειτουργικός έλεγχος αφορά τη συνεργασία των διαφορετικών τμημάτων του συστήματος και όχι νέα αξιολόγηση της ακρίβειας του μοντέλου, η οποία εξετάζεται ξεχωριστά στο Κεφάλαιο 6. Ελέγχεται αρχικά η ορθή φόρτωση των τεσσάρων βασικών σελίδων και η μετάβαση μεταξύ τους μέσω της γραμμής πλοήγησης. Στη συνέχεια ελέγχεται η επιλογή ενός υποστηριζόμενου αρχείου εικόνας, η εμφάνιση της προεπισκόπησής του και η δυνατότητα αλλαγής της επιλεγμένης εικόνας πριν από την υποβολή.

Το σημαντικότερο μέρος του ελέγχου αφορά την πλήρη διαδικασία αναγνώρισης. Μετά την επιλογή της εικόνας και την ενεργοποίηση του κουμπιού «Αναγνώριση φορεσιάς», το αρχείο αποστέλλεται μέσω αιτήματος POST στη διαδρομή /recognition. Το Flask backend ανοίγει την εικόνα και την προωθεί στη συνάρτηση πρόβλεψης. Μετά την ολοκλήρωση της προεπεξεργασίας και του inference, οι τρεις επικρατέστερες κλάσεις επιστρέφονται στη σελίδα και εμφανίζονται ως 1η, 2η και 3η πιθανότερη περιοχή. Με τον τρόπο αυτό ελέγχεται η συνολική αλυσίδα από την αλληλεπίδραση του χρήστη μέχρι την παρουσίαση της εξόδου του μοντέλου.

Παράλληλα ελέγχεται η λειτουργία του εκπαιδευτικού τμήματος της εφαρμογής, με μετάβαση στη σελίδα «Περιοχές & Φορεσιές», επιλογή διαφορετικών γεωγραφικών περιοχών και εμφάνιση του αντίστοιχου περιεχομένου. Με τον έλεγχο αυτό επιβεβαιώνεται ότι η εφαρμογή λειτουργεί ως ενιαίο σύστημα και ότι η λειτουργία αναγνώρισης, η πλοήγηση και το συμπληρωματικό εκπαιδευτικό περιεχόμενο είναι προσβάσιμα μέσα από την ίδια διεπαφή.

Η τοπική εκτέλεση αποτελεί το περιβάλλον ανάπτυξης και δοκιμής της εφαρμογής και δεν ισοδυναμεί με δημοσίευσή της σε διαδικτυακό διακομιστή παραγωγής. Παρ' όλα αυτά, είναι επαρκής για τον λειτουργικό έλεγχο της υλοποίησης στο πλαίσιο της πτυχιακής εργασίας, καθώς επιτρέπει την εκτέλεση του Flask backend, τη φόρτωση του εκπαιδευμένου μοντέλου και την αλληλεπίδραση με όλες τις λειτουργίες της εφαρμογής μέσω του φυλλομετρητή.

Συνεπώς, η τελική εφαρμογή συνδυάζει δύο συμπληρωματικές λειτουργίες: την αξιοποίηση του μοντέλου Machine Learning για την κατάταξη μιας εικόνας φορεσιάς στις τρεις πιθανότερες γεωγραφικές περιοχές και την παρουσίαση οργανωμένου περιεχομένου σχετικά με τις κατηγορίες που συμμετέχουν στο πρόβλημα ταξινόμησης. Ο συνδυασμός αυτός μεταφέρει το αποτέλεσμα της πειραματικής διαδικασίας από το περιβάλλον του notebook σε ένα ολοκληρωμένο και διαδραστικό λογισμικό σύστημα.

7.8 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΕΦΑΛΑΙΟΥ

Στο παρόν κεφάλαιο παρουσιάστηκε η ανάπτυξη της διαδικτυακής εφαρμογής που δημιουργήθηκε για την αξιοποίηση του εκπαιδευμένου μοντέλου αναγνώρισης ελληνικών παραδοσιακών φορεσιών. Η εφαρμογή υλοποιήθηκε με χρήση Python και Flask για το backend, ενώ για τη διεπαφή χρησιμοποιήθηκαν HTML, CSS και JavaScript. Παράλληλα, το PyTorch χρησιμοποιήθηκε για τη φόρτωση του εκπαιδευμένου μοντέλου και την εκτέλεση της διαδικασίας inference σε εικόνες που επιλέγονται από τον χρήστη.

Η αρχιτεκτονική της εφαρμογής οργανώθηκε με διαχωρισμό των βασικών λειτουργιών σε διαφορετικά αρχεία. Το app.py αναλαμβάνει τη διαχείριση των διαδρομών και των αιτημάτων HTTP, το model.py τη φόρτωση και αρχικοποίηση του εκπαιδευμένου μοντέλου, ενώ το predict.py υλοποιεί την προεπεξεργασία της εικόνας και τη διαδικασία πρόβλεψης. Η συγκεκριμένη οργάνωση επιτρέπει σαφέστερο διαχωρισμό μεταξύ της διαδικτυακής λειτουργίας της εφαρμογής και της λογικής του μοντέλου Machine Learning.

Ιδιαίτερη έμφαση δόθηκε στη διατήρηση της ίδιας διαδικασίας προεπεξεργασίας που χρησιμοποιείται και κατά την αξιολόγηση του μοντέλου. Η εισερχόμενη εικόνα μετατρέπεται σε RGB, προσαρμόζεται στο απαιτούμενο μέγεθος, μετατρέπεται σε tensor και κανονικοποιείται με τις αντίστοιχες παραμέτρους του μοντέλου. Στη συνέχεια εκτελείται η πρόβλεψη και η έξοδος μετατρέπεται μέσω Softmax σε τιμές που χρησιμοποιούνται για την κατάταξη των εννέα γεωγραφικών κλάσεων.

Αντί να παρουσιάζεται αποκλειστικά η πρώτη πρόβλεψη, η εφαρμογή εξάγει τις τρεις κλάσεις με τις υψηλότερες τιμές. Οι κλάσεις αυτές εμφανίζονται στον χρήστη ως 1η, 2η και 3η πιθανότερη περιοχή, χωρίς να είναι απαραίτητη η εμφάνιση των αριθμητικών πιθανοτήτων στη διεπαφή. Με αυτόν τον τρόπο διατηρείται η πληροφορία της σχετικής κατάταξης των προβλέψεων, ενώ η παρουσίαση παραμένει απλή και άμεσα κατανοητή.

Παράλληλα με τη λειτουργία αναγνώρισης, αναπτύχθηκε συμπληρωματικό εκπαιδευτικό περιεχόμενο για τις εννέα γεωγραφικές περιοχές που συμμετέχουν στο πρόβλημα ταξινόμησης. Η εφαρμογή περιλαμβάνει ξεχωριστές σελίδες για την αναγνώριση, την παρουσίαση των περιοχών και των φορεσιών, καθώς και για πληροφορίες σχετικά με το έργο. Με αυτόν τον τρόπο η εφαρμογή δεν λειτουργεί μόνο ως διεπαφή προς το μοντέλο, αλλά και ως οργανωμένο περιβάλλον παρουσίασης του αντικειμένου της πτυχιακής εργασίας.

Ο λειτουργικός έλεγχος της εφαρμογής πραγματοποιήθηκε σε τοπικό περιβάλλον μέσω του Flask development server. Ελέγχθηκε η πλοήγηση μεταξύ των σελίδων, η επιλογή και προεπισκόπηση εικόνας, η αποστολή της στο backend, η εκτέλεση της πρόβλεψης και η επιστροφή των Top-3 αποτελεσμάτων. Με τον τρόπο αυτό επιβεβαιώθηκε η συνεργασία των επιμέρους τμημάτων του συστήματος ως ενιαία εφαρμογή.

Συνολικά, το Κεφάλαιο 7 περιέγραψε τη μετάβαση από το εκπαιδευμένο μοντέλο σε μία πλήρως λειτουργική διαδικτυακή εφαρμογή. Η υλοποίηση αυτή αποτελεί το τελικό στάδιο σύνδεσης της πειραματικής διαδικασίας με ένα περιβάλλον που μπορεί να χρησιμοποιηθεί άμεσα από τον τελικό χρήστη. Στο επόμενο κεφάλαιο παρουσιάζονται τα συνολικά συμπεράσματα της εργασίας, οι περιορισμοί της παρούσας προσέγγισης και πιθανές κατευθύνσεις για μελλοντική επέκταση του συστήματος.

ΚΕΦΑΛΑΙΟ 8 - ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΑΙ ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ

8.1 ΣΥΝΟΛΙΚΗ ΑΠΟΤΙΜΗΣΗ ΤΗΣ ΕΡΓΑΣΙΑΣ

Η παρούσα πτυχιακή εργασία επικεντρώθηκε στην ανάπτυξη ενός ολοκληρωμένου συστήματος αναγνώρισης ελληνικών παραδοσιακών φορεσιών με τεχνικές Μηχανικής Μάθησης και, ειδικότερα, Βαθιάς Μάθησης και Υπολογιστικής Όρασης. Το πρόβλημα διατυπώθηκε ως επιβλεπόμενη ταξινόμηση εικόνων σε εννέα γεωγραφικές κατηγορίες: Ήπειρος, Κρήτη, Μακεδονία, Νησιά Αιγαίου, Νησιά Ιονίου, Πελοπόννησος, Στερεά Ελλάδα, Θεσσαλία και Θράκη. Η εργασία κάλυψε ολόκληρη τη διαδικασία ανάπτυξης ενός συστήματος Machine Learning, από τη δημιουργία και την προετοιμασία του συνόλου δεδομένων μέχρι την εκπαίδευση και αξιολόγηση διαφορετικών νευρωνικών δικτύων και, τελικά, την ενσωμάτωση εκπαιδευμένου μοντέλου σε διαδικτυακή εφαρμογή.

Ένα από τα βασικά στάδια της εργασίας ήταν η δημιουργία του συνόλου δεδομένων. Συγκεντρώθηκαν συνολικά 411 εικόνες, οι οποίες οργανώθηκαν στις εννέα κατηγορίες του προβλήματος. Πριν από την εκπαίδευση πραγματοποιήθηκε έλεγχος της δομής και της ποιότητας των δεδομένων, καθώς και διερεύνηση για ακριβώς όμοιες και οπτικά παρόμοιες εικόνες. Ιδιαίτερη σημασία δόθηκε στον διαχωρισμό του συνόλου δεδομένων σε σύνολα εκπαίδευσης, επικύρωσης και ελέγχου με τρόπο που να αποφεύγει τη μεταφορά συσχετιζόμενων εικόνων μεταξύ διαφορετικών υποσυνόλων και, κατά συνέπεια, τον κίνδυνο διαρροής δεδομένων.

Το τελικό σύνολο διαχωρίστηκε σε 286 εικόνες εκπαίδευσης, 63 εικόνες επικύρωσης και 62 εικόνες ελέγχου. Κατά την εκπαίδευση εφαρμόστηκαν τεχνικές επαύξησης δεδομένων, όπως μικρές περιστροφές, μετατοπίσεις, αλλαγές κλίμακας και περιορισμένες μεταβολές φωτεινότητας και αντίθεσης. Οι μετασχηματισμοί εφαρμόζονταν δυναμικά μόνο στο training set, ενώ στα validation και test sets χρησιμοποιήθηκε σταθερή διαδικασία προεπεξεργασίας. Η διάκριση αυτή επέτρεψε την εισαγωγή ελεγχόμενης μεταβλητότητας κατά την εκπαίδευση χωρίς να μεταβάλλονται τα δεδομένα που χρησιμοποιήθηκαν για την αξιολόγηση.

Για την αντιμετώπιση του προβλήματος εξετάστηκαν τρεις αρχιτεκτονικές προεκπαιδευμένων συνελικτικών νευρωνικών δικτύων: ResNet18, EfficientNet-B0 και ConvNeXt-Tiny. Για καθεμία από αυτές μελετήθηκαν δύο διαφορετικές στρατηγικές αξιοποίησης της Μεταφοράς Μάθησης, το Feature Extraction και το Partial Fine-Tuning. Με τον τρόπο αυτό προέκυψαν έξι διακριτές πειραματικές διαμορφώσεις, οι οποίες εκπαιδεύτηκαν και αξιολογήθηκαν υπό κοινό πειραματικό πλαίσιο. Η προσέγγιση αυτή επέτρεψε τη σύγκριση όχι μόνο διαφορετικών αρχιτεκτονικών αλλά και διαφορετικού βαθμού προσαρμογής των προεκπαιδευμένων χαρακτηριστικών στο συγκεκριμένο σύνολο δεδομένων.

Η αξιολόγηση των πειραμάτων δεν περιορίστηκε αποκλειστικά στην Accuracy. Χρησιμοποιήθηκαν συμπληρωματικές μετρικές, όπως Precision, Recall και F1-score, καθώς και ανάλυση ανά κλάση και πίνακες σύγχυσης, ώστε να εξεταστεί αναλυτικότερα η συμπεριφορά των μοντέλων στις διαφορετικές γεωγραφικές κατηγορίες. Παράλληλα, η χρήση της Top-3 Accuracy επέτρεψε να εξεταστεί κατά πόσο η πραγματική περιοχή μιας εικόνας περιλαμβάνεται μεταξύ των τριών επικρατέστερων προβλέψεων του μοντέλου. Η συγκεκριμένη πληροφορία αξιοποιήθηκε και στη σχεδίαση της διαδικτυακής εφαρμογής, όπου παρουσιάζονται οι τρεις πιθανότερες περιοχές με σειρά κατάταξης.

Το τελευταίο στάδιο της εργασίας αφορούσε τη μεταφορά του εκπαιδευμένου συστήματος από το πειραματικό περιβάλλον σε λειτουργική διαδικτυακή εφαρμογή. Για το backend χρησιμοποιήθηκε το Flask, ενώ η διεπαφή αναπτύχθηκε με HTML, CSS και JavaScript. Ο χρήστης μπορεί να επιλέξει μία εικόνα παραδοσιακής φορεσιάς και το σύστημα αναλαμβάνει την προεπεξεργασία της, την εκτέλεση inference και την επιστροφή της 1ης, 2ης και 3ης πιθανότερης γεωγραφικής περιοχής. Παράλληλα, η εφαρμογή περιλαμβάνει εκπαιδευτικό περιεχόμενο για τις εννέα περιοχές, συνδέοντας την τεχνική λειτουργία του μοντέλου με το θεματικό αντικείμενο της εργασίας.

8.2 ΒΑΣΙΚΑ ΣΥΜΠΕΡΑΣΜΑΤΑ

Τα πειράματα που πραγματοποιήθηκαν καταδεικνύουν ότι η Μεταφορά Μάθησης αποτελεί εφαρμόσιμη προσέγγιση για το συγκεκριμένο πρόβλημα, ιδιαίτερα λαμβάνοντας υπόψη το περιορισμένο μέγεθος του διαθέσιμου συνόλου δεδομένων.

Αντί για εκπαίδευση ενός βαθιού νευρωνικού δικτύου από τυχαία αρχικοποίηση, αξιοποιήθηκαν μοντέλα με βάρη προεκπαιδευμένα στο ImageNet και προσαρμόστηκαν στις εννέα κλάσεις της εργασίας. Με τον τρόπο αυτό αξιοποιούνται οπτικές αναπαραστάσεις που έχουν ήδη αποκτηθεί από ένα πολύ μεγαλύτερο σύνολο εικόνων.

Η σύγκριση των έξι πειραματικών διαμορφώσεων έδειξε επίσης ότι η επιλογή της αρχιτεκτονικής και η στρατηγική προσαρμογής των προεκπαιδευμένων βαρών επηρεάζουν την τελική συμπεριφορά του συστήματος. Το Feature Extraction και το Partial Fine-Tuning αποτελούν δύο διαφορετικές προσεγγίσεις: στην πρώτη περίπτωση διατηρείται παγωμένο το προεκπαιδευμένο τμήμα του δικτύου και εκπαιδεύεται κυρίως ο νέος ταξινομητής, ενώ στη δεύτερη επιτρέπεται η προσαρμογή και επιλεγμένων βαθύτερων επιπέδων. Τα αποτελέσματα των συγκεκριμένων πειραμάτων, όπως παρουσιάστηκαν αναλυτικά στο Κεφάλαιο 6, επιτρέπουν την αξιολόγηση της αποτελεσματικότητας κάθε επιλογής στο συγκεκριμένο dataset, χωρίς αυτό να σημαίνει ότι η ίδια κατάταξη των μοντέλων μπορεί να γενικευθεί αυτομάτως σε διαφορετικά σύνολα δεδομένων.

Ιδιαίτερη σημασία έχει επίσης η παρατήρηση ότι η ταξινόμηση παραδοσιακών φορεσιών αποκλειστικά από την εικόνα αποτελεί απαιτητικό πρόβλημα. Διαφορετικές γεωγραφικές κατηγορίες μπορεί να παρουσιάζουν κοινά ή παρόμοια οπτικά χαρακτηριστικά, όπως χρώματα, υφάσματα, διακοσμητικά μοτίβα και μορφολογικά στοιχεία. Ταυτόχρονα, μέσα στην ίδια γεωγραφική κατηγορία μπορεί να υπάρχουν σημαντικές διαφοροποιήσεις μεταξύ επιμέρους τοπικών ενδυμασιών. Επομένως, η γεωγραφική κατηγορία δεν αντιστοιχεί πάντοτε σε ένα μοναδικό και οπτικά ομοιογενές πρότυπο.

Η επιλογή παρουσίασης Top-3 αποτελεσμάτων στην τελική εφαρμογή είναι συμβατή με αυτή τη φύση του προβλήματος. Η εφαρμογή δεν περιορίζει την πληροφορία αποκλειστικά στην κλάση που κατατάσσεται πρώτη, αλλά παρουσιάζει και τις δύο επόμενες επικρατέστερες επιλογές. Οι πιθανότητες εξακολουθούν να χρησιμοποιούνται εσωτερικά για την κατάταξη, χωρίς όμως να εμφανίζονται ως ποσοστά στον τελικό χρήστη. Έτσι, η διεπαφή παρουσιάζει την έξοδο ως σειρά πιθανότερων περιοχών και όχι ως απόλυτη βεβαιότητα για τη γεωγραφική προέλευση μιας φορεσιάς.

8.3 ΠΕΡΙΟΡΙΣΜΟΙ ΤΗΣ ΠΑΡΟΥΣΑΣ ΠΡΟΣΕΓΓΙΣΗΣ

Ο σημαντικότερος περιορισμός της εργασίας αφορά το μέγεθος του συνόλου δεδομένων. Οι 411 εικόνες αποτελούν σχετικά μικρό αριθμό για ένα πρόβλημα ταξινόμησης εικόνων σε εννέα κατηγορίες με βαθιά νευρωνικά δίκτυα. Παρότι η χρήση Transfer Learning και Data Augmentation περιορίζει σε έναν βαθμό τις απαιτήσεις σε δεδομένα, δεν μπορεί να υποκαταστήσει πλήρως ένα μεγαλύτερο και περισσότερο αντιπροσωπευτικό σύνολο πραγματικών εικόνων.

Ένας δεύτερος περιορισμός σχετίζεται με την εσωτερική ποικιλία των κατηγοριών. Η ταξινόμηση πραγματοποιείται σε επίπεδο ευρύτερης γεωγραφικής περιοχής, ενώ στην πραγματικότητα οι ελληνικές παραδοσιακές φορεσιές χαρακτηρίζονται από σημαντικές τοπικές διαφοροποιήσεις. Συνεπώς, δύο φορεσιές της ίδιας ευρύτερης περιοχής μπορεί να παρουσιάζουν σημαντικές οπτικές διαφορές, ενώ φορεσιές διαφορετικών περιοχών μπορεί να διαθέτουν κοινά χαρακτηριστικά. Η συγκεκριμένη ιδιότητα αυξάνει τη δυσκολία του προβλήματος και μπορεί να οδηγήσει σε σύγχυση μεταξύ ορισμένων κλάσεων.

Επιπλέον, οι εικόνες του dataset προέρχονται από συγκεκριμένη διαδικασία συλλογής και επομένως δεν μπορούν να θεωρηθούν αντιπροσωπευτικές όλων των πιθανών συνθηκών στις οποίες θα μπορούσε να χρησιμοποιηθεί το σύστημα. Εικόνες από κινητό τηλέφωνο, διαφορετικό φωτισμό, διαφορετικές γωνίες λήψης, σύνθετο φόντο, μερικώς ορατή φορεσιά ή παρουσία περισσότερων ατόμων μπορούν να διαφέρουν σημαντικά από τα δεδομένα στα οποία εκπαιδεύτηκαν τα μοντέλα. Για τον λόγο αυτό, οι επιδόσεις που μετρήθηκαν στο test set δεν πρέπει να ερμηνεύονται ως εγγύηση αντίστοιχης επίδοσης σε κάθε πραγματικό σενάριο χρήσης.

Περιορισμό αποτελεί επίσης το γεγονός ότι τα πειράματα αφορούν συγκεκριμένες αρχιτεκτονικές, στρατηγικές προσαρμογής και υπερπαραμέτρους. Η εργασία συγκρίνει τις έξι πειραματικές διαμορφώσεις που υλοποιήθηκαν, χωρίς να πραγματοποιείται εξαντλητική αναζήτηση όλων των δυνατών αρχιτεκτονικών ή συνδυασμών υπερπαραμέτρων. Επομένως, η καλύτερη διαμόρφωση μεταξύ των πειραμάτων της εργασίας δεν πρέπει να θεωρηθεί καθολικά βέλτιστη λύση για το πρόβλημα.

Τέλος, η διαδικτυακή εφαρμογή αναπτύχθηκε και ελέγχθηκε σε τοπικό περιβάλλον μέσω του Flask development server.

Η υλοποίηση αποδεικνύει τη δυνατότητα ενσωμάτωσης του μοντέλου σε ολοκληρωμένο λογισμικό σύστημα, αλλά δεν περιλαμβάνει ανάπτυξη σε δημόσιο production server, διαχείριση μεγάλου αριθμού ταυτόχρονων χρηστών ή αξιολόγηση της απόδοσης της υπηρεσίας υπό πραγματικό διαδικτυακό φόρτο.

8.4 ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ

Η σημαντικότερη μελλοντική επέκταση αφορά τη διεύρυνση και περαιτέρω βελτίωση του συνόλου δεδομένων. Η συλλογή περισσότερων εικόνων ανά γεωγραφική περιοχή και η αύξηση της ποικιλίας ως προς τον φωτισμό, τη στάση του ατόμου, το φόντο, τη γωνία λήψης και τον τύπο της φορεσιάς θα μπορούσαν να δημιουργήσουν ένα περισσότερο αντιπροσωπευτικό dataset. Ιδιαίτερη σημασία θα είχε η συλλογή εικόνων από ανεξάρτητες πηγές και διαφορετικές πραγματικές συνθήκες, ώστε να εξεταστεί με μεγαλύτερη αξιοπιστία η ικανότητα γενίκευσης των μοντέλων.

Μία δεύτερη κατεύθυνση αφορά τη μετάβαση από την ταξινόμηση σε εννέα ευρείες γεωγραφικές περιοχές σε λεπτομερέστερη αναγνώριση συγκεκριμένων τοπικών φορεσιών. Ένα μελλοντικό σύστημα θα μπορούσε, για παράδειγμα, να επιχειρεί αρχικά την αναγνώριση της ευρύτερης περιοχής και στη συνέχεια να πραγματοποιεί δεύτερο επίπεδο ταξινόμησης μεταξύ επιμέρους τοπικών κατηγοριών. Μια τέτοια ιεραρχική προσέγγιση θα ανταποκρινόταν περισσότερο στην πραγματική ποικιλομορφία των ελληνικών παραδοσιακών ενδυμασιών, αλλά θα απαιτούσε σημαντικά μεγαλύτερο και λεπτομερέστερα επισημασμένο σύνολο δεδομένων.

Επιπλέον, θα μπορούσαν να εξεταστούν πρόσθετες αρχιτεκτονικές Υπολογιστικής Όρασης και διαφορετικές στρατηγικές Fine-Tuning. Με μεγαλύτερο σύνολο δεδομένων θα ήταν επίσης δυνατή η συστηματικότερη διερεύνηση υπερπαραμέτρων, διαφορετικών πολιτικών Data Augmentation και διαφορετικού αριθμού επιπέδων που προσαρμόζονται κατά το Fine-Tuning.

Η αξιολόγηση αυτών των επιλογών μέσω οργανωμένων πειραμάτων θα μπορούσε να προσδιορίσει με μεγαλύτερη ακρίβεια ποιες παράμετροι επηρεάζουν περισσότερο την απόδοση στο συγκεκριμένο πρόβλημα.

Μια ακόμη ενδιαφέρουσα επέκταση θα ήταν η αξιοποίηση τεχνικών ερμηνευσιμότητας της Βαθιάς Μάθησης. Μέθοδοι οπτικής επεξήγησης θα μπορούσαν να χρησιμοποιηθούν για να επισημαίνουν τα τμήματα μιας εικόνας που επηρέασαν περισσότερο την απόφαση του μοντέλου. Έτσι θα μπορούσε να εξεταστεί εάν το δίκτυο βασίζεται πράγματι σε χαρακτηριστικά της φορεσιάς, όπως συγκεκριμένα μοτίβα ή μορφολογικά στοιχεία, ή εάν επηρεάζεται από άσχετα χαρακτηριστικά της εικόνας, όπως το φόντο. Η δυνατότητα αυτή θα ήταν χρήσιμη τόσο για την τεχνική ανάλυση του μοντέλου όσο και για την εκπαιδευτική διάσταση της εφαρμογής.

Σε επίπεδο διαδικτυακής εφαρμογής, μελλοντική επέκταση θα μπορούσε να αποτελέσει η ανάπτυξη του συστήματος σε κατάλληλο production περιβάλλον, ώστε να είναι προσβάσιμο μέσω του διαδικτύου χωρίς να απαιτείται τοπική εκτέλεση. Παράλληλα, θα μπορούσαν να προστεθούν περισσότερες πληροφορίες για κάθε περιοχή και φορεσιά, πρόσθετο πολυμεσικό υλικό και δυνατότητες αναζήτησης ή φιλτραρίσματος του εκπαιδευτικού περιεχομένου. Η responsive σχεδίαση που έχει ήδη υλοποιηθεί μπορεί να αποτελέσει βάση για περαιτέρω προσαρμογή της εμπειρίας χρήσης σε κινητές συσκευές.

Τέλος, ιδιαίτερο ενδιαφέρον παρουσιάζει η δυνατότητα συνδυασμού της αναγνώρισης με περισσότερο εξειδικευμένο εκπαιδευτικό περιεχόμενο. Μετά την πρόβλεψη μιας περιοχής, το σύστημα θα μπορούσε να συνδέει άμεσα το αποτέλεσμα με σχετικές πληροφορίες, φωτογραφίες και χαρακτηριστικά των αντίστοιχων

φορεσιών. Με αυτόν τον τρόπο, η τεχνολογία αναγνώρισης εικόνας δεν θα αποτελούσε μόνο μηχανισμό ταξινόμησης, αλλά και σημείο εισόδου σε ένα ευρύτερο διαδραστικό περιβάλλον εξερεύνησης της ελληνικής παραδοσιακής ενδυμασίας.

8.5 ΤΕΛΙΚΟ ΣΥΜΠΕΡΑΣΜΑ

Η παρούσα εργασία ανέδειξε στην πράξη τη διαδικασία σχεδιασμού και υλοποίησης ενός συστήματος Υπολογιστικής Όρασης για ένα εξειδικευμένο πρόβλημα ταξινόμησης εικόνων. Η δημιουργία του dataset, η προεπεξεργασία και ο ελεγχόμενος διαχωρισμός των δεδομένων, η αξιοποίηση προεκπαιδευμένων νευρωνικών δικτύων, η σύγκριση διαφορετικών στρατηγικών Transfer Learning, η αξιολόγηση των αποτελεσμάτων και η τελική ενσωμάτωση του μοντέλου σε διαδικτυακή εφαρμογή αποτέλεσαν διαδοχικά και αλληλένδετα στάδια της ίδιας διαδικασίας.

Παρά τους περιορισμούς που προκύπτουν κυρίως από το μέγεθος και τη φύση του διαθέσιμου συνόλου δεδομένων, η εργασία παρέχει μία ολοκληρωμένη βάση πάνω στην οποία μπορούν να πραγματοποιηθούν περαιτέρω πειράματα και επεκτάσεις. Το τελικό σύστημα συνδέει τη Μηχανική Μάθηση, την Υπολογιστική Όραση και την ανάπτυξη διαδικτυακών εφαρμογών σε μία ενιαία υλοποίηση, αναδεικνύοντας παράλληλα τις δυνατότητες αλλά και τις προκλήσεις που παρουσιάζει η εφαρμογή σύγχρονων τεχνικών Τεχνητής Νοημοσύνης σε ένα αντικείμενο που σχετίζεται με την ελληνική πολιτιστική κληρονομιά.

ΒΙΒΛΙΟΓΡΑΦΙΑ ΚΑΙ ΠΗΓΕΣ

ΒΙΒΛΙΟΓΡΑΦΙΑ

[1] Ι. Βλαχάβας, Π. Κεφαλάς, Ν. Βασιλειάδης, Φ. Κόκκορας και Η. Σακελλαρίου, *Τεχνητή Νοημοσύνη*, 4η έκδ., Θεσσαλονίκη: Εκδόσεις Πανεπιστημίου Μακεδονίας.

[2] Κ. Διαμαντάρας και Δ. Μπότσης, *Μηχανική Μάθηση*. Αθήνα: Εκδόσεις Κλειδάριθμος.

ΠΗΓΕΣ

[3] Μηχανική όραση – Wikipedia

[4] Transfer Learning – Wikipedia

[5] Difference Between Fine-Tuning and Transfer Learning – GeeksforGeeks

[6] ImageNet – Wikipedia

[7] Residual Networks (ResNet) – Deep Learning – GeeksforGeeks

[8] EfficientNet Architecture – GeeksforGeeks

[9] ConvNeXt – GeeksforGeeks

[10] Macro Average vs Weighted Average – GeeksforGeeks

[11] Σταματίνα Μπογέα – Παραδοσιακές Ενδυμασίες

[12] Hamming Distance – GeeksforGeeks

[13] Data Leakage in Machine Learning – GeeksforGeeks

[14] Data Augmentation for Images – GeeksforGeeks